

1. Qt 概述
 - 1.1 简介
 - 1.2 优点
2. 创建 Qt 项目
 - 2.1 mian 程序解释
3. 命名规范与快捷键
 - 3.1 pro 文件的解释 (qmake 构建才有.pro 文件)
 - 3.1.1 Qt 模块
 - 3.2 头文件
 - 3.3 命令规范
 - 3.4 快捷键
4. QT窗口类
 - 4.1 基础窗口类
 - 4.1.1 QWidget
 - 4.2 窗口的相对坐标系
5. QT的数据类型
 - 5.1 QT的基本数据类型
 - 5.2 QT中终端窗口中输出日志
 - 5.3 QT的字符串类型
 - 5.3.1 QByteArray
 - 5.3.2 QString
 - 5.4 QVariant
 - 5.5 位置和尺寸
 - 5.5.1 QPoint
 - 5.5.2 QLine
 - 5.5.3 QSize
 - 5.5.4 QRect
 - 5.6 日期和时间
 - 5.6.1 QDate
 - 5.6.2 QTime
 - 5.6.3 QDateTime
6. 组件类
 - 6.1 QPushButton(按钮控件)
 - 6.1.1 按钮基类 QAbstractButton
 - 6.1.2 QPushButton
 - 6.1.3 QToolButton
 - 6.1.4 QRadioButton
 - 6.1.5 QCheckBox
 - 6.1.6 基本简介
 - 6.1.7 信号
 - 6.1.8 槽函数
 - 6.1.9 示例
 - 6.2 QUdpSocket(网络控件)
 - 6.2.1 基本简介
 - 6.3 QMainWindow(主窗口类)
 - 6.3.1 基本简介
 - 6.3.2 菜单栏(一个窗口最多一个)
 - 6.3.3 工具栏(一个窗口可以有多个)
 - 6.3.4 状态栏(最多只有一个)
 - 6.3.5 铆接部件(浮动控件 可以有多个) 和 核心控件(只能有一个)
 - 6.4 对话框(QDialog)
 - 6.4.1 模态对话框(打开以后, 不能操作其他窗口——阻塞其他窗口)——自定义对话框

- 6.4.2 非模态对话框(打开以后，能操作其他窗口——不阻塞其他窗口)——自定义对话框
- 6.4.3 消息对话框——QMessageBox(模态对话框)
- 6.4.4 QFileDialog
- 6.4.5 QFontDialog
 - 6.4.5.1 QFont 字体类
 - 6.4.5.2 QFontDialog的API
- 6.4.6 QColorDialog
 - 6.4.6.1 颜色类 QColor
 - 6.4.6.2 QColorDialog的API
- 6.4.7 QInputDialog
- 6.4.8 QProgressDialog
- 6.4.9 其他对话框
- 7. 容器控件
 - 7.1 QWidget
 - 7.2 Frame
 - 7.3 Group Box
 - 7.4 Scroll Area
 - 7.5 Tool Box
 - 7.6 Tab Widget
 - 7.7 Stacked Widget
- 8. 对象树
 - 8.1 示例
 - 8.2 qt 窗口坐标系
- 9. 信号和槽
 - 9.1 基本概念
 - 9.2 技巧
 - 9.3 自定义信号和槽
 - 9.4 自定义信号和槽重载问题解决
 - 9.5 拓展
 - 9.6 Lambda 表达式（用于一个信号连接多个槽函数 或者 多个信号连接一个槽函数）
- 10. 资源文件
 - 10.1 添加 ui 文件
 - 10.1.1 新建时添加 ui 文件
 - 10.1.2 已经创建的文件中添加 ui 文件
 - 10.2 使用 ui 文件
 - 10.3 添加资源文件
 - 10.3.1 创建资源文件
 - 10.3.2 回到资源文件界面
 - 10.3.3 添加资源
 - 10.3.4 使用
- 11. 界面布局
 - 11.1 布局的样式
 - 11.2 控件布局
 - 11.3 弹簧
 - 11.4 API实现
 - 11.4.1 QLayout
 - 11.4.2 QHBoxLayout
 - 11.4.3 QVBoxLayout
 - 11.4.4 GridLayout
- 12. ui 控件
 - 12.1 按钮控件
 - 12.1.1 工具按钮
 - 12.1.2 单选按钮

- 12.1.3 复选按钮
- 12.2 QListWidget 控件
- 12.3 QTreeWidget
- 12.4 QTableWidget
- 12.5 其他控件
 - 12.5.1 scroll Area(下拉条)
 - 12.5.2 Tool Box(类似抽屉)
 - 12.5.3 Tab Widget (类似网页切换)
 - 12.5.4 Stacked Widget (栈控件——翻页的切换页面)
 - 12.5.5 combo Box(下拉框) font Combo Box(字体下拉框)
- 12.6 自定义控件封装
 - 12.6.1 创建 ui 文件
 - 12.6.2 引入到其他窗口界面中
 - 12.6.3 给自己定义控件添加功能
- 13. QEvent(事件)
 - 13.1 鼠标事件
 - 13.2 定时器事件 (概念与 stm32 中学的基本类似)
 - 13.3 事件分发器(event())
 - 13.4 事件过滤器
 - 13.5 QPainter(绘图)
 - 13.5.1 绘图事件
 - 13.5.2 绘图高级设置
 - 13.5.3 手动调用绘图事件(利用绘图类画图片中的图)
 - 13.5.4 绘图设备
- 14. QFile(对文件进行读写操作)
 - 14.1 QFileInfo对文件信息的读取
- 15. 翻金币项目
 - 15.1 主窗口
 - 15.2 自定义按钮类
 - 15.3 选择关卡类
 - 15.4 翻金市场景
 - 15.5 金币类
 - 15.6 关卡数据类
- 16. 打包
 - 16.1 步骤1
 - 16.2 步骤2(只做到这一步, 需要使用者有对应的qt环境)
 - 16.3 步骤3(这一步之后, 就可以将这个文件夹压缩, 发给使用者)
 - 16.4 步骤4 (可省略 使用第三方, 形成安装程序 固定生成的是setup.exe的安装包)

1. Qt 概述

1.1 简介

Qt 是一个 跨平台 的 C++图形用户界面应用程序框架。

支持 WIndow、Linux、OS。

安装: <https://www.qt.io/>

安装教程: https://blog.csdn.net/qq_51355375/article/details/143222677

1.2 优点

- 跨平台
- 接口简单
- 一定程度简化内存回收机制
- 开发效率高
- 支持嵌入式开发

2. 创建 Qt 项目

- 第一步，点击 Qt Creator
- 第二步，New Project 创建新工程
- 第二步（其他方式），点击 Open Project 打开一个工程
- 第三步，选择 Application-》Qt Widgets Application

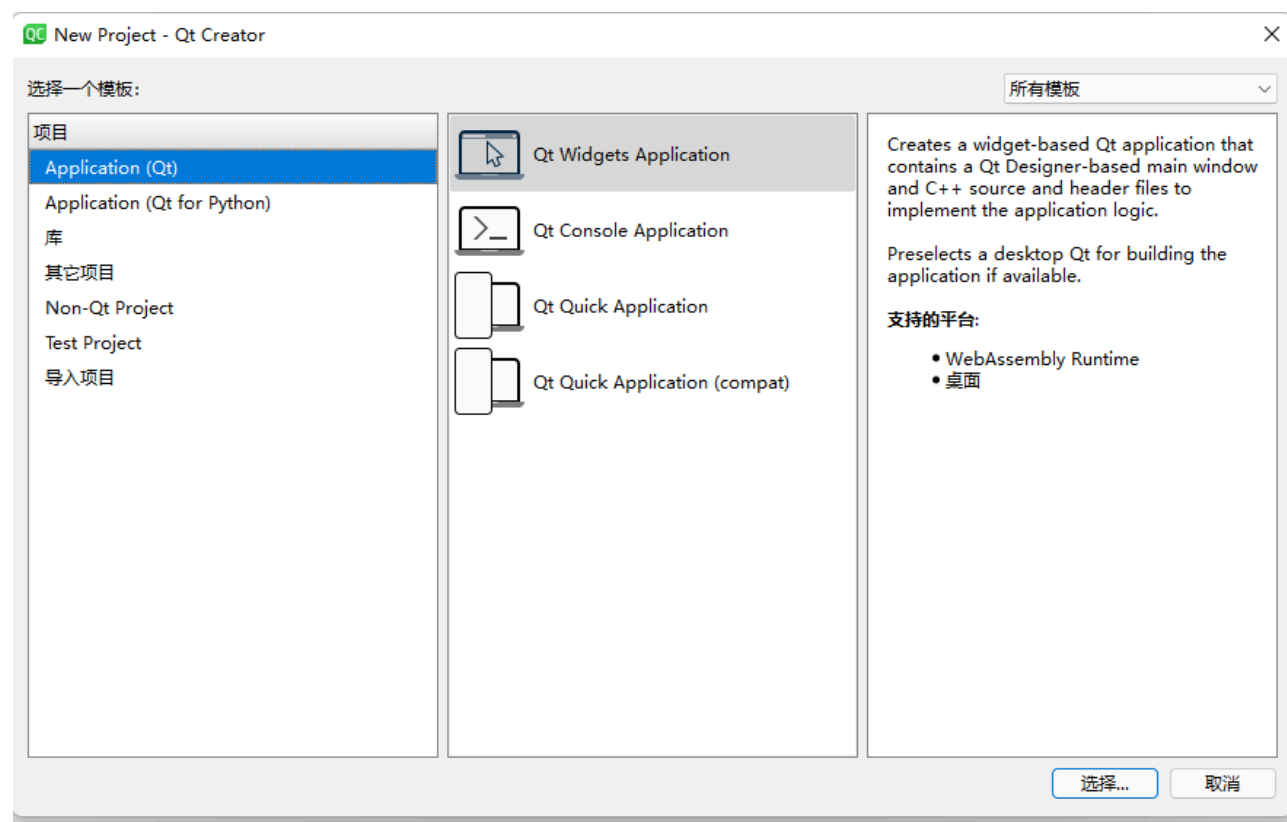


Figure 1

- 第四步，切记不能有中文或者空格。

Qt Widgets Application

Location

构建系统

Details

Translation

构建套件(Kit)

汇总

Project Location

This wizard generates a Qt Widgets Application project. The application derives by default from QApplication and includes an empty widget.

项目名称 二者均不能有中文或者空格

名称:

创建路径: 浏览...

☐ 设为默认的项目路径 路径

下一步(N)

取消

Figure 2

QWidget

QMainWindow

QWidget

QDialog

widget.cpp

第五步, ☐ Generate form

widget.ui

QMainWidow (界面会出现菜单栏、状态栏等) 与 QDialog (以弹出的对话框的形式出现) 都继承与 QWidget (空窗口)

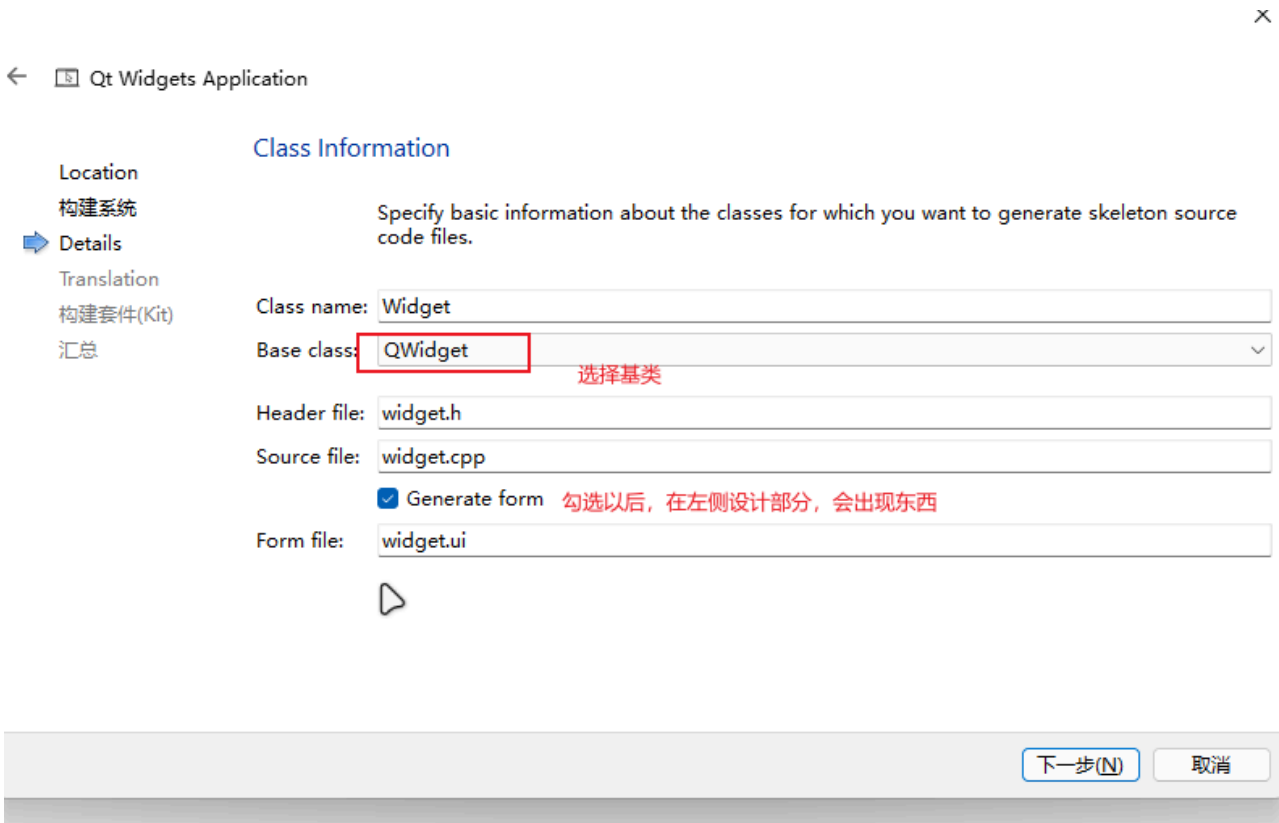


Figure 3

第六步, 选择自己所需要的环境。

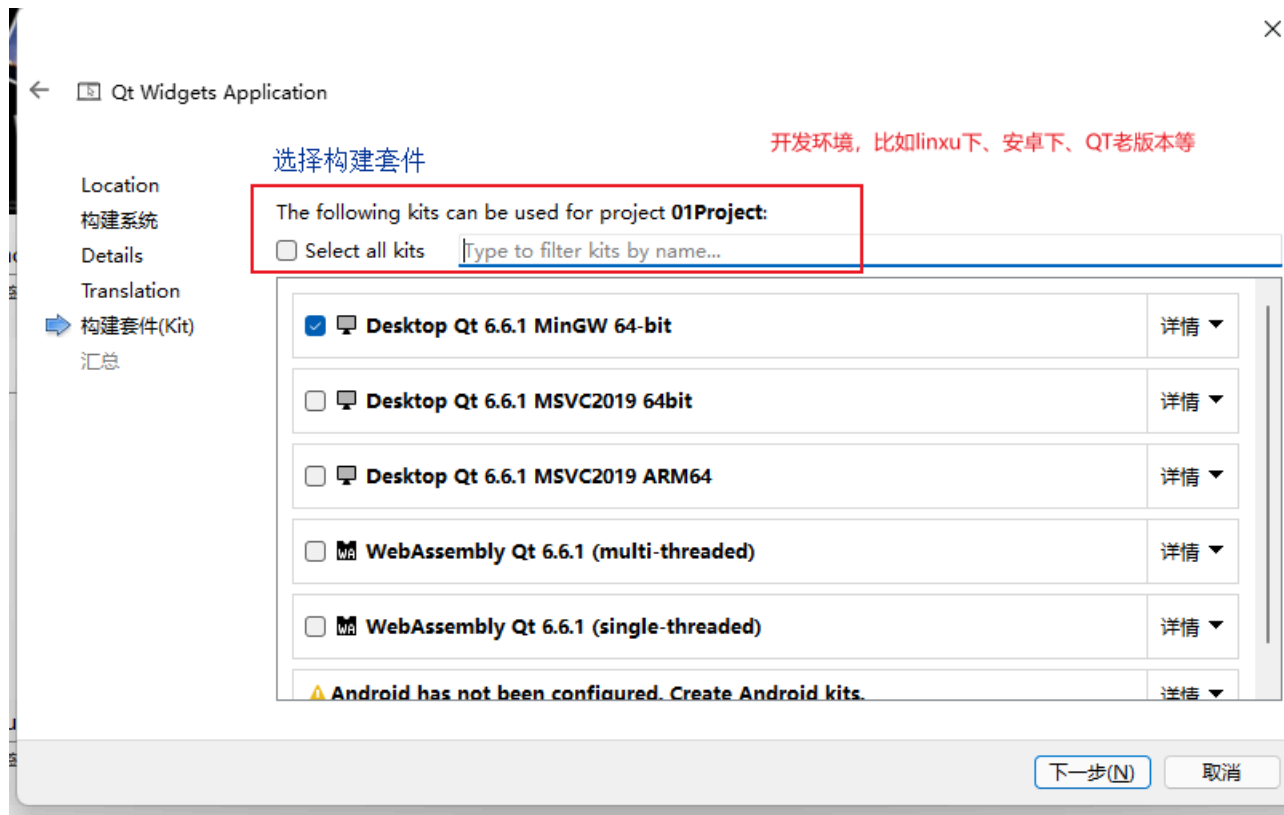


Figure 4

第七步, 选择自己所需要版本控制。没有就不选。

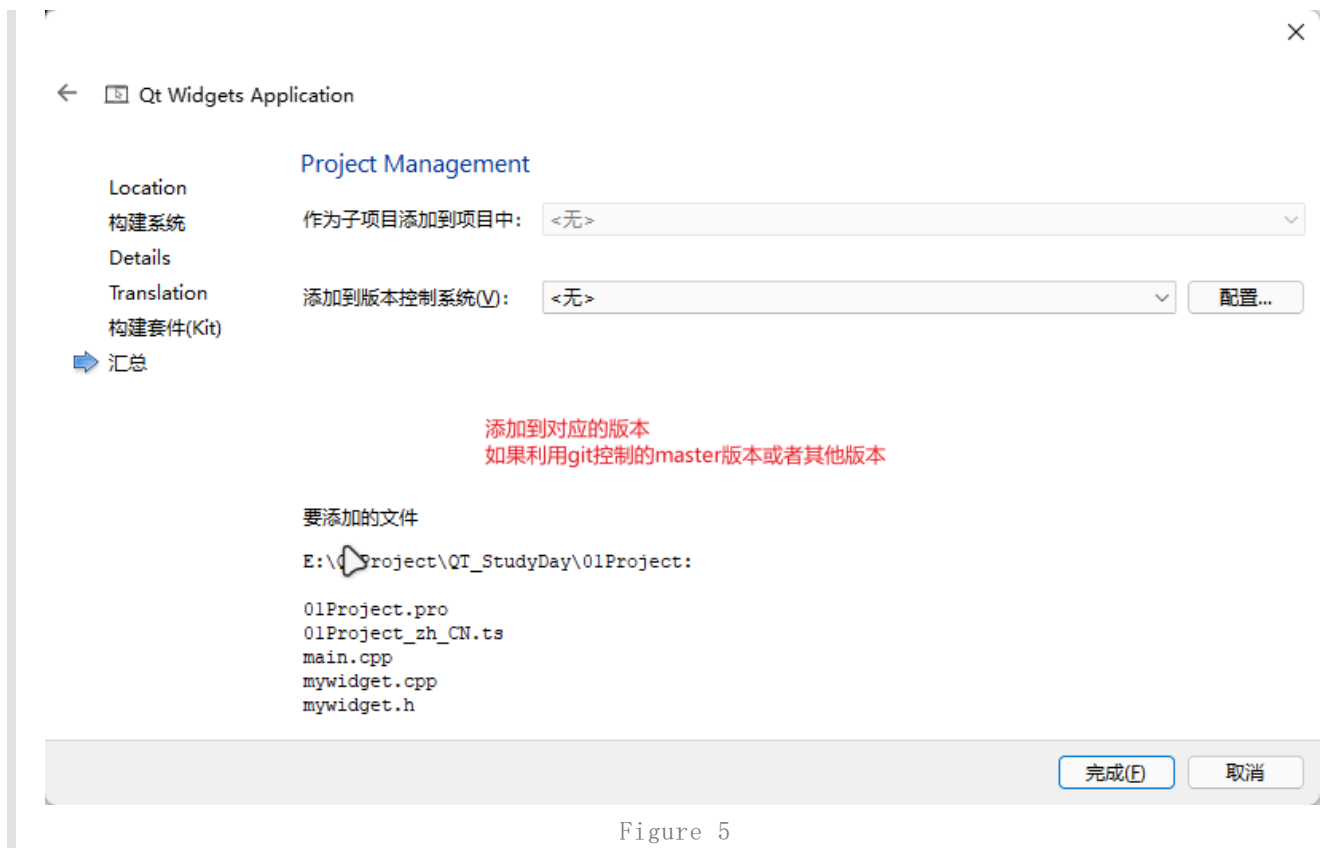


Figure 5

.pro 相应的工程文件（点击之后，能直接打开软件类似 keil5 的.uvprojx）

2.1 mian 程序解释

```

#include "mywidget.h"

#include <QApplication>
#include <QLocale>
#include <QTranslator>

/* 程序入口 argc: 命令行变量的数量 argv: 命令行变量数组 */
int main(int argc, char *argv[])
{
    QApplication a(argc, argv);    //a应用程序对象，在Qt中，应用程序对象，有且只有一个

    QTranslator translator;
    const QStringList uiLanguages = QLocale::system().uiLanguages();
    for (const QString &locale : uiLanguages) {
        const QString baseName = "01Project_" + QLocale(locale).name();
        if (translator.load(":/i18n/" + baseName)) {
            a.installTranslator(&translator);
            break;
        }
    }

    MyWidget w;                    //窗口对象，MyWidget的父类-》QWidget
    w.show();                      //窗口对象 默认不会显示 需要调用show方法才能显示 类似keil5中江协OLED中的更新函数
    return a.exec();              //让应用程序对象进入消息循环机制 类似keil5的main中while的作用，不同的是，点击x会退出循环
}

```

I

Figure 6

```

1  #ifndef MYWIDGET_H
2  #define MYWIDGET_H
3
4  #include <QWidget>
5
6  class MyWidget : public QWidget
7  {
8  |   Q_OBJECT
9
10 |   public:
11 |       MyWidget(QWidget *parent = nullptr);
12 |       ~MyWidget();
13 |   };
14 #endif // MYWIDGET_H
15

```

I

Figure 7

3. 命名规范与快捷键

3.1 pro 文件的解释（qmake 构建才有.pro 文件）

配置项	含义
QT += core gui	添加 Qt 模块 core：核心模块 gui：图形模块
greaterThan(QT_MAJOR_VERSION, 4): QT += widgets	Qt 版本大于 4，添加 widgets 模块
CONFIG += c++17	经常用的值，release：以发布模式编译程序，debug：以调试模式编译程序，warn_on：编译器输出尽可能多的警告，C++11：启用 c++11 标准支持
TARGET	指定生成的可执行程序的名字
TEMPLATE	指定如何运行当前程序，默认值为 app（Application），表示当前程序是应用程序，可直接编译运行，常用值还有 lib，表示将当前程序编译成库文件
DEFINES	在程序中定义一个宏，如同 c 文件中的#define xxx
SOURCES	指定项目中的所有 cpp 源文件
HEADERS	指定项目中的所有 h 头文件
FORMS	指定项目的所有 ui 文件
INCLUDEPATH	指定引入外部资源的头文件路径
LIBS	指定引入外部资源的指定库文件
qnx: target.path = /tmp/\${TARGET}/bin	当前平台为QNX（嵌入式实时操作系统），路径就为 /tmp/\${TARGET}/bin
else: unix:! android: target.path = /opt/\${TARGET}/bin	当前平台是unix而不是android，路径就为/opt/\${TARGET}/bin
! isEmpty(target.path): INSTALLS += target	如果 target.path 不为空，那么就添加 target 这个规则

Table 1

```
1 QT      += core gui
2
3 greaterThan(QT_MAJOR_VERSION, 4): QT += widgets
4
5 CONFIG += c++17
6
7 # You can make your code fail to compile if it uses deprecated APIs.
8 # In order to do so, uncomment the following line.
```

```

9  #DEFINES += QT_DISABLE_DEPRECATED_BEFORE=0x060000    # disables all the APIs
   deprecated before Qt 6.0.0
10
11  SOURCES += \
12      main.cpp \
13      mywidget.cpp
14
15  HEADERS += \
16      mywidget.h
17
18  TRANSLATIONS += \
19      01Project_zh_CN.ts
20  CONFIG += lrelease
21  CONFIG += embed_translations
22
23  # Default rules for deployment.
24  qnx: target.path = /tmp/${TARGET}/bin
25  else: unix:!android: target.path = /opt/${TARGET}/bin
26  !isEmpty(target.path): INSTALLS += target

```

Fence 1

3.1.1 Qt 模块

Qt Widgets: Qt5 以及之后的版本，才有将 Widgets 模块独立，之前版本将 Widgets 放在 GUI 模块中。

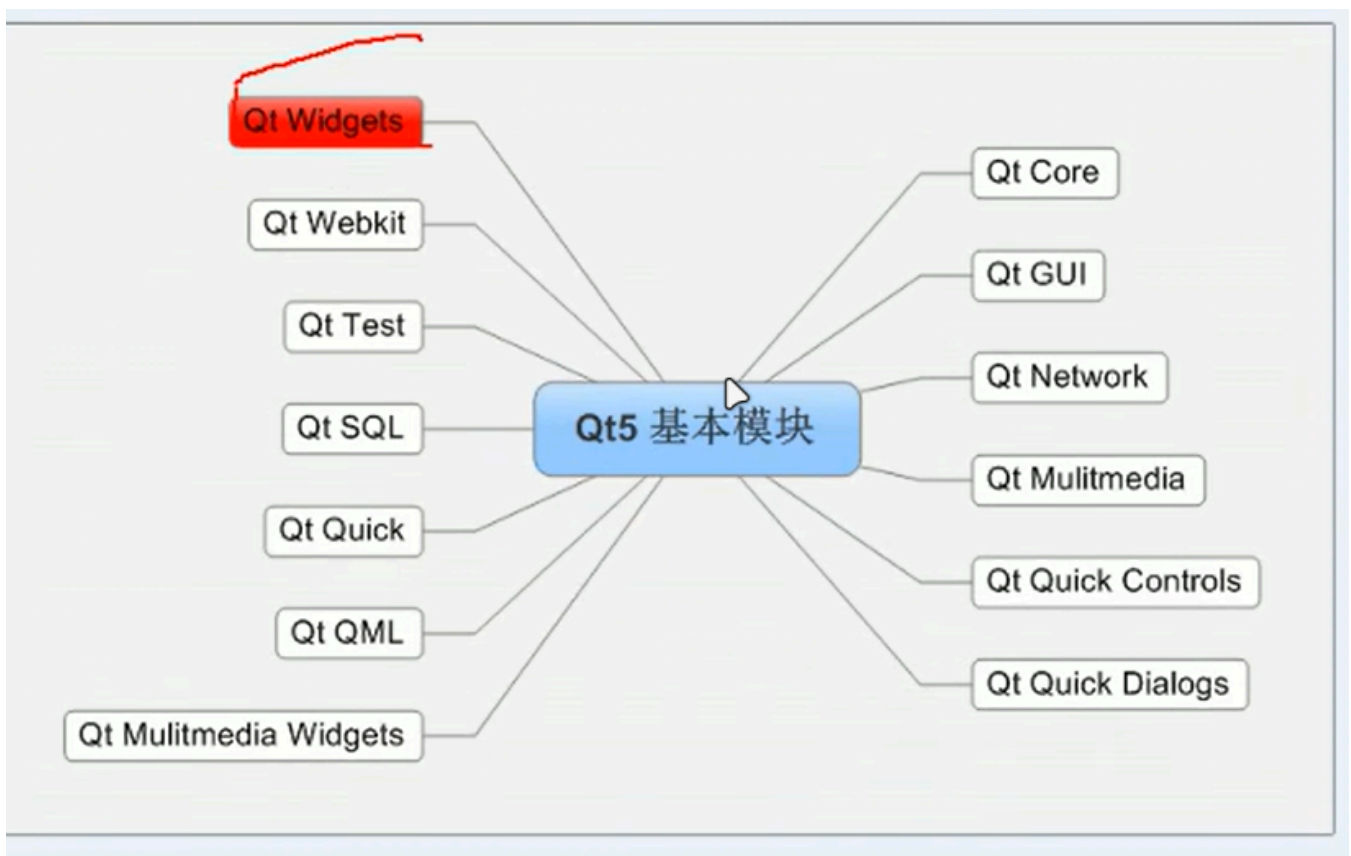


Figure 8

3.2 头文件

```
1  #pragma once          //防止头文件重复声明 (C++中)
2
3  #ifndef __XX_H_        //也是防止头文件重复声明
4  #define __XX_H_
5      xxxx
6  #endif
7
```

Fence 2

```
1  /* Mywidget.h */
2  #ifndef MYWIDGET_H
3  #define MYWIDGET_H
4
5  #include <Qwidget>      //Qwidget 窗口类
6
7  class Mywidget : public Qwidget    //继承与Qwidget类
8  {
9      Q_OBJECT            //Q_OBJECT宏，允许类中使用信号和槽的机制
10
11  public:
12      Mywidget(Qwidget *parent = nullptr);    //有参构造函数
13      ~Mywidget();                            //析构函数
14  };
15  #endif // MYWIDGET_H
16
17
18
19  /* Mywidget.cpp */
20  #include "mywidget.h"
21
22  Mywidget::Mywidget(Qwidget *parent)
23      : Qwidget(parent)    //使用初始化列表，初始化Mywidge类
24  {}
25
26  Mywidget::~~Mywidget() {}
```

Fence 3

3.3 命名规范

类名：首字母大写，单词与单词之间的首字母大写(大驼峰)

函数名、变量名：首字母小写、单词与单词之间首字母大写(小驼峰)

指针变量 -> ptr 指针名 eg. ptrstusql

成员变量 -> m_成员变量名 eg. m_age

3.4 快捷键

注释: CTRL+/
运行: CTRL+r

编译: CTRL+b

字体缩放: CTRL+鼠标滚轮

查找: CTRL+f

整行移动: CTRL+↓ 或者 ↑

帮助文档: 选中+F1

第一种: F1

第二种: 左侧帮助按钮

第三种: "E:\Qt\6.6.1\mingw_64\bin\assistant.exe"

自动对齐: CTRL+i

同名之间的.h 和.cpp 切换: F4

选取列: alt+左键

快速将.h 中的函数在.cpp 中定义: alt+enter

重写虚函数不提示: 帮助-> 关于插件-> c++-> ClangCodeModel-> 取消勾选-> 重启 qt

4. QT窗口类

4.1 基础窗口类

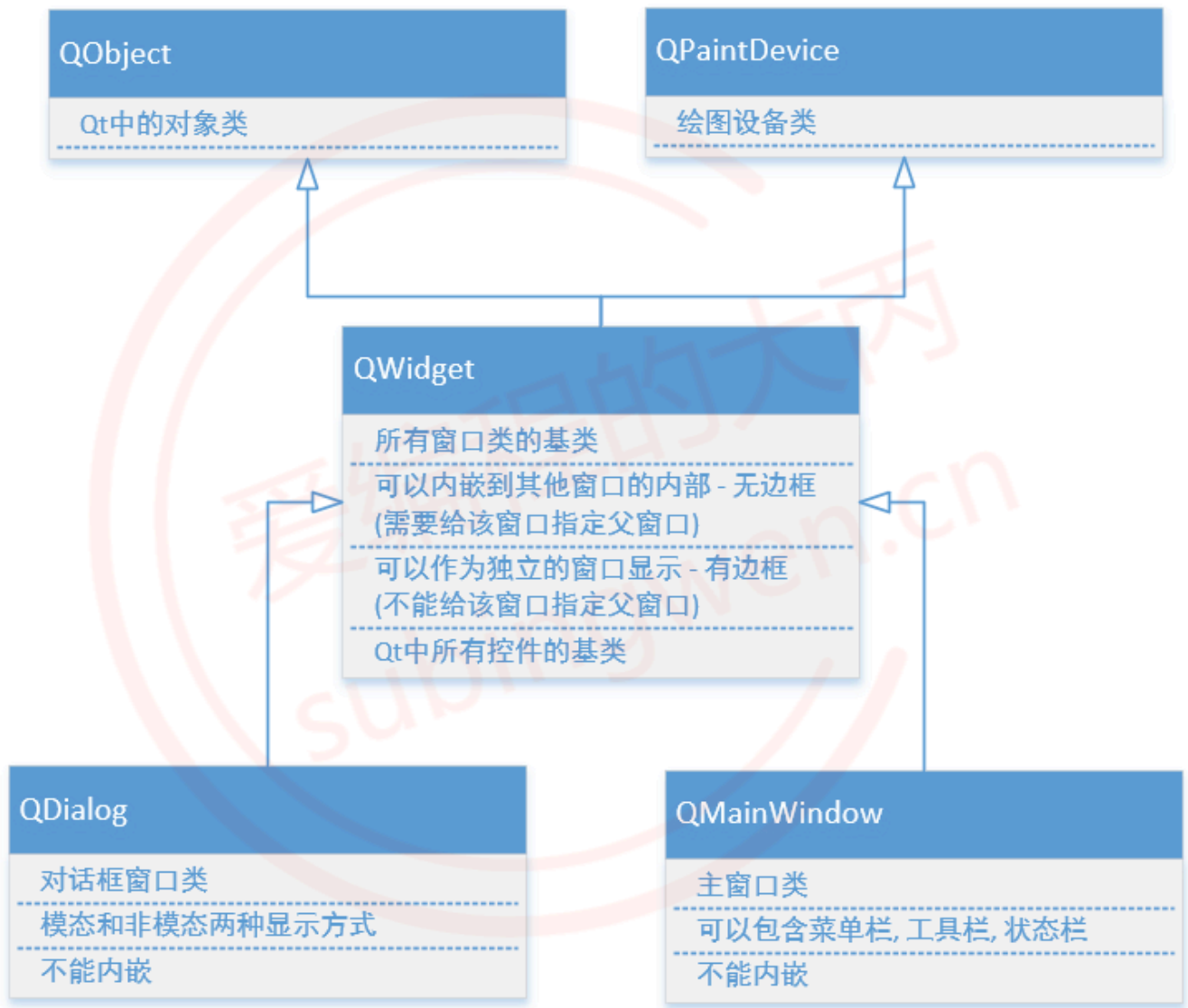


Figure 9

- 常用的窗口类有3个
 - 在创建Qt窗口的时候, 需要让自己的窗口类继承上述三个窗口类的其中一个
- QWidget
 - 所有窗口类的基类
 - Qt中的控件(按钮, 输入框, 单选框...)也属于窗口, 基类都是QWidget
 - 可以内嵌到其他窗口中: 没有边框
 - 可以不内嵌单独显示: 独立的窗口, 有边框
- QDialog
 - 对话框类
 - 不能内嵌到其他窗口中
- QMainWindow
 - 有工具栏, 状态栏, 菜单栏, 后边的章节会具体介绍这个窗口
 - 不能内嵌到其他窗口中

4.1.1 QWidget

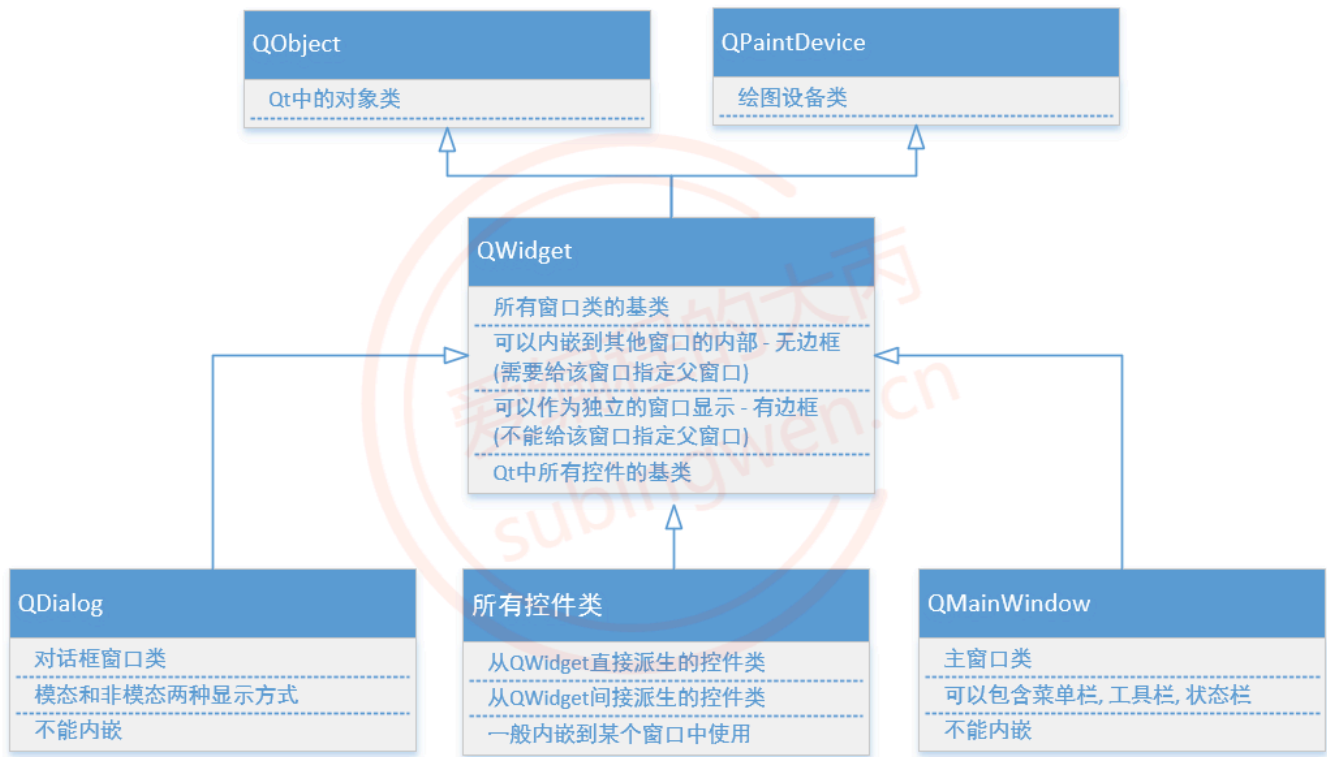


Figure 10

```
1 //设置父对象
2 // 构造函数
3 QWidget::QWidget(QWidget *parent = nullptr, Qt::WindowFlags f = Qt::windowFlags());
4
5 // 公共成员函数
6 // 给当前窗口设置父对象
7 void QWidget::setParent(QWidget *parent);
8 void QWidget::setParent(QWidget *parent, Qt::WindowFlags f);
9 // 获取当前窗口的父对象, 没有父对象返回 nullptr
10 QWidget *QWidget::parentWidget() const;
11
12
13
14 //窗口位置
15 //----- 窗口位置 -----
16 // 得到相对于当前窗口父窗口的几何信息, 边框也被计算在内
17 QRect QWidget::frameGeometry() const;
18 // 得到相对于当前窗口父窗口的几何信息, 不包括边框
19 const QRect &geometry() const;
20 // 设置当前窗口的几何信息(位置和尺寸信息), 不包括边框
21 void setGeometry(int x, int y, int w, int h);
22 void setGeometry(const QRect &);
23
24 // 移动窗口, 重新设置窗口的位置
25 void move(int x, int y);
26 void move(const QPoint &);
27
28
```

```

29 //窗口位置设定和位置获取的测试代码如下：
30 // 获取当前窗口的位置信息
31 void MainWindow::on_positionBtn_clicked()
32 {
33     QRect rect = this->frameGeometry();
34     qDebug() << "左上角: " << rect.topLeft()
35             << "右上角: " << rect.topRight()
36             << "左下角: " << rect.bottomLeft()
37             << "右下角: " << rect.bottomRight()
38             << "宽度: " << rect.width()
39             << "高度: " << rect.height();
40 }
41
42 // 重新设置当前窗口的位置以及宽度，高度
43 void MainWindow::on_geometryBtn_clicked()
44 {
45     int x = 100 + rand() % 500;
46     int y = 100 + rand() % 500;
47     int width = this->width() + 10;
48     int height = this->height() + 10;
49     setGeometry(x, y, width, height);
50 }
51
52 // 通过 move() 方法移动窗口
53 void MainWindow::on_moveBtn_clicked()
54 {
55     QRect rect = this->frameGeometry();
56     move(rect.topLeft() + QPoint(10, 20));
57 }
58
59
60
61
62
63 //窗口尺寸
64 //----- 窗口尺寸 -----
65 // 获取当前窗口的尺寸信息
66 QSize size() const
67 // 重新设置窗口的尺寸信息
68 void resize(int w, int h);
69 void resize(const QSize &);
70 // 获取当前窗口的最大尺寸信息
71 QSize maximumSize() const;
72 // 获取当前窗口的最小尺寸信息
73 QSize minimumSize() const;
74 // 设置当前窗口固定的尺寸信息
75 void QWidget::setFixedSize(const QSize &s);
76 void QWidget::setFixedSize(int w, int h);
77 // 设置当前窗口的最大尺寸信息
78 void setMaximumSize(const QSize &);
79 void setMaximumSize(int maxw, int maxh);
80 // 设置当前窗口的最小尺寸信息
81 void setMinimumSize(const QSize &);

```

```

82 void setMinimumSize(int minw, int minh);
83
84
85 // 获取当前窗口的高度
86 int height() const;
87 // 获取当前窗口的最小高度
88 int minimumHeight() const;
89 // 获取当前窗口的最大高度
90 int maximumHeight() const;
91 // 给窗口设置固定的高度
92 void QWidget::setFixedHeight(int h);
93 // 给窗口设置最大高度
94 void setMaximumHeight(int maxh);
95 // 给窗口设置最小高度
96 void setMinimumHeight(int minh);
97
98 // 获取当前窗口的宽度
99 int width() const;
100 // 获取当前窗口的最小宽度
101 int minimumWidth() const;
102 // 获取当前窗口的最大宽度
103 int maximumWidth() const;
104 // 给窗口设置固定宽度
105 void QWidget::setFixedWidth(int w);
106 // 给窗口设置最大宽度
107 void setMaximumWidth(int maxw);
108 // 给窗口设置最小宽度
109 void setMinimumWidth(int minw);
110
111
112
113
114 //窗口标题和图标
115 //----- 窗口图标 -----
116 // 得到当前窗口的图标
117 QIcon windowIcon() const;
118 // 构造图标对象，参数为图片的路径
119 QIcon::QIcon(const QString &fileName);
120 // 设置当前窗口的图标
121 void setWindowIcon(const QIcon &icon);
122
123 //----- 窗口标题 -----
124 // 得到当前窗口的标题
125 QString windowTitle() const;
126 // 设置当前窗口的标题
127 void setWindowTitle(const QString &);
128
129
130
131 //信号
132 // QWidget::setContextMenuPolicy(Qt::ContextMenuPolicy policy);
133 // 窗口的右键菜单策略 contextMenuPolicy() 参数设置为 Qt::CustomContextMenu，按下鼠标右键发射该信号

```

```

134 [signal] void QWidget::customContextMenuRequested(const QPoint &pos);
135 // 窗口图标发生变化，发射此信号
136 [signal] void QWidget::windowIconChanged(const QIcon &icon);
137 // 窗口标题发生变化，发射此信号
138 [signal] void QWidget::windowTitleChanged(const QString &title);
139
140
141
142 //槽函数
143 //----- 窗口显示 -----
144 // 关闭当前窗口
145 [slot] bool QWidget::close();
146 // 隐藏当前窗口
147 [slot] void QWidget::hide();
148 // 显示当前创建以及其子窗口
149 [slot] void QWidget::show();
150 // 全屏显示当前窗口，只对windows有效
151 [slot] void QWidget::showFullScreen();
152 // 窗口最大化显示，只对windows有效
153 [slot] void QWidget::showMaximized();
154 // 窗口最小化显示，只对windows有效
155 [slot] void QWidget::showMinimized();
156 // 将窗口回复为最大化/最小化之前的状态，只对windows有效
157 [slot] void QWidget::showNormal();
158
159 //----- 窗口状态 -----
160 // 判断窗口是否可用
161 bool QWidget::isEnabled() const; // 非槽函数
162 // 设置窗口是否可用，不可用窗口无法接收和处理窗口事件
163 // 参数true->可用，false->不可用
164 [slot] void QWidget::setEnabled(bool);
165 // 设置窗口是否可用，不可用窗口无法接收和处理窗口事件
166 // 参数true->不可用，false->可用
167 [slot] void QWidget::setEnabled(bool disable);
168 // 设置窗口是否可见，参数为true->可见，false->不可见
169 [slot] virtual void QWidget::setVisible(bool visible);
170
171
172
173

```

4.2 窗口的相对坐标系

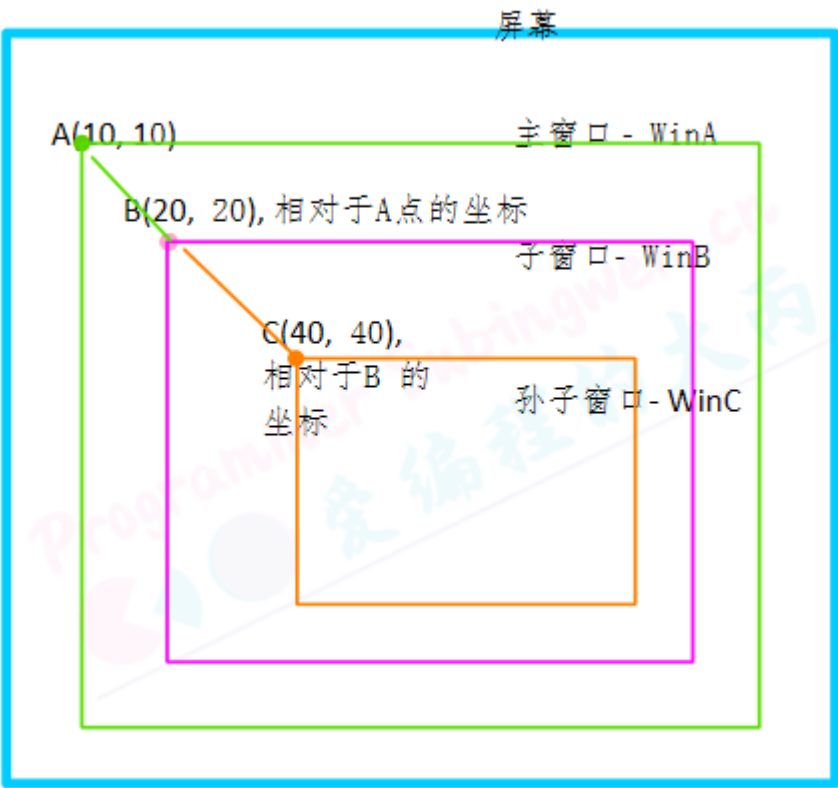


Figure 11

1 在一个Qt窗口中一般都有很多子窗口内嵌到这个父窗口中，其中每个窗口都有自己的坐标原点，子窗口的位置也就是其使用的坐标点就是它的父窗口坐标体系中的坐标点`

Fence 5

- 在Qt的某一个窗口中有可能有若干个控件, 这个控件都是嵌套的关系
 - A窗口包含B窗口, B窗口包含C窗口
- 每个窗口都有坐标原点, 在左上角
 - 子窗口的位置是基于父窗口的坐标体系来确定的, 也就是说通过父窗口左上角的坐标点来确定自己的位置
- Qt中窗口显示的时候使用的相对坐标, 相对于自己的父窗口

5. QT的数据类型

5.1 QT的基本数据类型

- 因为Qt是一个C++ 框架, 因此C++中所有的语法和数据类型在Qt中都是被支持的, 但是Qt中也定义了一些属于自己的数据类型, 下边给大家介绍一下这些基础的数类型。
- QT基本数据类型定义在 `#include` 中，QT基本数据类型有：

Qt类型名称	注释	备注
qint8	signed char	有符号8位数据类型

Qt类型名称	注释	备注
qint16	signed short	有符号16位数据类型
qint32	signed int	有符号32位数据类型
qint64	long long int 或(int64)	有符号64位数据类型，Windows中定义为_int64
quint8	unsigned char	无符号8位数据类型
quint16	unsigned short	无符号16位数据类型
quint32	unsigned int	无符号32位数据类型
quint64	unsigned long long int 或 unsigned int64	无符号64位数据类型，Windows中定义为unsigned int64
qintptr	qint32 或 qint64	指针类型，32位系统为qint32，64位系统为qint64
quintptr	quint32 或 quint64	指针类型，32位系统为quint32，64位系统为 quint64
qlonglong	long long int 或 int64	有符号64位数据类型，Windows中定义为__int64
qulonglong	unsigned long long int 或 unsigned int64	无符号64位数据类型，Windows中定义为unsigned int64
qptrdiff	qint32 或 qint64	指针差类型，32位系统为qint32，64位系统为 qint64
qreal	double 或 float	默认为double，除非配置了-qreal float选项
uchar	unsigned char	无符号字符类型
uint	unsigned int	无符号整型
ulong	unsigned long	无符号长整型
ushort	unsigned short	无符号短整型

Table 2

- 虽然在Qt中有属于自己的整形或者浮点型, 但是在变成过程中这些一般不用, 常用的类型关键字还是 C/C++中的 int, float, double 等。

5.2 QT中终端窗口中输出日志

- 使用上面的方法只能在项目调试过程中进行日志输出, 如果不是通过IDE进行程序调试（使用qDebug），而是直接执行可执行程序在这种情况下是没有日志输出窗口的, 因此也就看不到任何的日志输出。
默认情况下日志信息是不会打印到终端窗口的, 如果想要实现这样的效果, 必须在项目文件中添加相关的属性信息
- 打开项目文件 (*.pro或者) 找到配置项 config, 添加 console 控制台属性:

```

1  CONFIG += c++11 console
2
3  或者
4
5  # 设置日志输出到终端
6  # 在Qt中, qDebug()默认是输出到QDebug的默认消息处理器的, 通常是IDE的输出窗口。
7  # 如果你想在终端中看到输出, 确保在运行时没有重定向QDebug的输出。
8  # 你可以通过宏定义来控制, 或者在代码中设置。
9
10 # 方法一: 通过编译选项设置宏定义
11 target_compile_definitions(${PROJECT_NAME} PRIVATE QT_MESSAGELOG_FILE= "")
12
13 # 方法二: 在代码中设置
14 # 你也可以在你的代码中通过QLoggingCategory来设置日志输出, 例如在main.cpp中:
15 # #include <QLoggingCategory>
16 # int main(int argc, char *argv[]) {
17 #     QLoggingCategory::setFilterRules("*.debug=true");
18 #     // 其他代码
19 # }
20
21 # 方法三: 运行时设置环境变量
22 # 在运行可执行文件之前, 设置QT_MESSAGE_PATTERN环境变量, 例如:
23 # export QT_MESSAGE_PATTERN="%{message}\n"
24 # 这将使日志信息以简单的消息格式输出到终端

```

Fence 6

5.3 QT的字符串类型

5.3.1 QByteArray

- 在Qt中QByteArray可以看做是c语言中 char*的升级版。我们在使用这种类型的时候可通过这个类的构造函数申请一块动态内存, 用于存储我们需要处理的字符串数据。

- ```

1 //构造函数
2 // 构造空对象, 里边没有数据
3 QByteArray::QByteArray();
4 // 将data中的size个字符进行构造, 得到一个字节数组对象
5 // 如果 size==-1 函数内部自动计算字符串长度, 计算方式为: strlen(data)
6 QByteArray::QByteArray(const char *data, int size = -1);
7 // 构造一个长度为size个字节, 并且每个字节值都为ch的字节数组
8 QByteArray::QByteArray(int size, char ch);
9
10 //数据操作
11 // 在尾部追加数据
12 // 其他重载的同名函数可参考Qt帮助文档, 此处略
13 QByteArray &QByteArray::append(const QByteArray &ba);
14 void QByteArray::push_back(const QByteArray &other);
15
16 // 头部添加数据
17 // 其他重载的同名函数可参考Qt帮助文档, 此处略
18 QByteArray &QByteArray::prepend(const QByteArray &ba);
19 void QByteArray::push_front(const QByteArray &other);

```

```

20
21 // 插入数据, 将ba插入到数组第 i 个字节的位置(从0开始)
22 // 其他重载的同名函数可参考Qt帮助文档, 此处略
23 QByteArray &QByteArray::insert(int i, const QByteArray &ba);
24
25 // 删除数据
26 // 从大字符串中删除len个字符, 从第pos个字符的位置开始删除
27 QByteArray &QByteArray::remove(int pos, int len);
28 // 从字符数组的尾部删除 n 个字节
29 void QByteArray::chop(int n);
30 // 从字节数组的 pos 位置将数组截断 (前边部分留下, 后边部分被删除)
31 void QByteArray::truncate(int pos);
32 // 将对象中的数据清空, 使其为null
33 void QByteArray::clear();
34
35 // 字符串替换
36 // 将字节数组中的 子字符串 before 替换为 after
37 // 其他重载的同名函数可参考Qt帮助文档, 此处略
38 QByteArray &QByteArray::replace(const QByteArray &before, const QByteArray
&after);
39
40
41 //字符串查找和判断
42 // 在尾部追加数据
43 // 其他重载的同名函数可参考Qt帮助文档, 此处略
44 QByteArray &QByteArray::append(const QByteArray &ba);
45 void QByteArray::push_back(const QByteArray &other);
46
47 // 头部添加数据
48 // 其他重载的同名函数可参考Qt帮助文档, 此处略
49 QByteArray &QByteArray::prepend(const QByteArray &ba);
50 void QByteArray::push_front(const QByteArray &other);
51
52 // 插入数据, 将ba插入到数组第 i 个字节的位置(从0开始)
53 // 其他重载的同名函数可参考Qt帮助文档, 此处略
54 QByteArray &QByteArray::insert(int i, const QByteArray &ba);
55
56 // 删除数据
57 // 从大字符串中删除len个字符, 从第pos个字符的位置开始删除
58 QByteArray &QByteArray::remove(int pos, int len);
59 // 从字符数组的尾部删除 n 个字节
60 void QByteArray::chop(int n);
61 // 从字节数组的 pos 位置将数组截断 (前边部分留下, 后边部分被删除)
62 void QByteArray::truncate(int pos);
63 // 将对象中的数据清空, 使其为null
64 void QByteArray::clear();
65
66 // 字符串替换
67 // 将字节数组中的 子字符串 before 替换为 after
68 // 其他重载的同名函数可参考Qt帮助文档, 此处略
69 QByteArray &QByteArray::replace(const QByteArray &before, const QByteArray
&after);
70

```

```

71
72 //遍历
73 // 使用迭代器
74 iterator QByteArray::begin();
75 iterator QByteArray::end();
76
77 // 使用数组的方式进行遍历
78 // i的取值范围 0 <= i < size()
79 char QByteArray::at(int i) const;
80 char QByteArray::operator[](int i) const;
81
82
83
84 //查看字节数
85 // 返回字节数组对象中字符的个数
86 int QByteArray::length() const;
87 int QByteArray::size() const;
88 int QByteArray::count() const;
89
90 // 返回字节数组对象中 子字符串ba 出现的次数
91 int QByteArray::count(const QByteArray &ba) const;
92 int QByteArray::count(const char *ba) const;
93 // 返回字节数组对象中 字符串ch 出现的次数
94 int QByteArray::count(char ch) const;
95
96
97 //类型转换
98 // 将QByteArray类型的字符串 转换为 char* 类型
99 char *QByteArray::data();
100 const char *QByteArray::data() const;
101
102 // int, short, long, float, double -> QByteArray
103 // 其他重载的同名函数可参考Qt帮助文档，此处略
104 QByteArray &QByteArray::setNum(int n, int base = 10);
105 QByteArray &QByteArray::setNum(short n, int base = 10);
106 QByteArray &QByteArray::setNum(qlonglong n, int base = 10);
107 QByteArray &QByteArray::setNum(float n, char f = 'g', int prec = 6);
108 QByteArray &QByteArray::setNum(double n, char f = 'g', int prec = 6);
109 [static] QByteArray QByteArray::number(int n, int base = 10);
110 [static] QByteArray QByteArray::number(qlonglong n, int base = 10);
111 //g表示使用科学技术法
112 [static] QByteArray QByteArray::number(double n, char f = 'g', int prec = 6);
113
114
115 // QByteArray -> int, short, long, float, double
116 int QByteArray::toInt(bool *ok = Q_NULLPTR, int base = 10) const;
117 short QByteArray::toShort(bool *ok = Q_NULLPTR, int base = 10) const;
118 long QByteArray::toLong(bool *ok = Q_NULLPTR, int base = 10) const;
119 float QByteArray::toFloat(bool *ok = Q_NULLPTR) const;
120 double QByteArray::toDouble(bool *ok = Q_NULLPTR) const;
121
122 // std::string -> QByteArray
123 [static] QByteArray QByteArray::fromStdString(const std::string &str);

```

```

124 // QByteArray -> std::string
125 std::string QByteArray::toStdString() const;
126
127 // 所有字符转换为大写
128 QByteArray QByteArray::toUpper() const;
129 // 所有字符转换为小写
130 QByteArray QByteArray::toLower() const;
131
132

```

Fence 7

### 5.3.2 QString

- QString也是封装了字符串,但是内部的编码为utf8, UTF-8属于Unicode字符集,它固定使用多个字节 (window为2字节, linux为3字节) 来表示一个字符, 这样可以将世界上几乎所有语言的常用字符收录其中。

- ```

1 //构造函数
2 // 构造一个空字符串对象
3 QString::QString();
4 // 将 char* 字符串 转换为 QString 类型
5 QString::QString(const char *str);
6 // 将 QByteArray 转换为 QString 类型
7 QString::QString(const QByteArray &ba);
8 // 其他重载的同名构造函数可参考Qt帮助文档, 此处略
9
10
11 //数据操作
12 // 尾部追加数据
13 // 其他重载的同名函数可参考Qt帮助文档, 此处略
14 QString &QString::append(const QString &str);
15 QString &QString::append(const char *str);
16 QString &QString::append(const QByteArray &ba);
17 void QString::push_back(const QString &other);
18
19 // 头部添加数据
20 // 其他重载的同名函数可参考Qt帮助文档, 此处略
21 QString &QString::prepend(const QString &str);
22 QString &QString::prepend(const char *str);
23 QString &QString::prepend(const QByteArray &ba);
24 void QString::push_front(const QString &other);
25
26 // 插入数据, 将 str 插入到字符串第 position 个字符的位置(从0开始)
27 // 其他重载的同名函数可参考Qt帮助文档, 此处略
28 QString &QString::insert(int position, const QString &str);
29 QString &QString::insert(int position, const char *str);
30 QString &QString::insert(int position, const QByteArray &str);
31
32 // 删除数据
33 // 从大字符串中删除len个字符, 从第pos个字符的位置开始删除
34 QString &QString::remove(int position, int n);
35
36 // 从字符串的尾部删除 n 个字符

```

```

37 void QString::chop(int n);
38 // 从字节串的 position 位置将字符串截断（前边部分留下，后边部分被删除）
39 void QString::truncate(int position);
40 // 将对象中的数据清空，使其为null
41 void QString::clear();
42
43 // 字符串替换
44 // 将字节数组中的 子字符串 before 替换为 after
45 // 参数 cs 为是否区分大小写，默认区分大小写
46 // 其他重载的同名函数可参考Qt帮助文档，此处略
47 QString &QString::replace(const QString &before, const QString &after,
48                             Qt::CaseSensitivity cs = Qt::CaseSensitive);
49
50
51
52 //字符串查找和判断
53 // 参数 cs 为是否区分大小写，默认区分大小写
54 // 其他重载的同名函数可参考Qt帮助文档，此处略
55
56 // 判断字符串中是否包含子字符串 str，包含返回true，否则返回false
57 bool QString::contains(const QString &str, Qt::CaseSensitivity cs =
58                         Qt::CaseSensitive) const;
59
60 // 判断字符串是否以字符串 ba 开始，是返回true，不是返回false
61 bool QString::startsWith(const QString &s, Qt::CaseSensitivity cs =
62                           Qt::CaseSensitive) const;
63
64 // 判断字符串是否以字符串 ba 结尾，是返回true，不是返回false
65 bool QString::endsWith(const QString &s, Qt::CaseSensitivity cs =
66                         Qt::CaseSensitive) const;
67
68 //遍历
69 // 使用迭代器
70 iterator QString::begin();
71 iterator QString::end();
72
73 // 使用数组的方式进行遍历
74 // i的取值范围 0 <= position < size()
75 const QChar QString::at(int position) const
76 const QChar QString::operator[](int position) const;
77
78
79
80
81 //查看字节数
82 // 返回字节数组对象中字符的个数（字符个数和字节个数是不同的概念）
83 int QString::length() const;
84 int QString::size() const;
85 int QString::count() const;

```

```

86
87 // 返回字节串对象中 子字符串 str 出现的次数
88 // 参数 cs 为是否区分大小写，默认区分大小写
89 int QString::count(const QStringRef &str, Qt::CaseSensitivity cs =
    Qt::CaseSensitive) const;
90
91
92
93
94
95 //类型转换
96 // 将int, short, long, float, double 转换为 QString 类型
97 // 其他重载的同名函数可参考Qt帮助文档，此处略
98 QString &QString::setNum(int n, int base = 10);
99 QString &QString::setNum(short n, int base = 10);
100 QString &QString::setNum(long n, int base = 10);
101 QString &QString::setNum(float n, char format = 'g', int precision = 6);
102 QString &QString::setNum(double n, char format = 'g', int precision = 6);
103 [static] QString QString::number(long n, int base = 10);
104 [static] QString QString::number(int n, int base = 10);
105 [static] QString QString::number(double n, char format = 'g', int precision =
    6);
106
107 // 将 QString 转换为 int, short, long, float, double 类型
108 int QString::toInt(bool *ok = Q_NULLPTR, int base = 10) const;
109 short QString::toShort(bool *ok = Q_NULLPTR, int base = 10) const;
110 long QString::toLong(bool *ok = Q_NULLPTR, int base = 10) const
111 float QString::toFloat(bool *ok = Q_NULLPTR) const;
112 double QString::toDouble(bool *ok = Q_NULLPTR) const;
113
114 // 将标准C++中的 std::string 类型 转换为 QString 类型
115 [static] QString QString::fromStdString(const std::string &str);
116 // 将 QString 转换为 标准C++中的 std::string 类型
117 std::string QString::toStdString() const;
118
119 // QString -> QByteArray
120 // 转换为本地编码，跟随操作系统
121 QByteArray QString::toLocal8Bit() const;
122 // 转换为 Latin-1 编码的字符串 不支持中文
123 QByteArray QString::toLatin1() const;
124 // 转换为 utf8 编码格式的字符串（常用）
125 QByteArray QString::toUtf8() const;
126
127 // 所有字符转换为大写
128 QString QString::toUpper() const;
129 // 所有字符转换为小写
130 QString QString::toLower() const;
131
132
133
134
135

```

5.4 QVariant

- QVariant这个类很神奇，或者说方便。很多时候，需要几种不同的数据类型需要传递，如果用结构体，又不大方便，容器保存的也只是一种数据类型，而QVariant则可以统统搞定。

QVariant 这个类型充当着最常见的数据类型的联合。QVariant 可以保存很多Qt的数据类型，包括QBrush、QColor、QCursor、QDateTime、QFont、QKeySequence、QPalette、QPen、QPixmap、QPoint、QRect、QRegion、QSize和QString，并且还有C++基本类型，如 int、float等。

(类似泛型机制)

- 标准类型转换成QVariant

```
1 // 这类转换需要使用QVariant类的构造函数，由于比较多，大家可自行查阅Qt帮助文档，在这里简单写几个
2 QVariant::QVariant(int val);
3 QVariant::QVariant(bool val);
4 QVariant::QVariant(double val);
5 QVariant::QVariant(const char *val);
6 QVariant::QVariant(const QByteArray &val);
7 QVariant::QVariant(const QString &val);
8 .....
9
10 // 使用设置函数也可以将支持的类型的数据设置到QVariant对象中
11 // 这里的 T 类型，就是QVariant支持的类型
12 void QVariant::setValue(const T &value);
13 // 该函数行为和 setValue() 函数完全相同
14 [static] QVariant QVariant::fromValue(const T &value);
15 // 例子：
16 #if 1
17 QVariant v;
18 v.setValue(5);
19 #else
20 QVariant v = QVariant::fromValue(5);
21 #endif
22
23 int i = v.toInt();           // i is now 5
24 QString s = v.toString();    // s is now "5"
25
```

Fence 9

- 判断 QVariant中封装的实际数据类型

```
1 //Qt5.0
2 // 该函数的返回值是一个枚举类型，可通过这个枚举判断出实际是什么类型的数据
3 Type QVariant::type() const;
4
5
6 //Qt6.0
7 [since 6.0] QMetaType QVariant::metaType() const
8
9 //返回值Type 或者 QMetaType的部分枚举定义，全部信息可以自行查阅Qt帮助文档
```

Fence 10

- 将QVariant对象转换为实际的数据类型

- ```

1 // 如果要实现该操作，可以使用QVariant类提供的 toxxx() 方法，全部转换可以参考Qt帮助文档
2 // 在此列举几个常用函数：
3 bool QVariant::toBool() const;
4 QByteArray QVariant::toByteArray() const;
5 double QVariant::toDouble(bool *ok = Q_NULLPTR) const;
6 float QVariant::toFloat(bool *ok = Q_NULLPTR) const;
7 int QVariant::toInt(bool *ok = Q_NULLPTR) const;
8 QString QVariant::toString() const;
9
10
11

```

Fence 11

- 自定义数据类型
- 除了标准类型, 我们自定义的类型也可以使用QVariant类进行封装, 被QVariant存储的数据类型需要有一个默认的构造函数和一个拷贝构造函数。为了实现这个功能, 首先必须使用Q\_DECLARE\_METATYPE()宏。通常会将这个宏放在类的声明所在头文件的下面, 原型为:

Q\_DECLARE\_METATYPE(Type)

使用的具体步骤如下:

1. 第一步: 在头文件中声明

```

1 // *.h
2 struct MyTest
3 {
4 int id;
5 QString name;
6 };
7 // 自定义类型注册
8 Q_DECLARE_METATYPE(MyTest)
9
10

```

Fence 12

2. 第二步: 在源文件中定义

```

1 MyTest t;
2 t.name = "张三丰";
3 t.num = 666;
4 // 值的封装
5 QVariant vt = QVariant::fromValue(t);
6
7 // 值的读取
8 if(vt.canConvert<MyTest>())
9 {
10 MyTest t = vt.value<MyTest>();
11 qDebug() << "name: " << t.name << ", num: " << t.num;
12 }
13

```

以上操作用到的QVariant类的API如下:

```

1 // 如果当前QVariant对象可用转换为对应的模板类型 T, 返回true, 否则返回false
2 bool QVariant::canConvert() const;
3 // 将当前QVariant对象转换为实际的 T 类型
4 T QVariant::value() const;
5

```

## 5.5 位置和尺寸

### 5.5.1 QPoint

- QPoint类封装了我们常用用到的坐标点 (x, y), 常用的 API如下:

```

1 // 构造函数
2 // 构造一个坐标原点, 即(0, 0)
3 QPoint::QPoint();
4 // 参数为 x轴坐标, y轴坐标
5 QPoint::QPoint(int xpos, int ypos);
6
7 // 设置x轴坐标
8 void QPoint::setX(int x);
9 // 设置y轴坐标
10 void QPoint::setY(int y);
11
12 // 得到x轴坐标
13 int QPoint::x() const;
14 // 得到x轴坐标的引用
15 int &QPoint::rx();
16 // 得到y轴坐标
17 int QPoint::y() const;
18 // 得到y轴坐标的引用
19 int &QPoint::ry();
20
21 // 直接通过坐标对象进行算术运算: 加减乘除
22 QPoint &QPoint::operator*=(float factor);
23 QPoint &QPoint::operator*=(double factor);
24 QPoint &QPoint::operator*=(int factor);
25 QPoint &QPoint::operator+=(const QPoint &point);
26 QPoint &QPoint::operator-=(const QPoint &point);
27 QPoint &QPoint::operator/=(qreal divisor);
28
29 // 其他API请自行查询Qt帮助文档, 不要犯懒哦哦哦哦哦.....
30
31

```

## 5.5.2 QLine

- QLine是一个直线类, 封装了两个坐标点 (两点确定一条直线), 常用API如下:

```
1 // 构造函数
2 // 构造一个空对象
3 QLine::QLine();
4 // 构造一条直线, 通过两个坐标点
5 QLine::QLine(const QPoint &p1, const QPoint &p2);
6 // 从点 (x1, y1) 到 (x2, y2)
7 QLine::QLine(int x1, int y1, int x2, int y2);
8
9 // 给直线对象设置坐标点
10 void QLine::setPoints(const QPoint &p1, const QPoint &p2);
11 // 起始点(x1, y1), 终点(x2, y2)
12 void QLine::setLine(int x1, int y1, int x2, int y2);
13 // 设置直线的起点坐标
14 void QLine::setP1(const QPoint &p1);
15 // 设置直线的终点坐标
16 void QLine::setP2(const QPoint &p2);
17
18 // 返回直线的起始点坐标
19 QPoint QLine::p1() const;
20 // 返回直线的终点坐标
21 QPoint QLine::p2() const;
22 // 返回值直线的中心点坐标, (p1() + p2()) / 2
23 QPoint QLine::center() const;
24
25 // 返回值直线起点的 x 坐标
26 int QLine::x1() const;
27 // 返回值直线终点的 x 坐标
28 int QLine::x2() const;
29 // 返回值直线起点的 y 坐标
30 int QLine::y1() const;
31 // 返回值直线终点的 y 坐标
32 int QLine::y2() const;
33
34 // 用给定的坐标点平移这条直线
35 void QLine::translate(const QPoint &offset);
36 void QLine::translate(int dx, int dy);
37 // 用给定的坐标点平移这条直线, 返回平移之后的坐标点
38 QLine QLine::translated(const QPoint &offset) const;
39 QLine QLine::translated(int dx, int dy) const;
40
41 // 直线对象进行比较
42 bool QLine::operator!=(const QLine &line) const;
43 bool QLine::operator==(const QLine &line) const;
44
45 // 其他API请自行查询Qt帮助文档, 不要犯懒哦哦哦哦哦.....
46
47
```

### 5.5.3 QSize

- 在QT中QSize类用来形容长度和宽度, 常用的API如下:

```
1 // 构造函数
2 // 构造空对象, 对象中的宽和高都是无效的
3 QSize::QSize();
4 // 使用宽和高构造一个有效对象
5 QSize::QSize(int width, int height);
6
7 // 设置宽度
8 void QSize::setWidth(int width)
9 // 设置高度
10 void QSize::setHeight(int height);
11
12 // 得到宽度
13 int QSize::width() const;
14 // 得到宽度的引用
15 int &QSize::rwidth();
16 // 得到高度
17 int QSize::height() const;
18 // 得到高度的引用
19 int &QSize::rheight();
20
21 // 交换高度和宽度的值
22 void QSize::transpose();
23 // 交换高度和宽度的值, 返回交换之后的尺寸信息
24 QSize QSize::transposed() const;
25
26 // 进行算法运算: 加减乘除
27 QSize &QSize::operator*=(qreal factor);
28 QSize &QSize::operator+=(const QSize &size);
29 QSize &QSize::operator-=(const QSize &size);
30 QSize &QSize::operator/=(qreal divisor);
31
32 // 其他API请自行查询Qt帮助文档, 不要犯懒哦哦哦哦哦.....
33
34
```

Fence 17

### 5.5.4 QRect

- 在Qt中使用 QRect类来描述一个矩形, 常用的API如下:

```
1 // 构造函数
2 // 构造一个空对象
3 QRect::QRect();
4 // 基于左上角坐标, 和右下角坐标构造一个矩形对象
5 QRect::QRect(const QPoint &topLeft, const QPoint &bottomRight);
```

```

6 // 基于左上角坐标, 和 宽度, 高度构造一个矩形对象
7 QRect::QRect(const QPoint &topLeft, const QSize &size);
8 // 通过 左上角坐标(x, y), 和 矩形尺寸(width, height) 构造一个矩形对象
9 QRect::QRect(int x, int y, int width, int height);
10
11 // 设置矩形的尺寸信息, 左上角坐标不变
12 void QRect::setSize(const QSize &size);
13 // 设置矩形左上角坐标为(x,y), 大小为(width, height)
14 void QRect::setRect(int x, int y, int width, int height);
15 // 设置矩形宽度
16 void QRect::setWidth(int width);
17 // 设置矩形高度
18 void QRect::setHeight(int height);
19
20 // 返回值矩形左上角坐标
21 QPoint QRect::topLeft() const;
22 // 返回矩形右上角坐标
23 // 该坐标点值为: QPoint(left() + width() -1, top())
24 QPoint QRect::topRight() const;
25 // 返回矩形左下角坐标
26 // 该坐标点值为: QPoint(left(), top() + height() - 1)
27 QPoint QRect::bottomLeft() const;
28 // 返回矩形右下角坐标
29 // 该坐标点值为: QPoint(left() + width() -1, top() + height() - 1)
30 QPoint QRect::bottomRight() const;
31 // 返回矩形中心点坐标
32 QPoint QRect::center() const;
33
34 // 返回矩形上边缘y轴坐标
35 int QRect::top() const;
36 int QRect::y() const;
37 // 返回值矩形下边缘y轴坐标
38 int QRect::bottom() const;
39 // 返回矩形左边缘 x轴坐标
40 int QRect::x() const;
41 int QRect::left() const;
42 // 返回矩形右边缘x轴坐标
43 int QRect::right() const;
44
45 // 返回矩形的高度
46 int QRect::width() const;
47 // 返回矩形的宽度
48 int QRect::height() const;
49 // 返回矩形的尺寸信息
50 QSize QRect::size() const;
51
52
53

```

## 5.6 日期和时间

### 5.6.1 QDate

- QDate类可以封装日期信息也可以通过这个类得到日期相关的信息, 包括:年, 月, 日。

```
1 // 构造函数
2 QDate::QDate();
3 QDate::QDate(int y, int m, int d);
4
5 // 公共成员函数
6 // 重新设置日期对象中的日期
7 bool QDate::setDate(int year, int month, int day);
8 // 给日期对象添加 ndays 天
9 QDate QDate::addDays(qint64 ndays) const;
10 // 给日期对象添加 nmonths 月
11 QDate QDate::addMonths(int nmonths) const;
12 // 给日期对象添加 nyears 月
13 QDate QDate::addYears(int nyears) const;
14
15 // 得到日期对象中的年/月/日
16 int QDate::year() const;
17 int QDate::month() const;
18 int QDate::day() const;
19 void QDate::getDate(int *year, int *month, int *day) const;
20
21 // 日期对象格式化
22 /*
23 d - The day as a number without a leading zero (1 to 31)
24 dd - The day as a number with a leading zero (01 to 31)
25 ddd - The abbreviated localized day name (e.g. 'Mon' to 'Sun'). Uses the system
26 locale to localize the name, i.e. QLocale::system().
27 dddd - The long localized day name (e.g. 'Monday' to 'Sunday'). Uses the system
28 locale to localize the name, i.e. QLocale::system().
29 M - The month as a number without a leading zero (1 to 12)
30 MM - The month as a number with a leading zero (01 to 12)
31 MMM - The abbreviated localized month name (e.g. 'Jan' to 'Dec'). Uses the system
32 locale to localize the name, i.e. QLocale::system().
33 MMMM - The long localized month name (e.g. 'January' to 'December'). Uses the
34 system locale to localize the name, i.e. QLocale::system().
35 yy - The year as a two digit number (00 to 99)
36 yyyy - The year as a four digit number. If the year is negative, a minus sign is
37 prepended, making five characters.
38 */
39 QString QDate::toString(const QString &format) const;
40
41 // 操作符重载 ==> 日期比较
42 bool QDate::operator!=(const QDate &d) const;
43 bool QDate::operator<(const QDate &d) const;
44 bool QDate::operator<=(const QDate &d) const;
45 bool QDate::operator==(const QDate &d) const;
46 bool QDate::operator>(const QDate &d) const;
47 bool QDate::operator>=(const QDate &d) const;
```

```

43
44 // 静态函数 -> 得到本地的当前日期
45 [static] QDate QDate::currentDate();
46

```

Fence 19

## 5.6.2 QTime

- QTime类可以封装时间信息也可以通过这个类得到时间相关的信息, 包括:时, 分, 秒, 毫秒。

```

1 // 构造函数
2 QTime::QTime();
3 /*
4 h ==> 取值范围: 0 ~ 23
5 m and s ==> 取值范围: 0 ~ 59
6 ms ==> 取值范围: 0 ~ 999
7 */
8 QTime::QTime(int h, int m, int s = 0, int ms = 0);
9
10 // 公共成员函数
11 // Returns true if the set time is valid; otherwise returns false.
12 bool QTime::setHMS(int h, int m, int s, int ms = 0);
13 QTime QTime::addSecs(int s) const;
14 QTime QTime::addMSecs(int ms) const;
15
16 // 示例代码
17 QTime n(14, 0, 0); // n == 14:00:00
18 QTime t;
19 t = n.addSecs(70); // t == 14:01:10
20 t = n.addSecs(-70); // t == 13:58:50
21 t = n.addSecs(10 * 60 * 60 + 5); // t == 00:00:05
22 t = n.addSecs(-15 * 60 * 60); // t == 23:00:00
23
24 // 从时间对象中取出 时/分/秒/毫秒
25 // Returns the hour part (0 to 23) of the time. Returns -1 if the time is invalid.
26 int QTime::hour() const;
27 // Returns the minute part (0 to 59) of the time. Returns -1 if the time is invalid.
28 int QTime::minute() const;
29 // Returns the second part (0 to 59) of the time. Returns -1 if the time is invalid.
30 int QTime::second() const;
31 // Returns the millisecond part (0 to 999) of the time. Returns -1 if the time is
 invalid.
32 int QTime::msec() const;
33
34
35 // 时间格式化
36 /*
37 -- 时 --
38 h ==> The hour without a leading zero (0 to 23 or 1 to 12 if AM/PM display)
39 hh ==> The hour with a leading zero (00 to 23 or 01 to 12 if AM/PM display)
40 H ==> The hour without a leading zero (0 to 23, even with AM/PM display)
41 HH ==> The hour with a leading zero (00 to 23, even with AM/PM display)
42 -- 分 --

```

```

43 m ==> The minute without a leading zero (0 to 59)
44 mm ==> The minute with a leading zero (00 to 59)
45 -- 秒 --
46 s ==> The whole second, without any leading zero (0 to 59)
47 ss ==> The whole second, with a leading zero where applicable (00 to 59)
48 -- 毫秒 --
49 zzz ==> The fractional part of the second, to millisecond precision,
50 including trailing zeroes where applicable (000 to 999).
51 -- 上午或者下午
52 AP or A ==> 使用AM/PM(大写) 描述上下午, 中文系统显示汉字
53 ap or a ==> 使用am/pm(小写) 描述上下午, 中文系统显示汉字
54 */
55 QString QTime::toString(const QString &format) const;
56
57 // 阶段性计时
58 // 过时的API函数
59 // 开始计时
60 void QTime::start();
61 // 计时结束
62 int QTime::elapsed() const;
63 // 重新计时
64 int QTime::restart();
65
66 // 推荐使用的API函数
67 // QElapsedTimer 类
68 void QElapsedTimer::start();
69 qint64 QElapsedTimer::restart();
70 qint64 QElapsedTimer::elapsed() const;
71
72
73 // 操作符重载 ==> 时间比较
74 bool QTime::operator!=(const QTime &t) const;
75 bool QTime::operator<(const QTime &t) const;
76 bool QTime::operator<=(const QTime &t) const;
77 bool QTime::operator==(const QTime &t) const;
78 bool QTime::operator>(const QTime &t) const;
79 bool QTime::operator>=(const QTime &t) const;
80
81 // 静态函数 -> 得到当前时间
82 [static] QTime QTime::currentTime();
83

```

Fence 20

### 5.6.3 QDateTime

- QDateTime类可以封装日期和时间信息也可以通过这个类得到日期和时间相关的信息, 包括:年, 月, 日, 时, 分, 秒, 毫秒。其实这个类就是QDate 和 QTime 这两个类的结合体。

```

1 // 构造函数
2 QDateTime::QDateTime();
3 QDateTime::QDateTime(const QDate &date, const QTime &time, Qt::TimeSpec spec =
4 Qt::LocalTime);

```

```

5 // 公共成员函数
6 // 设置日期
7 void QDateTime::setDate(const QDate &date);
8 // 设置时间
9 void QDateTime::setTime(const QTime &time);
10 // 给当前日期对象追加 年/月/日/秒/毫秒, 参数可以是负数
11 QDateTime QDateTime::addYears(int nyears) const;
12 QDateTime QDateTime::addMonths(int nmonths) const;
13 QDateTime QDateTime::addDays(qint64 ndays) const;
14 QDateTime QDateTime::addSecs(qint64 s) const;
15 QDateTime QDateTime::addMsecs(qint64 msecs) const;
16
17 // 得到对象中的日期
18 QDate QDateTime::date() const;
19 // 得到对象中的时间
20 QTime QDateTime::time() const;
21
22 // 日期和时间格式, 格式字符参考QDate 和 QTime 类的 toString() 函数
23 QString QDateTime::toString(const QString &format) const;
24
25
26 // 操作符重载 ==> 日期时间对象的比较
27 bool QDateTime::operator!=(const QDateTime &other) const;
28 bool QDateTime::operator<(const QDateTime &other) const;
29 bool QDateTime::operator<=(const QDateTime &other) const;
30 bool QDateTime::operator==(const QDateTime &other) const;
31 bool QDateTime::operator>(const QDateTime &other) const;
32 bool QDateTime::operator>=(const QDateTime &other) const;
33
34 // 静态函数
35 // 得到当前时区的日期和时间(本地设置的时区对应的日期和时间)
36 [static] QDateTime QDateTime::currentDateTime();
37
38

```

Fence 21

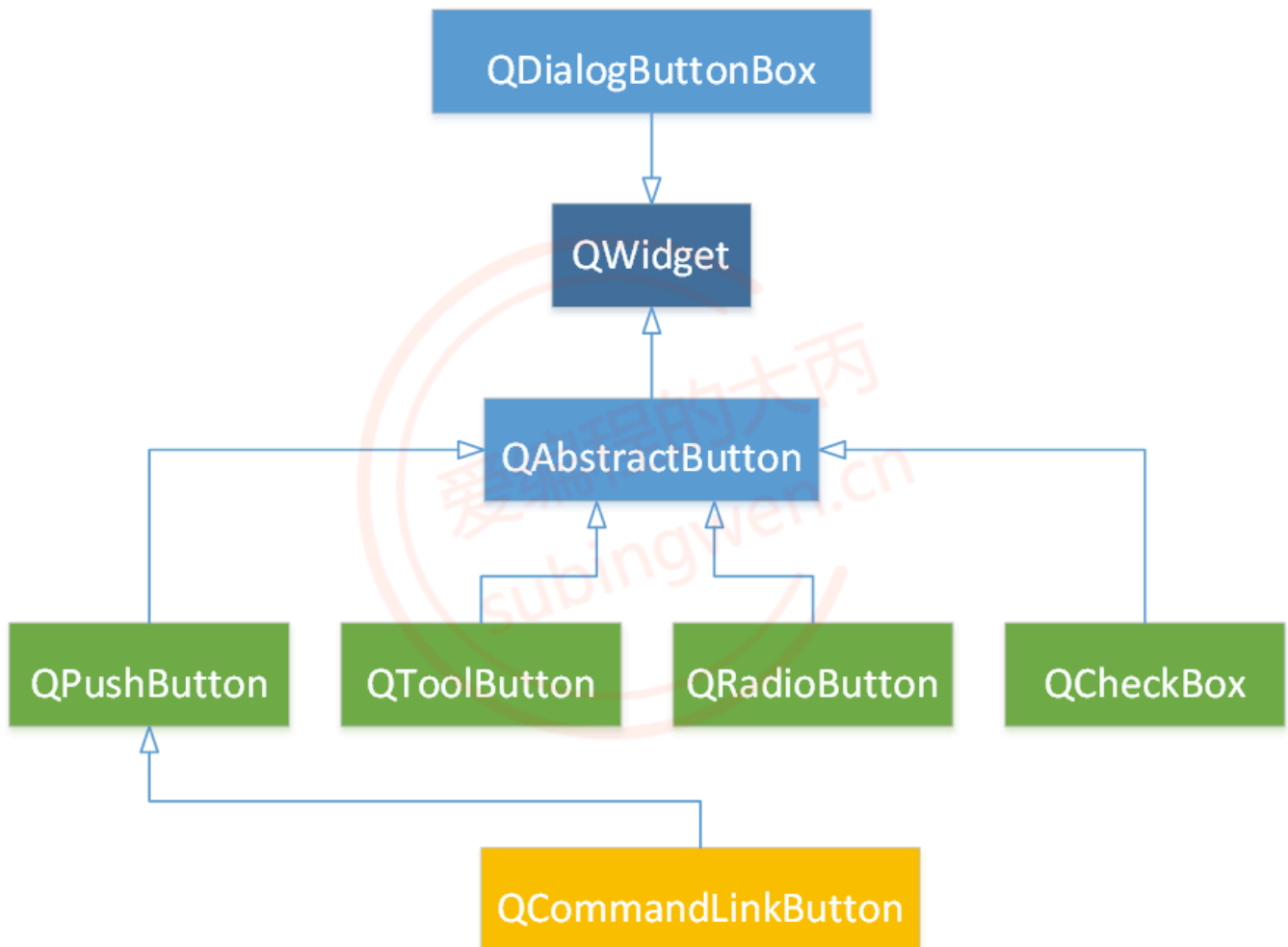
## 6. 组件类

### 6.1 QPushButton(按钮控件)

#### 6.1.1 按钮基类 QAbstractButton



Figure 12



- 相关API

```

1 //
2 // 参数text的内容显示到按钮上
3 void QAbstractButton::setText(const QString &text);
4 // 得到按钮上显示的文本内容，函数的返回就是
5 QString QAbstractButton::text() const;
6
7 // 得到按钮设置的图标
8 QIcon icon() const;
9 // 给按钮设置图标
10 void setIcon(const QIcon &icon);
11
12 // 得到按钮图标大小

```

```

13 QSize iconSize() const
14 // 设置按钮图标的大小
15 [slot]void setIconSize(const QSize &size);
16
17
18 //按钮的 check 属性
19 // 判断按钮是否设置了checkable属性，如果设置了点击按钮，按钮一直处于选中状态
20 // 默认这个属性是关闭的，not checkable
21 bool QAbstractButton::isChecked() const;
22 // 设置按钮的checkable属性
23 // 参数为true：点击按钮，按钮被选中，松开鼠标，按钮不弹起
24 // 参数为false：点击按钮，按钮被选中，松开鼠标，按钮弹起
25 void QAbstractButton::setCheckable(bool);
26
27
28 // 判断按钮是不是被按下的选中状态
29 bool QAbstractButton::isChecked() const;
30 // 设置按钮的选中状态：true-选中，false-没选中
31 // 设置该属性前，必须先进行 checkable属性的设置
32 void QAbstractButton::setChecked(bool);
33
34
35
36 //信号
37 /*
38 这些信号都按钮被点击之后发射出来的，只是在细节上有细微的区别，其中最常用的是 clicked()，通过鼠
 标的不同瞬间状态可以发射出pressed() 和 released() 信号，如果鼠标设置了 check属性，一般通过
 toggled()信号判断当前按钮是选中状态还是非选中状态。
39 */
40 /*
41 当按钮被激活时(即，当鼠标光标在按钮内时按下然后释放)，当键入快捷键时，或者当click()或
 animateClick()被调用时，这个信号被发出。值得注意的是，如果调用setDown()、setChecked()或
 toggle()，则不会触发此信号。
42 */
43 [signal] void QAbstractButton::clicked(bool checked = false);
44 // 在按下按钮的时候发射这个信号
45 [signal] void QAbstractButton::pressed();
46 // 在释放这个按钮的时候发射直观信号
47 [signal] void QAbstractButton::released();
48 // 每当可检查按钮改变其状态时，就会发出此信号。checked在选中按钮时为true，在未选中按钮时为
 false。
49 [signal] void QAbstractButton::toggled(bool checked);
50
51
52 //槽函数
53 // 执行一个动画点击:按钮被立即按下，并在毫秒后释放(默认是100毫秒)。
54 [slot] void QAbstractButton::animateClick(int msec = 100);
55 // 执行一次按钮点击，相当于使用鼠标点击了按钮
56 [slot] void QAbstractButton::click();
57
58 // 参考 1.2 中的函数介绍
59 [slot] void QAbstractButton::setChecked(bool);
60 // 设置按钮上图标大小

```

```

61 [slot]void setIconSize(const QSize &size);
62 // 切换可检查按钮的状态。 checked <==> unchecked
63 [slot] void QAbstractButton::toggle();
64

```

- 对应按钮来说, 一般有三种常见状态, 分别为: Normal, <sup>Fence 22</sup>Hover, Pressed。

- Normal: 普通状态, 没有和鼠标做任何接触
- Hover: 悬停状态, 鼠标位于按钮之上, 但是并未按下
- Pressed: 按压状态, 鼠标键在按钮上处于按下状态

默认情况下, 鼠标在按钮上按下, 按钮从 Normal 切换到 Pressed状态,

鼠标释放, 按钮从 Pressed恢复到Normal状态。

当我们给按钮设置了 check 属性之后, 情况就有所不同了, 在按钮上释放鼠标键, 按钮依然会处在 Pressed状态, 再次点击按钮, 按钮才能恢复到 Normal 状态。具有check属性的按钮就相当于一个开关, 每点击一次才能实现一次状态的切换。

## 6.1.2 QPushButton

- 常用API

```

1 // 构造函数
2 /*
3 参数:
4 - icon: 按钮上显示的图标
5 - text: 按钮上显示的标题
6 - parent: 按钮的父对象, 可以不指定
7 */
8 QPushButton::QPushButton(const QIcon &icon, const QString &text, QWidget *parent
9 = nullptr);
10 QPushButton::QPushButton(const QString &text, QWidget *parent = nullptr);
11 QPushButton::QPushButton(QWidget *parent = nullptr);
12
13 // 判断按钮是不是默认按钮
14 bool isDefault() const;
15 // 一般在对话框窗口中使用, 将按钮设置为默认按钮, 自动关联 Enter 键
16 void setDefault(bool);
17
18 /*
19 将弹出菜单菜单与此按钮关联起来。这将把按钮变成一个菜单按钮,
20 在某些样式中会在按钮文本的右边产生一个小三角形。
21 */
22 void QPushButton::setMenu(QMenu *menu);
23
24 /*
25 显示(弹出)相关的弹出菜单。如果没有这样的菜单, 这个函数什么也不做。
26 这个函数直到弹出菜单被用户关闭后才返回。
27 */
28 [slot] void QPushButton::showMenu();
29

```

- 示例:

通过API的介绍, 我们可以知道, 使用QPushButton这种类型的按钮, 有三种使用方式:

- 作为普通按钮, 可以显示文本信息和图标
- 设置check属性, 使其可以处于持续的被选中状态
- 关联一个菜单, 点击按钮菜单弹出

具体操作可以参考如下代码:

```

1 MainWindow::MainWindow(QWidget *parent)
2 : QMainWindow(parent)
3 , ui(new Ui::MainWindow)
4 {
5 ui->setupUi(this);
6
7 // 普通按钮, 没有checked属性
8 ui->normalBtn->setText("我是小猪佩奇");
9 ui->normalBtn->setIcon(QIcon(":/Peppa-Pig.png"));
10 ui->normalBtn->setIconSize(QSize(30, 30));
11 connect(ui->normalBtn, &QPushButton::clicked, this, [=]()
12 {
13 qDebug() << "我是一个普通按钮, 图标是小猪佩奇...";
14 });
15
16 // 有checked属性的按钮
17 ui->checkedBtn->setCheckable(true);
18 connect(ui->checkedBtn, &QPushButton::toggled, this, [=](bool b1)
19 {
20 qDebug() << "我是一个checked按钮, 当前状态为:" << b1;
21 });
22
23 // 关联菜单
24 ui->menuBtn->setText("你喜欢哪种美女?");
25 QMenu* menu = new QMenu;
26 QAction* act = menu->addAction("可爱的");
27 menu->addAction("粘人的");
28 menu->addAction("胸大的");
29 menu->addAction("屁股翘的");
30 ui->menuBtn->setMenu(menu);
31 connect(act, &QAction::triggered, this, [=]{
32 qDebug() << "我是一个可爱的女人, 今晚约吗?";
33 });
34 }
35

```

### 6.1.3 QToolButton

- 常用API

```

1 ////////////////////////////////// 构造函数 //////////////////////////////////
2 QToolButton::QToolButton(QWidget *parent = nullptr);

```

```

3
4 // 公共成员函数 //////////////////////////////////////
5 /*
6 1. 将给定的菜单与此工具按钮相关联。
7 2. 菜单将根据按钮的弹出模式显示。
8 3. 菜单的所有权没有转移到“工具”按钮(不能建立父子关系)
9 */
10 void QToolButton::setMenu(QMenu *menu);
11 // 返回关联的菜单，如果没有定义菜单，则返回nullptr。
12 QMenu *QToolButton::menu() const;
13
14 /*
15 弹出菜单的弹出模式是一个枚举类型：QToolButton::ToolButtonPopupMode，取值如下：
16 - QToolButton::DelayedPopup:
17 - 延时弹出，按压工具按钮一段时间后才能弹出，比如：浏览器的返回按钮
18 - 长按按钮菜单弹出，但是按钮的 clicked 信号不会被发射，短按不会弹出菜单，但是会有
19 clicked信号
20 - QToolButton::MenuButtonPopup:
21 - 在这种模式下，工具按钮会显示一个特殊的箭头，表示有菜单。
22 - 当按下按钮的箭头部分时，将显示菜单。按下按钮部分发射 clicked 信号
23 - QToolButton::InstantPopup:
24 - 当按下工具按钮时，菜单立即显示出来。
25 - 在这种模式下，按钮本身的动作不会被触发(不会发射clicked信号
26 */
27 // 设置弹出菜单的弹出方式
28 void setPopupMode(QToolButton::ToolButtonPopupMode mode);
29 // 获取弹出菜单的弹出方式
30 QToolButton::ToolButtonPopupMode popupMode() const;
31
32 /*
33 QToolButton可以帮助我们在按钮上绘制箭头图标，是一个枚举类型，取值如下：
34 - Qt::NoArrow: 没有箭头
35 - Qt::UpArrow: 箭头向上
36 - Qt::DownArrow: 箭头向下
37 - Qt::LeftArrow: 箭头向左
38 - Qt::RightArrow: 箭头向右
39 */
40 // 显示一个箭头作为QToolButton的图标。默认情况下，这个属性被设置为Qt::NoArrow。
41 void setArrowType(Qt::ArrowType type);
42 // 获取工具按钮上显示的箭头图标样式
43 Qt::ArrowType arrowType() const;
44
45 // 槽函数 //////////////////////////////////////
46 // 给按钮关联一个QAction对象，主要目的是美化按钮
47 [slot] void QToolButton::setDefaultAction(QAction *action);
48 // 返回给按钮设置的QAction对象
49 QAction *QToolButton::defaultAction() const;
50
51 /*
52 图标的显示样式是一个枚举类型->Qt::ToolButtonStyle，取值如下：
53 - Qt::ToolButtonIconOnly: 只有图标，不显示文本信息
54 - Qt::ToolButtonTextOnly: 不显示图标，只显示文本信息
55 - Qt::ToolButtonTextBesideIcon: 文本信息在图标的后边显示

```

```

55 - Qt::ToolButtonTextUnderIcon: 文本信息在图标的下边显示
56 - Qt::ToolButtonFollowStyle: 跟随默认样式(只显示图标)
57 */
58 // 设置的这个属性决定工具按钮是只显示一个图标、只显示文本，还是在图标旁边/下面显示文本。
59 [slot] void QToolButton::setToolButtonStyle(Qt::ToolButtonStyle style);
60 // 返回工具按钮设置的图标显示模式
61 Qt::ToolButtonStyle toolButtonStyle() const;
62
63
64 // 显示相关的弹出菜单。如果没有这样的菜单，这个函数将什么也不做。这个函数直到弹出菜单被用户关闭才会返回。
65 [slot] void QToolButton::showMenu();
66

```

Fence 25

### • 示例:

通过API的介绍, 我们可以知道, 使用QToolButton这种类型的按钮, 有三种使用方式::

- 作为普通按钮, 可以显示文本信息和图标
- 按钮的图标可以使用不同的方式设置, 并且制定图标和文本信息的显示模式
- 设置check属性, 使其可以处于持续的被选中状态
- 关联一个菜单, 点击按钮菜单弹出, 并且可以设置菜单的弹出方式

具体操作可以参考如下代码:

```

1 MainWindow::MainWindow(QWidget *parent)
2 : QMainWindow(parent)
3 , ui(new Ui::MainWindow)
4 {
5 ui->setupUi(this);
6
7 // 普通按钮, 没有checked属性
8 ui->normalBtn->setText("我是个屌丝");
9 ui->normalBtn->setIconSize(QSize(50, 50));
10 ui->normalBtn->setIcon(QIcon(":/mario.png"));
11 connect(ui->normalBtn, &QToolButton::clicked, this, [=]()
12 {
13 qDebug() << "我是一个普通按钮, 是一个屌丝...";
14 });
15 // 设置图标和文本的显示模式
16 ui->normalBtn->setToolButtonStyle(Qt::ToolButtonTextBesideIcon);
17 // 基于QAction的方式给按钮设置图标和文本信息
18 QAction* actBtn = new QAction(QIcon(":/mushroom_life.png"), "奥利给");
19 ui->actionBtn->setDefaultAction(actBtn);
20 connect(ui->actionBtn, &QToolButton::triggered, this, [=](QAction* act)
21 {
22 act->setText("我是修改之后的马里奥...");
23 act->setIcon(QIcon(":/mario.png"));
24 });
25 // 设置图标和文本的显示模式
26 ui->actionBtn->setToolButtonStyle(Qt::ToolButtonTextUnderIcon);

```

```

27
28 // 基于自带样式，给按钮设置箭头图标
29 ui->arrowBtn->setArrowType(Qt::UpArrow);
30 ui->arrowBtn->setText("向上");
31 // 设置图标和文本的显示模式
32 ui->arrowBtn->setToolButtonStyle(Qt::ToolButtonTextUnderIcon);
33
34
35 // 有checked属性的按钮
36 ui->checkedBtn->setCheckable(true);
37 connect(ui->checkedBtn, &QToolButton::toggled, this, [=](bool b1)
38 {
39 qDebug() << "我是一个checked按钮，当前状态为:" << b1;
40 });
41
42 // 关联菜单
43 ui->menuBtn->setText("你喜欢哪种美女?");
44 QMenu* menu = new QMenu;
45 QAction* act = menu->addAction("可爱的");
46 menu->addAction("粘人的");
47 menu->addAction("胸大的");
48 menu->addAction("屁股翘的");
49 ui->menuBtn->setMenu(menu);
50 connect(act, &QAction::triggered, this, [=]{
51 qDebug() << "我是一个可爱的女人，今晚约吗?";
52 });
53
54 ui->popmenu->setMenu(menu);
55 /*
56 弹出菜单的弹出模式是一个枚举类型：QToolButton::ToolButtonPopupMode，取值如下：
57 - QToolButton::DelayedPopup:
58 - 延时弹出，按压工具按钮一段时间后才能弹出，比如：浏览器的返回按钮
59 - 长按按钮菜单弹出，但是按钮的 clicked 信号不会被发射
60 - QToolButton::MenuButtonPopup:
61 - 在这种模式下，工具按钮会显示一个特殊的箭头，表示有菜单。
62 - 当按下按钮的箭头部分时，将显示菜单。按下按钮部分发射 clicked 信号
63 - QToolButton::InstantPopup:
64 - 当按下工具按钮时，菜单立即显示出来。
65 - 在这种模式下，按钮本身的动作不会被触发(不会发射clicked信号
66 */
67 ui->popmenu->setPopupMode(QToolButton::MenuButtonPopup);
68 // 测试关联了菜单的按钮是否会发射clicked信号
69 connect(ui->popmenu, &QToolButton::clicked, this, [=]()
70 {
71 qDebug() << "我是popMenu按钮，好痒呀...";
72 });
73 }
74

```

## 6.1.4 QRadioButton

- 常用API

```
1 // 构造函数
2 /*
3 参数:
4 - text: 按钮上显示的标题
5 - parent: 按钮的父对象
6 */
7 QRadioButton::QRadioButton(const QString &text, QWidget *parent = nullptr);
8 QRadioButton::QRadioButton(QWidget *parent = nullptr);
9
```

Fence 27

- 示例:

单选按钮一般是以组的形式来使用的, 如果在一个窗口中需要有多组单选按钮, 应该如何处理呢?

在同一窗口中, Qt会认为所有的单选按钮都属于同一组, 如果需要多个单选按钮组, 应该将他们放到不同的子窗口中。



Figure 13

通过上图可以看到有两个单选按钮组, 在制作的时候分别将单选按钮放到了不同的容器窗口 (组框) 中, 这样就被人为分隔为两组了。

| 对象                 | 类            |     |
|--------------------|--------------|-----|
| ▼ MainWindow       | QMainWindow  |     |
| ▼ centralwidget    | QWidget      |     |
| ▼ groupBox         | QGroupBox    | 第一组 |
| radio_cpp          | QRadioButton |     |
| radio_go           | QRadioButton |     |
| radio_java         | QRadioButton |     |
| radio_python       | QRadioButton |     |
| ▼ groupBox_2       | QGroupBox    | 第二组 |
| radio_nogirl       | QRadioButton |     |
| radio_nosalary     | QRadioButton |     |
| radio_notbeautiful | QRadioButton |     |
| radio_996          | QRadioButton |     |

Figure 14

如果我们使用鼠标点击了某个单选按钮, 按钮还是会发射出 clicked()信号, 简单的按钮测试代码如下所示:

```

1 void MainWindow::on_radio_996_clicked()
2 {
3 qDebug() << "996";
4 }
5
6 void MainWindow::on_radio_nosalary_clicked()
7 {
8 qDebug() << "没有加班费";
9 }
10
11 void MainWindow::on_radio_nogirl_clicked()
12 {
13 qDebug() << "公司没有妹子...";
14 }
15
16 // clicked 信号传递的参数可以接收, 也可以不接收
17 // 这个参数对应这中类型的按钮没啥用
18 void MainWindow::on_radio_notbeautiful_clicked(bool checked)
19 {
20 qDebug() << "前台小姐姐不好看!!!";
21 }
22
23

```

## 6.1.5 QCheckBox

- 常用API

```
1 // 构造函数
2 /*
3 参数:
4 - text: 按钮上显示的文本信息
5 - parent: 按钮的父对象
6 */
7 QCheckBox::QCheckBox(const QString &text, QWidget *parent = nullptr);
8 QCheckBox::QCheckBox(QWidget *parent = nullptr);
9
10 // 判断当前复选框是否为三态复选框, 默认情况下为两种状态: 未选中, 选中
11 bool isTristate() const;
12 // 设置当前复选框为三态复选框: 未选中, 选中, 半选中
13 void setTristate(bool y = true);
14
15 /*
16 参数 state, 枚举类型 Qt::CheckState:
17 - Qt::Unchecked --> 当前复选框没有被选中
18 - Qt::PartiallyChecked --> 当前复选框处于半选中状态, 部分被选中(三态复选框)
19 - Qt::Checked --> 当前复选框处于选中状态
20 */
21 // 设置复选框按钮的状态
22 void QCheckBox::setCheckState(Qt::CheckState state);
23 // 获取当前复选框的状态
24 Qt::CheckState QCheckBox::checkState() const;
25
26
27
28 // 当复选框的状态改变时, 即当用户选中或取消选中复选框时, 他的信号就会发出。
29 // 参数 state 表示的是复选框的三种状态中某一种, 可参考 Qt::CheckState
30 [signal] void QCheckBox::stateChanged(int state);
31
32
```

Fence 29

- 示例:

下面针对于复选框按钮的三种状态, 为大家展示一下对应的操作流程, 首先第一步搭建一个有树状关系的界面:

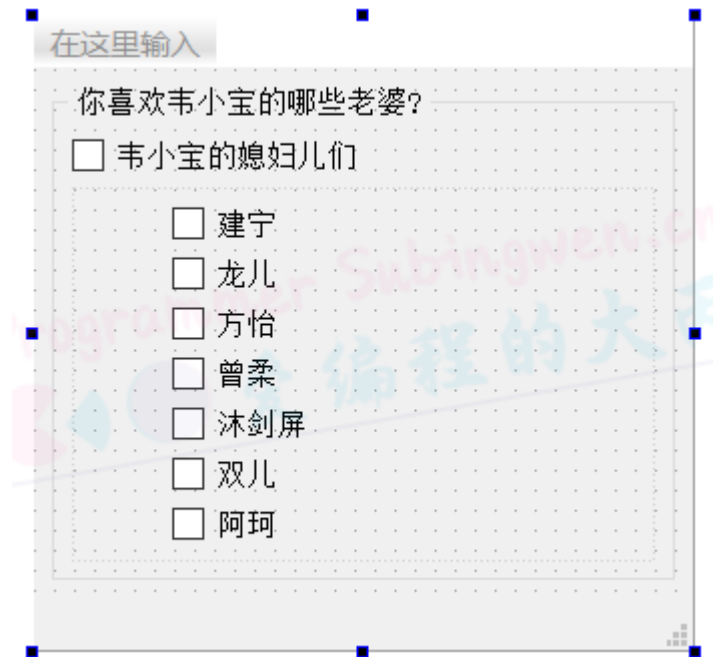


Figure 15

这些复选框按钮的关系以及 objectName 如下:

| 对象              | 类           |
|-----------------|-------------|
| ▼ MainWindow    | QMainWindow |
| ▼ centralwidget | QWidget     |
| ▼ groupBox      | QGroupBox   |
| ▼ widget        | QWidget     |
| ake             | QCheckBox   |
| fangyi          | QCheckBox   |
| jianning        | QCheckBox   |
| longer          | QCheckBox   |
| mujianping      | QCheckBox   |
| shuanger        | QCheckBox   |
| zengrou         | QCheckBox   |
| wives           | QCheckBox   |

子节点

父节点

Figure 16

```

1 // mainwindow.h
2 QT_BEGIN_NAMESPACE
3 namespace Ui { class MainWindow; }
4 QT_END_NAMESPACE
5
6 class MainWindow : public QMainWindow
7 {
8 Q_OBJECT
9
10 public:
11 MainWindow(QWidget *parent = nullptr);

```

```

12 ~MainWindow();
13
14 private slots:
15 // 添加槽函数，处理复选框按钮状态变化
16 void statusChanged(int state);
17
18 private:
19 Ui::MainWindow *ui;
20 int m_number = 0; // 添加一个计数器，记录有几个子节点被选中了
21 };
22
23
24 // mainwindow.cpp
25 // 窗口的构造函数
26 MainWindow::MainWindow(QWidget *parent)
27 : QMainWindow(parent)
28 , ui(new Ui::MainWindow)
29 {
30 ui->setupUi(this);
31
32 // 设置根节点的三态属性
33 ui->wives->setTristate(true);
34 // 处理根节点的鼠标点击事件
35 connect(ui->wives, &QCheckBox::clicked, this, [=](bool b1)
36 {
37 if(b1)
38 {
39 // 子节点全部设置为选中状态
40 ui->jianning->setChecked(true);
41 ui->fangyi->setChecked(true);
42 ui->longer->setChecked(true);
43 ui->zengrou->setChecked(true);
44 ui->mujianping->setChecked(true);
45 ui->shuanger->setChecked(true);
46 ui->ake->setChecked(true);
47 }
48 else
49 {
50 // 子节点全部设置为非选中状态
51 ui->jianning->setChecked(false);
52 ui->fangyi->setChecked(false);
53 ui->longer->setChecked(false);
54 ui->zengrou->setChecked(false);
55 ui->mujianping->setChecked(false);
56 ui->shuanger->setChecked(false);
57 ui->ake->setChecked(false);
58 }
59 });
60
61 // 处理子节点的状态变化，对应的槽函数相同
62 connect(ui->jianning, &QCheckBox::stateChanged, this, &MainWindow::statusChanged);
63 connect(ui->fangyi, &QCheckBox::stateChanged, this, &MainWindow::statusChanged);
64 connect(ui->longer, &QCheckBox::stateChanged, this, &MainWindow::statusChanged);

```

```

65 connect(ui->zengrou, &QCheckBox::stateChanged, this, &Mainwindow::statusChanged);
66 connect(ui->mujianping, &QCheckBox::stateChanged, this,
 &Mainwindow::statusChanged);
67 connect(ui->shuanger, &QCheckBox::stateChanged, this, &Mainwindow::statusChanged);
68 connect(ui->ake, &QCheckBox::stateChanged, this, &Mainwindow::statusChanged);
69 }
70
71 // 槽函数
72 void Mainwindow::statusChanged(int state)
73 {
74 if(state == Qt::Checked)
75 {
76 m_number ++; // 选中一个子节点，计数器加1
77 }
78 else
79 {
80 m_number --; // 取消选中一个子节点，计数器减1
81 }
82
83 // 根据计数器值判断根节点是否需要做状态的更新
84 if(m_number == 7)
85 {
86 ui->wives->setCheckState(Qt::Checked);
87 }
88 else if(m_number == 0)
89 {
90 ui->wives->setCheckState(Qt::Unchecked);
91 }
92 else
93 {
94 ui->wives->setCheckState(Qt::PartiallyChecked);
95 }
96 }
97
98

```

Fence 30

## 6.1.6 基本简介

| Header:   | #include                                                                                               |
|-----------|--------------------------------------------------------------------------------------------------------|
| CMake:    | find_package(Qt6 REQUIRED COMPONENTS Widgets)<br>target_link_libraries(mytarget PRIVATE Qt6:: Widgets) |
| qmake:    | QT += widgets                                                                                          |
| Inherits: | QAbstractButton                                                                                        |

|               |                    |
|---------------|--------------------|
| Header:       | #include           |
| Inherited By: | QCommandLinkButton |

Table 3

- **头文件:** `#include <QPushButton>` 这行代码用于在 C++源文件中包含 QPushButton 类的定义，以便可以使用该类创建和操作按钮。
- **CMake 配置:**
  - `find_package(Qt6 REQUIRED COMPONENTS Widgets)` 这行代码用于在 CMake 项目中查找 Qt6 库，并指定需要使用 Widgets 模块。
  - `target_link_libraries(mytarget PRIVATE Qt6::Widgets)` 这行代码则用于将 Qt6 的 Widgets 模块链接到名为 `mytarget` 的目标项目。
- **qmake 配置:** `QT += widgets` 这行代码用于在 qmake 项目文件 (.pro) 中指定需要使用 Qt 的 widgets 模块。
- **继承关系:** `QAbstractButton` 是 QPushButton 类的基类，这意味着 QPushButton 继承了 QAbstractButton 的所有公共和受保护的成员。
- **被继承关系:** `QCommandLinkButton` 类继承自 QPushButton，这意味着 QCommandLinkButton 类具备 QPushButton 类的所有功能，并且还可以在其基础上添加额外的功能。

6.1.7 信号

该信号全部继承于 `QAbstractButton` 类

- Signals:
  - `void clicked(bool checked = false)` //当按钮被点击时发出该信号。如果按钮是可选中的，`checked` 表示当前的选中状态。
  - `void pressed()` //当按钮被按下时发出该信号。
  - `void released()` //当按钮被释放时发出该信号（即用户松开按钮时触发）。
  - `void toggled(bool checked)` //当按钮的选中状态改变（切换）时发出该信号，`checked` 表示新的选中状态。

6.1.8 槽函数

- public Slots:
- 继承于 `QAbstractButton` 类
  - `void animateClick()` //触发按钮的点击动画默认持续时间（通常为 100 ms），并会触发按钮的 `clicked()` 信号。
  - `void click()` //程序模拟按钮被点击的动作，会触发 `clicked()` 信号，但不会有动画。

- void setChecked(bool) //设置按钮的选中状态，仅适用于可切换的按钮（如 QCheckBox 或 QRadioButton）。
- void setIconSize(const QSize &size) //设置按钮图标显示尺寸。
- void toggle() //切换按钮的选中状态（从选中到未选中或从未选中到选中）。
- 自身的槽函数
  - void showMenu(); //用于显示与按钮关联的弹出菜单（QMenu）

## 6.1.9 示例

```

1 //mywidget.h
2 #include <QPushButton> //QPushButton 按钮类
3
4
5 //mywidget.cpp
6 #include "mywidget.h"
7
8
9 MyWidget::MyWidget(QWidget *parent)
10 : QWidget(parent)
11 {
12 /* 第一种重载 */
13 QPushButton *btn = new QPushButton; //创建在堆区的按钮指针
14 // btn->show(); //show()方法是以顶层的形式创建按钮（新创建一个窗口）
15 btn->setParent(this); //btn对象 绑定父组件（依赖于mywidget窗口中）
16
17 btn->setText("这是第一个按钮"); //按钮上显示文本 设置按钮的属性一般为set...();
18
19 /* 第二种重载 */
20 QPushButton *btn01 = new QPushButton("这是第二个按钮", this);
21 //弊端：按照控件大小创建窗口
22 //新创建的控件会覆盖原控件，需要调整位置
23
24 /* 移动控件位置 */
25 btn->move(100, 100);
26
27 /* 设置控件窗口标题 */
28 this->setWindowTitle("按钮");
29
30 /* 重置窗口初始大小 */
31 resize(500, 400);
32
33 /* 固定窗口大小 */
34 this->setFixedSize(500, 400);
35
36 /* 重置按钮大小 */
37 btn01->resize(100, 50);
38
39

```

```
40 |
41 | }
```

Fence 31

- 注意：在 Qt 中，如果你通过 `setParent(this)` 将按钮设置为某个父组件（例如 `mywidget`）的子组件，那么你通常不需要手动删除这个按钮对象。Qt 的内存管理机制会自动处理这种情况：当父组件被销毁时，所有子组件也会被自动销毁。但是，如果你在没有设置父组件的情况下创建了一个按钮对象，并且希望在某个时刻手动删除它，那么你需要调用 `delete` 来释放内存。
- 查看编码格式：工具->外部->配置->文本编辑器->行为->编码格式

## 6.2 QUdpSocket(网络控件)

### 6.2.1 基本简介

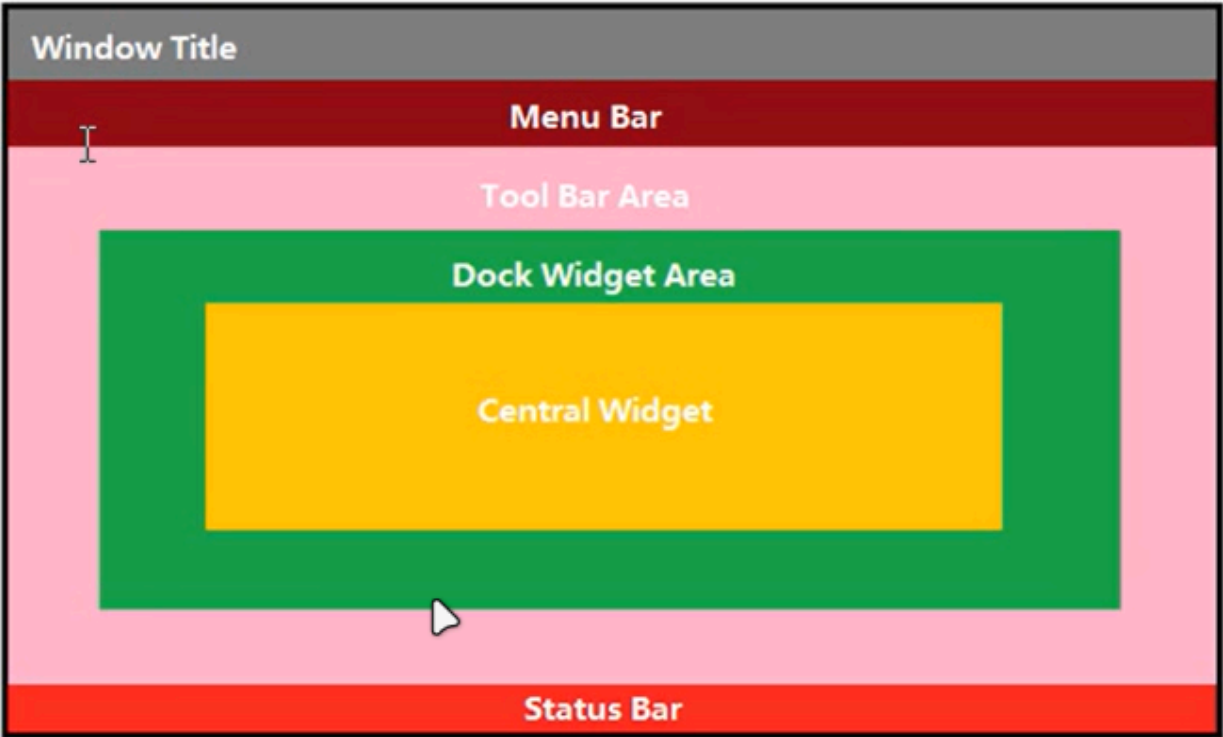
| Header:   | #include                                                                                                                         |
|-----------|----------------------------------------------------------------------------------------------------------------------------------|
| CMake:    | <code>find_package(Qt6 REQUIRED COMPONENTS Network)</code><br><code>target_link_libraries(mytarget PRIVATE Qt6:: Network)</code> |
| qmake:    | <code>QT += network</code>                                                                                                       |
| Inherits: | <code>QAbstractSocket</code>                                                                                                     |

Table 4

- 头文件： `#include <QUdpSocket>` 用于在项目中包含 QUdpSocket 类的定义。
- CMake 配置：
  - `find_package(Qt6 REQUIRED COMPONENTS Network)` 用于查找 Qt6 库中的 Network 模块，该模块包含 QUdpSocket 等网络相关的类。
  - `target_link_libraries(mytarget PRIVATE Qt6::Network)` 用于将 Qt6 的 Network 模块链接到你的项目目标 `mytarget`，以便在项目中使用 QUdpSocket 类。
- qmake 配置： `QT += network` 用于在 qmake 项目文件.pro 中添加 Network 模块，使 QUdpSocket 类可以被项目使用。
- 继承关系：QUdpSocket 继承自 QAbstractSocket，这意味着 QUdpSocket 类具有 QAbstractSocket 类的所有功能，并且可以用于处理 UDP 套接字的特定需求。

## 6.3 QMainWindow(主窗口类)

- **QMainWindow** 是一个为用户提供主窗口程序的类，包含一个菜单栏（**menu bar**）、多个工具栏（**tool bars**）、多个锚接部件（**dock widgets**）、一个状态栏（**status bar**）及一个中心部件（**central widget**），是许多应用程序的基础，如文本编辑器、图片编辑器等。



### 6.3.1 基本简介

|           |                                                                                                        |
|-----------|--------------------------------------------------------------------------------------------------------|
| Header:   | #include                                                                                               |
| CMake:    | find_package(Qt6 REQUIRED COMPONENTS Widgets)<br>target_link_libraries(mytarget PRIVATE Qt6:: Widgets) |
| qmake:    | QT += widgets                                                                                          |
| Inherits: | QWidget                                                                                                |

Table 5

- **头文件:** `#include <QMainWindow>` 用于在项目中包含 QMainWindow 类的定义，使得可以创建主窗口应用程序。
- **CMake 配置:**
  - `find_package(Qt6 REQUIRED COMPONENTS Widgets)` 用于查找 Qt6 库中的 Widgets 模块，该模块提供了构建图形用户界面的基础类。
  - `target_link_libraries(mytarget PRIVATE Qt6::Widgets)` 用于将 Qt6 的 Widgets 模块链接到你的项目目标 `mytarget`，以便在项目中使用 QMainWindow 及其他相关类。
- **qmake 配置:** `QT += widgets` 用于在 qmake 项目文件.pro 中添加 Widgets 模块，使 QMainWindow 类可以被项目使用。

- **继承关系**: QMainWindow 继承自 QWidget, 这意味着 QMainWindow 类拥有 QWidget 类的所有功能, 并且提供更多用于构建应用程序主窗口的特性, 如工具栏、状态栏和菜单栏等。

### 6.3.2 菜单栏(一个窗口最多一个)

```
1 //窗口菜单栏
2 QMenuBar *bar = menuBar();
3 //将菜单栏放入窗口中
4 setMenuBar(bar);
5
6 //创建菜单
7 QMenu *fileMenu = bar->addMenu("文件");
8 QMenu *editMenu = bar->addMenu("编辑");
9
10 //创建菜单项
11 fileMenu->addAction("新建");
12 fileMenu->addAction("打开");
13
14 //创建分隔线
15 fileMenu->addSeparator();
16
17 fileMenu->addAction("退出");
18
19
```

Fence 32

### 6.3.3 工具栏(一个窗口可以有多个)

```
1 /* 工具栏 可以有多个 */
2 //创建工具栏 QMainWindow中没有实现工具栏的方法
3 QToolBar *toolBar = new QToolBar(this);
4 //添加工具栏 设定工具生成的初始位置
5 addToolBar(Qt::LeftToolBarArea, toolBar);
6
7 //设置可以移动访问 只能停在左边或者右边
8 toolBar->setAllowedAreas(Qt::LeftToolBarArea | Qt::RightToolBarArea);
9
10 //设置是否浮动 (如果停靠的边不合规, 将放回上一次的位置)
11 toolBar->setFloatable(false);
12
13 //设置能否移动 (移动总开关)
14 toolBar->setMovable(true);
15
16 //创建工具项
17 toolBar->addAction("运行");
18 toolBar->addSeparator(); //添加分割线
19 toolBar->addAction(newAct); //和菜单栏的新建 与 打开 共用一套数据
20 toolBar->addAction(openAct);
21
22 //添加控件
23 QPushButton *btn = new QPushButton("你好", toolBar);
```

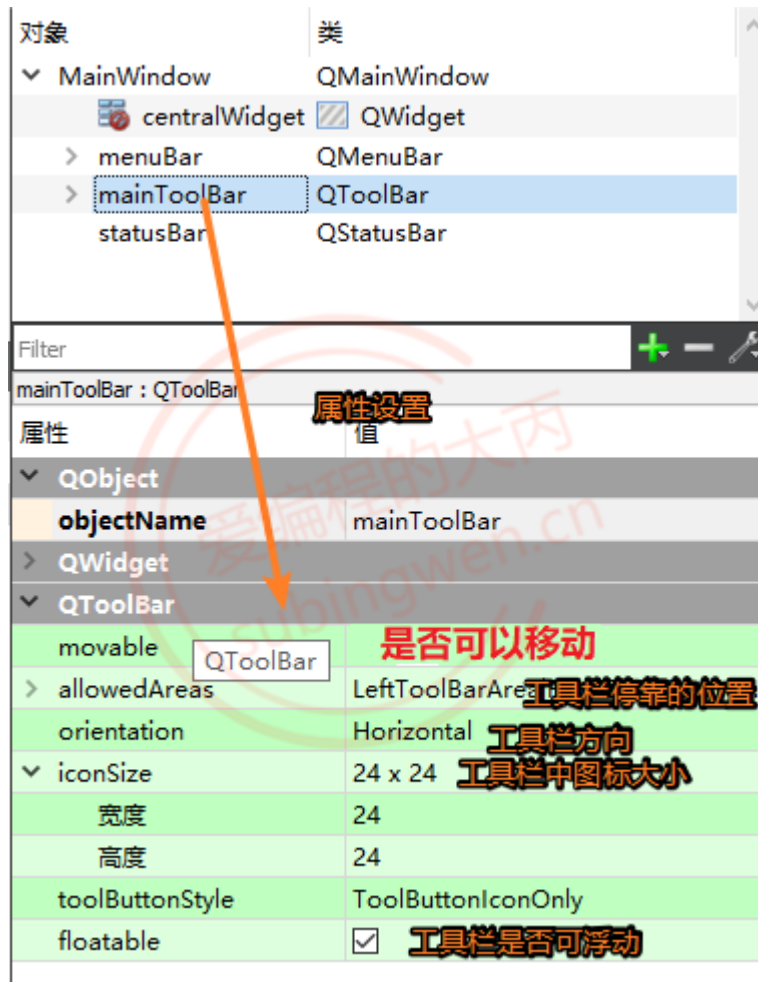


Figure 17

### 6.3.4 状态栏(最多只有一个)

```

1 /* 状态栏 最多只有一个 */
2 //创建状态栏 有成员函数可以实现
3 QStatusBar *stb = statusBar();
4 //放入窗口
5 setStatusBar(stb);
6
7 //添加标签
8 QLabel *lab = new QLabel("提示信息",this);
9 QLabel *lab1 = new QLabel("提示信息2",this);
10
11 //默认左侧
12 stb->addWidget(lab);
13 //设置在右侧
14 stb->addPermanentWidget(lab1);

```

6.3.5 铆接部件(浮动控件 可以有多个) 和 核心控件(只能有一个)

```
1 /* 铆接部件（浮动控件） 可以有多个 核心部件 只能有一个*/
2 //创建铆接部件
3 QDockWidget *dow = new QDockWidget("浮动",this);
4 //添加到窗口 同时指定位置(位置是以核心部件为中心)
5 addDockWidget(Qt::BottomDockWidgetArea,dow);
6
7 //固定位置移动范围 只能上或者下
8 dow->setAllowedAreas(Qt::BottomDockWidgetArea | Qt::TopDockWidgetArea);
9
10 //创建文本部件
11 QTextEdit *te = new QTextEdit(this);
12 //添加成核心部件
13 setCentralWidget(te);
```

Fence 35

小技巧：只能有一个的用 set 添加 可以有多个用 add 添加 父对象提供实现的成员函数，用成员函数 没有提供用 new

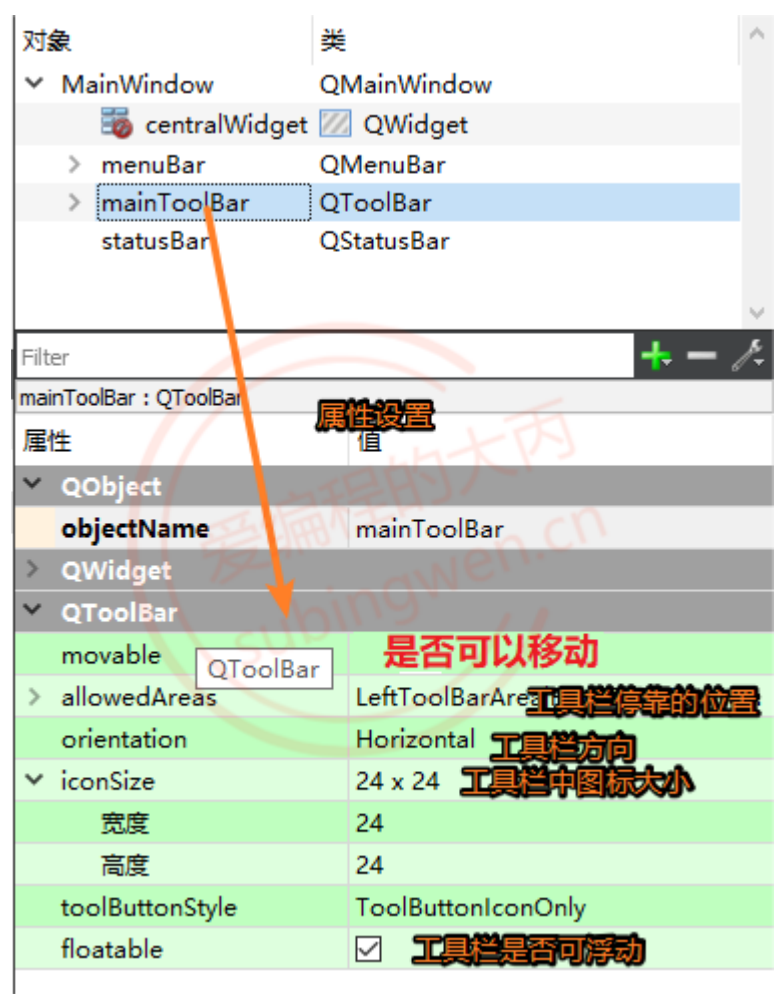


Figure 18

## 6.4 对话框(QDialog)

标准对话框，是 Qt 内置的一系列对话框，用于简化开发。实际上，有很多对话框都是通用的，比如打开文件、设置颜色、打印设置等。这些对话框在所有程序中几乎都相应，因此没有必要在每一个程序中都实现这一个对话框。

Qt 的内置对话框大致分为以下几类：

- **QColorDialog**: 选择颜色；
- **QFileDialog**: 选择文件或目录；
- **QFontDialog**: 选择字体；
- **QInputDialog**: 允许用户输入一个值，并将其返回；
- **QMessageBox**: 消息对话框，用于显示信息、询问问题等；
- **QPageSetupDialog**: 为打印机提供纸张相关的选项；
- **QPrintDialog**: 打印机设置；
- **QPrintPreviewDialog**: 打印预览；
- **QProgressDialog**: 显示操作进程。

```
1 //常用API
2 // 构造函数
3 QDialog::QDialog(QWidget *parent = nullptr, Qt::WindowFlags f = Qt::WindowFlags());
4
5 // 模态显示窗口
6 [virtual slot] int QDialog::exec();
7 // 隐藏模态窗口，并且解除模态窗口的阻塞，将 exec() 的返回值设置为 QDialog::Accepted
8 [virtual slot] void QDialog::accept();
9 // 隐藏模态窗口，并且解除模态窗口的阻塞，将 exec() 的返回值设置为 QDialog::Rejected
10 [virtual slot] void QDialog::reject();
11 // 关闭对话框并将其结果代码设置为r。finished()信号将发出r；
12 // 如果r是QDialog::Accepted 或 QDialog::Rejected，则还将分别发出accept()或Rejected()信号。
13 [virtual slot] void QDialog::done(int r);
14
15 [signal] void QDialog::accepted();
16 [signal] void QDialog::rejected();
17 [signal] void QDialog::finished(int result);
18
19
20
21 //例子
22 /*场景介绍：
23 1. 有两个窗口，主窗口和一个对话框子窗口
24 2. 对话框窗口先显示，根据用户操作选择是否显示主窗口
25 */
26 // 对话框窗口中三个普通按钮按下之后对应的槽函数
27 void MyDialog::on_acceptBtn_clicked()
28 {
29 this->accept(); // exec()函数返回值为QDialog::Accepted
30 }
31
```

```

32 void MyDialog::on_rejectBtn_clicked()
33 {
34 this->reject(); // exec()函数返回值为QDialog::Rejected
35 }
36
37 void MyDialog::on_donBtn_clicked()
38 {
39 // exec()函数返回值为 done() 的参数，并根据参数发射出对应的信号
40 this->done(666);
41 }
42
43
44 // 创建对话框对象
45 MyDialog dlg;
46 int ret = dlg.exec();
47 if(ret == QDialog::Accepted)
48 {
49 qDebug() << "accept button clicked...";
50 // 显示主窗口
51 MainWindow* w = new MainWindow;
52 w->show();
53 }
54 else if(ret == QDialog::Rejected)
55 {
56 qDebug() << "reject button clicked...";
57 // 不显示主窗口
58
59
60 }
61 else
62 {
63 // ret == 666
64 qDebug() << "done button clicked...";
65 // 根据需求进行逻辑处理
66
67
68 }
69
70

```

### 6.4.1 模态对话框(打开以后，不能操作其他窗口——阻塞其他窗口)——自定义对话框

```
1 //连接新建触发的信号 与 创建窗口
2 connect(ui->actionNew,&QAction::triggered,[=]() {
3 //模态对话框 会阻塞原窗口进程
4 //阻塞了进程 生命周期暂停 点击x的时候才释放 不要new
5 QDialog dlg(this);
6 dlg.setWindowTitle("新建");
7 dlg.exec();
8
9 qDebug() << "新窗口创建了" ;
10
11 });
12
```

Fence 37

### 6.4.2 非模态对话框(打开以后，能操作其他窗口——不阻塞其他窗口)——自定义对话框

```
1 connect(ui->new2_2,&QAction::triggered,[=]() {
2 //非模态对话框 不会阻塞原窗口进程
3 //需要创建到堆区 才能保证窗口不会只出现一瞬间 释放在父对象x掉以后释放
4 QDialog *dlg = new QDialog(this);
5 dlg->setTitle("新建");
6 dlg->setAttribute(Qt::WA_DeleteOnClose); // 值为55 保证点击关闭时，释放小控
7 //件内存 如果不做这个处理，连续的创建会导致内存泄露
8 dlg->show();
9
10 qDebug() << "新窗口创建了" ;
11 });
```

Fence 38

### 6.4.3 消息对话框——QMessageBox(模态对话框)

- 底层封装的函数为静态函数（不需要对象，能直接调用）
- QMessageBox 对话框类是 QDialog 类的子类, 通过这个类可以显示一些简单的提示框, 用于展示警告、错误、问题等信息。关于这个类我们只需要掌握一些静态方法的使用就可以了。

```
1 // 显示一个模态对话框，将参数 text 的信息展示到窗口中
2 [static] void QMessageBox::about(QWidget *parent, const QString &title, const QString
3 &text);
4
5 /*
6 参数：
7 - parent: 对话框窗口的父窗口
8 - title: 对话框窗口的标题
9 - text: 对话框窗口中显示的提示信息
10 - buttons: 对话框窗口中显示的按钮(一个或多个)
11 - defaultButton
12 1. defaultButton指定按下Enter键时使用的按钮。
```

2. `defaultButton`必须引用在参数 `buttons` 中给定的按钮。
3. 如果`defaultButton`是`QMessageBox::NoButton`, `QMessageBox`会自动选择一个合适的默认值。

```
12 */
13 // 显示一个信息模态对话框
14 [static] QMessageBox::StandardButton QMessageBox::information(
15 QWidget *parent, const QString &title,
16 const QString &text,
17 QMessageBox::StandardButtons buttons = Ok,
18 QMessageBox::StandardButton defaultButton = NoButton);
19
20 // 显示一个错误模态对话框
21 [static] QMessageBox::StandardButton QMessageBox::critical(
22 QWidget *parent, const QString &title,
23 const QString &text,
24 QMessageBox::StandardButtons buttons = Ok,
25 QMessageBox::StandardButton defaultButton = NoButton);
26
27 // 显示一个问题模态对话框
28 [static] QMessageBox::StandardButton QMessageBox::question(
29 QWidget *parent, const QString &title,
30 const QString &text,
31 QMessageBox::StandardButtons buttons = StandardButtons(Yes | No),
32 QMessageBox::StandardButton defaultButton = NoButton);
33
34 // 显示一个警告模态对话框
35 [static] QMessageBox::StandardButton QMessageBox::warning(
36 QWidget *parent, const QString &title,
37 const QString &text,
38 QMessageBox::StandardButtons buttons = Ok,
39 QMessageBox::StandardButton defaultButton = NoButton);
40
41 //例子
42 void MainWindow::on_msgbox_clicked()
43 {
44 QMessageBox::about(this, "about", "这是一个简单的消息提示框!!!");
45 QMessageBox::critical(this, "critical", "这是一个错误对话框-critical...");
46 int ret = QMessageBox::question(this, "question",
47 "你要保存修改的文件内容吗???",
48 QMessageBox::Save|QMessageBox::Cancel,
49 QMessageBox::Cancel);
50 if(ret == QMessageBox::Save)
51 {
52 QMessageBox::information(this, "information", "恭喜你保存成功了, o(*￣▽￣*)o!!!");
53 }
54 else if(ret == QMessageBox::Cancel)
55 {
56 QMessageBox::warning(this, "warning", "你放弃了保存, TT__TT !!!");
57 }
58 }
59
60
61
62
63
```

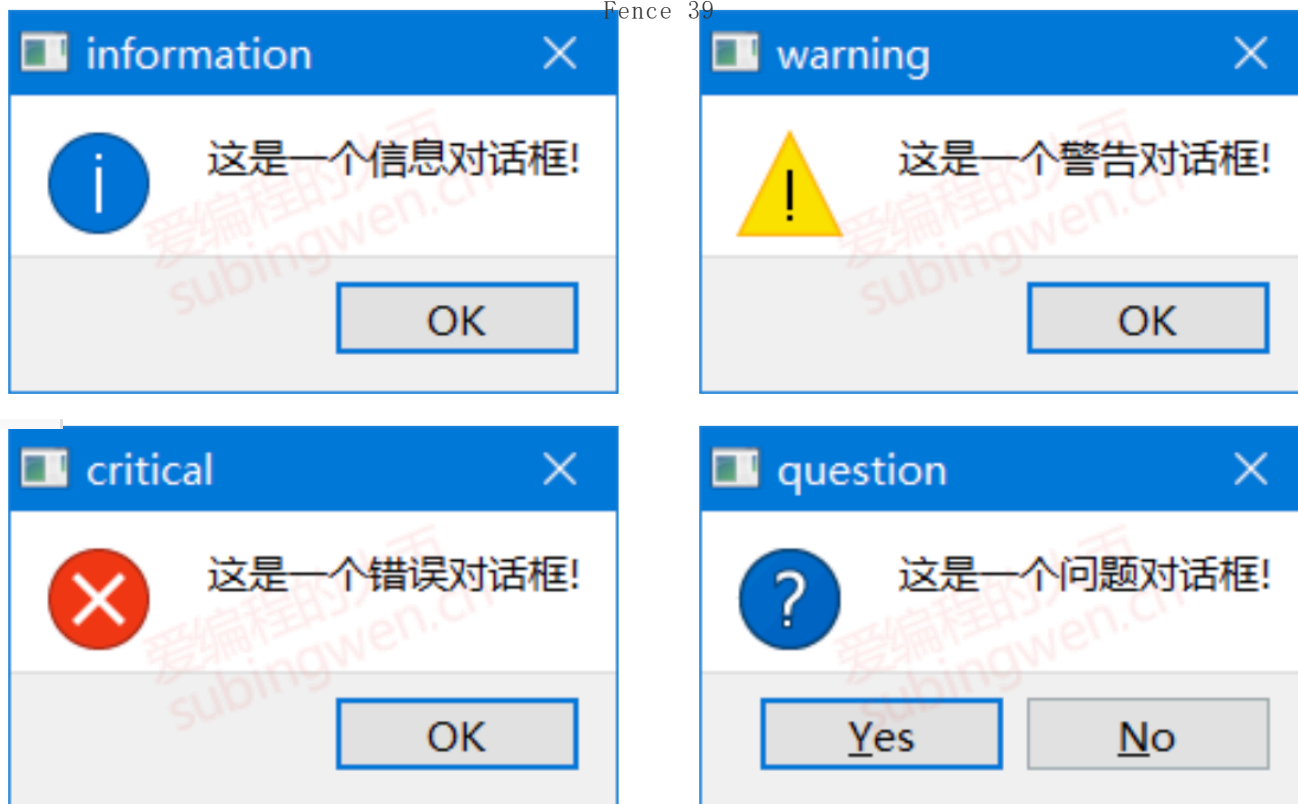


Figure 19

```

1 connect(ui->actionNew,&QAction::triggered,[=]() {
2 /* 消息对话框 */
3 //错误对话框
4 // QMessageBox::critical(this,"错误提示框","xxx内容错误");
5
6 //信息提示框
7 // QMessageBox::information(this,"信息提示框","xxx内容");
8
9 //提问提示框
10 //参数1 父对象 参数2 窗口标题 参数3 提示内容 参数4 按钮内容 参数5 默认关联回车
 的按钮类型
11 //返回值为StandardButton 用于确定返回值类型
12 // QMessageBox::question(this,"提问提示框","是否取消");
13 //QMessageBox::question(this,"提问提示框","是否保存",QMessageBox::Save |
 QMessageBox::Cancel,QMessageBox::Cancel);
14 // if(QMessageBox::Save == QMessageBox::question(this,"提问提示框","是否保
 存",QMessageBox::Save | QMessageBox::Cancel,QMessageBox::Cancel))
15 // {
16 // qDebug() << "保存";
17 // }
18
19
20 //警告提示框
21 // QMessageBox::warning(this,"警告提示框","xxx警告");
22
23

```

```

24
25 //自定义提示框
26 QMessageBox qmb(this);
27 qmb.setWindowTitle("自定义提示框");
28 qmb.setText("是否保存");
29 // qmb.setIcon(); //设置图标
30
31 //添加自定义按钮
32 QPushButton *p1 = qmb.addButton("保存",QMessageBox::AcceptRole);
33 QPushButton *p2 = qmb.addButton("取消",QMessageBox::RejectRole);
34
35 qmb.setDefaultButton(p2); //回车关联取消
36 qmb.exec(); //显示并设置成模态
37
38 if(qmb.clickedButton() == p1){
39 qDebug() << "保存";
40 }
41
42
43 });
44
45

```

Fence 40

## 6.4.4 QFileDialog

- QFileDialog 对话框类是 QDialog 类的子类, 通过这个类可以选择要打开/保存的文件或者目录。关于这个类我们只需要掌握一些静态方法的使用就可以了。

```

1 //API
2 /*
3 通用参数:
4 - parent: 当前对话框窗口的父对象也就是父窗口
5 - caption: 当前对话框窗口的标题
6 - dir: 当前对话框窗口打开的默认目录
7 - options: 当前对话框窗口的一些可选项,枚举类型,一般不需要进行设置,使用默认值即可
8 - filter: 过滤器,在对话框中只显示满足条件的文件,可以指定多个过滤器,使用 ;; 分隔
9 - 样式举例:
10 - Images (*.png *.jpg)
11 - Images (*.png *.jpg);;Text files (*.txt)
12 - selectedFilter: 如果指定了多个过滤器,通过该参数指定默认使用哪一个,不指定默认使用第一个
 过滤器
13 */
14 // 打开一个目录,得到这个目录的绝对路径
15 [static] QString QFileDialog::getExistingDirectory(
16 QWidget *parent = nullptr,
17 const QString &caption = QString(),
18 const QString &dir = QString(),
19 QFileDialog::Options options = ShowDirsOnly);
20
21 // 打开一个文件,得到这个文件的绝对路径

```

```

22 [static] QString QFileDialog::getOpenFileName(
23 QWidget *parent = nullptr,
24 const QString &caption = QString(),
25 const QString &dir = QString(),
26 const QString &filter = QString(),
27 QString *selectedFilter = nullptr,
28 QFileDialog::Options options = Options());
29
30 // 打开多个文件，得到这多个文件的绝对路径
31 [static] QStringList QFileDialog::getOpenFileNames(
32 QWidget *parent = nullptr,
33 const QString &caption = QString(),
34 const QString &dir = QString(),
35 const QString &filter = QString(),
36 QString *selectedFilter = nullptr,
37 QFileDialog::Options options = Options());
38
39 // 打开一个目录，使用这个目录来保存指定的文件
40 [static] QString QFileDialog::getSaveFileName(
41 QWidget *parent = nullptr,
42 const QString &caption = QString(),
43 const QString &dir = QString(),
44 const QString &filter = QString(),
45 QString *selectedFilter = nullptr,
46 QFileDialog::Options options = Options());
47
48
49

```

Fence 41

- 打开一个已存在的本地目录

```

1 void MainWindow::on_fileDlg_clicked()
2 {
3 QString dirName = QFileDialog::getExistingDirectory(this, "打开目录",
4 "e:\\temp");
5 QMessageBox::information(this, "打开目录", "您选择的目录是: " + dirName);
6 }

```

Fence 42

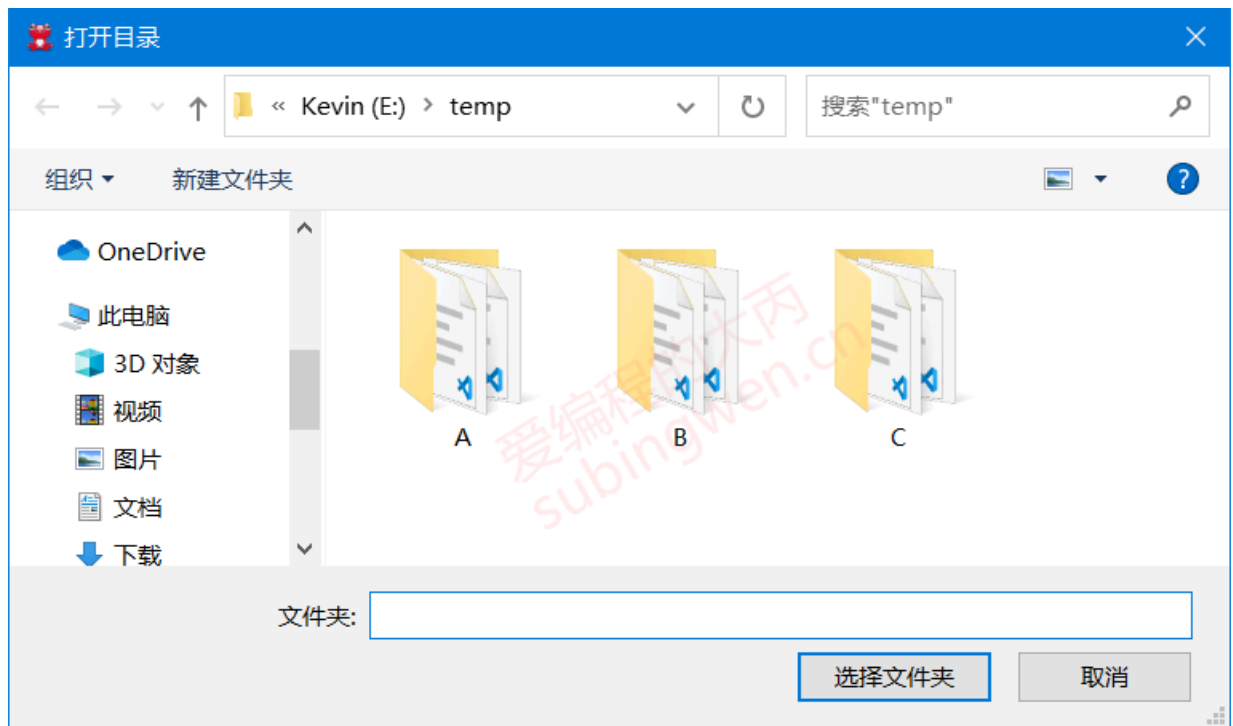


Figure 20

- 打开一个本地文件

```
1 void MainWindow::on_filedlg_clicked()
2 {
3 QString arg("Text files (*.txt)");
4 QString fileName = QFileDialog::getOpenFileName(
5 this, "Open File", "e:\\temp",
6 "Images (*.png *.jpg);;Text files (*.txt)", &arg);
7 QMessageBox::information(this, "打开文件", "您选择的文件是: " + fileName);
8 }
9
10
```

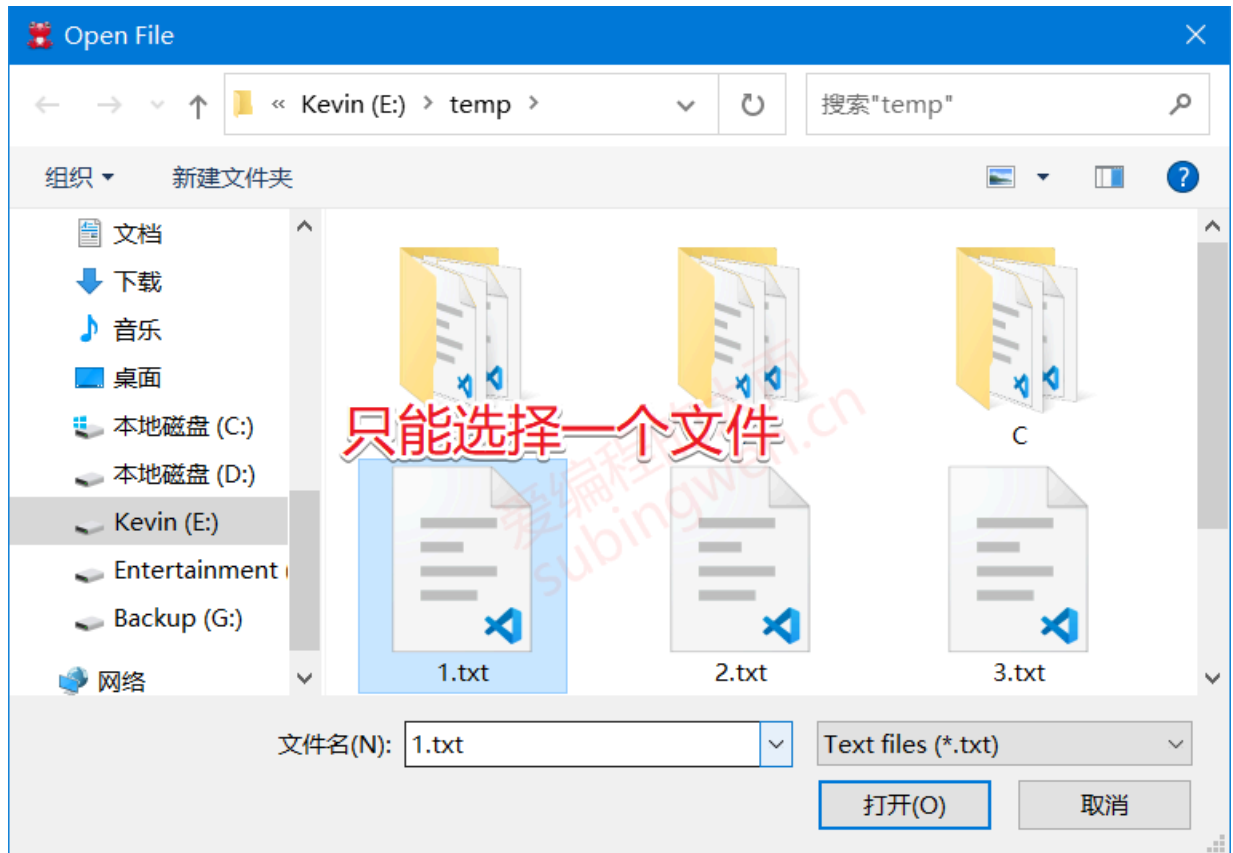


Figure 21

- 打开多个本地文件

```
1 void MainWindow::on_fileDlg_clicked()
2 {
3 QStringList fileNames = QFileDialog::getOpenFileNames(
4 this, "Open File", "e:\\temp",
5 "Images (*.png *.jpg);;Text files (*.txt)");
6 QString names;
7 for(int i=0; i<fileNames.size(); ++i)
8 {
9 names += fileNames.at(i) + " ";
10 }
11 QMessageBox::information(this, "打开文件(s)", "您选择的文件是: " + names);
12 }
13
14
```

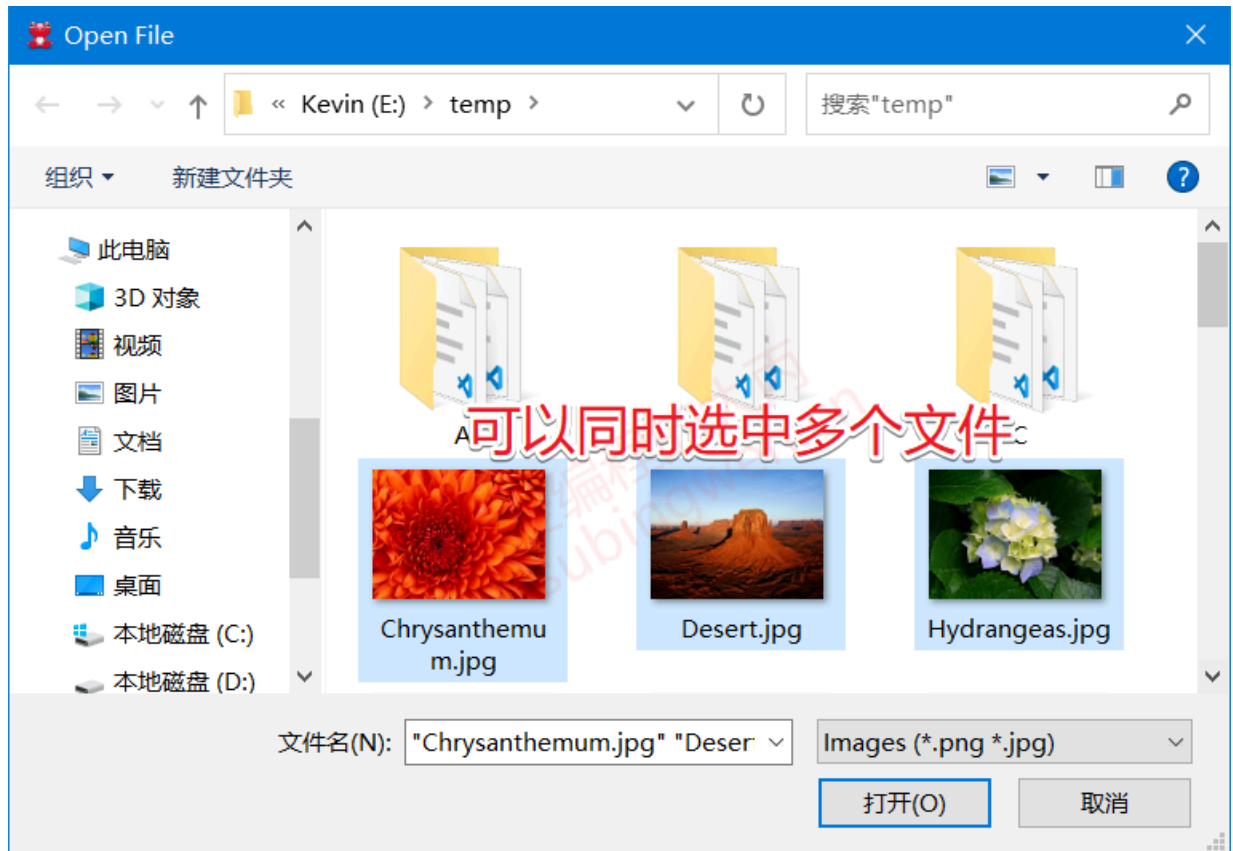


Figure 22

- 打开保存文件对话框

```

1 void MainWindow::on_fileDlg_clicked()
2 {
3 QString fileName = QFileDialog::getSaveFileName(this, "保存文件",
4 "e:\\temp");
5 QMessageBox::information(this, "保存文件", "您指定的保存数据的文件是: " +
6 fileName);
7 //只是获取一个路径，该路径下的文件是否存在，并不关心，如果不存在，在后续操作中可以更具这个路径
8 创建

```

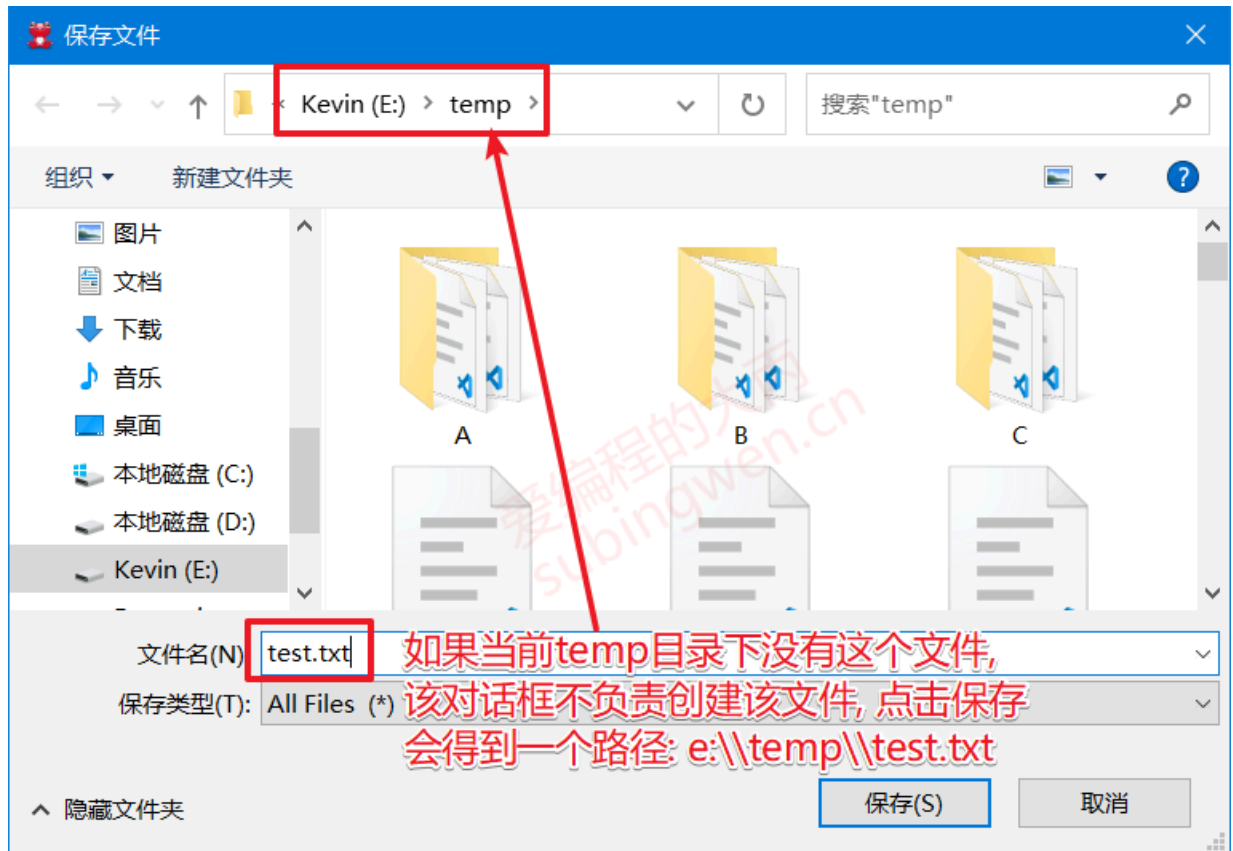


Figure 23

## 6.4.5 QFontDialog

- QFontDialog类是QDialog的子类, 通过这个类我们可以得到一个进行字体属性设置的对话框窗口, 和前边介绍的对话框类一样, 我们只需要调用这个类的静态成员函数就可以得到想要的窗口了。

### 6.4.5.1 QFont 字体类

- 关于字体的属性信息, 在QT框架中被封装到了一个叫QFont的类中,

```

1 // 构造函数
2 QFont::QFont();
3 /*
4 参数:
5 - family: 本地字库中的字体名, 通过 office 等文件软件可以查看
6 - pointSize: 字体的字号
7 - weight: 字体的粗细, 有效范围为 0 ~ 99
8 - italic: 字体是否倾斜显示, 默认不倾斜
9 */
10 QFont::QFont(const QString &family, int pointSize = -1, int weight = -1, bool
 italic = false);
11
12 // 设置字体
13 void QFont::setFamily(const QString &family);
14 // 根据字号设置字体大小
15 void QFont::setPointSize(int pointSize);
16 // 根据像素设置字体大小
17 void QFont::setPixelSize(int pixelSize);
18 // 设置字体的粗细程度, 有效范围: 0 ~ 99

```

```

19 void QFont::setWeight(int weight);
20 // 设置字体是否加粗显示
21 void QFont::setBold(bool enable);
22 // 设置字体是否要倾斜显示
23 void QFont::setItalic(bool enable);
24
25 // 获取字体相关属性(一般规律: 去掉设置函数的 set 就是获取相关属性对应的函数名)
26 QString QFont::family() const;
27 bool QFont::italic() const;
28 int QFont::pixelSize() const;
29 int QFont::pointSize() const;
30 bool QFont::bold() const;
31 int QFont::weight() const;
32
33
34
35 //如果一个QFont对象被创建, 并且进行了初始化, 我们可以将这个属性设置给某个窗口, 或者设置给当前应用程序对象。
36 // QWidget 类
37 // 得到当前窗口使用的字体
38 const QWidget::QFont& font() const;
39 // 给当前窗口设置字体, 只对当前窗口类生效
40 void QWidget::setFont(const QFont &);
41
42 // QApplication 类
43 // 得到当前应用程序对象使用的字体
44 [static] QFont QApplication::font();
45 // 给当前应用程序对象设置字体, 作用于当前应用程序的所有窗口
46 [static] void QApplication::setFont(const QFont &font, const char *className = nullptr);
47
48
49

```

Fence 46

#### 6.4.5.2 QFontDialog的API

```

1 /*
2 参数:
3 - ok: 传出参数, 用于判断是否获得了有效字体信息, 指定一个布尔类型变量地址
4 - initial: 字体对话框中默认选中并显示该字体信息, 用于对话框的初始化
5 - parent: 字体对话框窗口的父对象
6 - title: 字体对话框的窗口标题
7 - options: 字体对话框选项, 使用默认属性即可, 一般不设置
8 */
9 [static] QFont QFontDialog::getFont(
10 bool *ok, const QFont &initial,
11 QWidget *parent = nullptr, const QString &title = QString(),
12 QFontDialog::FontDialogOptions options = FontDialogOptions());
13
14 [static] QFont QFontDialog::getFont(bool *ok, QWidget *parent = nullptr);
15
16
17

```

```

18
19 //例子
20 //通过字体对话框选择字体，并将选择的字体设置给当前窗口
21 void MainWindow::on_fontdlg_clicked()
22 {
23 #if 1
24 // 方式1
25 bool ok;
26 QFont ft = QFontDialog::getFont(
27 &ok, QFont("微软雅黑", 12, QFont::Bold), this, "选择字体");
28 qDebug() << "ok value is: " << ok;
29 #else
30 // 方式2
31 QFont ft = QFontDialog::getFont(NULL);
32 #endif
33 // 将选择的字体设置给当前窗口对象
34 this->setFont(ft);
35 }
36

```

Fence 47

## 6.4.6 QColorDialog

- QColorDialog类是QDialog的子类, 通过这个类我们可以得到一个选择颜色的对话框窗口, 和前边介绍的对话框类一样, 我们只需要调用这个类的静态成员函数就可以得到想要的窗口了。

### 6.4.6.1 颜色类 QColor

- 关于颜色的属性信息, 在QT框架中被封装到了一个叫QColor的类中, 下边为大家介绍一下这个类的API, 了解一下关于这个类的使用。

各种颜色都是基于红, 绿, 蓝这三种颜色调配而成的, 并且颜色还可以进行透明度设置, 默认是不透明的。

- ```

1 // 构造函数
2 QColor::QColor(Qt::GlobalColor color);
3 QColor::QColor(int r, int g, int b, int a = ...);
4 QColor::QColor();
5
6 // 参数设置 red, green, blue, alpha, 取值范围都是 0-255
7 void QColor::setRed(int red); // 红色
8 void QColor::setGreen(int green); // 绿色
9 void QColor::setBlue(int blue); // 蓝色
10 void QColor::setAlpha(int alpha); // 透明度, 默认不透明(255)
11 void QColor::setRgb(int r, int g, int b, int a = 255);
12
13 int QColor::red() const;
14 int QColor::green() const;
15 int QColor::blue() const;
16 int QColor::alpha() const;
17 void QColor::getRgb(int *r, int *g, int *b, int *a = nullptr) const;
18
19

```

Fence 48

6.4.6.2 QColorDialog的API

```
1 // 弹出颜色选择对话框，并返回选中的颜色信息
2 /*
3 参数：
4     - initial: 对话框中默认选中的颜色，用于窗口初始化
5     - parent: 给对话框窗口指定父对象
6     - title: 对话框窗口的标题
7     - options: 颜色对话框窗口选项，使用默认属性即可，一般不需要设置
8 */
9 [static] QColor QColorDialog::getColor(
10     const QColor &initial = Qt::white,
11     QWidget *parent = nullptr, const QString &title = QString(),
12     QColorDialog::ColorDialogOptions options = ColorDialogOptions());
13
14
15
16 //例子
17 /*
18 场景描述：
19     1. 在窗口上放一个标签控件
20     2. 通过颜色对话框选择一个颜色，将选中的颜色显示到标签控件上
21     3. 将选中的颜色的 RGBA 值分别显示出来
22 */
23 void MainWindow::on_colordlg_clicked()
24 {
25     QColor color = QColorDialog::getColor();
26     QBrush brush(color);
27     QRect rect(0, 0, ui->color->width(), ui->color->height());
28     QPixmap pix(rect.width(), rect.height());
29     QPainter p(&pix);
30     p.fillRect(rect, brush);
31     ui->color->setPixmap(pix);
32     QString text = QString("red: %1, green: %2, blue: %3, 透明度: %4")
33         .arg(color.red()).arg(color.green()).arg(color.blue()).arg(color.alpha());
34     ui->colorlabel->setText(text);
35 }
36
```

Fence 49

6.4.7 QInputDialog

- QInputDialog类是QDialog的子类, 通过这个类我们可以得到一个输入对话框窗口, 根据实际需求我们可以在这个输入窗口中输入整形, 浮点型, 字符串类型的数据, 并且还可以显示下拉菜单供使用者选择。
和前边介绍的对话框类一样, 我们只需要调用这个类的静态成员函数就可以得到想要的窗口了。

- ```
1 // 得到一个可以输入浮点数的对话框窗口，返回对话框窗口中输入的浮点数
2 /*
3 参数：
4 - parent: 对话框窗口的父窗口
5 - title: 对话框窗口显示的标题信息
6 - label: 对话框窗口中显示的文本信息(用于描述对话框的功能)
7 - value: 对话框窗口中显示的浮点值，默认为 0
```

```

8 - min: 对话框窗口支持显示的最小数值
9 - max: 对话框窗口支持显示的最大数值
10 - decimals: 浮点数的精度, 默认保留小数点以后1位
11 - ok: 传出参数, 用于判断是否得到了有效数据, 一般不会使用该参数
12 - flags: 对话框窗口的窗口属性, 使用默认值即可
13 */
14 [static] double QInputDialog::getDouble(
15 QWidget *parent, const QString &title,
16 const QString &label, double value = 0,
17 double min = -2147483647, double max = 2147483647,
18 int decimals = 1, bool *ok = nullptr,
19 Qt::WindowFlags flags = Qt::WindowFlags());
20
21 // 得到一个可以输入整形数的对话框窗口, 返回对话框窗口中输入的整形数
22 /*
23 参数:
24 - parent: 对话框窗口的父窗口
25 - title: 对话框窗口显示的标题信息
26 - label: 对话框窗口中显示的文本信息(用于描述对话框的功能)
27 - value: 对话框窗口中显示的整形值, 默认为 0
28 - min: 对话框窗口支持显示的最小数值
29 - max: 对话框窗口支持显示的最大数值
30 - step: 步长, 通过对话框提供的按钮调节数值每次增长/递减的量
31 - ok: 传出参数, 用于判断是否得到了有效数据, 一般不会使用该参数
32 - flags: 对话框窗口的窗口属性, 使用默认值即可
33 */
34 [static] int QInputDialog::getInt(
35 QWidget *parent, const QString &title,
36 const QString &label, int value = 0,
37 int min = -2147483647, int max = 2147483647,
38 int step = 1, bool *ok = nullptr,
39 Qt::WindowFlags flags = Qt::WindowFlags());
40
41 // 得到一个带下来菜单的对话框窗口, 返回选择的菜单项上边的文本信息
42 /*
43 参数:
44 - parent: 对话框窗口的父窗口
45 - title: 对话框窗口显示的标题信息
46 - label: 对话框窗口中显示的文本信息(用于描述对话框的功能)
47 - items: 字符串列表, 用于初始化窗口中的下拉菜单, 每个字符串对应一个菜单项
48 - current: 通过菜单项的索引指定显示下拉菜单中的哪个菜单项, 默认显示第一个(编号为0)
49 - editable: 设置菜单项上的文本信息是否可以进行编辑, 默认为true, 即可以编辑
50 - ok: 传出参数, 用于判断是否得到了有效数据, 一般不会使用该参数
51 - flags: 对话框窗口的窗口属性, 使用默认值即可
52 - inputMethodHints: 设置显示模式, 默认没有指定任何特殊显示格式, 显示普通文本字符串
53 - 如果有特殊需求, 可以参数帮助文档进行相关设置
54 */
55 [static] QString QInputDialog::getItem(
56 QWidget *parent, const QString &title,
57 const QString &label, const QStringList &items,
58 int current = 0, bool editable = true, bool *ok = nullptr,
59 Qt::WindowFlags flags = Qt::WindowFlags(),
60 Qt::InputMethodHints inputMethodHints = Qt::ImhNone);

```

```

61
62 // 得到一个可以输入多行数据的对话框窗口，返回用户在窗口中输入的文本信息
63 /*
64 参数：
65 - parent: 对话框窗口的父窗口
66 - title: 对话框窗口显示的标题信息
67 - label: 对话框窗口中显示的文本信息(用于描述对话框的功能)
68 - text: 指定显示到多行输入框中的文本信息，默认是空字符串
69 - ok: 传出参数，用于判断是否得到了有效数据，一般不会使用该参数
70 - flags: 对话框窗口的窗口属性，使用默认值即可
71 - inputMethodHints: 设置显示模式，默认没有指定任何特殊显示格式，显示普通文本字符串
72 - 如果有特殊需求，可以参数帮助文档进行相关设置
73 */
74 [static] QString QInputDialog::getMultiLineText(
75 QWidget *parent, const QString &title, const QString &label,
76 const QString &text = QString(), bool *ok = nullptr,
77 Qt::WindowFlags flags = Qt::WindowFlags(),
78 Qt::InputMethodHints inputMethodHints = Qt::ImhNone);
79
80 // 得到一个可以输入单行信息的对话框窗口，返回用户在窗口中输入的文本信息
81 /*
82 参数：
83 - parent: 对话框窗口的父窗口
84 - title: 对话框窗口显示的标题信息
85 - label: 对话框窗口中显示的文本信息(用于描述对话框的功能)
86 - mode: 指定单行编辑框中数据的反馈模式，是一个 QLineEdit::EchoMode 类型的枚举值
87 - QLineEdit::Normal: 显示输入的字符。这是默认值
88 - QLineEdit::NoEcho: 不要展示任何东西。这可能适用于连密码长度都应该保密的密码。
89 - QLineEdit::Password: 显示与平台相关的密码掩码字符，而不是实际输入的字符。
90 - QLineEdit::PasswordEchoOnEdit: 在编辑时按输入显示字符，否则按密码显示字符。
91 - text: 指定显示到单行输入框中的文本信息，默认是空字符串
92 - ok: 传出参数，用于判断是否得到了有效数据，一般不会使用该参数
93 - flags: 对话框窗口的窗口属性，使用默认值即可
94 - inputMethodHints: 设置显示模式，默认没有指定任何特殊显示格式，显示普通文本字符串
95 - 如果有特殊需求，可以参数帮助文档进行相关设置
96 */
97 [static] QString QInputDialog::getText(
98 QWidget *parent, const QString &title, const QString &label,
99 QLineEdit::EchoMode mode = QLineEdit::Normal,
100 const QString &text = QString(), bool *ok = nullptr,
101 Qt::WindowFlags flags = Qt::WindowFlags(),
102 Qt::InputMethodHints inputMethodHints = Qt::ImhNone);
103

```

## ◦ 整形输入框

```

1 void MainWindow::on_inputdlg_clicked()
2 {
3 int ret = QInputDialog::getInt(this, "年龄", "您的当前年龄: ", 10, 1, 100,
4 2);
5 QMessageBox::information(this, "年龄", "您的当前年龄: " +
6 QString::number(ret));
7 }
8

```

Fence 51

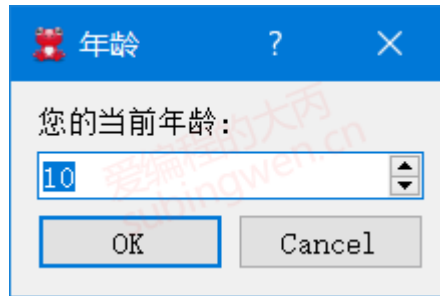


Figure 24

- 单行字符串输入框

```

1 void MainWindow::on_inputdlg_clicked()
2 {
3 QString text = QInputDialog::getText(this, "密码", "请输入新的密码",
4 QLineEdit::Password, "helloworld");
5 QMessageBox::information(this, "密码", "您设置的密码是: " + text);
6 }
7

```

Fence 52

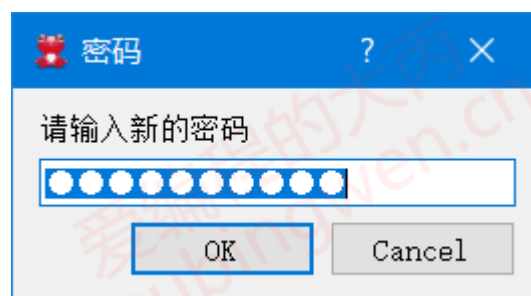


Figure 25

- 浮点型输入框

```

1 void MainWindow::on_inputdlg_clicked()
2 {
3 double ret = QInputDialog::getDouble(this, "工资", "您的工资: ", 2000, 1000,
4 6000, 2);
5 QMessageBox::information(this, "工资", "您的当前工资: " +
6 QString::number(ret));
7 }

```

Fence 53

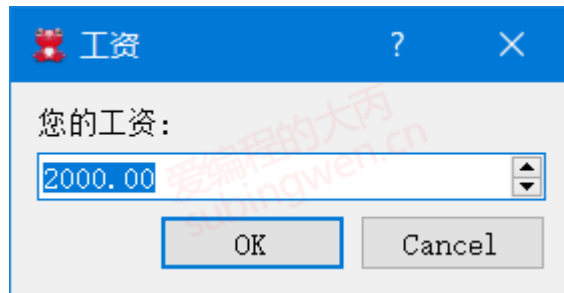


Figure 26

◦ 带下拉菜单的输入框

```

1 void MainWindow::on_inputdlg_clicked()
2 {
3 QStringList items;
4 items << "苹果" << "橙子" << "橘子" << "葡萄" << "香蕉" << "哈密瓜";
5 QString item = QInputDialog::getItem(this, "请选择你喜欢的水果", "你最喜欢的水
6 果:", items, 1, false);
7 QMessageBox::information(this, "水果", "您最喜欢的水果是: " + item);
8 }
9

```

Fence 54

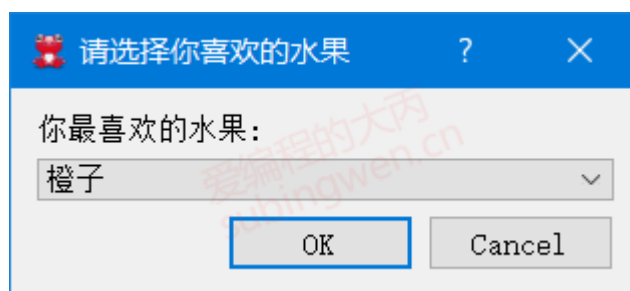


Figure 27

◦ 多行字符串输入框

```

1 void MainWindow::on_inputdlg_clicked()
2 {
3 QString info = QInputDialog::getMultiLineText(this, "表白", "您最想对漂亮小姐
4 姐说什么呢?", "呦吼吼...");
5 QMessageBox::information(this, "知心姐姐", "您最想对小姐姐说: " + info);
6 }

```

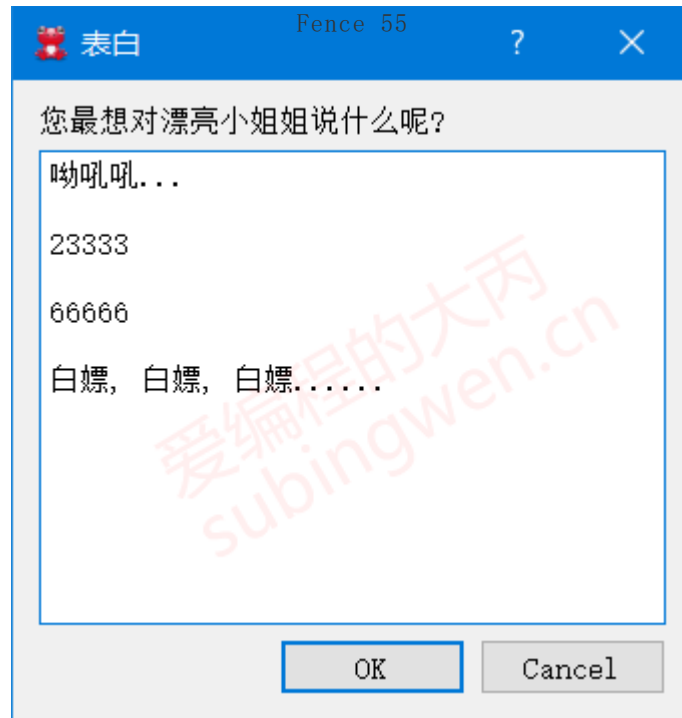


Figure 28

## 6.4.8 QProgressDialog

- QProgressDialog类是QDialog的子类, 通过这个类我们可以得到一个带进度条的对话框窗口, 这种类型的对话框窗口一般常用于文件拷贝、数据传输等实时交互的场景中。

```
1 // 构造函数
2 /*
3 参数:
4 - labelText: 对话框中显示的提示信息
5 - cancelButtonText: 取消按钮上显示的文本信息
6 - minimum: 进度条最小值
7 - maximum: 进度条最大值
8 - parent: 当前窗口的父对象
9 - f: 当前进度窗口的flag属性, 使用默认属性即可, 无需设置
10 */
11 QProgressDialog::QProgressDialog(
12 QWidget *parent = nullptr,
13 Qt::WindowFlags f = Qt::WindowFlags());
14
15 QProgressDialog::QProgressDialog(
16 const QString &labelText, const QString &cancelButtonText,
17 int minimum, int maximum, QWidget *parent = nullptr,
18 Qt::WindowFlags f = Qt::WindowFlags());
19
20
21 // 设置取消按钮显示的文本信息
22 [slot] void QProgressDialog::setCancelButtonText(const QString &cancelButtonText);
23
24 // 公共成员函数和槽函数
25 QString QProgressDialog::labelText() const;
```

```

26 void QProgressDialog::setLabelText(const QString &text);
27
28 // 得到进度条最小值
29 int QProgressDialog::minimum() const;
30 // 设置进度条最小值
31 void QProgressDialog::setMinimum(int minimum);
32
33 // 得到进度条最大值
34 int QProgressDialog::maximum() const;
35 // 设置进度条最大值
36 void QProgressDialog::setMaximum(int maximum);
37
38 // 设置进度条范围(最大和最小值)
39 [slot] void QProgressDialog::setRange(int minimum, int maximum);
40
41 // 得到进度条当前的值
42 int QProgressDialog::value() const;
43 // 设置进度条当前的值
44 void QProgressDialog::setValue(int progress);
45
46
47 bool QProgressDialog::autoReset() const;
48 // 当value() = maximum()时, 进程对话框是否调用reset(), 此属性默认为true。
49 void QProgressDialog::setAutoReset(bool reset);
50
51
52 bool QProgressDialog::autoClose() const;
53 // 当value() = maximum()时, 进程对话框是否调用reset()并且隐藏, 此属性默认为true。
54 void QProgressDialog::setAutoClose(bool close);
55
56 // 判断用户是否按下了取消键, 按下了返回true, 否则返回false
57 bool wasCanceled() const;
58
59
60 // 重置进度条
61 // 重置进度对话框。wasCancelled()变为true, 直到进程对话框被重置。进度对话框被隐藏。
62 [slot] void QProgressDialog::cancel();
63 // 重置进度对话框。如果autoClose()为真, 进程对话框将隐藏。
64 [slot] void QProgressDialog::reset();
65
66 // 信号
67 // 当单击cancel按钮时, 将发出此信号。默认情况下, 它连接到cancel()槽。
68 [signal] void QProgressDialog::canceled();
69
70 // 设置窗口的显示状态(模态, 非模态)
71 /*
72 参数:
73 Qt::NonModal -> 非模态
74 Qt::WindowModal -> 模态, 阻塞父窗口
75 Qt::ApplicationModal -> 模态, 阻塞应用程序中的所有窗口
76 */
77 void QWidget::setWindowModality(Qt::WindowModality windowModality);
78

```

```

79
80
81
82 //例子
83 /*
84 场景描述:
85 1. 基于定时器模拟文件拷贝的场景
86 2. 点击窗口按钮, 进度条窗口显示, 同时启动定时器
87 3. 通过定时器信号, 按照固定频率更新对话框窗口进度条
88 4. 当进度条当前值 == 最大值, 关闭定时器, 关闭并析构进度对话框
89 */
90 void MainWindow::on_progressdlg_clicked()
91 {
92 // 1. 创建进度条对话框窗口对象
93 QProgressDialog *progress = new QProgressDialog(
94 "正在拷贝数据...", "取消拷贝", 0, 100, this);
95 // 2. 初始化并显示进度条窗口
96 progress->setWindowTitle("请稍后");
97 progress->setWindowModality(Qt::WindowModal);
98 progress->show();
99
100 // 3. 更新进度条
101 static int value = 0;
102 QTimer *timer = new QTimer;
103 connect(timer, &QTimer::timeout, this, [=]()
104 {
105 progress->setValue(value);
106 value++;
107 // 当value > 最大值的时候
108 if(value > progress->maximum())
109 {
110 timer->stop();
111 value = 0;
112 delete progress;
113 delete timer;
114 }
115 });
116
117 connect(progress, &QProgressDialog::canceled, this, [=]()
118 {
119 timer->stop();
120 value = 0;
121 delete progress;
122 delete timer;
123 });
124
125 timer->start(50);
126 }
127
128

```

## 6.4.9 其他对话框

```
1 connect(ui->actionNew,&QAction::triggered,[=]() {
2 //其他对话框
3 //颜色对话框
4 // QColor color = QColorDialog::getColor(QColor(255,55,0));
5 //QColor()还有第四个参数,为透明度
6 // qDebug() << "r=" << color.red() << "g=" << color.green() << "b=" <<
7 color.blue();
8 // qDebug() << "html =" << color.name();
9
10 //文本对话框
11 //参数1 父对象 参数2 窗口标题 参数3 文件路径(需要转义字符\ 参数4 过滤文本格式
12 // QString fileName = QFileDialog::getOpenFileName(this,"打开文件","E:\\笔记","
13 (*.txt)");
14 // qDebug() << fileName;
15
16 //字体对话框
17 bool flag; //是否选择字体
18 QFont font = QFontDialog::getFont(&flag,QFont("宋体",20,1),this,"字体选择");
19 qDebug() << "字体: " << font.family().toUtf8().data() << "字号: " <<
20 font.pointSize() << "是否加粗: " << font.bold() << "是否倾斜:" << font.italic();
21 qDebug() << flag;
22 });
```

Fence 57

## 7. 容器控件

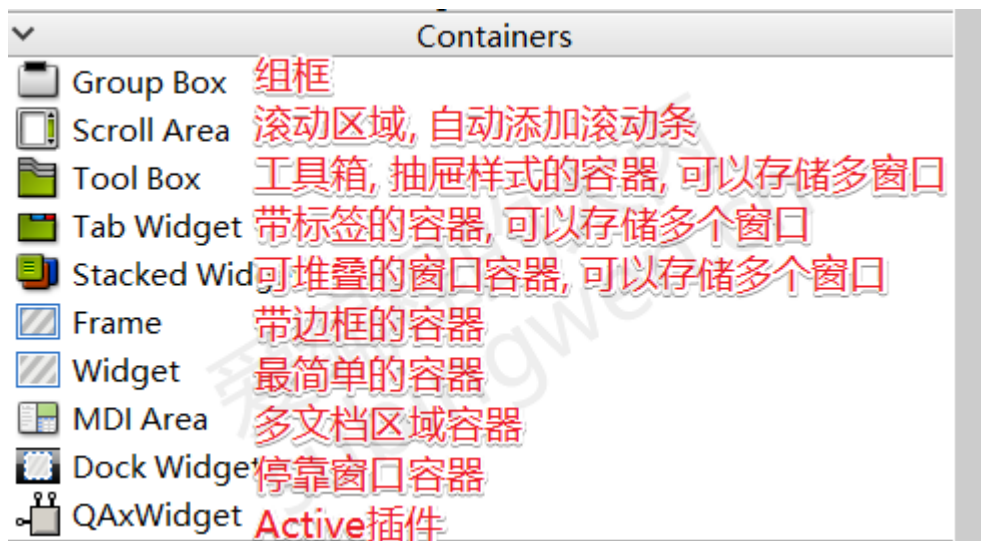


Figure 29

7.1 QWidget

选中属性，按F1可以直接跳转到对于帮助文档

| 属性                      | 值                                   |                      |
|-------------------------|-------------------------------------|----------------------|
| > QObject               |                                     |                      |
| ▼ QWidget               |                                     |                      |
| windowModality          | NonModal                            | 窗口的显示模式: 模态, 非模态     |
| enabled                 | <input checked="" type="checkbox"/> | 窗口是否有效, 无效则无法处理触发的事件 |
| > geometry              | [(0, 0), 329 x 316]                 | 窗口当前的几何信息, 位置和宽度, 高度 |
| > sizePolicy            | [Preferred, Preferred, 0, 0]        | 窗口的尺寸策略, 默认自动缩放      |
| > minimumSize           | 0 x 0                               | 设置窗口的最新尺寸信息          |
| > maximumSize           | 16777215 x 16777215                 | 设置窗口的最大尺寸信息          |
| > sizeIncrement         | 0 x 0                               | 设置窗口的尺寸增量            |
| > baseSize              | 0 x 0                               |                      |
| palette                 | 继承                                  |                      |
| > font                  | A [SimSun, 9]                       | 设置窗口使用的字体            |
| cursor                  | 鼠标 箭头                               | 设置窗口中的光标样式           |
| mouseTracking           | <input type="checkbox"/>            | 设置窗口是否追踪鼠标移动事件       |
| tabletTracking          | <input type="checkbox"/>            | 设置窗口是否追踪键盘 Tab 事件    |
| focusPolicy             | NoFocus                             | 设置窗口的焦点策略            |
| contextMenuPolicy       | DefaultContextMenu                  | 设置窗口的右键菜单策略          |
| acceptDrops             | <input type="checkbox"/>            |                      |
| > windowTitle           | MainWindow                          | 设置窗口的标题              |
| > windowIcon            |                                     | 设置窗口的图标              |
| windowOpacity           | 1.000000                            | 设置窗口的透明度, 默认不透明      |
| > toolTip               |                                     | 设置窗口的提示信息            |
| toolTipDuration         | -1                                  | 设置窗口提示信息持续的时长        |
| > statusTip             |                                     |                      |
| > whatsThis             |                                     |                      |
| > accessibleName        |                                     |                      |
| > accessibleDescription |                                     |                      |
| layoutDirection         | LeftToRight                         |                      |
| autoFillBackground      | <input type="checkbox"/>            |                      |
| styleSheet              |                                     | 给窗口设置样式表, 进行窗口的美化    |
| > locale                | Chinese, China                      |                      |

Figure 30

## 7.2 Frame

- QFrame就是一个升级版的QWidget, 它继承了QWidget的属性, 并且做了拓展, 这种类型的容器窗口可以提供边框, 并且可以设置边框的样式、宽度以及边框的阴影。
- 相关API

```
1 /*
2 边框形状为布尔类型，可选项为：
3 - QFrame::NoFrame：没有边框
4 - QFrame::Box：绘制一个框
5 - QFrame::Panel：绘制一个面板，使内容显示为凸起或凹陷
6 - QFrame::StyledPanel：绘制一个外观取决于当前GUI样式的矩形面板。它可以上升也可以下沉。
7 - QFrame::HLine：画一条没有边框的水平线(用作分隔符)
8 - QFrame::VLine：画一条没有边框的垂直线(用作分隔符)
9 - QFrame::WinPanel：绘制一个矩形面板，可以像Windows 2000那样向上或向下移动。
10 指定此形状将线宽设置为2像素。WinPanel是为了兼容而提供的。
11 对于GUI风格的独立性，我们建议使用StyledPanel代替。
12 */
13 // 获取边框形状
14 Shape frameShape() const;
15 // 设置边框形状
16 void setFrameShape(Shape);
17
18
19 /*
20 Qt中关于边框的阴影(QFrame::Shadow)提供了3种样式，分别为：
21 - QFrame::Plain：简单的, 朴素的，框架和内容与周围环境显得水平；
22 使用调色板绘制QPalette::WindowText颜色(没有任何3D效果)
23 - QFrame::Raised：框架和内容出现凸起；使用当前颜色组的明暗颜色绘制3D凸起线
24 - QFrame::Sunken：框架及内容物凹陷；使用当前颜色组的明暗颜色绘制3D凹线
25 */
26 // 获取边框阴影样式
27 Shadow frameShadow() const;
28 // 设置边框阴影样式
29 void setFrameShadow(Shadow);
30
31 // 得到边框线宽度
32 int lineWidth() const;
33 // 设置边框线宽度，默认值为1
34 void setLineWidth(int);
35
36 // 得到中线的宽度
37 int midLineWidth() const;
38 // 设置中线宽度，默认值为0，这条线会影响边框阴影的显示
39 void setMidLineWidth(int);
40
41
```

- 相关属性

这个类的属性并不多, 都是关于边框的设置。

| ▼ QFrame            |        |              |
|---------------------|--------|--------------|
| <b>frameShape</b>   | Box    | 边框形状         |
| <b>frameShadow</b>  | Sunken | 边框阴影样式       |
| <b>lineWidth</b>    | 3      | 边框线宽         |
| <b>midLineWidth</b> | 1      | 中线宽度, 主要控制阴影 |

这个表格显示了一些边框样式和线宽以及阴影的组合:

| 0 |   |   |   | 1 |   |   |   | 2 |   |   |   | 3 |   |   |   | lineWidth()         |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---------------------|
| 0 | 1 | 2 | 3 | 0 | 1 | 2 | 3 | 0 | 1 | 2 | 3 | 0 | 1 | 2 | 3 | midLineWidth()      |
|   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   | Box, Plain          |
|   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   | Box, Raised         |
|   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   | Box, Sunken         |
|   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   | Panel, Plain        |
|   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   | Panel, Raised       |
|   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   | Panel, Sunken       |
|   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   | WinPanel, Plain     |
|   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   | WinPanel, Raised    |
|   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   | WinPanel, Sunken    |
|   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   | HLine, Plain        |
|   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   | HLine, Raised       |
|   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   | HLine, Sunken       |
|   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   | VLine, Plain        |
|   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   | VLine, Raised       |
|   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   | VLine, Sunken       |
|   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   | StyledPanel, Plain  |
|   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   | StyledPanel, Raised |
|   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   | StyledPanel, Sunken |

Figure 31

## 7.3 Group Box

- QGroupBox类的基类是QWidget, 在这种类型的窗口中可以绘制边框、给窗口指定标题, 并且还支持显示复选框。
- 相关API

```

1 // 构造函数
2 QGroupBox::QGroupBox(QWidget *parent = Q_NULLPTR);
3 QGroupBox::QGroupBox(const QString &title, QWidget *parent = Q_NULLPTR);
4
5 // 公共成员函数
6 bool QGroupBox::isChecked() const;
7 // 设置是否在组框中显示一个复选框
8 void QGroupBox::setCheckable(bool checkable);
9
10 /*
11 关于对齐方式需要使用枚举类型 Qt::Alignment, 其可选项为:
12 - Qt::AlignLeft: 左对齐(水平方向)
13 - Qt::AlignRight: 右对齐(水平方向)
14 - Qt::AlignHCenter: 水平居中
15 - Qt::AlignJustify: 在可用的空间内调整文本(水平方向)
16
17 - Qt::AlignTop: 上对齐(垂直方向)
18 - Qt::AlignBottom: 下对齐(垂直方向)
19 - Qt::AlignVCenter: 垂直居中
20 */
21 Qt::Alignment QGroupBox::alignment() const;
22 // 设置组框标题的对齐方式
23 void QGroupBox::setAlignment(int alignment);
24
25 QString QGroupBox::title() const;
26 // 设置组框的标题
27 void QGroupBox::setTitle(const QString &title);
28
29 bool QGroupBox::isChecked() const;
30 // 设置组框中复选框的选中状态
31 [slot] void QGroupBox::setChecked(bool checked);
32

```

Fence 59

- 属性设置

| ▼ QGroupBox |                                                  |
|-------------|--------------------------------------------------|
| ▼ title     | 你知道我在等你吗???                                      |
| 可翻译的        | <input checked="" type="checkbox"/> 设置组框的标题      |
| 澄清          |                                                  |
| 注释          |                                                  |
| ▼ alignment | AlignJustify, AlignTop                           |
| 水平的         | AlignJustify 设置组框标题的对齐方式                         |
| 垂直的         | AlignTop (水平或者垂直方向)                              |
| flat        | <input checked="" type="checkbox"/> 设置组框的绘制方式    |
| checkable   | <input checked="" type="checkbox"/> 设置是否给组框添加复选框 |
| checked     | <input type="checkbox"/> 设置添加的复选框的选择状态           |

Figure 32

组框中的flat属性没有对应的API函数, 只能在属性窗口中设置, 它控制的是窗口边框的绘制方式, 如果打开该属性, 组框的边框就消失了, 效果如下:

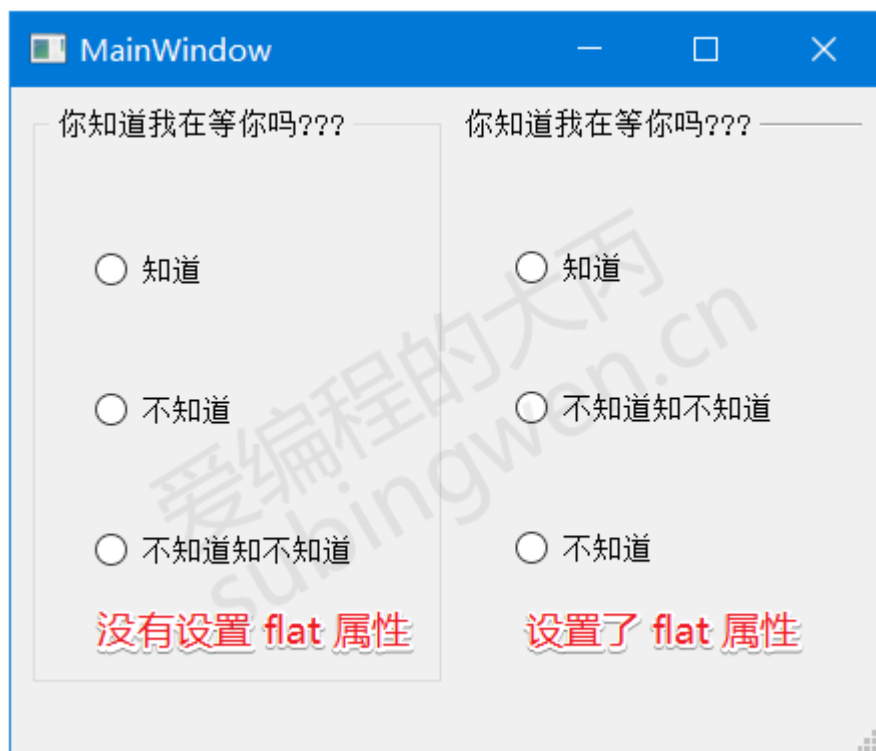


Figure 33

## 7.4 Scroll Area

- QScrollArea这种类型的容器, 里边可以放置一些窗口控件, 当放置的窗口控件大于当前区域导致无法全部显示的时候, 滚动区域容器会自动添加相应的滚动条(水平方向或者垂直方向), 保证放置到该区域中的所有窗口内容都可以正常显示出来。对于使用者不需要做太多事情, 只需要把需要显示的窗口放到滚动区域中就行了。
- 相关API

```

1 //在某些特定环境下，我们需要动态的往滚动区域内部添加要显示的窗口，或者动态的将显示的窗口移除，这
 时候就必须要调用对应的API函数来完成这部分操作了。主要API有两个 添加 - setwidget()，移除 -
 takewidget()

2
3
4 // 构造函数
5 QScrollArea::QScrollArea(QWidget *parent = Q_NULLPTR);
6
7 // 公共成员函数
8 // 给滚动区域设置要显示的子窗口widget
9 void QScrollArea::setwidget(QWidget *widget);
10 // 删除滚动区域中的子窗口，并返回被删除的子窗口对象
11 QWidget *QScrollArea::takewidget();
12
13 /*
14 关于显示位置的设定，是一个枚举类型，可选项为：
15 - Qt::AlignLeft: 左对齐
16 - Qt::AlignHCenter: 水平居中
17 - Qt::AlignRight: 右对齐
18 - Qt::AlignTop: 顶部对齐
19 - Qt::AlignVCenter: 垂直对其
20 - Qt::AlignBottom: 底部对其
21 */
22 // 获取子窗口在滚动区域中的显示位置
23 Qt::Alignment alignment() const;
24 // 设置滚动区域中子窗口的对其方式，默认显示的位置是右上
25 void setAlignment(Qt::Alignment);
26
27 // 判断滚动区域是否有自动调节小部件大小的属性
28 bool widgetResizable() const;
29 /*
30 1. 设置滚动区域是否应该调整视图小部件的大小，该属性默认为false，滚动区域按照小部件的默认大小进行
 显示。
31 2. 如果该属性设置为true，滚动区域将自动调整小部件的大小，避免滚动条出现在本可以避免的地方，
 或者利用额外的空间。
32 3. 不管这个属性是什么，我们都可以使用widget()->resize()以编程方式调整小部件的大小，
 滚动区域将自动调整自己以适应新的大小。
33
34 */
35
36 void setWidgetResizable(bool resizable);
37
38

```

- 属性

| ▼ QScrollArea          |                                                      |
|------------------------|------------------------------------------------------|
| <b>widgetResizable</b> | <input checked="" type="checkbox"/> 是否自动调整滚动区域内部窗口大小 |
| ▼ alignment            | AlignLeft, AlignVCenter                              |
| 水平的                    | AlignLeft 子窗口在滚动区域                                   |
| 垂直的                    | AlignVCenter 内部的对齐方式                                 |

Figure 34

- 窗口的动态添加和删除

- 关于窗口的滚动区域对象创建有两种方式, 第一种比较简单在编辑页面直接拖拽一个控件到UI界面, 然后布局即可。第二种方式是在程序中通过new操作创建一个实例对象, 然后通过通过代码的方式将其添加到窗口的某个布局中, 相对来说要麻烦一点。

下面通过第一种方式, 演示一下如果往滚动区域中添加多个子窗口。

- ```

1  MainWindow::MainWindow(QWidget *parent)
2      : QMainWindow(parent)
3      , ui(new Ui::MainWindow)
4  {
5      ui->setupUi(this);
6
7      // 创建一个垂直布局对象
8      QVBoxLayout* vlayout = new QVBoxLayout;
9
10     for(int i=0; i<11; ++i)
11     {
12         // 创建标签对象
13         QLabel* pic = new QLabel;
14         // 拼接图片在资源文件中的路径
15         QString name = QString(":/images/%1.png").arg(i+1);
16         // 给标签对象设置显示的图片
17         pic->setPixmap(QPixmap(name));
18         // 设置图片在便签内部的对齐方式
19         pic->setAlignment(Qt::AlignHCenter);
20         // 将标签添加到垂直布局中
21         vlayout->addWidget(pic);
22     }
23
24     // 创建一个窗口对象
25     QWidget* wg = new QWidget;
26     // 将垂直布局设置给窗口对象
27     wg->setLayout(vlayout);
28     // 将带有垂直布局的窗口设置到滚动区域中
29     ui->scrollArea->setWidget(wg);
30 }
31
32

```

Fence 61

- 关于以上代码做以下说明, 调用setWidget(wg)之后, wg会自动平铺填充满整个滚动区域, 因此:
 - 在程序中调用 void setWidgetResizable(bool resizable);不会有明显效果
 - 在程序中调用 void setAlignment(Qt::Alignment);不会看到任何效果

- 如果要设置显示的图片的对其方式要设置图片的载体对象即 标签对象
- 如果要动态移除滚动区域中的窗口, 直接使用滚动区域对象调用 takeWidget() 即可
- 滚动区域中只能通过setWidget(wg)添加一个子窗口, 如果要添加多个可使用布局的方式来实现

7.5 Tool Box

- QToolBox工具箱控件, 可以存储多个子窗口, 该控件可以实现类似QQ的抽屉效果, 每一个抽屉都可以设置图标和标题, 并且对应一个子窗口, 通过抽屉按钮就可以实现各个子窗口显示的切换。
- 相关API

```

1 // 构造函数
2 QToolBox::QToolBox(QWidget *parent = Q_NULLPTR, Qt::WindowFlags f =
  Qt::WindowFlags());
3
4 // 公共成员
5 /*
6 addItem(), insertItem()函数相关参数:
7     - widget: 添加到工具箱中的选项卡对应的子窗口对象
8     - icon: 工具箱新的选项卡上显示的图标
9     - text: 工具箱新的选项卡上显示的标题
10    - index: 指定在工具箱中插入的新的选项卡的位置
11 */
12 // 给工具箱尾部添加一个选项卡, 每个选项卡在工具箱中就是一个子窗口, 即参数widget
13 int QToolBox::addItem(QWidget *widget, const QString &text);
14 int QToolBox::addItem(QWidget *widget, const QIcon &icon, const QString &text);
15 // 在工具箱的指定位置添加一个选项卡, 即添加一个子窗口
16 int QToolBox::insertItem(int index, QWidget *widget, const QString &text);
17 int QToolBox::insertItem(int index, QWidget *widget, const QIcon &icon,
18                          const QString &text);
19 // 移除工具箱中索引index位置对应的选项卡, 注意: 只是移除对应的窗口对象并没有被销毁
20 void QToolBox::removeItem(int index);
21
22 // 设置索引index位置的选项卡是否可用, 参数 enabled=true为可用, enabled=false为禁用
23 void QToolBox::setItemEnabled(int index, bool enabled);
24 // 设置工具箱中index位置选项卡的图标
25 void QToolBox::setItemIcon(int index, const QIcon &icon);
26 // 设置工具箱中index位置选项卡的标题
27 void QToolBox::setItemText(int index, const QString &text);
28 // 设置工具箱中index位置选项卡的提示信息(需要鼠标在选项卡上悬停一定时长才能显示)
29 void QToolBox::setItemToolTip(int index, const QString &toolTip);
30
31 // 如果位置索引的项已启用, 则返回true;否则返回false。
32 bool QToolBox::setItemEnabled(int index) const;
33 // 返回位置索引处项目的图标, 如果索引超出范围, 则返回空图标。
34 QIcon QToolBox::itemIcon(int index) const;
35 // 返回位于位置索引处的项的文本, 如果索引超出范围, 则返回空字符串。
36 QString QToolBox::itemText(int index) const;
37 // 返回位于位置索引处的项的工具提示, 如果索引超出范围, 则返回空字符串。
38 QString QToolBox::itemToolTip(int index) const;
39

```

```

40 // 得到当前工具箱中显示的选项卡对应的索引
41 int QToolBox::currentIndex() const;
42 // 返回指向当前选项卡对应的子窗口的指针, 如果没有这样的项, 则返回0。
43 QWidget *QToolBox::currentWidget() const;
44 // 返回工具箱中子窗口的索引, 如果widget对象不存在, 则返回-1
45 int QToolBox::indexOf(QWidget *widget) const;
46 // 返回工具箱中包含的项的数量。
47 int QToolBox::count() const;
48
49 // 信号
50 // 工具箱中当前显示的选项卡发生变化, 该信号被发射, index为当前显示的新的选项卡的对应的索引
51 [signal] void QToolBox::currentChanged(int index);
52
53 // 槽函数
54 // 通过工具箱中选项卡对应的索引设置当前要显示哪一个选项卡中的子窗口
55 [slot] void QToolBox::setCurrentIndex(int index);
56 // 通过工具箱中选项卡对应的子窗口对象设置当前要显示哪一个选项卡中的子窗口
57 [slot] void QToolBox::setCurrentWidget(QWidget *widget);
58
59
60

```

Fence 62

- 属性

▼ QToolBox	
currentIndex	1 工具箱中当前选项卡对应的索引, 从0开始
▶ currentItemText	Page 2 工具箱中当前选项卡上显示的标题
currentItemName	page_4 当前选项卡对应的子窗口的objectName
▶ currentItemIcon	当前选项卡上显示的图标, 默认没有, 可以设置
▶ currentItemToolTip	当前选项卡显示的提示信息
tabSpacing	20 工具箱中窗口折叠后, 选项卡之间的间隙

Figure 35

7.6 Tab Widget

- QTabWidget的一种带标签页的窗口, 在这种类型的窗口中可以存储多个子窗口, 每个子窗口的显示可以通过对应的标签进行切换。
- 相关API

```

1 // 构造函数
2 QTabWidget::QTabWidget(QWidget *parent = Q_NULLPTR);
3
4 // 公共成员函数
5 /*
6 添加选项卡addTab()或者插入选项卡insertTab()函数相关的参数如下:
7     - page: 添加或者插入的选项卡对应的窗口实例对象
8     - label: 添加或者插入的选项卡的标题

```

```

9      - icon: 添加或者插入的选项卡的图标
10     - index: 将新的选项卡插入到索引index的位置上
11 */
12 int QTabWidget::addTab(QWidget *page, const QString &label);
13 int QTabWidget::addTab(QWidget *page, const QIcon &icon, const QString &label);
14 int QTabWidget::insertTab(int index, QWidget *page, const QString &label);
15 int QTabWidget::insertTab(int index, QWidget *page,
16                           const QIcon &icon, const QString &label);
17 // 删除index位置的选项卡
18 void QTabWidget::removeTab(int index);
19
20 // 得到选项卡栏中的选项卡的数量
21 int count() const;
22 // 从窗口中移除所有页面，但不删除它们。调用这个函数相当于调用removeTab()，直到选项卡小部件为空
    为止。
23 void QTabWidget::clear();
24 // 获取当前选项卡对应的索引
25 int QTabWidget::currentIndex() const;
26 // 获取当前选项卡对应的窗口对象地址
27 QWidget *QTabWidget::currentWidget() const;
28 // 返回索引位置为index的选项卡页，如果索引超出范围则返回0。
29 QWidget *QTabWidget::widget(int index) const;
30
31 /*
32 标签上显示的文本样式为枚举类型 Qt::TextElideMode，可选项为：
33     - Qt::ElideLeft: 省略号应出现在课文的开头，例如：.....是的,我很帅。
34     - Qt::ElideRight: 省略号应出现在文本的末尾，例如：我帅吗.....。
35     - Qt::ElideMiddle: 省略号应出现在文本的中间，例如：我帅.....很帅。
36     - Qt::ElideNone: 省略号不应出现在文本中
37 */
38 // 获取标签上显示的文本模式
39 Qt::TextElideMode QTabWidget::elideMode() const;
40 // 如何省略标签栏中的文本，此属性控制在给定的选项卡栏大小没有足够的空间显示项时如何省略项。
41 void QTabWidget::setElideMode(Qt::TextElideMode);
42
43 // 得到选项卡上图标尺寸的信息
44 QSize QTabWidget::iconSize() const
45 // 设置选项卡上显示的图标大小
46 void QTabWidget::setIconSize(const QSize &size)
47
48 // 判断用户是否可以在选项卡区域内移动选项卡，可以返回true，否则返回false
49 bool QTabWidget::isMovable() const;
50 // 此属性用于设置用户是否可以在选项卡区域内移动选项卡。默认情况下，此属性为false;
51 void QTabWidget::setMovable(bool movable);
52
53 // 判断选项卡是否可以自动隐藏，如果可以自动隐藏返回true，否则返回false
54 bool QTabWidget::tabBarAutoHide() const;
55 // 如果为true，则当选项卡栏包含少于2个选项卡时，它将自动隐藏。默认情况下，此属性为false。
56 void QTabWidget::setTabBarAutoHide(bool enabled);
57
58 // 判断index对应的选项卡是否是被启用的，如果是被启用的返回true，否则返回false
59 bool QTabWidget::isTabEnabled(int index) const;
60 // 如果enable为true，则在索引位置的页面是启用的;否则，在位置索引处的页面将被禁用。

```

```

61 void QTabWidget::setTabEnabled(int index, bool enable);
62
63 // 得到index位置的标签对应的图标
64 QIcon QTabWidget::tabIcon(int index) const;
65 // 在位置索引处设置标签的图标。
66 void QTabWidget::setTabIcon(int index, const QIcon &icon);
67
68 /*
69 选项卡标签的位置通过枚举值进行指定，可使用的选项如下：
70     - QTabWidget::North: 北(上)，默认
71     - QTabWidget::South: 南(下)
72     - QTabWidget::West: 西(左)
73     - QTabWidget::East: 东(右)
74 */
75 // 得到选项卡中显示的标签的位置，即：东，西，南，北
76 TabPosition QTabWidget::tabPosition() const;
77 // 设置选项卡中标签显示的位置，默认情况下，此属性设置为North。
78 void QTabWidget::setTabPosition(TabPosition);
79
80 /*
81 选项卡标签的形状通过枚举值进行指定，可使用的选项如下：
82     - QTabWidget::Rounded: 标签以圆形的外观绘制。这是默认形状
83     - QTabWidget::Triangular: 选项卡以三角形外观绘制。
84 */
85 // 获得选项卡标签的形状
86 TabShape QTabWidget::tabShape() const;
87 // 设置选项卡标签的形状
88 void QTabWidget::setTabShape(TabShape s);
89
90 // 得到index位置的标签的标题
91 QString QTabWidget::tabText(int index) const;
92 // 设置选项卡index位置的标签的标题
93 void QTabWidget::setTabText(int index, const QString &label);
94
95
96 // 获取index对应的标签页上设置的提示信息
97 QString QTabWidget::tabToolTip(int index) const;
98 // 设置选项卡index位置的标签的提示信息(鼠标需要悬停在标签上一定时长才能显示)
99 void QTabWidget::setTabToolTip(int index, const QString &tip);
100
101
102 // 判断选项卡标签上是否有关闭按钮，如果有返回true，否则返回false
103 bool QTabWidget::tabsClosable() const;
104 // 设置选项卡的标签页上是否显示关闭按钮，该属性默认情况下为false
105 void QTabWidget::setTabsClosable(bool closeable);
106
107
108 // 判断选项卡栏中是否有滚动按钮，如果有返回true，否则返回false
109 bool QTabWidget::usesScrollButtons() const;
110 // 设置选项卡栏有许多标签时，它是否应该使用按钮来滚动标签。
111 // 当一个选项卡栏有太多的标签时，选项卡栏可以选择扩大它的大小，或者添加按钮，让标签在选项卡栏中滚动。
112 void QTabWidget::setUsesScrollButtons(bool useButtons);

```

```

113
114 // 判断窗口是否设置了文档模式，如果设置了返回true，否则返回false
115 bool QTabWidget::documentMode() const;
116 // 此属性保存选项卡小部件是否以适合文档页面的模式呈现。这与macOS上的文档模式相同。
117 // 不设置该属性，QTabWidget窗口是带边框的，如果设置了该属性边框就没有了。
118 void QTabWidget::setDocumentMode(bool set);
119
120
121 // 每当前页索引改变时，就会发出这个信号。参数是新的当前页索引位置，如果没有新的索引位置，则为-1
122 [signal] void QTabWidget::currentChanged(int index);
123 // 当用户单击索引处的选项卡时，就会发出这个信号。index指所单击的选项卡，如果光标下没有选项卡，则为-1。
124 [signal] void QTabWidget::tabBarClicked(int index)
125 // 当用户双击索引上的一个选项卡时，就会发出这个信号。
126 // index是单击的选项卡的索引，如果光标下没有选项卡，则为-1。
127 [signal] void QTabWidget::tabBarDoubleClicked(int index);
128 // 此信号在单击选项卡上的close按钮时发出。索引是应该被删除的索引。
129 [signal] void QTabWidget::tabCloseRequested(int index);
130
131

```

Fence 63

• 属性

▶ QWidget	
▼ QTabWidget	
tabPosition	North 标签的在窗口中的位置, 上北下南, 左西右东
tabShape	Rounded 标签的形状, 有圆形和三角形可选择
currentIndex	0 当前选中的标签对应的索引
▶ iconSize	16 x 16 标签上显示的图标大小
elideMode	ElideNone 标签上的文本信息省略方式
usesScrollButtons	☒ 标签页太多无法全部显示时, 是否添加滚动按钮
documentMode	☐ 文档模式是否开启, 如果开启窗口边框会被去掉
tabsClosable	☐ 标签页上是否添加关闭按钮
movable	☐ 标签页是否可以移动
tabBarAutoHide	☐ 当标签 < 2时, 标签栏是否自动隐藏
▶ currentTabText	Tab 1 当前标签上显示的文本信息
currentTabName	tab 当前标签对应的窗口的objectName
▶ currentTabIcon	当前标签上显示的图标
▶ currentTabToolTip	当前标签的提示信息
▶ currentTabWhatsThis	

Figure 36

• 例子

```

1 //mainwindow.h
2 QT_BEGIN_NAMESPACE
3 namespace Ui { class MainWindow; }
4 QT_END_NAMESPACE
5
6 class MainWindow : public QMainWindow
7 {
8     Q_OBJECT
9
10 public:
11     MainWindow(QWidget *parent = nullptr);
12     ~MainWindow();
13
14 private:
15     Ui::MainWindow *ui;
16     QQueue<QWidget*> m_widgets; // 存储标签对应的窗口对象
17     QQueue<QString> m_names;    // 存储标签标题
18 };
19
20
21 //mainwindow.cpp
22 MainWindow::MainWindow(QWidget *parent)
23     : QMainWindow(parent)
24     , ui(new Ui::MainWindow)
25 {
26     ui->setupUi(this);
27
28     // 点击了标签上的关闭按钮
29     connect(ui->tabwidget, &QTabWidget::tabCloseRequested, this, [=](int index)
30     {
31         // 保存信息
32         QWidget* wg = ui->tabwidget->widget(index);
33         QString title = ui->tabwidget->tabText(index);
34         m_widgets.enqueue(wg);
35         m_names.enqueue(title);
36         // 移除tab页
37         ui->tabwidget->removeTab(index);
38         ui->addBtn->setEnabled(true);
39
40     });
41
42     // 当标签被点击了之后的处理动作
43     connect(ui->tabwidget, &QTabWidget::tabBarClicked, this, [=](int index)
44     {
45         qDebug() << "我被点击了一下，我的标题是：" << ui->tabwidget->tabText(index);
46     });
47
48     // 切换标签之后的处理动作
49     connect(ui->tabwidget, &QTabWidget::currentChanged, this, [=](int index)
50     {
51         qDebug() << "当前显示的tab页，我的标题是：" << ui->tabwidget->tabText(index);
52     });
53

```

```

54 // 点击添加标签按钮点击之后的处理动作
55 connect(ui->addBtn, &QPushButton::clicked, this, [=]()
56 {
57     // 将被删除的标签页添加到窗口中
58     // 1. 知道窗口对象，窗口的标题
59     // 2. 知道添加函数
60     ui->tabWidget->addTab(m_widgets.dequeue(), m_names.dequeue());
61     if(m_widgets.empty())
62     {
63         ui->addBtn->setDisabled(true);
64     }
65 });
66 }
67
68

```

Fence 64

7.7 Stacked Widget

- QStackedWidget 栈类型窗口, 在这种类型的窗口中可以存储多个子窗口, 但是只有其中某一个可以被显示出来, 至于是哪个子窗口被显示, 需要在程序中进行控制, 在这种类型的窗口中没有直接切换子窗口的按钮或者标签。
- 相关API

```

1 // 构造函数
2 QStackedWidget::QStackedWidget(QWidget *parent = Q_NULLPTR);
3
4 // 公共成员函数
5 // 在栈窗口中后边添加一个子窗口，返回这个子窗口在栈窗口中的索引值(从0开始计数)
6 int QStackedWidget::addWidget(QWidget *widget);
7 // 将子窗口widget插入到栈窗口的index位置
8 int QStackedWidget::insertWidget(int index, QWidget *widget);
9 // 将子窗口widget从栈窗口中删除
10 void QStackedWidget::removeWidget(QWidget *widget);
11
12 // 返回栈容器窗口中存储的子窗口的个数
13 int QStackedWidget::count() const;
14 // 得到当前栈窗口中显示的子窗口的索引
15 int QStackedWidget::currentIndex() const;
16 // 得到当前栈窗口中显示的子窗口的指针(窗口地址)
17 QWidget *QStackedWidget::currentWidget() const;
18 // 基于索引index得到栈窗口中对应的子窗口的指针
19 QWidget *QStackedWidget::widget(int index) const;
20 // 基于子窗口的指针(实例地址)得到其在栈窗口中的索引
21 int QStackedWidget::indexOf(QWidget *widget) const;
22
23 // 信号
24 // 切换栈窗口中显示子窗口，该信息被发射出来，index为新的当前窗口对应的索引值
25 [signal] void QStackedWidget::currentChanged(int index);
26 // 当栈窗口的子窗口被删除，该信号被发射出来，index为被删除的窗口对应的索引值
27 [signal] void QStackedWidget::widgetRemoved(int index);

```

```

28
29 // 槽函数
30 // 基于子窗口的index索引指定当前栈窗口中显示哪一个子窗口
31 [slot] void QStackedWidget::setCurrentIndex(int index);
32 [slot] void QStackedWidget::setCurrentWidget(QWidget *widget);
33
34 //在这些函数中最常用的就是它的槽函数，并且名字和 QToolBox, QTabWidget 两个类提供的槽函数名字
    相同 分别为 setCurrentIndex(int), setCurrentWidget(QWidget*) 用来设置当前显示的窗口。
35
36

```

Fence 65

- 属性

▼ QStackedWidget		
currentIndex	0	当前显示的子窗口对应的索引
currentPageName	page_3	当前显示的窗口对应的objectName

Figure 37

- 例子

这里主要给大家演示一下QStackedWidget类型的容器中的子窗口如何切换, 如下图所示, 我们在一个栈窗口容器中添加了两个子窗口, 通过两个按钮对这两个窗口进行切换

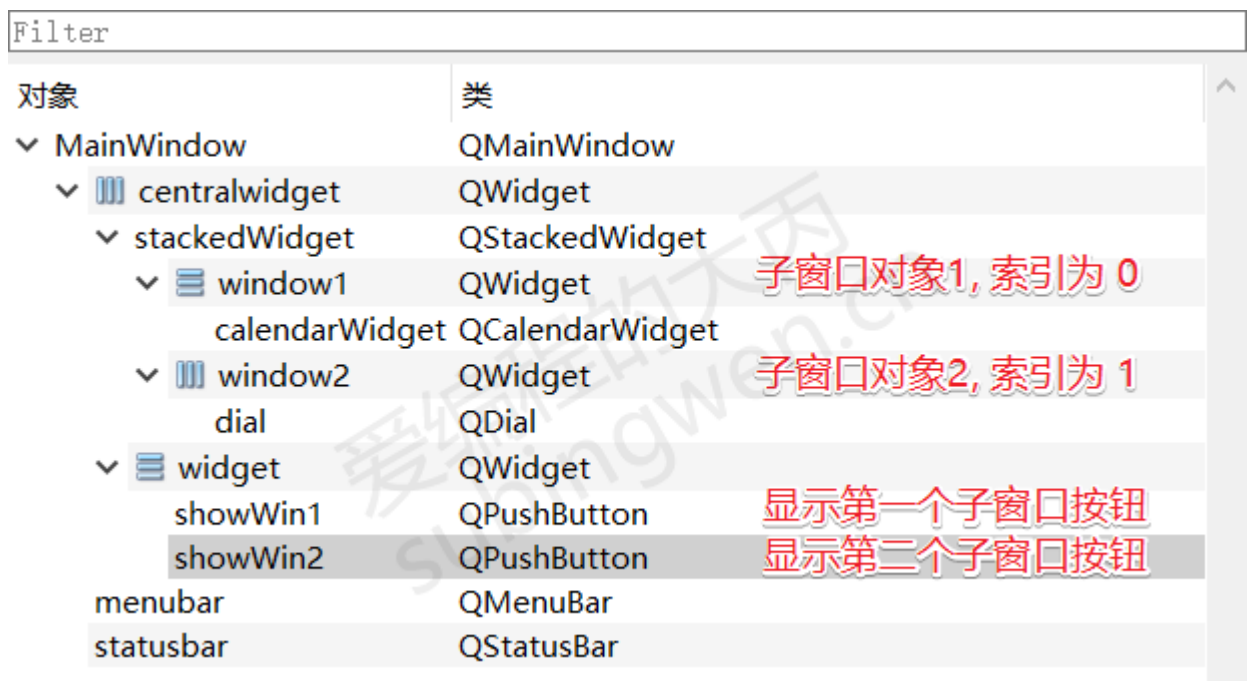


Figure 38

```

1 MainWindow::MainWindow(QWidget *parent)
2     : QMainWindow(parent)
3     , ui(new Ui::MainWindow)
4 {
5     ui->setupUi(this);
6     // 设置默认显示的窗口
7     ui->stackedWidget->setCurrentWidget(ui->window1);
8

```

```

9      connect(ui->showwin1, &QPushButton::clicked, this, [=]()
10      {
11          // 切换显示第一个子窗口
12          ui->stackedwidget->setCurrentIndex(0);
13      });
14
15      connect(ui->showwin2, &QPushButton::clicked, this, [=]()
16      {
17          // 切换显示第二个子窗口，调用这两个槽函数中的任何一个都可以
18          ui->stackedwidget->setCurrentWidget(ui->window2);
19      });
20  }
21

```

Fence 66

8. 对象树

- 在 Qt 中创建对象的时候会传递一个 Parent 对象指针，下面来解释这个 parent 到底是子什么的。
 - QObject 是以对象树的形式组织起来的。
 - 当前创建一个 QObject 对象时，会看到 QObject 的构造函数接收一个 QObject 指针作为参数，这个参数就是 parent，也就是父对象指针。
 - 这样当于，在创建 QObject 对象时，可以提供一个其父对象。我们创建的这个 QObject 对象会自动添加到其父对象的 children() 列表。
 - 当父对象析构的时候，这个列表中的所有对象也会被析构。（注意，**这里的父对象并不是继承意义上的父类！**）
 - 这种机制在 GUI 程序设计中相当有用。例如，一个按钮有一个 QShortcut（快捷键）对象作为其子对象。当我们删除按钮的时候，这个快捷键是应该删除。这是合理的。
 - 析构时，先调用自己（最大的父对象）的析构函数，再看有无子对象，有调用子对象的析构函数，子对象再检查自己有无子对象；无释放自己。（内存释放顺序为孙子对象-》子对象-》父对象。析构调用顺序为父对象-》子对象-》孙子对象）。
 - 对象放入对象树中，自动释放内存，——》一定程度简化了内存回收机制。

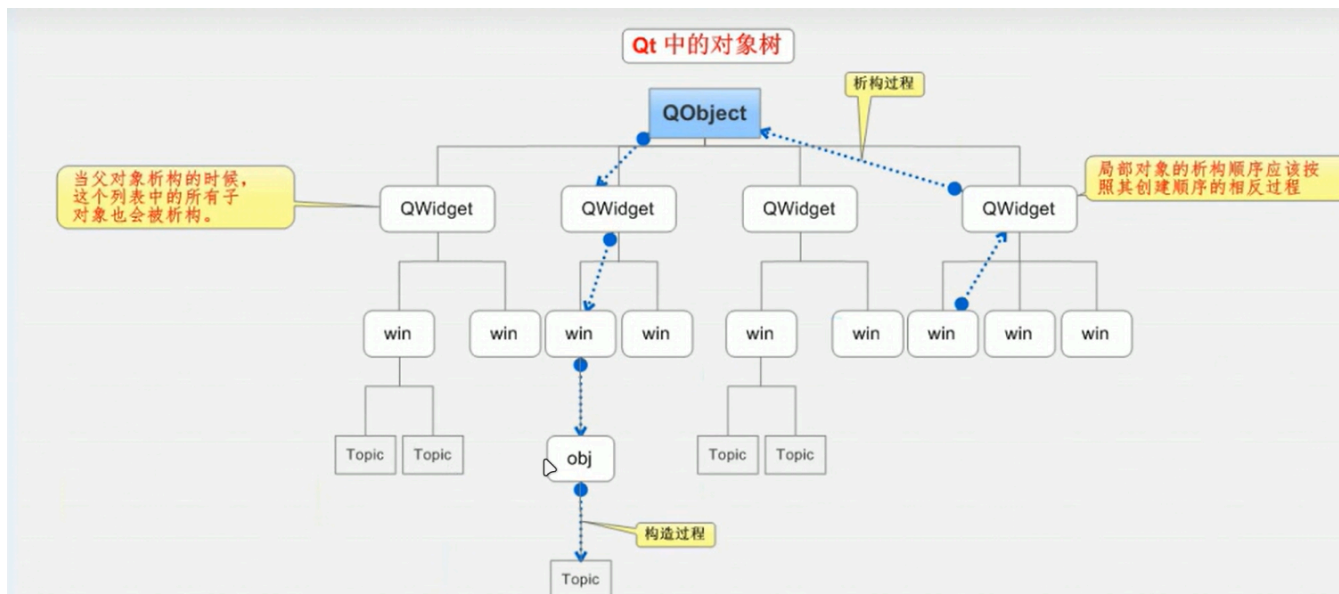


Figure 39

- QWidget 是能够定屏幕上显示的一切图形化的父类。
 - QWidget 继承自 QObject，因此也继承了这种对象关系链。一个子对象自动地成为父组件的一个子组件。因此，它会显示在父组件的坐标系之中，覆盖父组件的起示对象。例如，当用户关闭这个窗体对象，应用程序应该关闭那个控件，那么，我们现在只需要关闭一个父对象，这样这个控件就会被删除了。因为这些对象是及其相关的。

8.1 示例

```

1 //mypushbutton.h
2 #ifndef MYPUSHBUTTON_H
3 #define MYPUSHBUTTON_H
4
5 #include <QPushButton>
6 #include <QDebug> //控制台打印
7
8 class MyPushBotton : public QPushButton
9 {
10 public:
11     explicit MyPushBotton(QWidget *parent = nullptr); //explicit 声明构造
12     函数
13     ~MyPushBotton();
14
15 signals:
16 };
17
18 #endif // MYPUSHBUTTON_H
19
20
21 //mypushbutton.cpp
22 #include "mypushbutton.h"
23

```

```

24  MyPushBotton::MyPushBotton(QWidget *parent)
25      : QPushButton{parent}                                //QPushButton{parent} 传入父类进行
构造
26  {
27
28      qDebug() << "MyPushBotton的构造函数";
29  }
30
31  MyPushBotton::~MyPushBotton()
32  {
33      qDebug() << "MyPushBotton的析构函数";
34  }
35  }
36
37
38  //mypushbutton.cpp
39  #include "mywidget.h"
40  #include "mypushbutton.h"
41  #include <QDebug>
42
43
44  MyWidget::MyWidget(QWidget *parent)
45      : QWidget(parent)                                    //使用初始化列表，初始化Mywidge类
46  {
47      /* 验证对象树的构造和析构过程 */
48      MyPushBotton *myBtn = new MyPushBotton();
49      myBtn->setParent(this);                               //绑定父对象为Mywidget
50
51      myBtn->setText("自己的按钮");
52
53  }
54
55  MyWidget::~MyWidget() {
56      qDebug() << "Mywidget的析构函数";
57  }
58

```

8.2 qt 窗口坐标系

坐标体系：

以左上角为原点 $(0,0)$ ，X 向右增加，Y 向下增加。

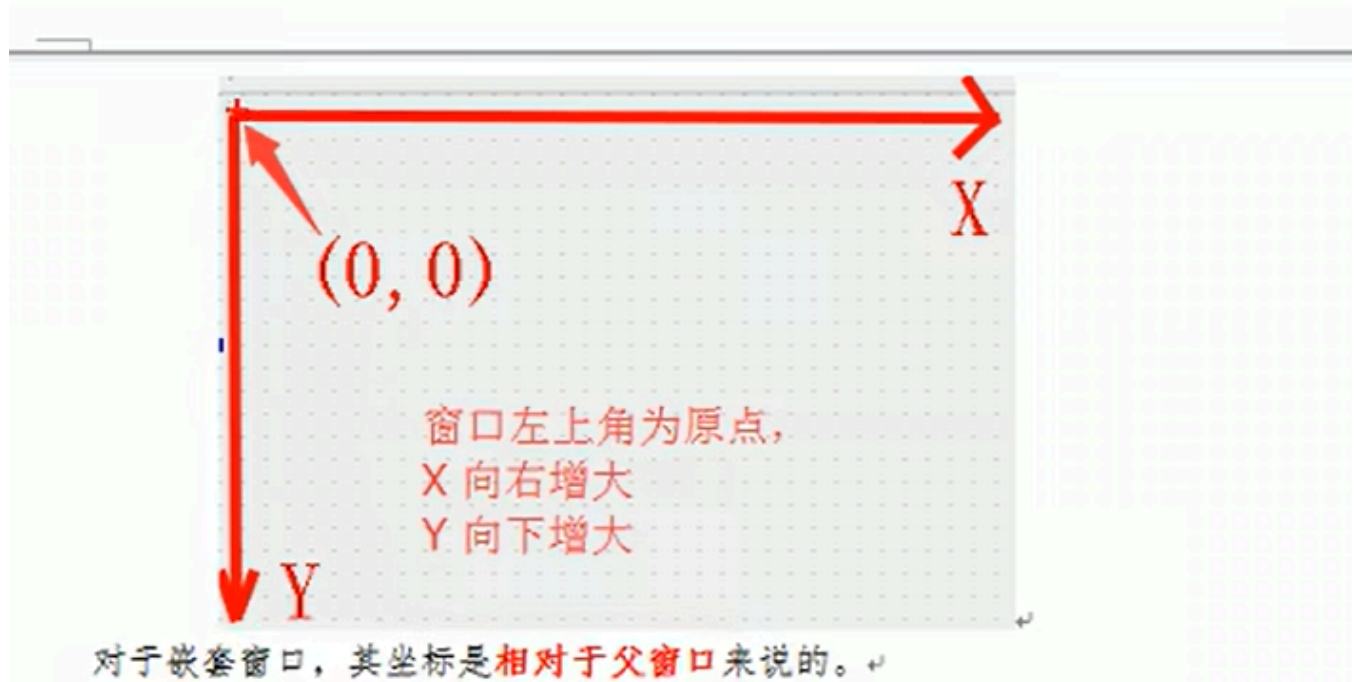


Figure 40

9. 信号和槽

9.1 基本概念

- 本质：信号-》事件 槽-》处理事件的方法或者函数 二者通过一定关系连接
- 优点：松散耦合。（信号发送方 与 信号处理方 本身没有联系 通过 connect 连接在一起）
- 使用：
 - `connect(信号发送方, 发送的信号（给函数的地址）, 信号接受方, 处理的槽函数（给函数地址）);`
 - eg. `connect(myBtn,&MyPushButton:: clicked, this,&MyWidget:: close);` // MyPushButton 可以换成继承的父类

9.2 技巧

- 使用 Qt 助手查找时，注意 Signals（信号）和 Public Slots（槽），如果没有找一下最后的函数，看一下继承与什么父类，在通过父类查找。

Public Slots

```
void animateClick()
void click()
void setChecked(bool)
void setIconSize(const QSize &size)
void toggle()
```

Signals

```
void clicked(bool checked = false)
void pressed()
void released()
void toggled(bool checked)
```

Figure 41

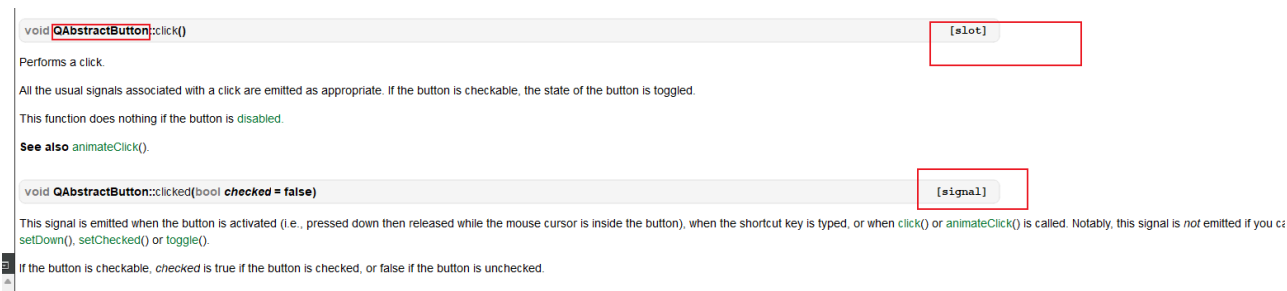


Figure 42

9.3 自定义信号和槽

- 自定义信号
 - 需要放到类中的 signals 下面
 - 无返回值 需要声明 不需要实现
 - 可有参数 可重载
- 自定义槽
 - 需要放到类中的 public 或者 public slots（低版本必须放在 public slots）下 可以私有 private slots
 - 无返回值 需要声明 需要实现
 - 可有参数 可重载
- 自定义触发
 - emit 关键字 调用信号 一般封装到函数中
 - 信号连接信号 触发信号

示例:

```
1  //student.h
2  #ifndef STUDENT_H
3  #define STUDENT_H
4
5  #include <QObject>
6
7  class Student : public QObject
8  {
9      Q_OBJECT
10 public:
11     explicit Student(QObject *parent = nullptr);
12
13     /* 槽函数 */
14     //有参 无返回 需实现 可重载
15     void dining();
16
17
18 signals:
19 };
20
21 #endif // STUDENT_H
22
23
24 //student.cpp
25 #include "student.h"
26 #include <QDebug>
27
28 Student::Student(QObject *parent)
29     : QObject{parent}
30 {}
31
32 void Student::dining(){
33
34     qDebug() << "冲出教室 吃饭! ";
35 }
36
37
38 //teacher.h
39 #ifndef TEACHER_H
40 #define TEACHER_H
41
42 #include <QObject>
43
44 class Teacher : public QObject
45 {
46     Q_OBJECT
47 public:
48     explicit Teacher(QObject *parent = nullptr);
49
50 signals:
51     // 有参无返回 不需要实现 可重载
```

```

52     void classIsOver();
53
54 };
55
56 #endif // TEACHER_H
57
58
59 //teacher.cpp
60 #include "teacher.h"
61
62 Teacher::Teacher(QObject *parent)
63     : QObject{parent}
64 {}
65
66
67 //widget.h
68 #ifndef WIDGET_H
69 #define WIDGET_H
70
71 #include <QWidget>
72 #include "student.h"
73 #include "teacher.h"
74
75 class widget : public QWidget
76 {
77     Q_OBJECT
78
79 public:
80     widget(QWidget *parent = nullptr);
81     ~widget();
82
83 private:
84     void By12();
85
86     Student *m_st;
87     Teacher *m_th;
88 };
89 #endif // WIDGET_H
90
91
92 //widget.cpp
93 #include "widget.h"
94
95
96 //12:00(触发条件)    老师 发出 下课信号        学生 冲向食堂
97
98 widget::widget(QWidget *parent)
99     : QWidget(parent)
100 {
101     //创建对象 同时将父对象告知
102     this->m_th = new Teacher(this);
103     this->m_st = new Student(this);
104

```

```

105     connect(this->m_th,&Teacher::classIsOver,this->m_st,&Student::dining);
106
107     this->By12();
108     //emit this->m_th->classIsOver();
109 }
110
111 void widget::By12()
112 {
113     //触发信号
114     emit this->m_th->classIsOver();
115
116 }
117
118 widget::~~widget() {}
119

```

Fence 68

注：先连接，在触发，才能生效

9.4 自定义信号和槽重载问题解决

- 连接时，使用函数指针 void(指针名) (参数列表) = 函数地址

示例：

```

1  //student.h
2  #ifndef STUDENT_H
3  #define STUDENT_H
4
5  #include <QObject>
6
7  class Student : public QObject
8  {
9      Q_OBJECT
10 public:
11     explicit Student(QObject *parent = nullptr);
12
13     /* 槽函数 */
14     //有参 无返回 需实现 可重载
15     void dining();
16     //重载
17     void dining(QString food);
18
19
20 signals:
21 };
22
23 #endif // STUDENT_H

```

```

24
25
26 //student.cpp
27 #include "student.h"
28 #include <QDebug>
29
30 Student::Student(QObject *parent)
31     : QObject{parent}
32 {}
33
34 void Student::dining(){
35
36     qDebug() << "冲出教室 吃饭! ";
37 }
38 void Student::dining(QString food)
39 {
40
41     //QString类型打印会出现""
42     //解决"" 先转换成QByteArray 在使用QByteArray的成员函数data()转换成char *
43     qDebug() << "冲出教室 吃饭! 吃:" << food.toUtf8().data();
44     //先转换成std::string 在换成 c_str
45     qDebug() << "冲出教室 吃饭! 吃:" << food.toStdString().c_str();
46
47 }
48
49 //teacher.h
50 #ifndef TEACHER_H
51 #define TEACHER_H
52
53 #include <QObject>
54
55 class Teacher : public QObject
56 {
57     Q_OBJECT
58 public:
59     explicit Teacher(QObject *parent = nullptr);
60
61 signals:
62     // 有参无返回 不需要实现 可重载
63     void classIsOver();
64     //重载
65     void classIsOver(QString food);
66 };
67
68 #endif // TEACHER_H
69
70
71 //teacher.cpp
72 #include "teacher.h"
73
74 Teacher::Teacher(QObject *parent)
75     : QObject{parent}
76 {}

```

```

77
78
79 //widget.h
80 #ifndef WIDGET_H
81 #define WIDGET_H
82
83 #include <QWidget>
84 #include "student.h"
85 #include "teacher.h"
86
87 class Widget : public QWidget
88 {
89     Q_OBJECT
90
91 public:
92     Widget(QWidget *parent = nullptr);
93     ~Widget();
94
95 private:
96     void By12();
97
98     Student *m_st;
99     Teacher *m_th;
100 };
101 #endif // WIDGET_H
102
103
104 //widget.cpp
105 #include "widget.h"
106
107
108 //12:00(触发条件) 老师 发出 下课信号 学生 冲向食堂
109
110 Widget::Widget(QWidget *parent)
111     : QWidget(parent)
112 {
113     //创建对象 同时将父对象告知
114     this->m_th = new Teacher(this);
115     this->m_st = new Student(this);
116
117     //出现重载版本后 编译器不知道用哪一个信号和槽
118     //使用函数指针 void(指针名) (参数列表) = 函数地址
119     void(Teacher:: *teacher)(QString) = &Teacher::classIsOver;
120     void(Student:: *student)(QString) = &Student::dining;
121     connect(this->m_th, teacher, this->m_st, student);
122
123     this->By12();
124 }
125
126 void Widget::By12()
127 {
128     //触发信号
129     emit this->m_th->classIsOver();

```

```

130
131     }
132
133     widget::~~widget() {}
134

```

Fence 69

9.5 拓展

- 信号链（信号连接信号）
connect(发送方 1, 发送信号 1, 发送方 2, 发送信号 2);
connect(发送方 2, 发送信号 2, 接受方, 槽函数);
- 断开信号
disconnect(发送方, 发送信号, 接受方, 槽函数); //参数和 connect 一样
- 一个信号 连接 多个槽
- 多个信号 连接 同一个槽
- 信号和槽的参数 类型必须一一对应
- 信号和槽的参数 数量可以不对应 但只能信号的参数个数 **多于** 槽的参数个数（且信号参数列表中存在槽的所有参数类型）
 - clicked () 参数列表只有 bool 所以不能直接连接一个及以上参数的槽函数 可以通过自定义函数进行间接连接 或者 Lambda 表达式
- Qt4 版本之前的连接方式
connect(发送方, SIGNAL(信号函数()), 接受方, SLOT(槽函数())); //缺点：不会做类型检测，会在运行后才能报错 **不推荐**

示例：

```

1 //同上
2 widget::widget(QWidget *parent)
3     : QWidget(parent)
4 {
5     //创建对象 同时将父对象告知
6     this->m_th = new Teacher(this);
7     this->m_st = new Student(this);
8
9     //出现重载版本后 编译器不知道用哪一个信号和槽
10    //使用函数指针 void(指针名) (参数列表) = 函数地址
11    void(Teacher:: *teacher)(QString) = &Teacher::classIsOver;
12    void(Student:: *student)(QString) = &Student::dining;
13    connect(this->m_th, teacher, this->m_st, student);
14
15    this->By12();
16    //emit this->m_th->classIsOver();
17

```

```

18 //信号连接信号
19 QPushButton *btn = new QPushButton("12: 00",this);
20
21 void(Teacher:: *teacher1)(void) = &Teacher::classIsOver;
22 void(Student:: *student1)(void) = &Student::dining;
23 connect(this->m_th,teacher1,this->m_st,student1);
24 connect(btn,&QPushButton::clicked,this->m_th,teacher1);
25
26 //断开信号
27 disconnect(btn,&QPushButton::clicked,this->m_th,teacher1);
28 }
29

```

Fence 70

9.6 Lambda 表达式（用于一个信号连接多个槽函数 或者 多个信号连接一个槽函数）

- 匿名函数 或者 闭包
- 用于定义并创建匿名的函数对象
- 低版本使用需要在 .pro 文件中 加 CONFIG += c++11
- [函数对象参数]（操作符重载函数参数）mutable -> 返回值类型{函数体}
 - 后面变成 最后加上()变为调用
 - 函数对象参数（捕获参数）形式：
 - 空：没有使用任何函数对象参数。
 - [=]：函数体内可以使用 Lambda 所在作用范围内所有可见的局部变量（包括 Lambda 所在类的 this），并且是 **值传递方式**（相当于编译器隐式为我们按值传递了所有局部变量）。
 - [&]：函数体内可以使用 Lambda 所在作用范围内所有可见的局部变量（包括 Lambda 所在类的 this），并且是 **引用传递方式**（相当于编译器隐式为我们按引用传递了所有局部变量）。
 - [this]：函数体内可以使用 Lambda 所在类中的成员变量。引用捕获
 - [*this]：值捕获
 - [a]：将 a 按值传递传递。按值传递传递，函数体内不能修改传递进来的 a 的拷贝，可以标记该传参 const 的。**要修改传递进来的 a 的拷贝，可以添加 mutable 修饰符。**
 - [&a]：将 a 按引用传递。
 - [a, &b]。将 a 按值进行传递，b 按引用进行传递。
 - [=, &a, &b]。除 a 和 b 按引用进行传递外，其他参数都按值进行传递。
 - [&, a, b]。除 a 和 b 按值进行传递外，其他参数都按引用进行传递。
 - 操作符重载函数参数：标识重载的操作符的参数，没有参数时，这部分可以省略。参数可以通过按值（如：(a, b)）和按引用（如：(&a, &b)）两种方式进行传递。
 - 可修改标示符：mutable 关键词，这部分可以省略。按值传递函数对象参数时，加上 mutable 修饰符后，可以修改按值传递进来的拷贝（注意是能修改拷贝，而不是原本的参数值）。

```

1   QPushButton * myBtn = new QPushButton(this);
2   QPushButton * myBtn2 = new QPushButton(this);
3   myBtn2->move(100, 100);
4
5   int m = 10;
6
7   //mutable 修改拷贝值，而不会改变原来的值
8   connect(myBtn, &QPushButton::clicked, this, [m]() mutable { m = 100 + 10;
9       qDebug() << m; });
10
11   connect(myBtn2, &QPushButton::clicked, this, [=] { qDebug() << m; });
12
13   qDebug() << m;

```

- 函数返回值：-> 返回值类型，标识函数返回值的类型。当返回值为 void，或者函数体中只有一处 return 的地方（比如编译器可以自动推断出返回值类型）时，这部分可以省略。如果有返回值，可以在-> 后面写一个返回值类型
- 是函数体：{}，标识函数的实现，这部分不能省略，但函数体可以为空。

```

1   // 匿名函数的定义，程序执行这个匿名函数是不会被调用的
2   [](){
3       qDebug() << "hello, 我是一个lambda表达式...";
4   };
5
6   // 匿名函数的定义+调用：
7   int ret = [](int a) -> int
8   {
9       return a+1;
10  }(100); // 100是传递给匿名函数的参数
11
12
13
14  //在Lambda表达式的捕获列表也就是 [] 内部添加不同的关键字，就可以在函数体中使用外部变量了。
15  // 在匿名函数外部定义变量
16  int a=100, b=200, c=300;
17  // 调用匿名函数
18  [](){
19      // 打印外部变量的值
20      qDebug() << "a:" << a << ", b: " << b << ", c:" << c; // error, 不能使用任何外部变量
21  }
22
23  [&](){
24      qDebug() << "hello, 我是一个lambda表达式...";
25      qDebug() << "使用引用的方式传递数据: ";
26      qDebug() << "a+1:" << a++ << ", b+c= " << b+c;
27  }();
28
29  // 值拷贝的方式使用外部数据
30  [=](int m, int n)mutable{
31      qDebug() << "hello, 我是一个lambda表达式...";
32      qDebug() << "使用拷贝的方式传递数据: ";

```

```
33 // 拷贝的外部数据在函数体内部是只读的，如果不添加 mutable 关键字是不能修改这些只读数据的值的
34 的
35 // 添加 mutable 允许修改的数据是拷贝到函数内部的副本，对外部数据没有影响
36 qDebug() << "a+1:" << a++ << ", b+c= " << b+c;
37 qDebug() << "m+1: " << ++m << ", n: " << n;
38 }(1, 2);
```

Fence 72

10. 资源文件

10.1 添加 ui 文件

10.1.1 新建时添加 ui 文件

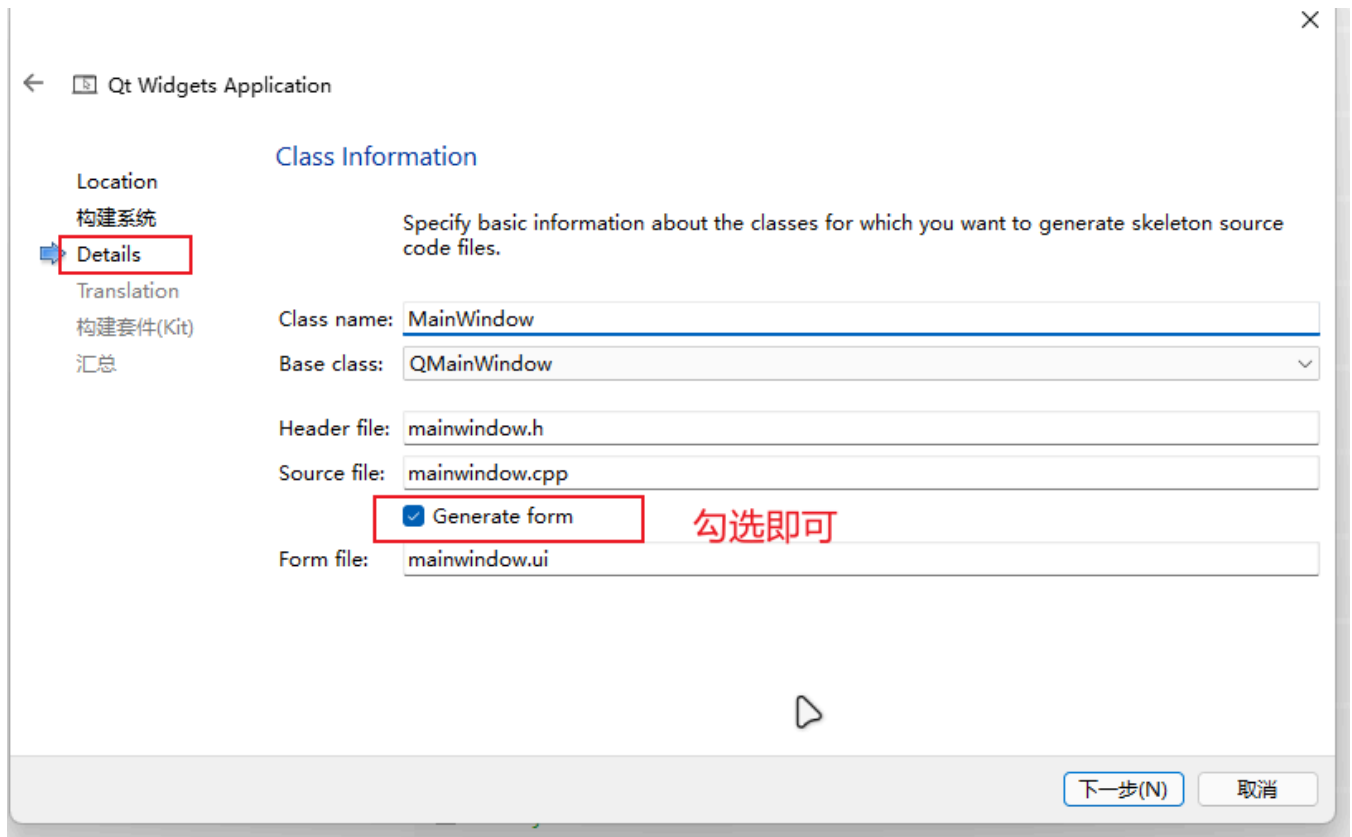


Figure 43

10.1.2 已经创建的文件中添加 ui 文件

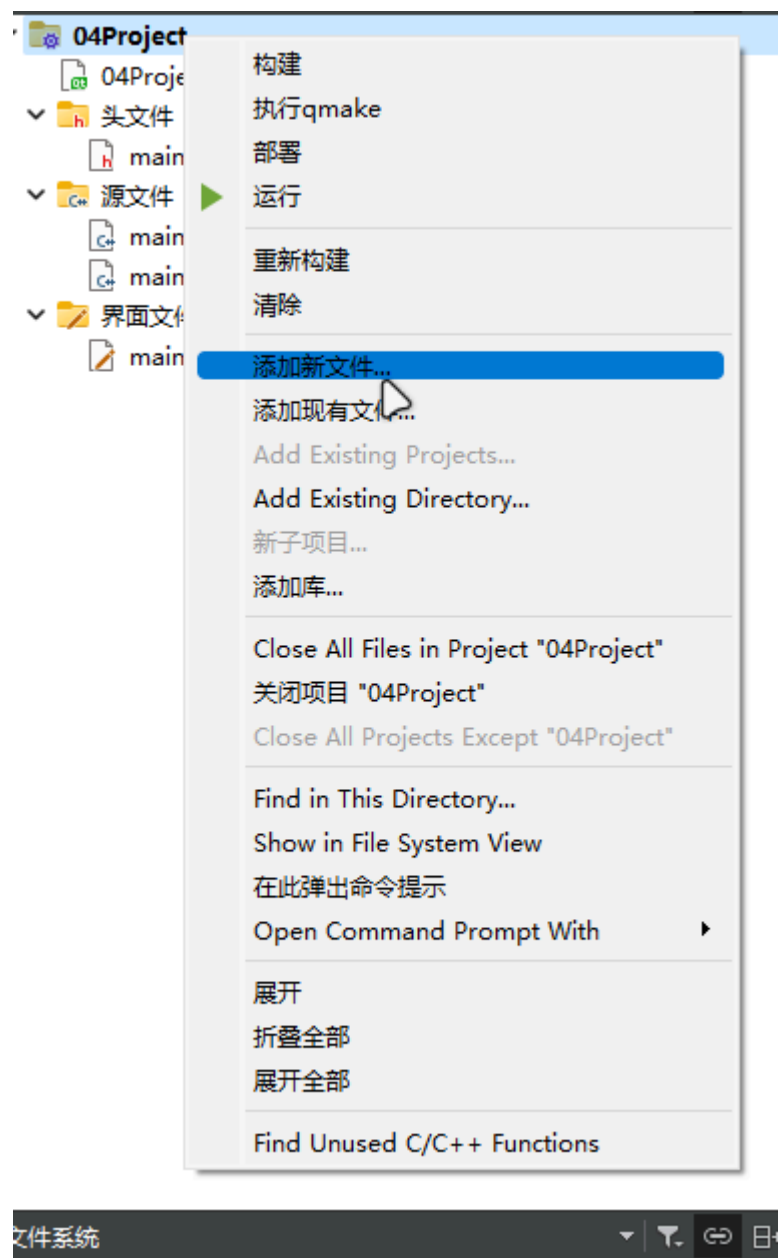


Figure 44

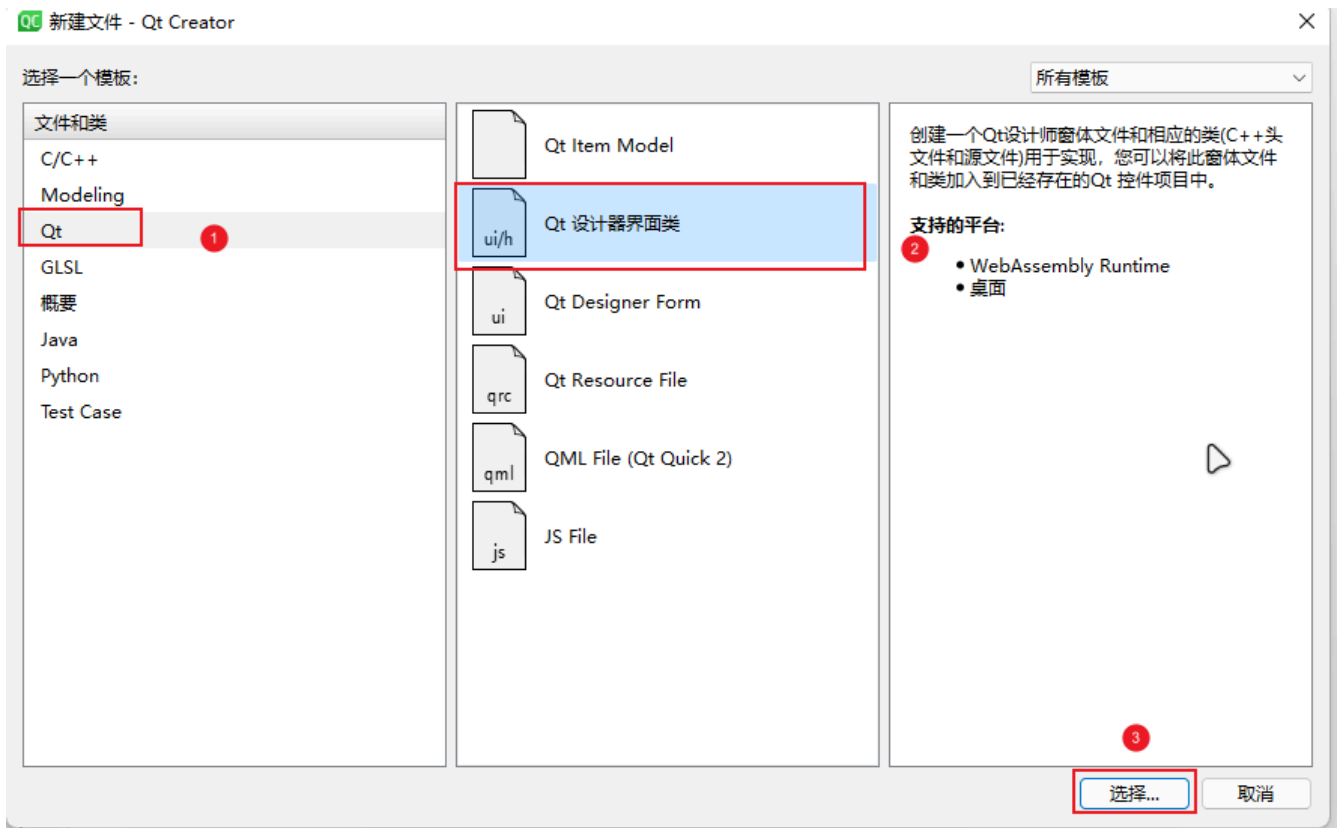


Figure 45

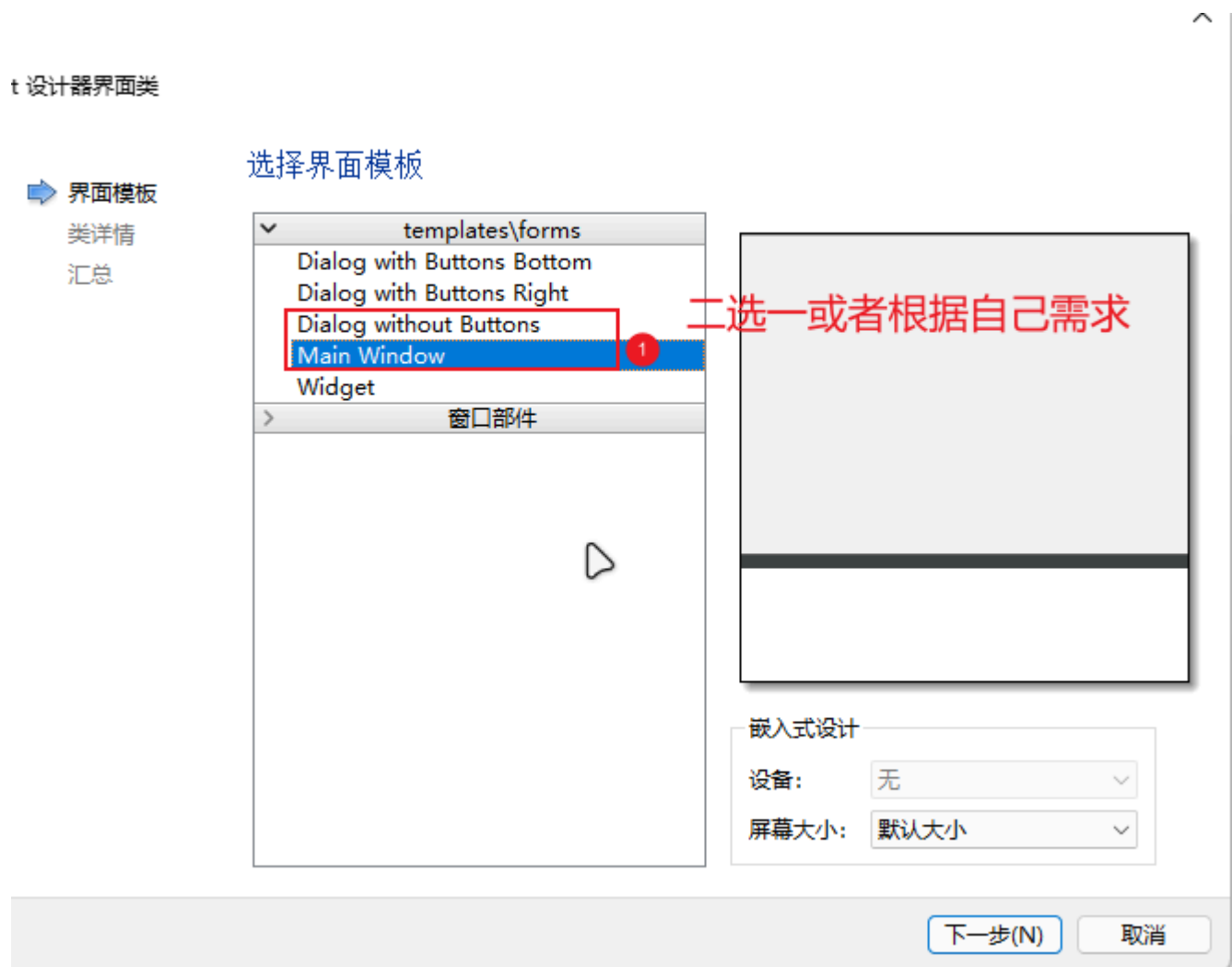


Figure 46

10.2 使用 ui 文件

使用时，注意有些地方需要敲回车确定

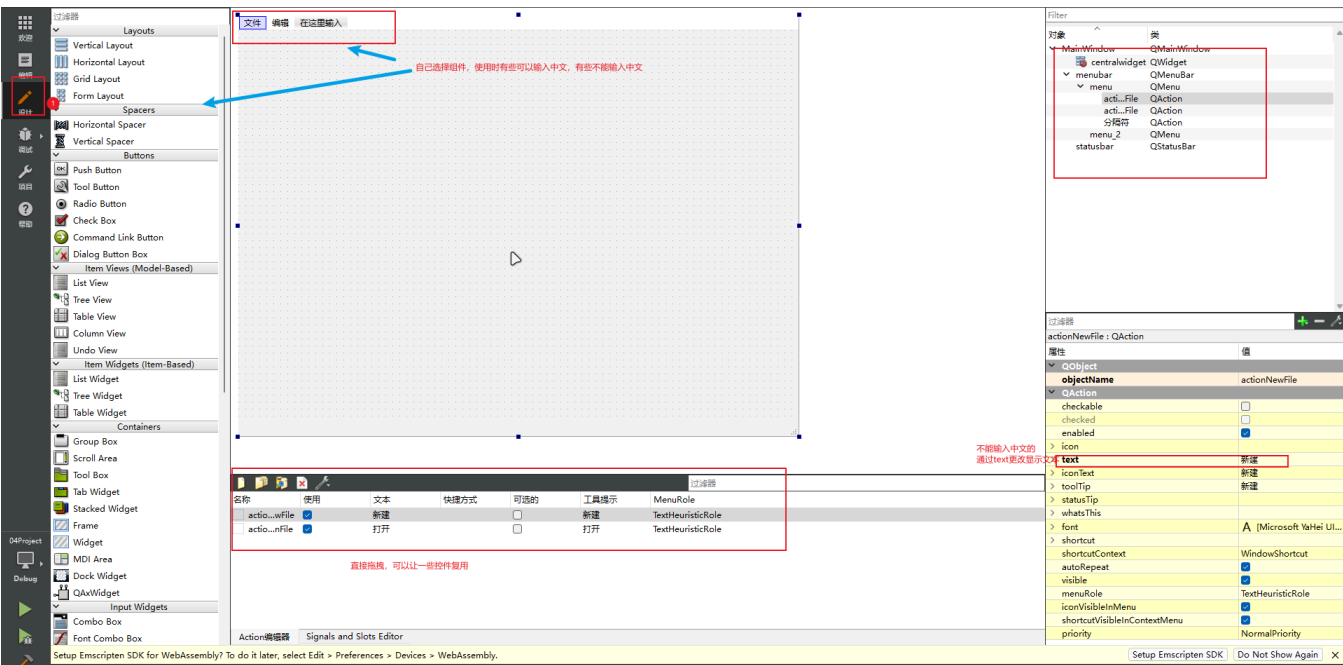


Figure 47

10.3 添加资源文件

10.3.1 创建资源文件

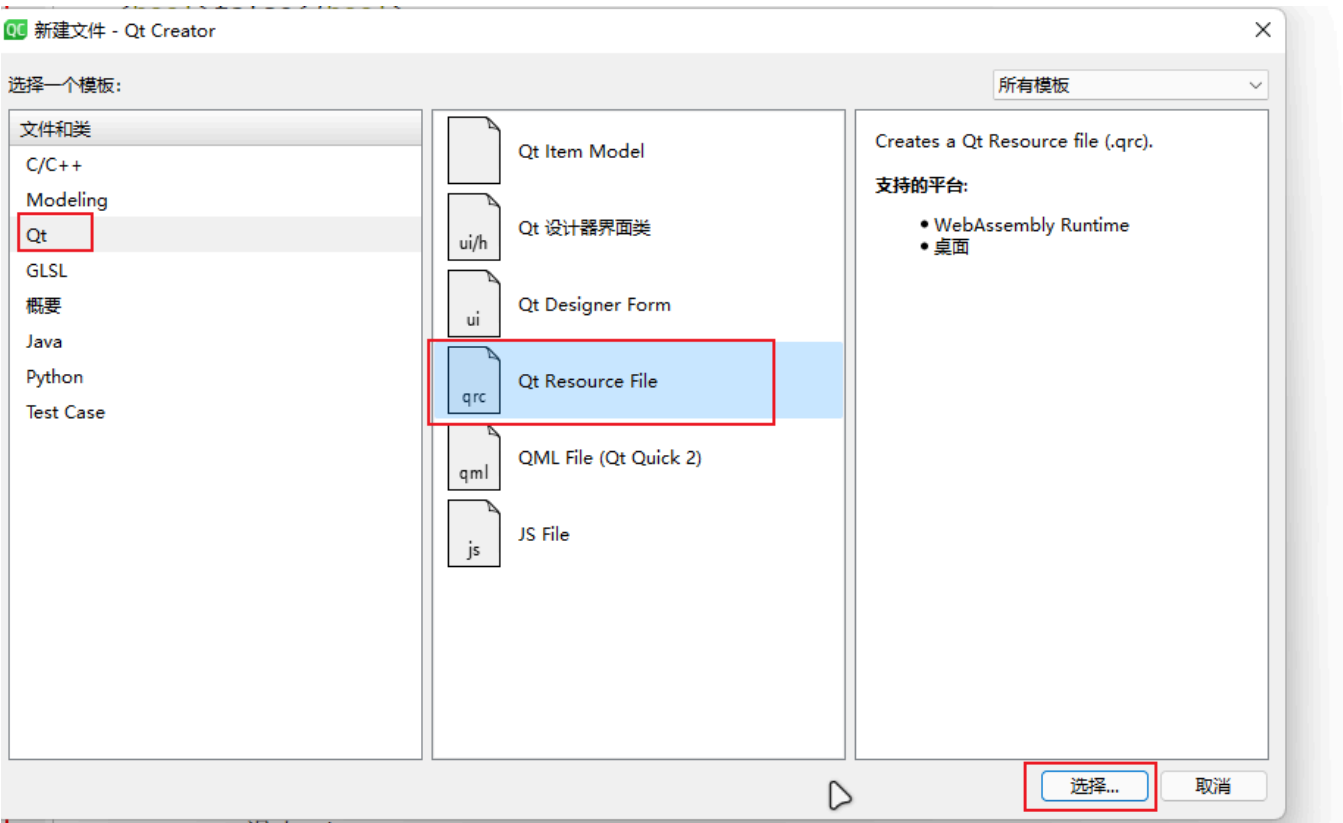


Figure 48

10.3.2 回到资源文件界面

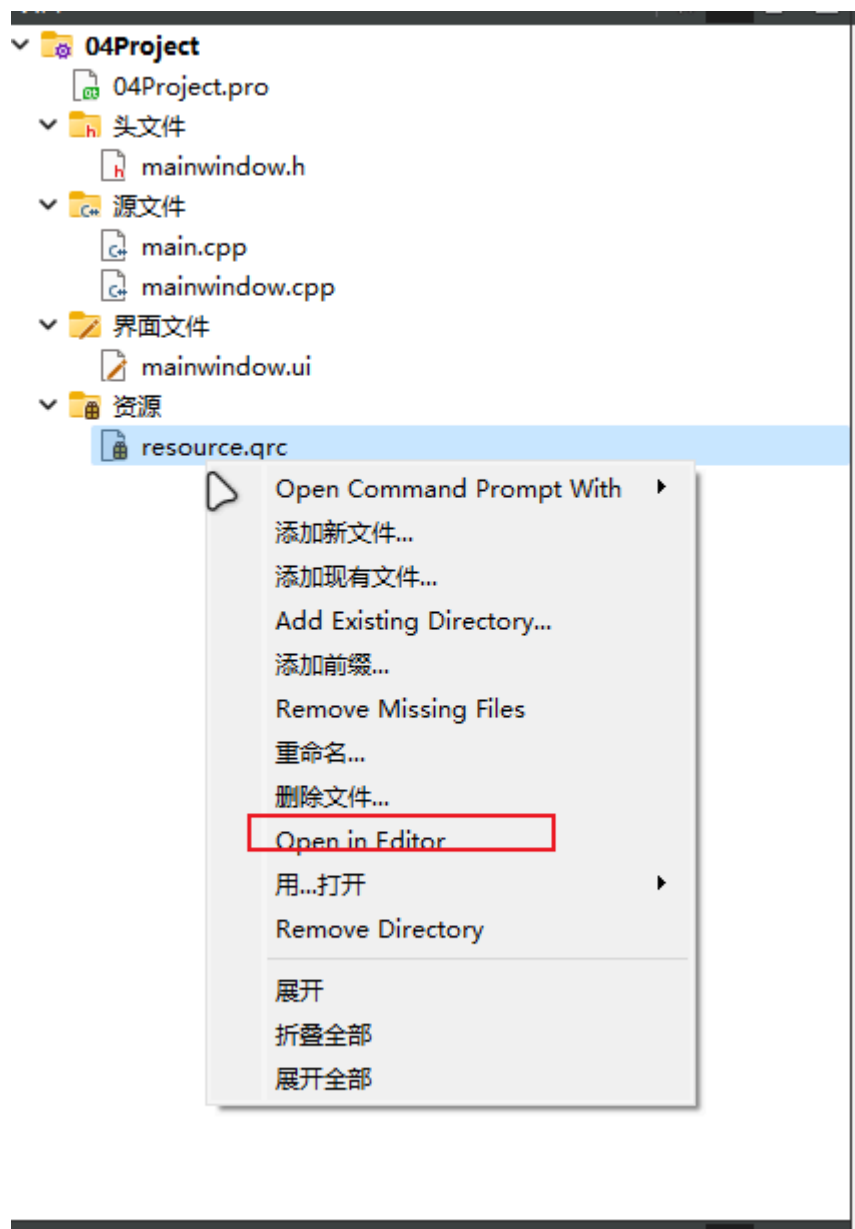


Figure 49

10.3.3 添加资源

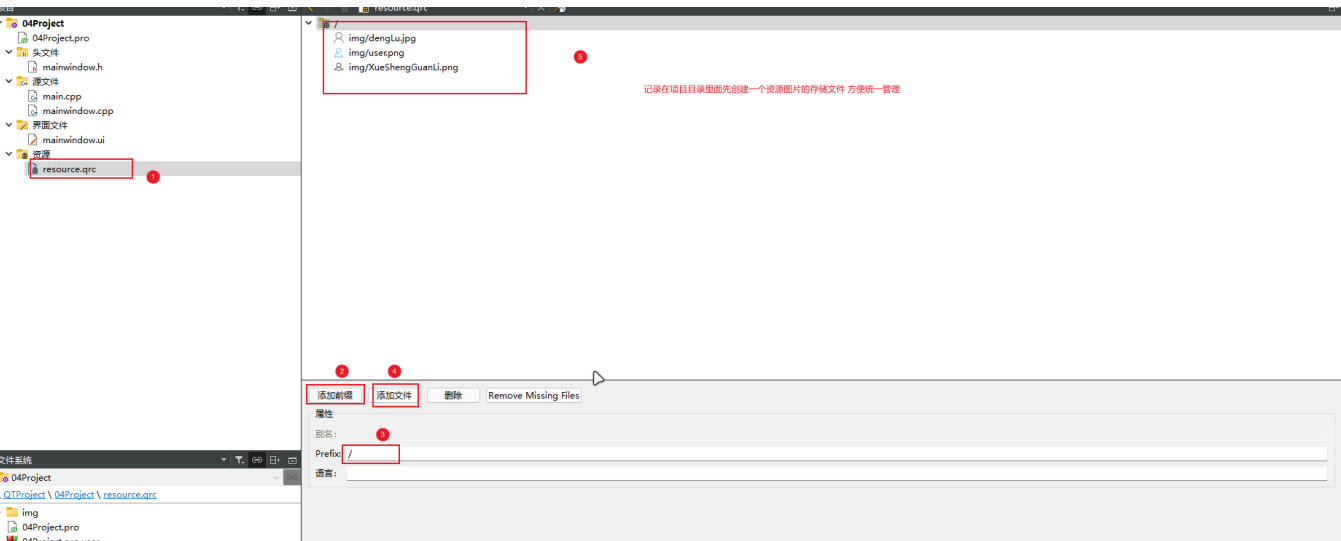


Figure 50

10.3.4 使用

```
1 //使用Qt资源文件 :+前缀名+资源名
2 //使用的对象-> setIcon(QIcon( "+前缀名+资源名"));
3 ui->actionNewFile->setIcon(QIcon(":/img/dengLu.jpg"));
4 ui->actionOpenFile->setIcon(QIcon(":/img/user.png"));
```

Fence 73

11. 界面布局

11.1 布局的样式

布局样式	描述	行数	列数
水平布局	布局中的所有的控件水平排列	1行	N列 (N>=1)
垂直布局	布局中的所有的控件垂直排列	N行(N>=1)	1列
网格（栅格）布局	布局中的所有的控件垂直+水平排列	N行	N列 (N>=1)

Table 6

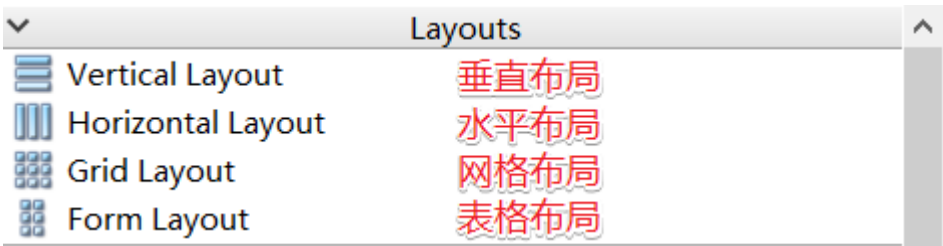


Figure 51

11.2 控件布局

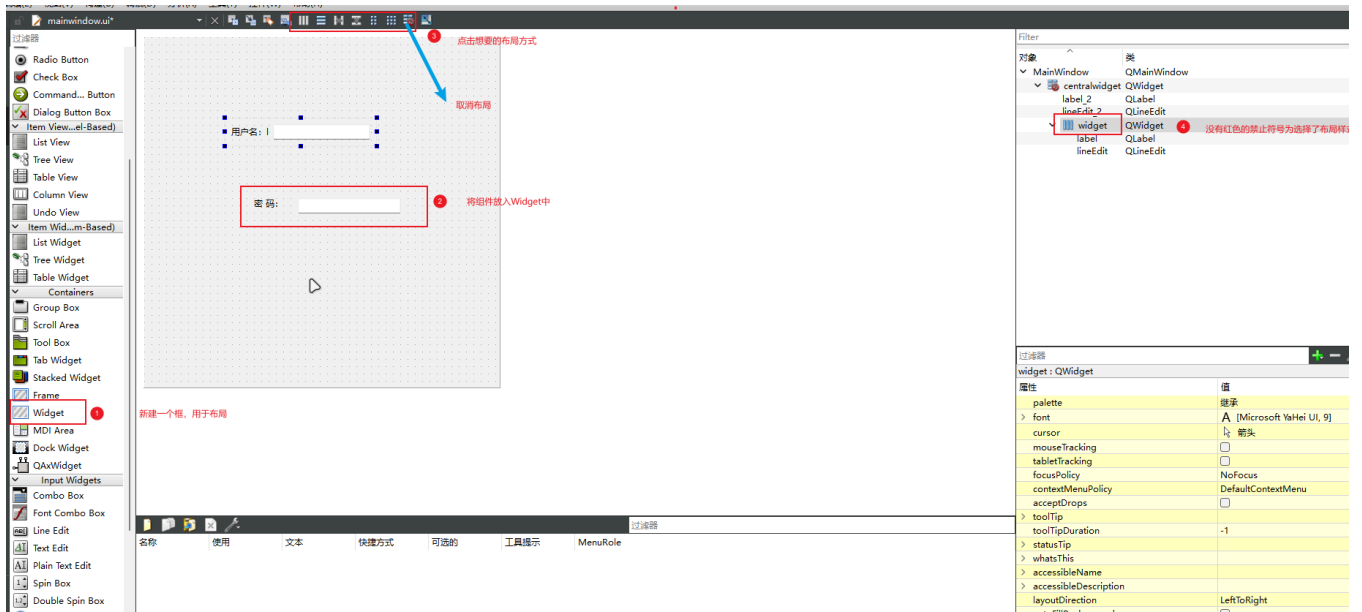


Figure 52

11.3 彈簧

用于自适应控件之间的距离。



Figure 53

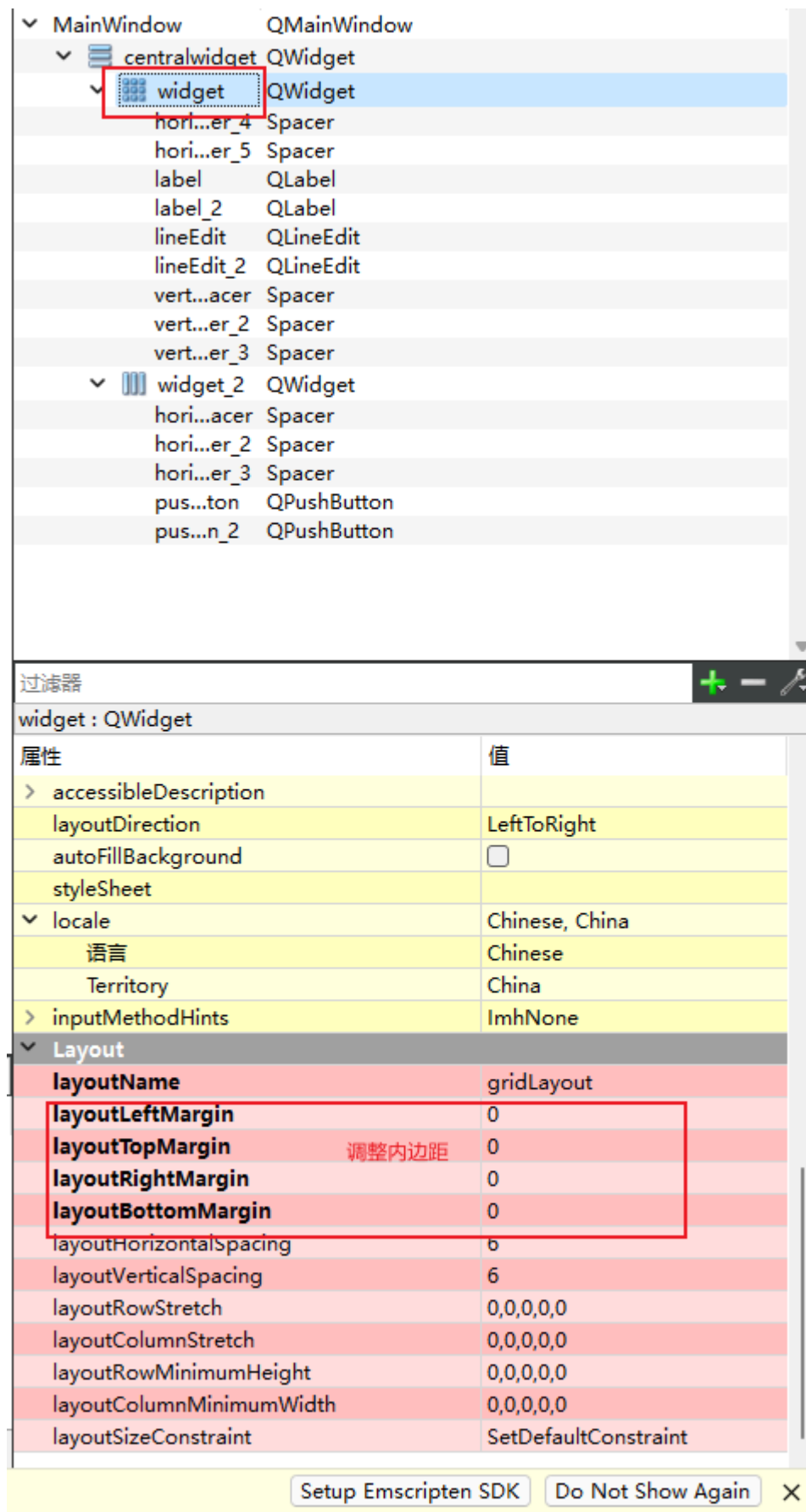


Figure 54

属性	值
> QObject	
> QWidget	
▼ Layout	
layoutName	horizontalLayout 布局的名字
layoutLeftMargin	9 布局左侧预留的边距(留白), 单位像素
layoutTopMargin	9 布局上侧预留的边距(留白), 单位像素
layoutRightMargin	9 布局右侧预留的边距(留白), 单位像素
layoutBottomMargin	9 布局下侧预留的边距(留白), 单位像素
layoutSpacing	6 布局中控件之间的间距大小, 单位像素
layoutStretch	0,0,0,0,0
layoutSizeConstraint	SetDefaultConstraint

Figure 55

label	QLabel
label_2	QLabel
lineEdit	QLineEdit
lineEdit_2	QLineEdit
vert...acer	Spacer
vert...er_2	Spacer
vert...er_3	Spacer
▼ widget_2	QWidget
hori...acer	Spacer
hori...er_2	Spacer
hori...er_3	Spacer
pus...ton	QPushButton
pus...n_2	QPushButton

过滤器

lineEdit_2 : QLineEdit

属性	值
QLineEdit	
inputMask	
text	
可翻译的	☒
澄清	
注释	
maxLength	32767 修改成密码框
frame	☒
echoMode	Password
cursorPosition	0
alignment	左对齐, 垂直中心对齐
水平的	左对齐
垂直的	垂直中心对齐
dragEnabled	☐
readOnly	☐
placeholderText	
可翻译的	☒
澄清	
注释	
cursorMoveStyle	LogicalMoveStyle
clearButtonEnabled	☐

Figure 56

- 弹簧属性

属性	值
▼ Spacer	
spacerName	horizontalSpacer_2 弹簧的名字
orientation	Horizontal 弹簧的方向: 水平 or 垂直
sizeType	Expanding 弹簧的类型
▼ sizeHint	122 x 20 弹簧的尺寸大小: 宽度 and 高度
宽度	122
高度	20

Figure 57

属性	值
▼ Spacer	
spacerName	horizontalSpacer
orientation	Horizontal
sizeType	Expanding
▼ sizeHint	Fixed 宽度 高度
	Fixed Minimum Maximum Preferred MinimumExpanding Expanding Ignored

弹簧大小固定, 没有弹性
 可以设置弹簧最小值, 有弹性
 可以设置弹簧最大值, 有弹性
 弹簧以合适大小显示, 有弹性
 可以设置最小值, 有弹性, 伸展到当前最大
 伸展到当前最大, 有弹性
 限制变少了, 有弹性

Figure 58

11.4 API实现

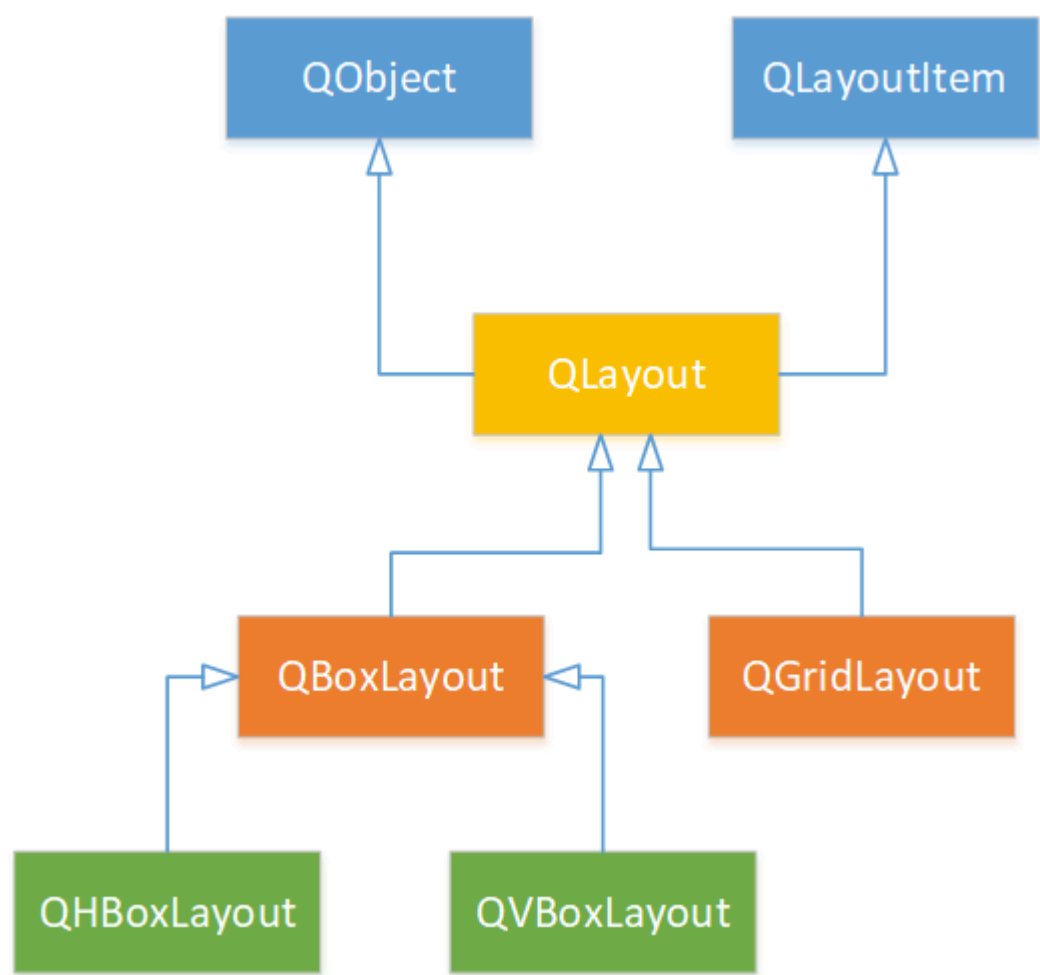


Figure 59

布局样式	类名
水平布局	QHBoxLayout
垂直布局	QVBoxLayout
网格（栅格）布局	QGridLayout

Table 7

11.4.1 QLayout

```
1 // 在布局最后面添加一个窗口
2 void QLayout::addWidget(QWidget *w);
3 // 将某个窗口对象从布局中移除，窗口对象如果不再使用需要自己析构
4 void QLayout::removeWidget(QWidget *widget);
5 // 设置布局的四个边界大小，即：左、上、右和下的边距。
6 void QLayout::setContentsMargins(int left, int top, int right, int bottom);
7 // 设置布局中各个窗口之间的间隙大小
8 void setSpacing(int);
9
```

11.4.2 QHBoxLayout

```
1 // 创建窗口对象
2 QWidget *window = new QWidget;
3 // 创建若干个子窗口对象
4 QPushButton *button1 = new QPushButton("One");
5 QPushButton *button2 = new QPushButton("Two");
6 QPushButton *button3 = new QPushButton("Three");
7 QPushButton *button4 = new QPushButton("Four");
8 QPushButton *button5 = new QPushButton("Five");
9
10 // 创建水平布局对象
11 QHBoxLayout *layout = new QHBoxLayout;
12 // 将子窗口添加到布局中
13 layout->addWidget(button1);
14 layout->addWidget(button2);
15 layout->addWidget(button3);
16 layout->addWidget(button4);
17 layout->addWidget(button5);
18
19 // 将水平布局设置给父窗口对象
20 window->setLayout(layout);
21 // 显示父窗口
22 window->show();
23
24
```

Fence 75

11.4.3 QVBoxLayout

```
1 // 创建窗口对象
2 QWidget *window = new QWidget;
3 // 创建若干个子窗口对象
4 QPushButton *button1 = new QPushButton("One");
5 QPushButton *button2 = new QPushButton("Two");
6 QPushButton *button3 = new QPushButton("Three");
7 QPushButton *button4 = new QPushButton("Four");
8 QPushButton *button5 = new QPushButton("Five");
9
10 // 创建垂直布局对象
11 QVBoxLayout *layout = new QVBoxLayout;
12 // 将子窗口添加到布局中
13 layout->addWidget(button1);
14 layout->addWidget(button2);
15 layout->addWidget(button3);
16 layout->addWidget(button4);
17 layout->addWidget(button5);
18
19 // 将垂直布局设置给父窗口对象
20 window->setLayout(layout);
21 // 显示父窗口
22 window->show();
```

11.4.4 QGridLayout

```
1 // 构造函数
2 QGridLayout::QGridLayout();
3 QGridLayout::QGridLayout(QWidget *parent);
4
5 // 添加窗口对象到网格布局中
6 /*
7 参数:
8     - widget: 添加到布局中的窗口对象
9     - row: 添加到布局中的窗口对象位于第几行 (从0开始)
10    - column: 添加到布局中的窗口对象位于第几列 (从0开始)
11    - alignment: 窗口在布局中的对齐方式, 没有特殊需求使用默认值即可
12 */
13 void QGridLayout::addWidget(
14     QWidget *widget, int row, int column,
15     Qt::Alignment alignment = Qt::Alignment());
16
17 /*
18 参数:
19     - widget: 添加到布局中的窗口对象
20     - fromRow: 添加到布局中的窗口对象位于第几行 (从0开始)
21     - fromColumn: 添加到布局中的窗口对象位于第几列 (从0开始)
22     - rowSpan: 添加的窗口从 fromRow 行开始跨越的行数
23     - columnSpan: 添加的窗口从 fromColumn 列开始跨越的列数
24     - alignment: 窗口在布局中的对齐方式, 没有特殊需求使用默认值即可
25 */
26 void QGridLayout::addWidget(
27     QWidget *widget, int fromRow, int fromColumn,
28     int rowSpan, int columnSpan,
29     Qt::Alignment alignment = Qt::Alignment());
30
31 // 设置 column 对应的列的最新宽度, 单位: 像素
32 void QGridLayout::setColumnMinimumWidth(int column, int minSize);
33
34 // 设置布局中水平方向窗口之间间隔的宽度
35 void QGridLayout::setHorizontalSpacing(int spacing);
36
37 // 设置布局中垂直方向窗口之间间隔的宽度
38 void QGridLayout::setVerticalSpacing(int spacing);
39
40
41 // 例子
42 // 创建父窗口对象
43 QWidget* window = new QWidget;
44 // 创建子窗口对象
45 QPushButton *button1 = new QPushButton("One");
46 QPushButton *button2 = new QPushButton("Two");
47 QPushButton *button3 = new QPushButton("Three");
```

```

48 QPushButton *button4 = new QPushButton("Four");
49 QPushButton *button5 = new QPushButton("Five");
50 QPushButton *button6 = new QPushButton("Six");
51 // 多行文本编辑框
52 QTextEdit* txedit = new QTextEdit;
53 txedit->setText("我占用了两行两列的空间哦。");
54
55 QGridLayout* layout = new QGridLayout;
56 // 按钮起始位置：第1行，第1列，该按钮占用空间情况为1行1列
57 layout->addWidget(button1, 0, 0);
58 // 按钮起始位置：第1行，第2列，该按钮占用空间情况为1行1列
59 layout->addWidget(button2, 0, 1);
60 // 按钮起始位置：第1行，第3列，该按钮占用空间情况为1行1列
61 layout->addWidget(button3, 0, 2);
62 // 编辑框起始位置：第2行，第1列，该按钮占用空间情况为2行2列
63 layout->addWidget(txedit, 1, 0, 2, 2);
64 // 按钮起始位置：第2行，第3列，该按钮占用空间情况为1行1列
65 layout->addWidget(button4, 1, 2);
66 // 按钮起始位置：第3行，第3列，该按钮占用空间情况为1行1列
67 layout->addWidget(button5, 2, 2);
68 // 按钮起始位置：第4行，第1列，该按钮占用空间情况为1行3列
69 layout->addWidget(button6, 3, 0, 1, 3);
70
71 // 网格布局设置给父窗口对象
72 window->setLayout(layout);
73 // 显示父窗口
74 window->show();
75

```

Fence 77

12. ui 控件

12.1 按钮控件

12.1.1 工具按钮

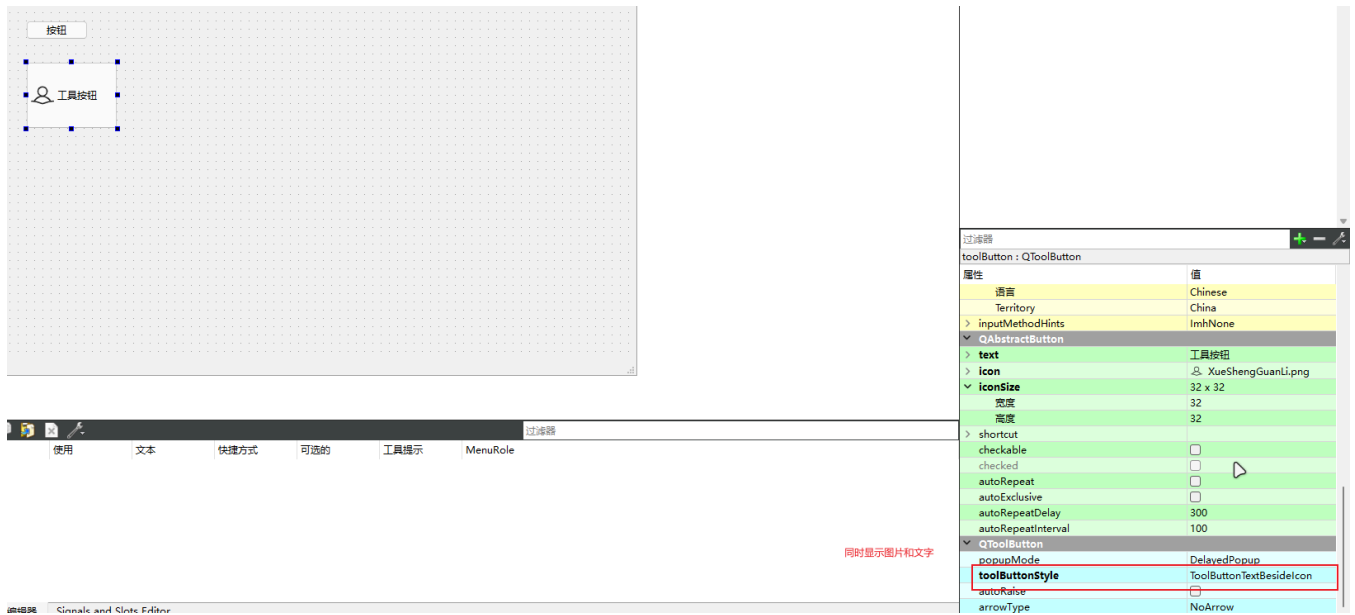


Figure 60

下面的 autoRaise 选上 变成突出风格

12.1.1.2 单选按钮



Figure 61

```

1 //默认男选择
2 ui->radioButton->setChecked(true);
3
4 connect(ui->radioButton_3,&QRadioButton::clicked,[=]() {
5     qDebug() << "多性别人士";
6 });

```

Fence 78

12.1.3 复选按钮

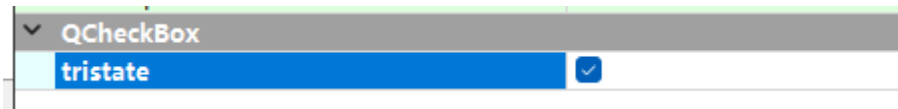


Figure 62

```

1 //stateChanged 获取按钮状态 1为半选 2为全选 0为不选
2 connect(ui->checkBox_3,&QCheckBox::stateChanged,[=](int state){
3     qDebug() << state;
4 });

```

Fence 79

12.2 QListWidget 控件

```

1 //设置List widget
2 //添加一行 但是可以设置文本位置
3 QListWidgetItem *item = new QListWidgetItem("锄禾日当午");
4 ui->listwidget->addItem(item);
5 item->setTextAlignment(Qt::AlignHCenter); //居中
6
7 //添加一个文本 但是不可以设置文本位置
8 QStringList stl; //qt风格的List<String>
9 stl << "汗滴留下土" << "谁知盘中餐" << "粒粒皆辛苦";
10 ui->listwidget->addItem(stl);

```

Fence 80

12.3 QTreeWidget

```

1 //treewidget 控件
2 //添加表头
3 ui->treewidget->setHeaderLabels(QStringList() << "英雄" << "英雄简介");
4
5 //添加头节点
6 QTreeWidgetItem * i1 = new QTreeWidgetItem(QStringList() << "打野");
7 QTreeWidgetItem * i2 = new QTreeWidgetItem(QStringList() << "射手");
8 QTreeWidgetItem * i3 = new QTreeWidgetItem(QStringList() << "法师");
9 ui->treewidget->addTopLevelItem(i1);
10 ui->treewidget->addTopLevelItem(i2);
11 ui->treewidget->addTopLevelItem(i3);
12
13 //添加子节点

```

```

14     QTreeWidgetItem *i11 = new QTreeWidgetItem(QStringList() << "兰陵王" << "隐身与爆发输出，兰陵王拥有强大的隐身能力，可以快速接近敌人并造成高额伤害。他的技能设计使得他在团战中也能发挥重要作用，是灵活多变的刺客型打野。");
15     QTreeWidgetItem *i12 = new QTreeWidgetItem(QStringList() << "韩信" << "灵活位移与控制，韩信的位移技能多样，能够快速穿梭战场，避开敌人的攻击。他擅长控制敌人位置，为队友创造输出机会，是极具机动性的打野英雄。");
16     QTreeWidgetItem *i13 = new QTreeWidgetItem(QStringList() << "李白" << "飘逸位移与群体削弱，李白以其高超的位移技巧和飘逸的战斗风格著称。他能够对多个敌人造成伤害，同时保持自身的灵活性，是团战中的重要输出点。");
17     i1->addChild(i11);
18     i1->addChild(i12);
19     i1->addChild(i13);
20
21     QTreeWidgetItem *i21 = new QTreeWidgetItem(QStringList() << "后羿" << "高爆发伤害，拥有远程消耗与控制能力，适合前期压制对手");
22     QTreeWidgetItem *i22 = new QTreeWidgetItem(QStringList() << "孙尚香" << "高机动性与输出，能够快速切入战场并造成大量伤害。");
23     QTreeWidgetItem *i23 = new QTreeWidgetItem(QStringList() << "公孙离" << "独特技能机制，集高伤害输出与灵活走位于一体，擅长爆发式击杀。");
24     i2->addChild(i21);
25     i2->addChild(i22);
26     i2->addChild(i23);
27
28     QTreeWidgetItem *i31 = new QTreeWidgetItem(QStringList() << "妲己" << "高爆发伤害，技能控制能力强，适合团队中消耗敌人和提供控制。");
29     QTreeWidgetItem *i32 = new QTreeWidgetItem(QStringList() << "安琪拉" << "持续输出能力强，拥有位移技能，能够灵活走位躲避敌人攻击。");
30     QTreeWidgetItem *i33 = new QTreeWidgetItem(QStringList() << "小乔" << "高伤害输出，技能带有范围伤害，适合在团战中造成大量AOE伤害。");
31     i3->addChild(i31);
32     i3->addChild(i32);
33     i3->addChild(i33);

```

Fence 81

12.4 QTableWidget

```

1 //tablewidget 控件
2 //设置列数
3 ui->tablewidget->setColumnCount(3);
4
5 //设置表头
6 ui->tablewidget->setHorizontalHeaderLabels(QStringList() << "姓名" << "性别" << "年龄");
7
8 //设置行数
9 ui->tablewidget->setRowCount(5);
10
11 //设置单元格
12 //0,0开始
13 //ui->tablewidget->setItem(0,0,new QTableWidgetItem("111"));
14 QString str[] = {

```

```

15         "姐己", "女", "20", "后羿", "男", "22", "鲁班大师", "男", "25", "孙尚香", "女", "23", "安琪
    拉", "女", "21"
16     };
17     for(int i = 0; i < 5; i++)
18     {
19         int t = i * 3;
20         ui->tablewidget->setItem(i,0,new QTableWidgetItem(str[t]));
21         ui->tablewidget->setItem(i,1,new QTableWidgetItem(str[t + 1]));
22         ui->tablewidget->setItem(i,2,new QTableWidgetItem(str[t + 2]));
23
24         //int 转 string QString::number()
25     }

```

Fence 82

12.5 其他控件

12.5.1 scroll Area(下拉条)

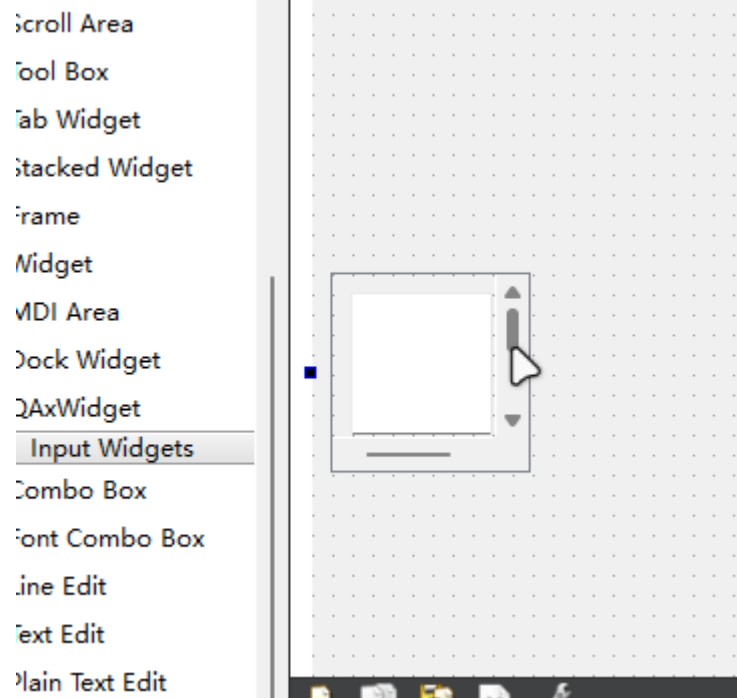


Figure 63

12.5.2 Tool Box(类似抽屉)

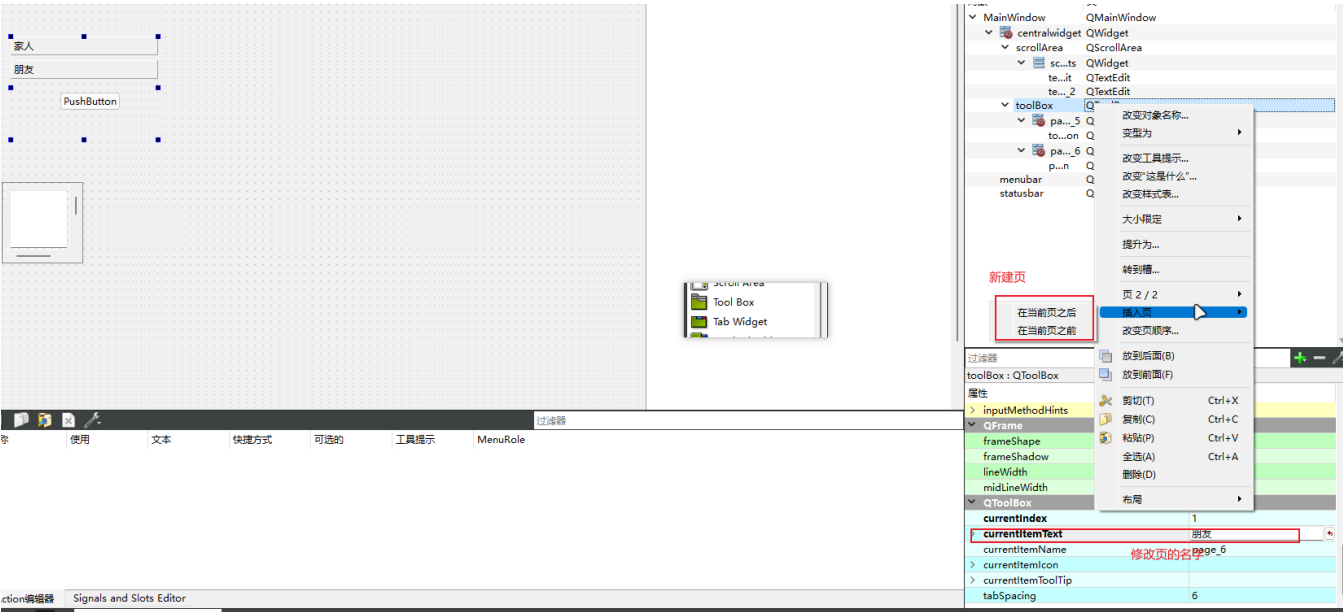


Figure 64

12.5.3 Tab Widget (类似网页切换)

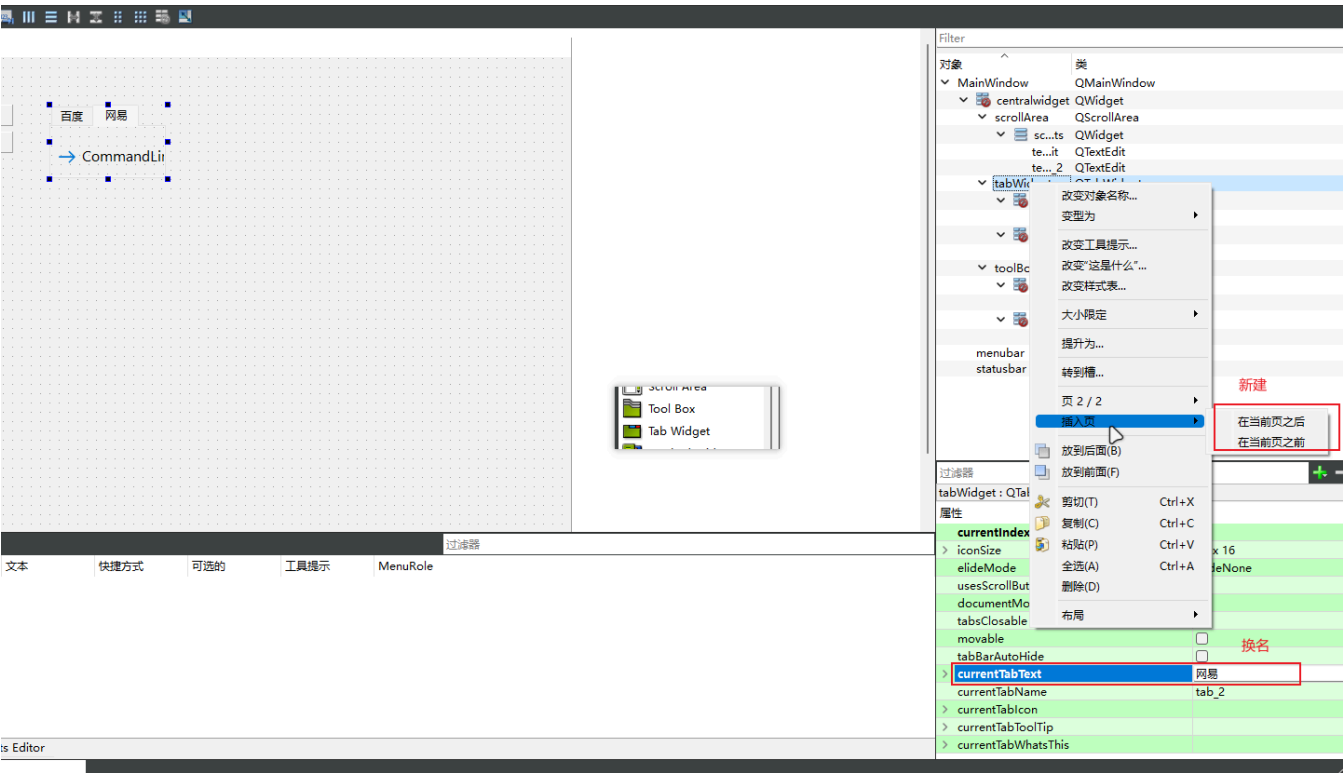


Figure 65

12.5.4 Stacked Widget (栈控件——翻页的切换页面)

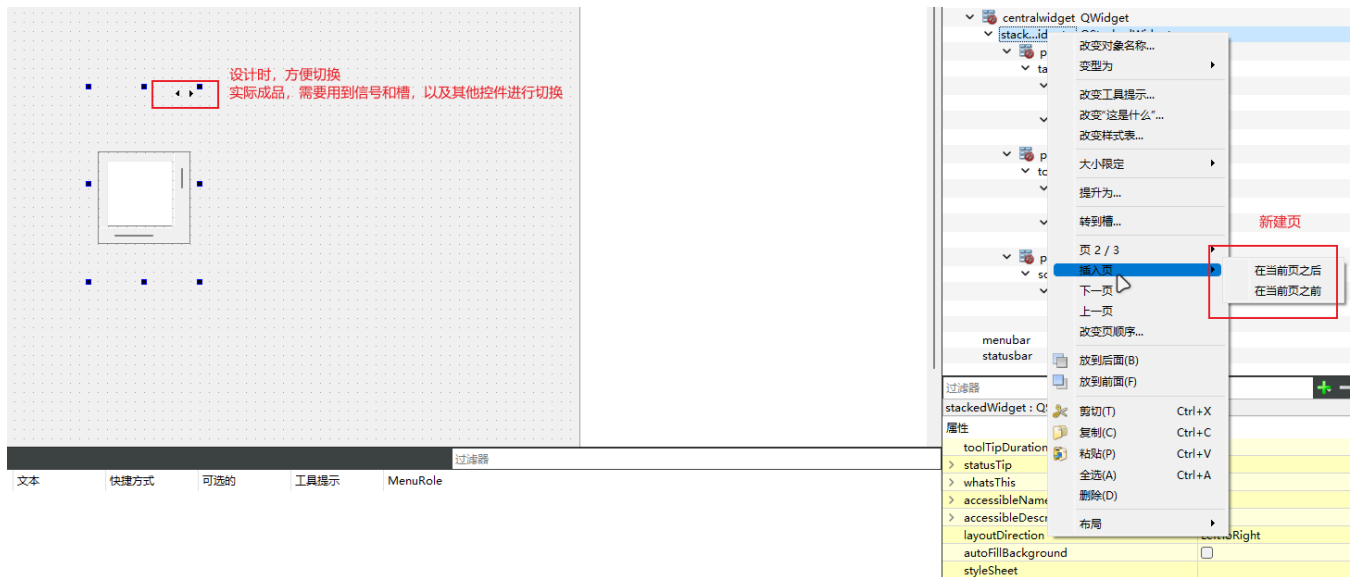


Figure 66

```
1 //栈控件的控制 查看ui->stackedwidget中的currentIndex属性，才能知道其他控件在多少页
2 //设置默认第一个页面
3 ui->stackedwidget->setCurrentIndex(2);
4 //Tool Box
5 connect(ui->pushButton_3,&QPushButton::clicked,[=]() {
6     ui->stackedwidget->setCurrentIndex(2);
7
8 });
9
10 //Scroll Area
11 connect(ui->pushButton_4,&QPushButton::clicked,[=]() {
12     ui->stackedwidget->setCurrentIndex(1);
13 });
14
15 //Tab widget
16 connect(ui->pushButton_5,&QPushButton::clicked,[=]() {
17     ui->stackedwidget->setCurrentIndex(0);
18 });
```

12.5.5 combo Box(下拉框) font Combo Box(字体下拉框)

```
1 //下拉框
2 ui->comboBox->addItem("1份宫保鸡丁");
3 ui->comboBox->addItem("2份宫保鸡丁");
4 ui->comboBox->addItem("3份宫保鸡丁");
5 ui->comboBox->addItem("没有宫保鸡丁");
6
7 connect(ui->pushButton_6,&QPushButton::clicked,[=]() {
8     ui->comboBox->setCurrentIndex(1);
9     // ui->comboBox->setCurrentText("2份宫保鸡丁");
10 });
```

Fence 84

- Text Edit 与 Plain Text Edit 都是文本框,前者可以修改文字颜色、倾斜,后者纯文本
- Spin Box 数字加减控件(类似快速跳转翻页)
- Double Spin Box 带小数的数字加减控件
- Time Edit 时间的加减控件
- Date Edit 日期加减控件
- Date/Time Edit 日期/时间加减控件
- Horizontal Scroll Bar 水平滑动条
- Vertical Scroll Bar 垂直滑动条
- Horizontal Slider 水平滑块
- Vertical Slider 垂直滑块
- Label 标签

- 显示图片 `ui->label->setPixmap(QPixmap(":/img/user.png"));`
- 显示动图 `QMovie *movie = new QMovie(".gif资源"); ui->label->setMovie(movie); movie->start(); //播放`

12.6 自定义控件封装

12.6.1 创建 ui 文件

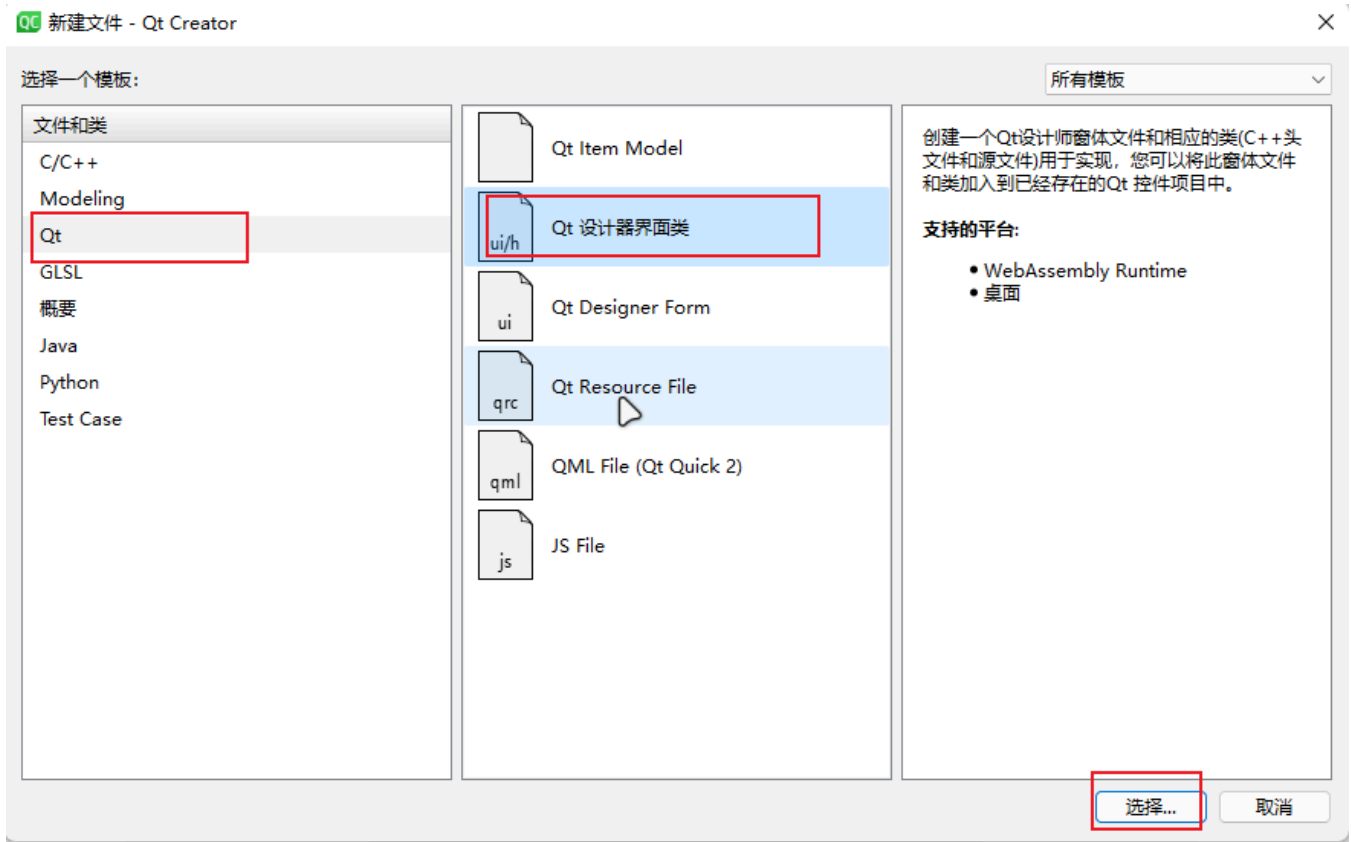


Figure 67

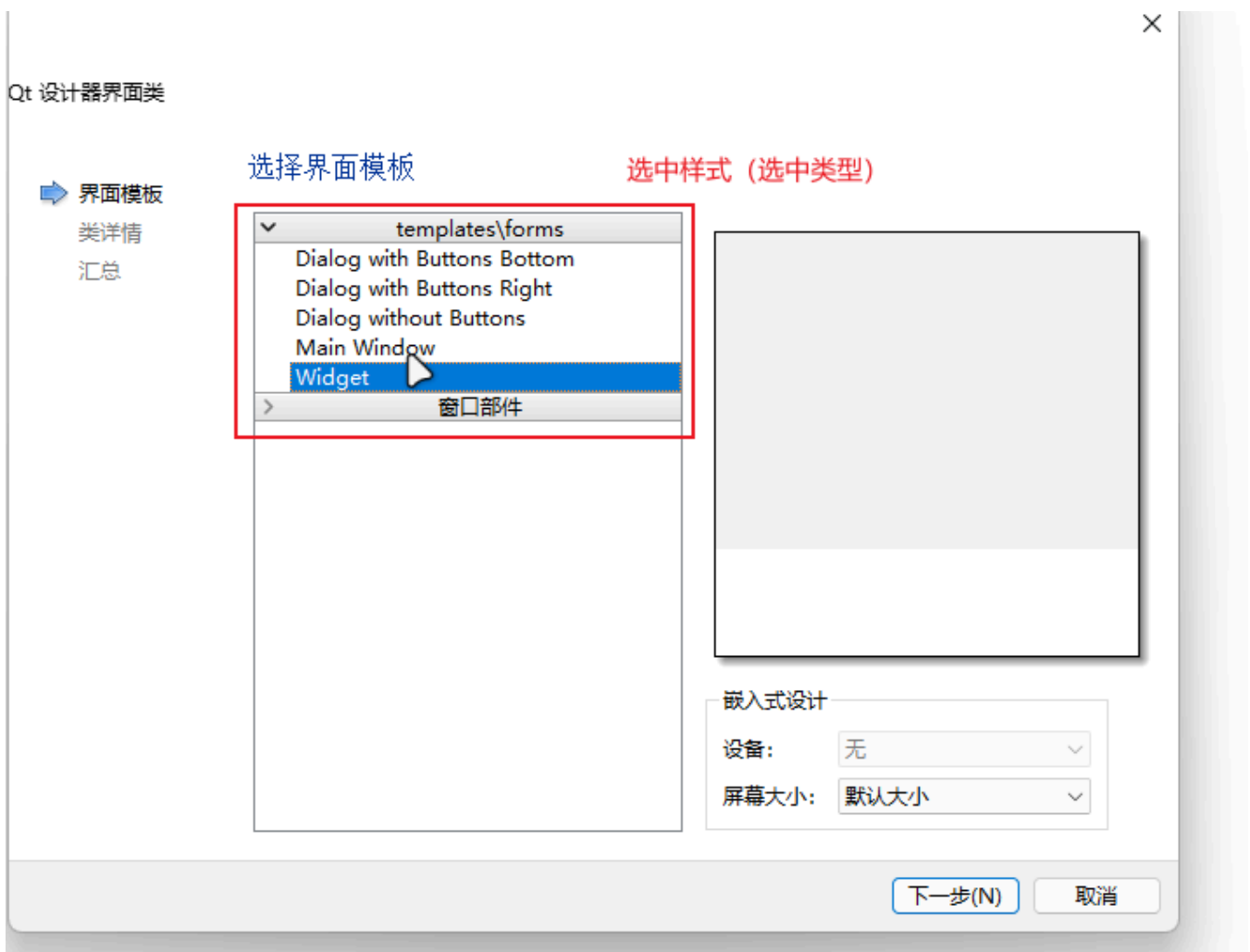


Figure 68



Figure 69

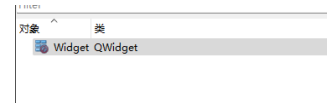
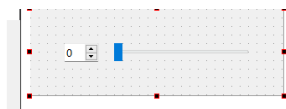


Figure 70

12.6.2 引入到其他窗口界面中

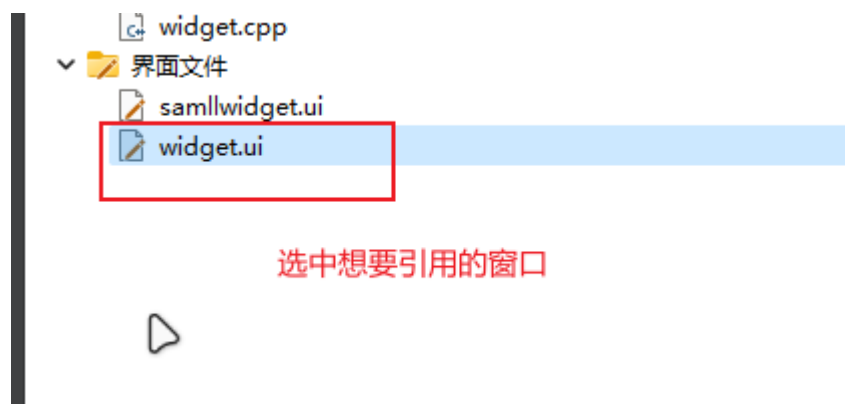


Figure 71

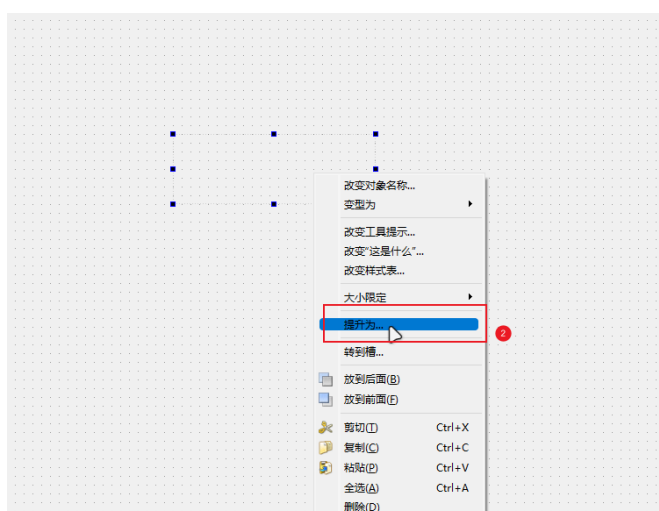
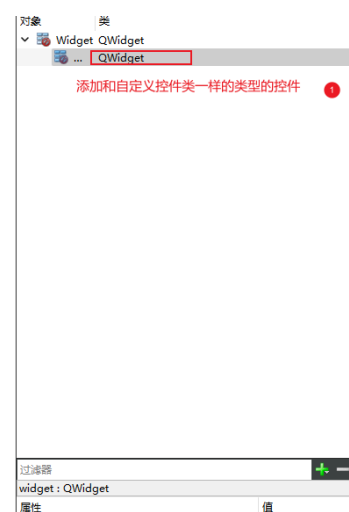


Figure 72



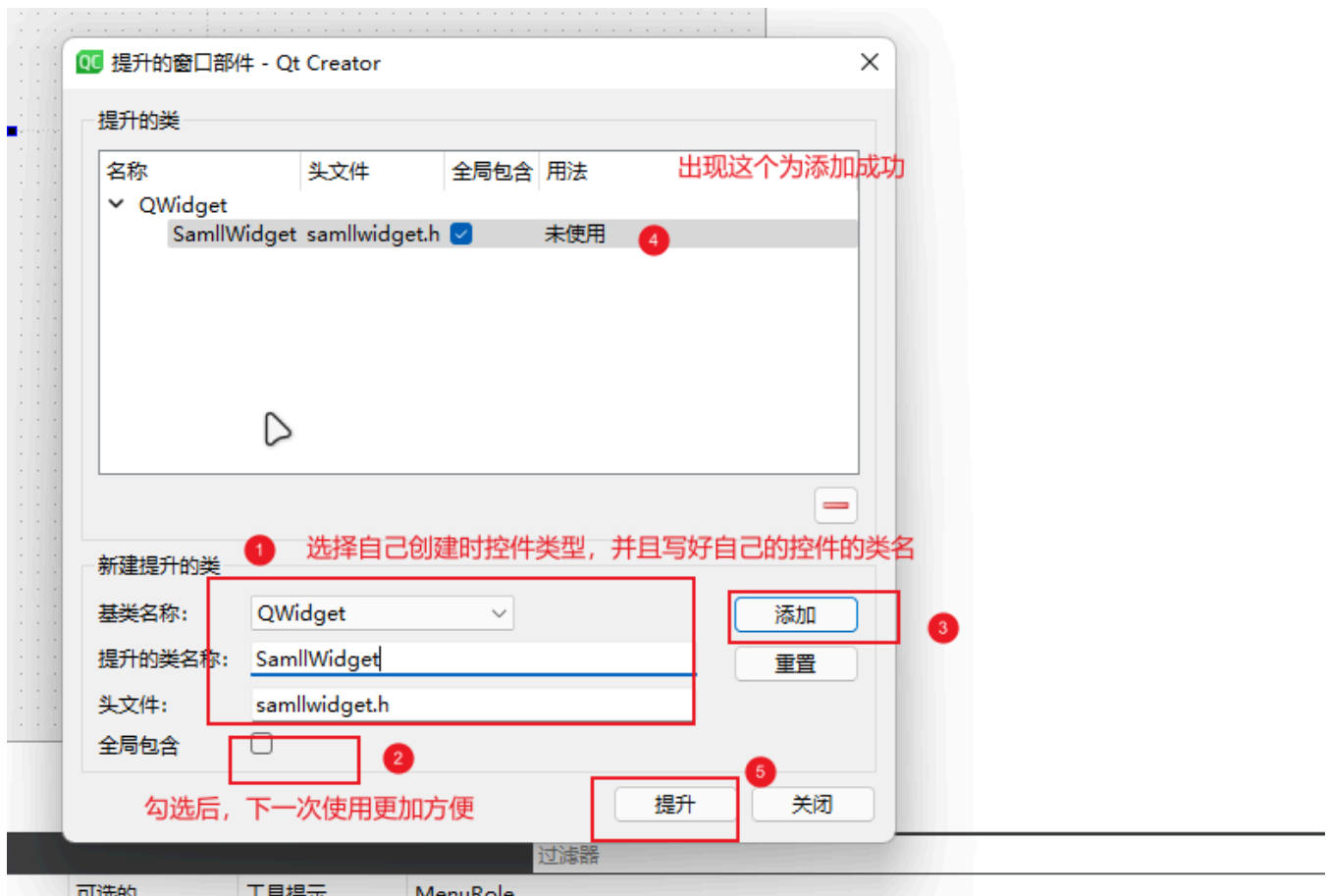


Figure 73

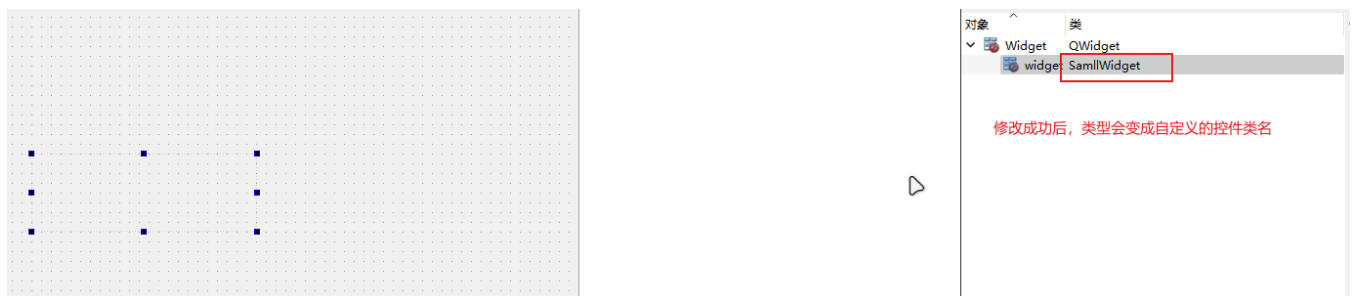


Figure 74

12.6.3 给自己定义控件添加功能

```

1 //自定义控件.h
2 namespace Ui {
3 class SamllWidget;
4 }
5
6 class SamllWidget : public QWidget
7 {
8     Q_OBJECT
9
10 public:
11     explicit SamllWidget(QWidget *parent = nullptr);
12     ~SamllWidget();
13 
```

```

14     //获取num
15     int getNum();
16
17     //设置num
18     void setNum(int num);
19
20     //触发信号
21     // void change(int num);
22
23 signals:
24     // 自定义信号
25     void numChanged(int num);
26
27 public slots:
28     // 触发信号
29     void change(int num);
30
31
32
33 private:
34
35     Ui::SamllWidget *ui;
36 };
37
38 //自定义控件.cpp
39 SamllWidget::SamllWidget(QWidget *parent)
40     : QWidget(parent)
41     , ui(new Ui::SamllWidget)
42 {
43     ui->setupUi(this);
44
45     //横向滑块设置范围
46     ui->horizontalSlider->setMaximum(100);
47     ui->horizontalSlider->setMinimum(0);
48
49     //设置spinBox的最大为100
50     ui->spinBox->setMaximum(100);
51
52     //给自定义控件设置功能    spinBox单向绑定horizontalSlider
53     connect(ui->spinBox,&QSpinBox::valueChanged,ui-
54 >horizontalSlider,&QSlider::setValue);
55
56     //horizontalSlider单向绑定spinBox
57     connect(ui->horizontalSlider,&QAbstractSlider::sliderMoved,ui-
58 >spinBox,&QSpinBox::setValue);
59 }
60
61 //获取num
62 int SamllWidget::getNum()
63 {
64     return ui->spinBox->value();
65 }

```

```

65 //设置num
66 void SamllWidget::setNum(int num)
67 {
68     ui->spinBox->setValue(num);
69 }
70
71 // 触发信号
72 void SamllWidget::change(int num)
73 {
74     emit numChanged(num); // 触发自定义信号
75 }
76

```

Fence 85

```

1 //主窗口
2 Widget::Widget(QWidget *parent)
3     : QWidget(parent)
4     , ui(new Ui::Widget)
5 {
6     ui->setupUi(this);
7
8     ui->lineEdit->setText(QString::number(ui->widget->getNum()));
9
10    //+
11    connect(ui->pushButton_2,&QPushButton::clicked,[=]() {
12        int num = 0;
13        num = ui->widget->getNum();
14        num++;
15        if(num >= 100)
16            num = 0;
17        ui->widget->setNum(num);
18        ui->lineEdit->setText(QString::number(num));
19    });
20
21    //-
22    connect(ui->pushButton_3,&QPushButton::clicked,[=]() {
23        int num = 0;
24        num = ui->widget->getNum();
25        num--;
26        if(num <= 0)
27            num = 100;
28        ui->widget->setNum(num);
29        ui->lineEdit->setText(QString::number(num));
30    });
31
32
33    connect(ui->widget,&SamllWidget::numChanged,[=](int num){
34        ui->lineEdit->setText(QString::number(num));
35    });
36 }

```

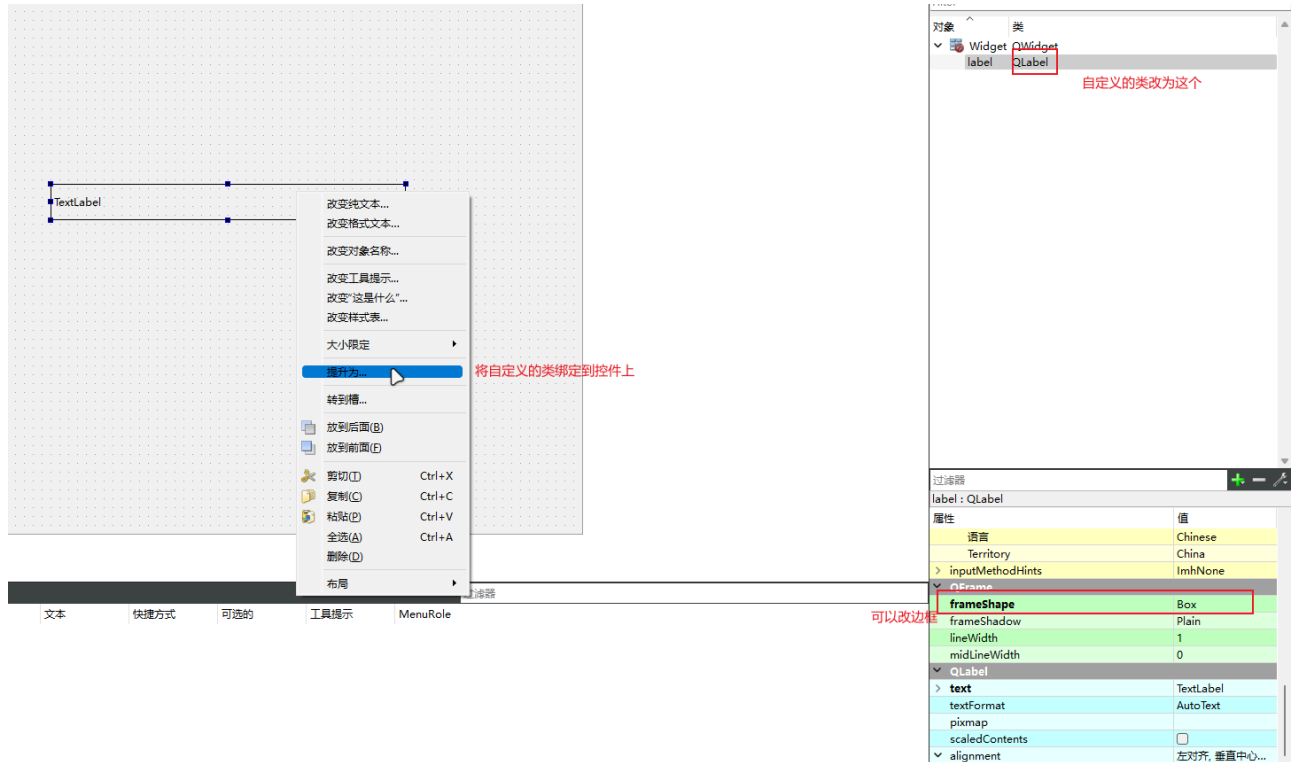
Fence 86

13. QEvent(事件)

事件都要重新实现，qt 文档中大多数在 Reimplemented Protected Functions 下面

13.1 鼠标事件

- 创建类，并且重写 enterEvent 和 leaveEvent 方法
- 改继承类型，改为和要操作的控件类型一致，如：我要操作 QLabel 控件，将继承改为 QLabel
- 在 ui 界面提升要操作的控件。



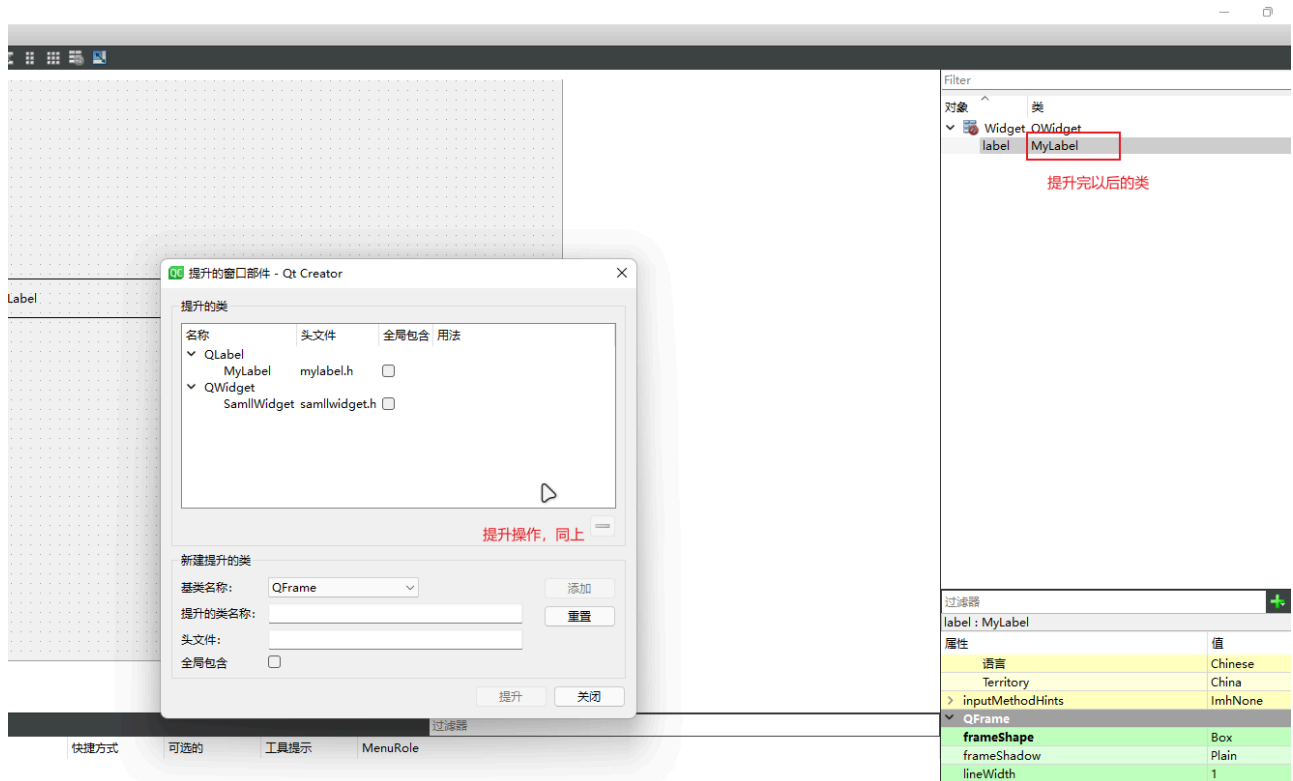


Figure 75

```

1 //MyLabel.h
2 #include <QLabel>
3
4 class MyLabel : public QLabel
5 {
6 public:
7     explicit MyLabel(QWidget *parent = nullptr);
8
9     //重写鼠标进入事件
10    void enterEvent(QEnterEvent *event);
11    //重写鼠标离开事件
12    void leaveEvent(QEvent *event);
13
14    //重写鼠标移动事件
15    void mouseMoveEvent(QMouseEvent *ev);
16    //重写鼠标按下事件
17    void mousePressEvent(QMouseEvent *ev);
18    //重写鼠标释放事件
19    void mouseReleaseEvent(QMouseEvent *ev);
20
21 signals:
22 };
23
24
25 //MyLabel.cpp
26 #include "mylabel.h"
27 #include <QDebug>
28 #include <QMouseEvent>
29 #include <QPointF>
30

```

```

31 MyLabel::MyLabel(QWidget *parent)
32     : QLabel{parent}
33 {
34     //设置鼠标追踪 不需要按下以后才有移动事件的触发
35     setMouseTracking(true);
36
37 }
38
39
40 //重写鼠标进入事件
41 void MyLabel::enterEvent(QEnterEvent *event)
42 {
43     qDebug() << "鼠标进入";
44     qDebug();
45 }
46
47 //重写鼠标离开事件
48 void MyLabel::leaveEvent(QEvent *event)
49 {
50     qDebug() << "鼠标离开";
51     qDebug();
52 }
53
54
55 //重写鼠标移动事件（过程值）
56 void MyLabel::mouseMoveEvent(QMouseEvent *ev)
57 {
58     QPointF f = ev->globalPosition();
59     //1为第一个arg()的占位符
60     //globalPosition是绝对位置 整个屏幕中的位置
61     //scenePosition是相对位置 主窗口中的位置
62     qDebug() << QString("鼠标移动到, x=%1,y=%2,globalx=%3,globaly=%4").arg(ev-
>scenePosition().x())
63         .arg(ev->scenePosition().y()).arg(f.x()).arg(f.y());
64     //qDebug() << f.rx() << " " << f.ry(); //和y() x()没有差别 只是rx和ry只读
65     qDebug();
66 }
67
68 //重写鼠标按下事件（瞬间值）
69 void MyLabel::mousePressEvent(QMouseEvent *ev)
70 {
71     // if(ev->button() == Qt::LeftButton)
72     // {
73     //     qDebug() << "左键按下";
74     // }
75     // else if(ev->button() == Qt::MiddleButton)
76     // {
77     //     qDebug() << "中键按下";
78     // }
79     // else if(ev->button() == Qt::RightButton)
80     // {
81     //     qDebug() << "右键按下";
82     // }

```

```

83
84     if(ev->buttons() == (Qt::LeftButton | Qt::MiddleButton))
85     {
86         qDebug() << "左键和中间按下";
87     }
88     qDebug();
89 }
90
91 //重写鼠标释放事件（瞬间值）
92 void MyLabel::mouseReleaseEvent(QMouseEvent *ev)
93 {
94     //buttons 用于检测按键组合 能用于检测按下事件和移动事件 释放事件中返回值依然是按下按键的值（排除引起该事件的按钮）
95     //button 用于检测按键
96     if(ev->buttons() == (Qt::LeftButton | Qt::MiddleButton))
97     {
98         qDebug() << "三键按下后，左键和中间按下,右键释放";
99     }
100    qDebug();
101 }
102
103

```

Fence 87

13.2 定时器事件（概念与 stm32 中学的基本类似）

- 在进行窗口程序的处理过程中, 经常要周期性的执行某些操作, 或者制作一些动画效果, 看似比较复杂的问题使用定时器就可以完美的解决这些问题, Qt中提供了两种定时器方式一种是使用Qt中的事件处理函数这个在后续章节会给大家做细致的讲解, 本节主要给大家介绍一下Qt中的定时器类 QTimer的使用方法。

要使用它, 只需创建一个QTimer类对象, 然后调用其 start() 函数开启定时器, 此后QTimer对象就会周期性的发出 timeout() 信号。

- ```

1 // 构造函数
2 // 如果指定了父对象, 创建的堆内存可以自动析构
3 QTimer::QTimer(QObject *parent = nullptr);
4
5 // 设置定时器时间间隔为 msec 毫秒
6 // 默认值是0, 一旦窗口系统事件队列中的所有事件都已经被处理完, 一个时间间隔为0的QTimer就会触发
7 void QTimer::setInterval(int msec);
8 // 获取定时器的时间间隔, 返回值单位: 毫秒
9 int QTimer::interval() const;
10
11 // 根据指定的时间间隔启动或者重启定时器, 需要调用 setInterval() 设置时间间隔
12 [slot] void QTimer::start();
13 // 启动或重新启动定时器, 超时间隔为msec毫秒。
14 [slot] void QTimer::start(int msec);
15 // 停止定时器。
16 [slot] void QTimer::stop();
17
18 // 设置定时器精度
19 /*

```

```

20 参数：
21 - Qt::PreciseTimer -> 精确的精度，毫秒级
22 - Qt::CoarseTimer -> 粗糙的精度，和1毫秒的误差在5%的范围内，默认精度
23 - Qt::VeryCoarseTimer -> 非常粗糙的精度，精度在1秒左右
24 */
25 void QTimer::setTimerType(Qt::TimerType atype);
26 Qt::TimerType QTimer::timerType() const; // 获取当前定时器的精度
27
28 // 如果定时器正在运行，返回true；否则返回false。
29 bool QTimer::isActive() const;
30
31 // 判断定时器是否只触发一次
32 bool QTimer::isSingleShot() const;
33 // 设置定时器是否只触发一次，参数为true定时器只触发一次，为false定时器重复触发，默认为false
34 void QTimer::setSingleShot(bool singleShot);
35
36
37
38
39 //这个类的信号只有一个，当定时器超时，该信号就会被发射出来。给这个信号通过connect()关联一个槽
 函数，就可以在槽函数中处理超时事件了。
40 [signal] void QTimer::timeout();
41
42
43
44 //静态方法 不创建QTimer对象
45 // 其他同名重载函数可以自己查阅帮助文档
46 /*
47 功能：在msec毫秒后发射一次信号，并且只发射一次
48 参数：
49 - msec： 在msec毫秒后发射信号
50 - receiver：接收信号的对象地址
51 - method： 槽函数地址
52 */
53 [static] void QTimer::singleShot(
54 int msec, const QObject *receiver,
55 PointerToMemberFunction method);
56
57
58
59 //例子
60 //周期性定时器
61 // 创建定时器对象
62 QTimer* timer = new QTimer(this);
63
64 // 修改定时器对象的精度
65 timer->setTimerType(Qt::PreciseTimer);
66
67 // 按钮 loopBtn 的点击事件
68 // 点击按钮启动或者关闭定时器，定时器启动，周期性得到当前时间
69 connect(ui->loopBtn, &QPushButton::clicked, this, [=]()
70 {
71 // 启动定时器

```

```

72 if(timer->isActive())
73 {
74 timer->stop(); // 关闭定时器
75 ui->loopBtn->setText("开始");
76 }
77 else
78 {
79 ui->loopBtn->setText("关闭");
80 timer->start(1000); // 1000ms == 1s
81 }
82 });
83
84 connect(timer, &QTimer::timeout, this, [=]()
85 {
86 QTime tm = QTime::currentTime();
87 // 格式化当前得到的系统时间
88 QString tmstr = tm.toString("hh:mm:ss.zzz");
89 // 设置要显示的时间
90 ui->curTime->setText(tmstr);
91 });
92
93
94
95 //一次性定时器
96 // 点击按钮 onceBtn 只发射一次信号
97 // 点击按钮一次，发射一个信号，得到某一个时间点的时间
98 connect(ui->onceBtn, &QPushButton::clicked, this, [=]()
99 {
100 // 获取2s以后的系统时间，不创建定时器对象，直接使用类的静态方法
101 QTimer::singleShot(2000, this, [=]() {
102 QTime tm = QTime::currentTime();
103 // 格式化当前得到的系统时间
104 QString tmstr = tm.toString("hh:mm:ss.zzz");
105 // 设置要显示的时间
106 ui->onceTime->setText(tmstr);
107 });
108 });
109

```

Fence 88

```

1 //widget.h
2 #include <QWidget>
3
4 QT_BEGIN_NAMESPACE
5 namespace Ui {
6 class widget;
7 }
8 QT_END_NAMESPACE
9

```

```

10 class Widget : public QWidget
11 {
12 Q_OBJECT
13
14 public:
15 Widget(QWidget *parent = nullptr);
16 ~Widget();
17
18 //重写定时器事件
19 void timerEvent(QTimerEvent *e);
20
21 private:
22 int id1; //定时器1的id
23 int id2; //定时器2的id
24 int num3 = 1; //定时器3的计数值
25 Ui::Widget *ui;
26 };
27
28 //widget.cpp
29 #include "widget.h"
30 #include "ui_widget.h"
31 #include <QTimer>
32 #include <QPushButton>
33
34 Widget::Widget(QWidget *parent)
35 : QWidget(parent)
36 , ui(new Ui::Widget)
37 {
38 ui->setupUi(this);
39
40 //启动一个定时器
41 id1 = startTimer(1000); //参数1 时间间隔（ms） 每隔xx毫秒调用定时器事
件 返回值为定时器的唯一id
42
43 id2 = startTimer(2000);
44
45 //第二种定时器方式（信号和槽，非事件）
46 QTimer *t = new QTimer(this);
47 t->start(500); //0.5秒
48 connect(t,&QTimer::timeout,[=]() {
49 ui->label_4->setText(QString::number(this->num3++));
50 });
51
52 //暂停
53 connect(ui->pushButton,&QPushButton::clicked,[=]() {
54 if(t->isActive()) //ui->pushButton->text() == "暂停" isActive()
是否在运行
55 {
56 t->stop(); //暂停
57 ui->pushButton->setText("继续");
58 }
59 else //ui->pushButton->text() == "继续"
60 {

```

```

61 t->start(); //重启
62 ui->pushButton->setText("暂停");
63 }
64 });
65
66 //清零
67 connect(ui->pushButton_2,&QPushButton::clicked,[=]() {
68 // t->interval(); //获取当前的毫秒数
69 this->num3 = 1;
70 ui->label_4->setText(QString::number(this->num3));
71 });
72 }
73
74 //重写定时器事件
75 void Widget::timerEvent(QTimerEvent *e)
76 {
77 //每隔1秒
78 if(e->timerId() == id1)
79 {
80 static int num1 = 1; //静态变量 让函数第一次调用的时候才赋值，后面
调用就用之前的值
81 ui->label_2->setText(QString::number(num1++));
82 }
83
84 //每隔2秒
85 if(e->timerId() == id2)
86 {
87 static int num2 = 1; //静态变量 让函数第一次调用的时候才赋值，后面
调用就用之前的值
88 ui->label_3->setText(QString::number(num2++));
89 }
90 }
91
92 Widget::~Widget()
93 {
94 delete ui;
95 }
96

```

Fence 89

### 13.3 事件分发器(event())

- bool event(QEvent \*ev);
- 返回值为 bool 类型。如果返回 true，代表用户自己处理了该事件，不在向下分发（不发给系统或者父类）。
- 事件分发器 介于 用户与事件之间，用户触发事件后，由事件分发器来告诉系统该事件是用户 自己处理（重写的 event 中处理） 还是系统处理（默认的对应该事件函数）。

```
1 //mylabel.h
```

```

2 //重写事件分发器event
3 bool event(QEvent *e);
4
5
6 //mylabel.cpp
7 //重写事件分发器 用于拦截所有事件
8 bool MyLabel::event(QEvent *e)
9 {
10 //处理拦截下的鼠标按下事件
11 if(QEvent::MouseButtonPress == e->type())
12 {
13 //类型转换
14 // QMouseEvent *ev = static_cast<QMouseEvent *>(e);
15 //static_cast<>() 基本类型之间的转换、上转型（从派生类到基类 子类到父类） 且不会进行继承关系的检查
16
17 QMouseEvent *ev = dynamic_cast<QMouseEvent *>(e);
18 //dynamic_cast<>() 向下转型（从基类到派生类 父类到子类） 且会进行继承检查
19
20 qDebug() << QString("用户自己的处理鼠标移动到，
21 x=%1,y=%2,globalx=%3,globaly=%4").arg(ev->scenePosition().x()).arg(ev->scenePosition().y());
22
23 return true;
24 }
25
26 //其他事件调用 父类的方法 默认处理
27 return QLabel::event(e);
28 }

```

Fence 90

## 13.4 事件过滤器

- 在 event 之前可以通过事件过滤器，进行一次高级过滤
- 使用步骤
  - 给控件安装事件过滤器（控件调用）
  - 重写 eventFilter()

```

1 //widget.h
2 //重写事件过滤器
3 bool eventFilter(QObject *obj, QEvent *ev);
4
5
6 //widget.cpp
7 Widget::Widget(QWidget *parent)
8 : QWidget(parent)
9 , ui(new Ui::Widget)
10 {
11
12 //给控件设置过滤器
13 //参数 父对象

```

```

14 ui->label->installEventFilter(this);
15
16 }
17 //重写事件过滤器
18 //obj 控件类型
19 //e 事件类型
20 bool Widget::eventFilter(QObject *obj, QEvent *e)
21 {
22 //过滤出label控件
23 if(ui->label == obj)
24 {
25 if(QEvent::MouseButtonDbClick == e->type())
26 {
27 //类型转换
28 QMouseEvent *ev = dynamic_cast<QMouseEvent *>(e);
29 //dynamic_cast<>() 向下转型（从基类到派生类 父类到子类） 且会进行继承检查
30
31 qDebug() << QString("过滤器过滤的鼠标事件，移动到，
32 x=%1,y=%2,globalx=%3,globaly=%4").arg(ev->scenePosition().x()).arg(ev->
33 >scenePosition().y());
34
35 return true;
36 }
37 }
38
39 //其他事件 默认处理 交给父类
40 return QWidget::eventFilter(obj,e);
41 }

```

Fence 91

## 13.5 QPainter(绘图)

### 13.5.1 绘图事件

```

1 //widget.h
2 #include <QWidget>
3
4 QT_BEGIN_NAMESPACE
5 namespace Ui {
6 class widget;
7 }
8 QT_END_NAMESPACE
9
10 class widget : public QWidget
11 {
12 Q_OBJECT
13
14 public:
15 widget(QWidget *parent = nullptr);

```

```

16 ~Widget();
17
18 //重写绘图事件
19 void paintEvent(QPaintEvent *event);
20
21 private:
22 Ui::Widget *ui;
23 };
24
25
26 //widget.cpp
27 #include "widget.h"
28 #include "ui_widget.h"
29 #include <QPainter> //回画家类
30
31 Widget::Widget(QWidget *parent)
32 : QWidget(parent)
33 , ui(new Ui::Widget)
34 {
35 ui->setupUi(this);
36 }
37
38 Widget::~Widget()
39 {
40 delete ui;
41 }
42
43 //重写绘图事件
44 void Widget::paintEvent(QPaintEvent *event)
45 {
46 //实例化画家对象 这里this 是告诉在哪里画（画画的地方）
47 QPainter painter(this);
48
49 //画线
50 painter.drawLine(QPoint(0,0),QPoint(100,100));
51
52 //画圆 本质是画椭圆 参数1: 圆心 参数2: x轴的焦点 参数3: y轴的焦点
53 painter.drawEllipse(QPoint(100,100),20,20);
54
55 //画矩形
56 //QRect构造重载版本1 QRect(QPoint(100,100),QPoint(200,200)) 给两个对角的点
57 //QRect构造重载版本2 QRect(QPoint(100,100),QSize(20,10)) 给一个左上角的点，通过
58 //QSize()给长，宽
59 //QRect构造重载版本3 QRect(100,100,100,100) 给一个左上角的点，直接给
60 //长，宽
61 //QRectF 使用浮点精度画矩形
62 painter.drawRect(QRect(100,100,100,100));
63
64 QPen pen;
65
66 //要想同时设置样式、颜色等，需要先设置笔的样式
67 pen.setColor(QColor(255,0,0));

```

```

67 pen.setStyle(Qt::DotLine);
68
69
70 //改变线条颜色 （换笔） 什么时候使用，那之后的线条就会变颜色
71 painter.setPen(pen);
72
73 //改变线条样式 但是不会改变文字的线条样式
74 painter.setPen(pen);
75
76
77 //设置文字样式
78 QFont font;
79 font.setBold(true); // 设置为粗体
80 font.setItalic(true); // 设置为斜体
81 font.setFamily("Microsoft YaHei"); // 设置字体为微软雅黑
82 font.setPointSize(14); // 设置字体大小为14
83 painter.setFont(font);
84
85
86 //设置画刷 填充封闭图形的颜色 注意字和填充的顺序
87 QBrush brush(QColor(255, 192, 203));
88 brush.setStyle(Qt::HorPattern); //设置画刷风格
89 painter.setBrush(brush);
90
91 painter.drawRect(QRect(300, 300, 100, 100)); //显示外面的框
92
93 //画文字 在一个矩形框里写字
94 painter.drawText(QRect(300, 300, 100, 100), "你好世界，你好中国！");
95 }

```

Fence 92

## 13.5.2 绘图高级设置

```

1 //widget.cpp
2 void Widget::paintEvent(QPaintEvent *event)
3 {
4 //高级设置
5 painter.drawEllipse(QPoint(100, 400), 50, 50);
6
7 //设置高锯齿 更精细 但是效率更低
8 painter.setRenderHint(QPainter::Antialiasing);
9
10 painter.drawEllipse(QPoint(200, 400), 50, 50);
11
12 //移动画家(将原点 (0, 0) 移动到 (100, 0))
13 painter.translate(100, 0);
14
15 painter.drawEllipse(QPoint(200, 400), 50, 50); //不会重叠
16
17 //保存画家状态
18 painter.save();
19 painter.translate(100, 0);
20

```

```

21
22 painter.drawEllipse(QPoint(200,400),50,50);
23
24 //还原画家状态
25 painter.restore();
26 // painter.translate(200,0);
27
28 painter.drawEllipse(QPoint(200,400),50,50); //与上一个重叠
29 }

```

Fence 93

### 13.5.3 手动调用绘图事件(利用绘图类画图片中的图)

```

1 //widget.cpp
2
3 #include "widget.h"
4 #include "ui_widget.h"
5 #include <QPainter> //回画家类
6 #include <QTimer>
7
8 Widget::Widget(QWidget *parent)
9 : QWidget(parent)
10 , ui(new Ui::Widget)
11 {
12 ui->setupUi(this);
13
14 QTimer *t = new QTimer(this);
15
16
17 connect(ui->pushButton,&QPushButton::clicked,[=]() {
18 if(t->isActive())
19 {
20 t->stop();
21 ui->pushButton->setText("启动");
22 }
23 else
24 {
25 t->start(500);
26 ui->pushButton->setText("暂停");
27 }
28 });
29
30
31 connect(t,&QTimer::timeout,[=]() {
32 posy += 20;
33 if(posy > this->height())
34 {
35 posy = 0;
36 }
37
38
39 //手动调用绘图事件 用update()更新
40 update();

```

```

41 });
42 }
43 //重写绘图事件
44 void widget::paintEvent(QPaintEvent *event)
45 {
46 //实例化画家对象 这里this 是告诉在哪里画（画画的地方）
47 QPainter painter(this);
48 //绘制资源图片
49 painter.drawPixmap(QRect(500, posy, 20, 20), QPixmap(":/img/user.png"));
50
51 }

```

Fence 94

### 13.5.4 绘图设备

绘图设备是指继承自 **QPaintDevice** 的子类。Qt一共提供了四个这样的类，分别是：

- **QPixmap**：专门为图像在屏幕上的显示做了优化。（不同设备或者操作系统，会有不同的优化）
- **QBitmap**：是 QPixmap 的一个子类，它的色深限定为1。可以使用 QPixmap 的 **isQBitmap()** 函数来确定这个 QPixmap 是否是一个 QBitmap。（只有黑白两种颜色）
- **QImage**：专门为图像的像素级访问做了优化。（可以获取像素点，并且可以对获取的像素点进行赋值）
- **QPicture**：可以用来记录和重现 QPainter 的各类命令。

- QPixmap、QImage、QPicture的使用：

```

1 //widget.h
2 #include <QWidget>
3
4 QT_BEGIN_NAMESPACE
5 namespace Ui {
6 class widget;
7 }
8 QT_END_NAMESPACE
9
10 class Widget : public QWidget
11 {
12 Q_OBJECT
13
14 public:
15 Widget(QWidget *parent = nullptr);
16 ~Widget();
17
18
19 //重写绘图事件
20 void paintEvent(QPaintEvent *event);
21 private:
22 Ui::Widget *ui;
23 };
24
25

```

```

26
27 //widget.cpp
28 #include "widget.h"
29 #include "ui_widget.h"
30 #include <QPixmap>
31 #include <QPainter>
32 #include <QImage>
33 #include <QPicture>
34
35 widget::widget(QWidget *parent)
36 : QWidget(parent)
37 , ui(new Ui::widget)
38 {
39 ui->setupUi(this);
40
41 //创建 QPixmap绘图设备 如果没有指定到widget上, 那么窗口什么也不会显示
42 QPixmap pix(300,400); //创建设备大小 300x400
43 pix.fill(Qt::white); //画布背景默认黑色 现在填充为白色
44 //pix = pix.scaled(pix.width() * 0.5, pix.height() * 0.5); //参数1: 原宽 * 缩
放倍数 参数2: 原高 * 缩放倍数
45
46 //创建画师
47 QPainter pain(&pix); //指定pix设备
48 pain.setPen(Qt::green); //设置成绿色
49 pain.drawRect(100,100,100,100); //画一个矩形
50
51 //保存设备 在该路径下创建pix.png进行保存
52 pix.save("E:\\QTProject\\13Project\\img\\pix.png");
53
54
55
56 //创建 QImage绘图设备 可以对像素点进行访问
57 QImage img(300,400,QImage::Format_RGB32); //宽 高 文件类型
58 img.fill(Qt::white); //画布背景默认黑色 现在填充为白色
59 QPainter pain1(&img);
60 pain1.setPen(Qt::blue); //设置成绿色
61 pain1.drawRect(100,100,100,100); //画一个矩形
62
63 //保存
64 img.save("E:\\QTProject\\13Project\\img\\img.png");
65
66
67
68 //创建 QPicture绘图设备 记录和重现 绘图指令(代码)
69 QPicture pic;
70 QPainter pain2;
71 pain2.begin(&pic);
72 pain2.setPen(Qt::yellow);
73 pain2.drawRect(100,100,100,100);
74 pain2.end();
75
76 //保存

```

```

77 pic.save("E:\\QTProject\\13Project\\img\\pic.tj"); //文件名为自定义的（可以保证
 数据隐私），里面存储的是begin()~end() 之间的代码
78
79 }
80
81 widget::~widget()
82 {
83 delete ui;
84 }
85
86 //重写绘图事件
87 void widget::paintEvent(QPaintEvent *event)
88 {
89 //指定到窗口
90 // QImage img;
91 // img.load(":/img/R-C.jpg"); //从文件中加载图片
92 // QPainter p(this);
93
94 // for(int i = 60; i < 100; i++)
95 // {
96 // for(int j = 60; j < 100; j++)
97 // {
98 // QRgb va = qRgb(255,0,0); //设置像素点的颜色
99 // img.setPixel(i,j,va); //访问像素点
100 // }
101 // }
102
103 // p.drawImage(0,0,img);
104
105 //重现代码
106 QPicture pic;
107 QPainter p(this);
108 pic.load("E:\\QTProject\\13Project\\img\\pic.tj"); //加载
109 p.drawPicture(0,0,pic); //复现绘图操作 不会完全覆盖 按照原图
 比例作画 drawPixmap(0,0,this->width(),this->height(),pic); 会拉伸图片
110 }

```

Fence 95

- QPixmap与QPixmap区别:

```

1 void PaintWidget::paintEvent(QPaintEvent *)
2 {
3 QPixmap pixmap("./Image/butterfly.png");
4 QPixmap pixmap1("./Image/butterfly1.png");
5
6 QPixmap bitmap("./Image/butterfly.png");
7 QPixmap bitmap1("./Image/butterfly1.png");
8
9 QPainter painter(this);
10 painter.drawPixmap(0, 0, pixmap);
11 painter.drawPixmap(200, 0, pixmap1);

```

```

12 painter.drawPixmap(0, 130, bitmap);
13 painter.drawPixmap(200, 130, bitmap1);
14 }

```

Fence 96

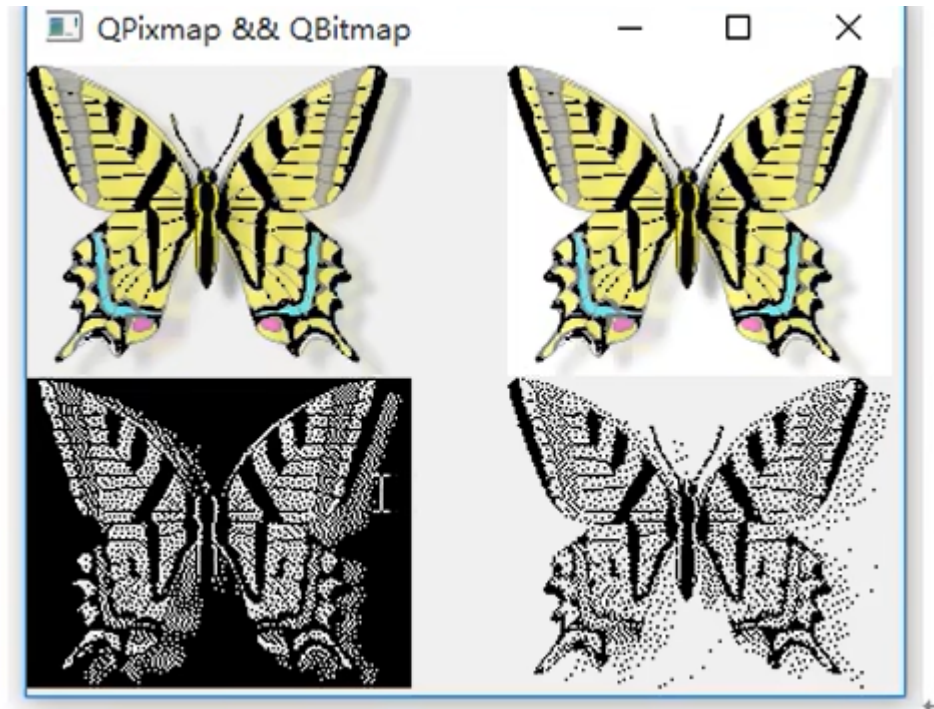


Figure 76

- 绘出了两张 .png 图片。butterfly1.png 是没有透明色的纯白背景，而 butterfly.png 是具有透明色的背景。我们分别使用 QPixmap 和 QImage 来加载它们。
- 注意它们的区别：
  - 白色的背景在 QImage 中消失了，而透明色在 QImage 中转换成了黑色；其他颜色则是使用点的疏密程度来体现的。

## 14. QFile(对文件进行读写操作)

```

1 //widget.cpp
2 #include "widget.h"
3 #include "ui_widget.h"
4 #include <QFile>
5 #include <QFileDialog>
6 // #include <QStringConverter>
7 // #include <QStringDecoder>
8 // #include <QMessageBox>
9
10 Widget::Widget(QWidget *parent)
11 : QWidget(parent)
12 , ui(new Ui::Widget)
13 {
14 ui->setupUi(this);
15
16 connect(ui->pushButton, &QPushButton::clicked, [=]() {

```

```

17 //创建堆区文件对话框
18 QString path = QFileDialog::getOpenFileName(this, "打开文件", "../14Project/text");
19
20 //放入lineEdit中
21 ui->lineEdit->setText(path);
22
23 //编码格式枚举类型 qt5 QTextCodec
24 //QTextCodec *codec = QTextCodec::codecForName("gbk");
25 //ui->textEdit->setText(codec->toUnicode(array));
26
27 //std::optional<T> 是一个在标准库中提供的模板类，用于表示一个值可以有意义（存在）或者没有意义（不存在）的情况。
28 //c++17提供std::optional<T> 是一个在标准库中提供的模板类，用于表示一个值可以有意义（存在）或者没有意义（不存在）的情况。
29 /* eg:
30 std::optional<int> findNumber(bool found) {
31 if (found) {
32 return 42; // 返回有效的值
33 }
34 return std::nullopt; // 返回无效值
35 }
36
37 int main() {
38 auto result = findNumber(true);
39 if (result) {
40 std::cout << "Found number: " << *result << "\n";
41 } else {
42 std::cout << "No number found.\n";
43 }
44 return 0;
45 }
46 */
47
48 //获取文本信息 默认获取的文本格式为utf-8（读文件）
49 QFile qf(path);
50 qf.open(QIODeviceBase::ReadOnly); //只读方式打开
51
52 QByteArray ba = qf.readAll(); //读取文件
53 //通过读取一行 qf.readLine() 读取全部
54 //QByteArray ba;
55 // while(!qf.atEnd())
56 // {
57 // ba += qf.readLine();
58 // }
59
60
61 qf.close(); //关闭文件
62 ui->textEdit->setText(QString::fromLocal8Bit(ba)); //会进行隐式转换
 QByteArray转存QString 将GBK转为UTF-8
63 // ui->textEdit->setText(ba);
64 //utf-8 转 GBK 数据.toLocal8Bit().data()
65

```

```

66
67 //写文件
68 QFile fw(path);
69 fw.open(QIODeviceBase::Append); //追加方式写入
70 fw.write("\n"); //换行
71 QString t("哈哈哈哈哈");
72 fw.write(t.toLocal8Bit().data()); //默认写入utf-8格式
73 fw.close();
74 });
75 }

```

Fence 97

## 14.1 QFileInfo对文件信息的读取

```

1 //widget.cpp续
2 //获取文本信息 默认获取的文本格式为utf-8（读文件）
3 QFile qf(path);
4 qf.open(QIODeviceBase::ReadOnly); //只读方式打开
5
6 QByteArray ba = qf.readAll(); //读取文件
7 //通过读取一行 qf.readLine() 读取全部
8 //QByteArray ba;
9 // while(!qf.atEnd())
10 // {
11 // ba += qf.readLine();
12 // }
13
14
15 qf.close(); //关闭文件
16 ui->textEdit->setText(QString::fromLocal8Bit(ba)); //会进行隐式转换
17 QByteArray转存QString 将GBK转为UTF-8
18 // ui->textEdit->setText(ba);
19 //utf-8 转 GBK 数据.toLocal8Bit().data()
20
21 //写文件
22 QFile fw(path);
23 fw.open(QIODeviceBase::Append); //追加方式写入
24 fw.write("\n"); //换行
25 QString t("哈哈哈哈哈");
26 fw.write(t.toLocal8Bit().data()); //默认写入utf-8格式
27 fw.close();
28
29
30
31 //读取文件的信息
32 QFileInfo fif(path);
33 qDebug() << "文件大小:" << fif.size() << " 后缀名:" << fif.suffix() << " 文件名" << fif.fileName() << " 文件路径:" << fif.filePath();
34
35 qDebug() << " 创建日期: " << fif.birthTime().toString("yyyy/MM/dd hh:mm:ss");

```

```
36 qDebug() << "最后修改日期:" << fif.metadataChangeTime().toString("yyyy/MM/dd
 hh:mm:ss");
```

Fence 98

## 15. 翻金币项目

### 15.1 主窗口

```
1 //mainscene.h
2 #ifndef MAINSCENE_H
3 #define MAINSCENE_H
4
5 #include <QMainWindow>
6 #include "chooselevelscene.h"
7
8 QT_BEGIN_NAMESPACE
9 namespace Ui {
10 class MainScene;
11 }
12 QT_END_NAMESPACE
13
14 class MainScene : public QMainWindow
15 {
16 Q_OBJECT
17
18 public:
19 MainScene(QWidget *parent = nullptr);
20 ~MainScene();
21
22
23 //重写绘图事件
24 void paintEvent(QPaintEvent *event);
25
26
27
28 //创建关卡场景
29 ChooseLevelScene *m_choose = NULL;
30 private:
31 Ui::MainScene *ui;
32 };
33 #endif // MAINSCENE_H
34
35
36
37 //mainscene.cpp
38 #include "mainscene.h"
39 #include "ui_mainscene.h"
40 #include "mypushbutton.h"
41 #include <QPainter>
42 #include <QPixmap>
```

```

43 #include <QImage>
44 #include <QTimer>
45 #include <QPushButton>
46 #include <QSoundEffect>
47 #include <QUrl>
48
49 MainScene::MainScene(QWidget *parent)
50 : QMainWindow(parent)
51 , ui(new Ui::MainScene)
52 {
53 ui->setupUi(this);
54
55 //设置窗口固定大小
56 this->setFixedSize(320,588);
57
58 //设置窗口图标
59 this->setWindowIcon(QIcon(":/img/Coin0001.png"));
60
61 //设置标题
62 this->setWindowTitle("翻金币");
63
64
65 /* 退出功能 */
66 connect(ui->quit,&QAction::triggered,this,&MainScene::close);
67
68
69 //准备按钮音效
70 QSoundEffect *sound = new QSoundEffect(this); //绑定父对象
71 sound->setSource(QUrl::fromLocalFile(":/img/TapButtonSound.wav"));
72
73
74 /* 设置start按钮 */
75 QString normalImg = ":/img/MenuSceneStartButton.png";
76 MyPushButton *startBtn = new MyPushButton(normalImg);
77 startBtn->setParent(this); //绑定父对象
78 int btnW = this->width() * 0.5 - startBtn->width() * 0.5;
79 int btnH = this->height() * 0.7;
80 startBtn->move(btnW,btnH); //移动按钮
81 this->setFocusPolicy(Qt::NoFocus); // 去除焦点边界 ‘虚线边框’
82
83 //实例化 关卡场景
84 this->m_choose = new ChooseLevelScene;
85
86 //监听back信号,进行返回
87 connect(this->m_choose,&ChooseLevelScene::chooseLevelSceneBack,[=]() {
88 QTimer::singleShot(500,this->m_choose,[=]() {
89 //保持前后窗口位置一致
90 this->setGeometry(this->m_choose->geometry());
91 this->show();
92 this->m_choose->hide();
93 });
94 });
95

```

```

96
97
98 /* 设置按钮按下效果 切换关卡 */
99 connect(startBtn,&MyPushButton::clicked,[=]() {
100 //弹起特效
101 startBtn->downAnim();
102 startBtn->upAnim();
103
104 //播放音频
105 sound->play();
106
107 //延时进入关卡场景
108 //参数1 延迟500ms 参数2 作用对象 参数3 延迟后进行的操作
109 QTimer::singleShot(500,this,[=]() {
110 //保持前后窗口位置一致
111 this->m_choose->setGeometry(this->geometry());
112 //自身隐藏
113 this->hide();
114
115 //显示关卡场景
116 this->m_choose->show();
117
118 });
119
120 });
121
122 }
123
124 MainScene::~MainScene()
125 {
126 delete ui;
127 }
128
129 //绘画事件
130 void MainScene::paintEvent(QPaintEvent *event)
131 {
132 /* 绘制背景图片 */
133 QPainter per(this); //创建画家 指定在窗口作画
134 QPixmap pix; //创建绘画设备
135 pix.load(":/img/PlayLevelSceneBg.png"); //加载文件
136 per.drawPixmap(0,0,this->width(),this->height(),pix); //作画 void
137 QPainter::drawPixmap(0,0,pix); //不会完全覆盖 按照原图比例作画
138
139 /* 绘制左上角标题图片 */
140 // pix.load(":/img/Title.png");
141 // pix = pix.scaled(pix.width() * 0.5, pix.height() * 0.5); //参数1: 原宽 *
142 // per.drawPixmap(10,30,pix); //缩放倍数 参数2: 原高 * 缩放倍数
143
144 QImage img;
145 img.load(":/img/Title.png");
146 img = img.scaled(img.width() * 0.5, img.height() * 0.5);
147 per.drawImage(10,30,img);

```

```
147 }
148
```

Fence 99

## 15.2 自定义按钮类

```
1 //mypushbutton.h
2 #ifndef MYPUSHBUTTON_H
3 #define MYPUSHBUTTON_H
4
5 #include <QPushButton>
6
7 class MyPushButton : public QPushButton
8 {
9 Q_OBJECT
10 public:
11 //explicit MyPushButton(QWidget *parent = nullptr);
12
13 //自己的构造函数
14 MyPushButton(QString normalImg, QString pressImg = "");
15
16 //按钮向下动的动画效果
17 void downAnim();
18
19 //按钮向上动的动画效果
20 void upAnim();
21
22 //重写鼠标按下事件
23 void mousePressEvent(QMouseEvent *ev);
24 //重写鼠标释放事件
25 void mouseReleaseEvent(QMouseEvent *ev);
26
27 QString m_normalImg; //正常按钮图片
28 QString m_pressImg; //按下按钮切换的图片
29 signals:
30
31
32 };
33
34 #endif // MYPUSHBUTTON_H
35
36
37 //mypushbutton.cpp
38 #include "mypushbutton.h"
39 #include <QMessageBox>
40 #include <QPropertyAnimation> //动画特效类
41
42 //normalImg 正常按钮 不要显示其他图片
43 //pressImg 按下按钮后显示的图片
44 MyPushButton::MyPushButton(QString normalImg, QString pressImg)
45 {
```

```

46 this->m_normalImg = normalImg;
47 this->m_pressImg = pressImg;
48
49 //处理正常状态显示的图片
50 QPixmap pix;
51 bool ret = pix.load(this->m_normalImg);
52
53 if(!ret)
54 {
55 QMessageBox::critical(this, "错误提示", "图片加载失败! ");
56 return;
57 }
58
59 //设置按钮固定大小
60 this->setFixedSize(pix.width(), pix.height());
61
62 //设置不规则图片样式
63 this->setStyleSheet("QPushButton{border:0px;}"); //里面为css
64
65 //设置图片
66 this->setIcon(QIcon(pix));
67
68 //设置图片大小
69 this->setIconSize(QSize(pix.width(), pix.height()));
70 }
71
72
73 //按钮向下动的动画效果
74 void MyPushButton::downAnim()
75 {
76 //参数1 作用的对象 参数2 动画属性 geometry 按照按钮的几何属性动画化
77 QPropertyAnimation * animation = new QPropertyAnimation(this, "geometry");
78
79 //设置动画时间间隔 200ms
80 animation->setDuration(200);
81
82 //设置起始位置 根据几何属性 参数可以是矩形框
83 animation->setStartValue(QRect(this->x(), this->y(), this->width(), this->
84 >height()));
85
86 //设置结束位置
87 animation->setEndValue(QRect(this->x(), this->y() + 10, this->width(), this->
88 >height()));
89
90 //设置弹跳曲线
91 animation->setEasingCurve(QEasingCurve::OutBounce);
92
93 //开始动画
94 animation->start();
95 }
96
97 //按钮向上动的动画效果

```

```

97 void MyPushButton::upAnim()
98 {
99 //参数1 作用的对象 参数2 动画属性 geometry 按照按钮的几何属性动画化
100 QPropertyAnimation * animation = new QPropertyAnimation(this,"geometry");
101
102 //设置动画时间间隔 200ms
103 animation->setDuration(200);
104
105 //设置起始位置 根据几何属性 参数可以是矩形框
106 animation->setStartValue(QRect(this->x(),this->y() + 10,this->width(),this->
>height()));
107
108 //设置结束位置
109 animation->setEndValue(QRect(this->x(),this->y(),this->width(),this->height()));
110
111 //设置弹跳曲线
112 animation->setEasingCurve(QEasingCurve::OutBounce);
113
114 //开始动画
115 animation->start();
116 }
117
118 //重载鼠标按下事件
119 void MyPushButton::mousePressEvent(QMouseEvent *ev)
120 {
121 //传入的状态不为空 说明需要有按下状态，切换到初始图片图片
122 if(this->m_pressImg != "")
123 {
124 //处理正常状态显示的图片
125 QPixmap pix;
126 bool ret = pix.load(this->m_pressImg);
127
128 if(!ret)
129 {
130 QMessageBox::critical(this,"点击事件错误提示","图片加载失败!");
131 return;
132 }
133
134 //设置按钮固定大小
135 this->setFixedSize(pix.width(),pix.height());
136
137 //设置不规则图片样式
138 this->setStyleSheet("QPushButton{border:0px;}"); //里面为css
139
140 //设置图片
141 this->setIcon(QIcon(pix));
142
143 //设置图片大小
144 this->setIconSize(QSize(pix.width(),pix.height()));
145 }
146
147 //其他操作让父类完成
148 return QPushButton::mousePressEvent(ev);

```

```

149 }
150
151 //重载鼠标释放事件
152 void MyPushButton::mouseReleaseEvent(QMouseEvent *ev)
153 {
154 //传入的状态不为空 说明需要有按下状态, 切换图片
155 if(this->m_pressImg != "")
156 {
157 //处理正常状态显示的图片
158 QPixmap pix;
159 bool ret = pix.load(this->m_normalImg);
160
161 if(!ret)
162 {
163 QMessageBox::critical(this, "释放错误提示", "图片加载失败!");
164 return;
165 }
166
167 //设置按钮固定大小
168 this->setFixedSize(pix.width(), pix.height());
169
170 //设置不规则图片样式
171 this->setStyleSheet("QPushButton{border:0px;}"); //里面为css
172
173 //设置图片
174 this->setIcon(QIcon(pix));
175
176 //设置图片大小
177 this->setIconSize(QSize(pix.width(), pix.height()));
178 }
179
180 //其他操作让父类完成
181 return QPushButton::mouseReleaseEvent(ev);
182 }
183
184

```

Fence 100

声明带默认参数, 实现不需要

## 15.3 选择关卡类

```

1 //chooselevelscene.h
2 #ifndef CHOOSELEVELSCENE_H
3 #define CHOOSELEVELSCENE_H
4
5 #include <QMainWindow>
6 #include "palyscene.h"
7
8 class ChooseLevelScene : public QMainWindow
9 {

```

```

10 Q_OBJECT
11 public:
12 explicit ChooseLevelScene(QWidget *parent = nullptr);
13
14 //重写绘图事件
15 void paintEvent(QPaintEvent *event);
16
17
18 PalyScene *paly = NULL;
19
20
21 signals:
22 //发出一个返回主窗口信号
23 void chooseLevelSceneBack();
24 };
25
26 #endif // CHOOSELEVELSCENE_H
27
28
29 //chooselevelscene.cpp
30 #include "chooselevelscene.h"
31 #include "mainscene.h"
32 #include "mypushbutton.h"
33 #include <QMenuBar>
34 #include <QMainWindow>
35 #include <QPixmap>
36 #include <QPainter>
37 #include <QTimer>
38 #include <QMouseEvent>
39 #include <QLabel>
40 #include <QDebug>
41 #include <QSoundEffect>
42 #include <QUrl>
43
44 ChooseLevelScene::ChooseLevelScene(QWidget *parent)
45 : QMainWindow{parent}
46 {
47 //设置窗口固定大小
48 this->setFixedSize(320,588);
49
50 //设置窗口图标
51 this->setWindowIcon(QIcon(":/img/Coin0001.png"));
52
53 //设置标题
54 this->setWindowTitle("翻金币-关卡选择");
55
56 //创建菜单栏
57 QMenuBar *bar = new QMenuBar(this);
58 this->setMenuBar(bar); //放入到当前窗口
59
60 //创建菜单
61 QMenu *startMenu = bar->addMenu("菜单");
62

```

```

63 //创建菜单项
64 QAction *exitAction = startMenu->addAction("退出");
65
66 //退出功能
67 connect(exitAction,&QAction::triggered,this,&QWidget::close);
68
69 //创建音效
70 QSoundEffect *choo = new QSoundEffect(this);
71 choo->setSource(QUrl::fromLocalFile(":/img/TapButtonSound.wav"));
72 QSoundEffect *chooBack = new QSoundEffect(this);
73 chooBack->setSource(QUrl::fromLocalFile(":/img/BackButtonSound.wav"));
74
75 //创建back按钮
76 MyPushButton *backBtn = new
MyPushButton(":/img/BackButton.png",":/img/BackButtonSelected.png");
77 backBtn->setParent(this);
78 backBtn->move(QPoint(this->width() - backBtn->width(), this->height() - backBtn-
>height()));
79
80
81 //返回主界面
82 connect(backBtn,&MyPushButton::clicked,[=]() {
83 chooBack->play();
84 //发出信号
85 emit this->chooseLevelSceneBack();
86 });
87
88
89
90
91 //创建关卡按钮
92 for(int i = 0; i < 20; i++)
93 {
94 MyPushButton *menuBtn = new MyPushButton(":/img/LevelIcon.png");
95 menuBtn->setParent(this);
96 menuBtn->move(i % 4 * 70 + 25, i / 4 * 70 + 130); //取余控制层
数，取商控制列数，乘数控制间隔，家数控制离窗口间距
97
98 //监听按钮
99 connect(menuBtn,&MyPushButton::clicked,[=]() { //通过new存放
在堆区，虽然名字都是menuBtn，但是地址不同
100 //设置动画
101 menuBtn->downAnim();
102 menuBtn->upAnim();
103
104 //设置音效
105 choo->play();
106
107 //进入游戏场景
108
109 paly = new PalyScene(i + 1); //实例化游戏场景
110 this->hide(); //隐藏本界面
111 //设置游戏窗口初始位置

```

```

112 paly->setGeometry(this->geometry());
113
114 paly->show(); //显示游戏场景
115
116
117
118
119 //监听游戏场景的Back信号
120 connect(paly,&PalyScene::palySceneBack,[=]() {
121 //保持前后窗口位置一致
122 this->setGeometry(paly->geometry());
123 this->show();
124 delete paly; //摧毁paly对象 让下一次进入新的关卡
125 paly = NULL;
126 });
127 });
128
129
130
131 //浮现文字
132 // menuBtn->setText(QString::number(i + 1)); //不能在图片上显示
133 QLabel *label = new QLabel(this);
134 label->move(i % 4 * 70 + 25, i / 4 * 70 + 130);
135 label->setFixedSize(menuBtn->width(),menuBtn->height()); //设置大小
136 label->setAlignment(Qt::AlignHCenter | Qt::AlignVCenter); //设置文本对齐方式
 水平和垂直居中
137 label->setText(QString::number(i + 1));
138
139 //因为Label标签挡住了按钮，所以需要鼠标穿透事件
140 label->setAttribute(Qt::WA_TransparentForMouseEvents); //鼠标穿透属性 值为
51
141 }
142
143
144 }
145
146
147 //重写绘图事件
148 void ChooseLevelScene::paintEvent(QPaintEvent *event)
149 {
150 //设置背景图
151 QPainter p(this);
152 QPixmap pix;
153 pix.load(":/img/OtherSceneBg.png");
154 p.drawPixmap(0,0,this->width(),this->height(),pix);
155
156 //设置背景标题
157 pix.load(":/img/Title.png");
158 p.drawPixmap(((this->width() - pix.width()) *
0.5),30,pix.width(),pix.height(),pix);
159
160 }
161

```

## 15.4 翻金市场景

```
1 //palyscene.h
2 #ifndef PALYSCENE_H
3 #define PALYSCENE_H
4
5 #include <QMainWindow>
6 #include "mycoin.h"
7
8 class Palyscene : public QMainWindow
9 {
10 Q_OBJECT
11 public:
12 //explicit Palyscene(QWidget *parent = nullptr);
13
14 Palyscene(int i);
15
16 //重写绘图事件
17 void paintEvent(QPaintEvent *event);
18
19
20
21 int m_id = 0; //记录游戏关卡
22 int m_coinGrou[4][4]; //用于维护每关数组的二维数组
23 MyCoin * m_coinArray[4][4]; //用于维护每个按钮的属性
24
25 bool m_iswinP = false;
26
27 signals:
28 void palySceneBack();
29 };
30
31 #endif // PALYSCENE_H
32
33
34 //palyscene.cpp
35 #include "palyscene.h"
36 #include "mypushbutton.h"
37 #include "mycoin.h"
38 #include "dataconfig.h"
39 #include <QPixmap>
40 #include <QPainter>
41 #include <QMenuBar>
42 #include <QImage>
43 #include <QLabel>
44 #include <QFont>
45 #include <QTimer>
46 #include <QPropertyAnimation>
```

```

47 #include <QSoundEffect>
48 #include <QUrl>
49
50 PalyScene::PalyScene(int i)
51 {
52 this->m_id = i;
53 QString str = QString("第 %1 关翻金币").arg(m_id);
54 setWindowTitle(str); //设置窗口标题
55
56 setFixedSize(320,588);
57 this->setWindowIcon(QIcon(":/img/Coin0001.png"));
58
59 //设置back按钮
60 MyPushButton *backBtn = new
MyPushButton(":/img/BackButton.png", ":/img/BackButtonSelected.png");
61 backBtn->setParent(this);
62 backBtn->move(QPoint(this->width() - backBtn->width(), this->height() - backBtn-
>height()));
63
64 //创建音效
65 QSoundEffect *palyBack = new QSoundEffect(this);
66 palyBack->setSource(QUrl::fromLocalFile(":/img/BackButtonSound.wav"));
67 QSoundEffect *palyCoin = new QSoundEffect(this);
68 palyCoin->setSource(QUrl::fromLocalFile(":/img/ConFlipSound.wav"));
69 QSoundEffect *palywin = new QSoundEffect(this);
70 palywin->setSource(QUrl::fromLocalFile(":/img/LevelWinSound.wav"));
71
72 //返回主界面
73 connect(backBtn,&MyPushButton::clicked,[=]() {
74 palyBack->play();
75 //发出信号
76 emit this->palySceneBack();
77 });
78
79 //创建菜单栏
80 QMenuBar *bar = new QMenuBar(this);
81 this->setMenuBar(bar); //放入到当前窗口
82
83 //创建菜单
84 QMenu *startMenu = bar->addMenu("菜单");
85
86 //创建菜单项
87 QAction *exitAction = startMenu->addAction("退出");
88
89 //退出功能
90 connect(exitAction,&QAction::triggered,this,&QWidget::close);
91
92
93 //显示当前关卡数
94 QLabel *label = new QLabel();
95 label->setParent(this);
96 //设置字体样式, 大小
97 QFont font;

```

```

98 font.setFamily("华文新魏");
99 font.setPointSize(20);
100 label->setFont(font);
101
102 QString str2 = QString("关卡: %1").arg(m_id);
103 label->setText(str2);
104 label->setGeometry(QRect(30,this->height() - 50,120,50)); //设置QLabel形状
105
106
107 //创建好胜利图片
108 QLabel *winLabel = new QLabel();
109 QPixmap tpix;
110 tpix.load(":/img/LevelCompletedDialogBg.png");
111 winLabel->setGeometry(0,0,tpix.width(),tpix.height());
112 winLabel->setPixmap(tpix);
113 winLabel->setParent(this);
114 winLabel->move((this->width() - tpix.width())*0.5,-tpix.height()); //移
到窗口外
115
116 //初始化维护数组
117 dataConfig da;
118 for(int i = 0; i < 4; i++)
119 {
120 for(int j = 0; j < 4; j++)
121 {
122 this->m_coinGrou[i][j] = da.m_data[this->m_id][i][j];
123 }
124 }
125
126 //显示金币背景图案
127 for(int i = 0; i < 4; i++)
128 {
129 for(int j = 0; j < 4; j++)
130 {
131 QLabel *la = new QLabel();
132 QPixmap pix = QPixmap(":/img/BoardNode(1).png");
133 la->setGeometry(0,0,pix.width(),pix.height());
134 la->setPixmap(pix);
135 la->setParent(this);
136
137 la->move(57 + i * 50,200 + j * 50);
138
139 // QLabel la(this); //创建在栈上，作用域完了，就消失
140
141
142 //创建金币并显示
143 QString str3;
144 if(this->m_coinGrou[i][j] == 1)
145 {
146 str3 = ":/img/Coin0001.png";
147 }
148 else
149 {

```

```

150 str3 = ":/img/Coin0008.png";
151 }
152
153 MyCoin *coin = new MyCoin(str3);
154 coin->setParent(this);
155 coin->move(59 + i * 50, 204 + j * 50);
156
157 //给金币属性赋值
158 coin->m_x = i;
159 coin->m_y = j;
160 coin->m_falg = this->m_coinGrou[i][j];
161
162
163 //更新维护按钮数组
164 this->m_coinArray[i][j] = coin;
165
166 //监听金币
167 connect(coin, &MyCoin::clicked, [=]() {
168 //消除用户快速点击时，动画没完成，出现贴图错误的第二中方式
169 // if(coin->m_isAnimation)
170 // return;
171 // else
172 // {
173 // coin->coinTurn();
174 // this->m_coinGrou[i][j] = this->m_coinGroup[i][j] == 0 ? 1 :
0;
175 // }
176
177
178 //防止胜利，用户快速点击其他按钮，导致出现bug
179 for(int i = 0; i < 4; i++)
180 {
181 for(int j = 0; j < 4; j++)
182 {
183 this->m_coinArray[i][j]->m_judge = true;
184 }
185 }
186
187 palyCoin->play();
188
189 coin->coinTurn();
190 //更新维护数组
191 this->m_coinGrou[i][j] = this->m_coinGrou[i][j] == 0 ? 1 : 0;
192
193
194 // QTimer::singleShot 不会阻塞进程。它用于在一定时间后执行指定的槽，所以胜利检
查需要写在这个下面 否则检查的是，周围还没有翻转的情况
195 QTimer::singleShot(300, this, [=]() {
196
197 //翻转右侧
198 if(coin->m_x + 1 <= 3)
199 {
200 this->m_coinArray[coin->m_x + 1][coin->m_y]->coinTurn();

```

```

201 this->m_coinGrou[coin->m_x + 1][coin->m_y] = this-
>m_coinGrou[coin->m_x + 1][coin->m_y] == 0 ? 1 : 0;
202 }
203 //翻转左侧
204 if(coin->m_x - 1 >= 0)
205 {
206 this->m_coinArray[coin->m_x - 1][coin->m_y]->coinTurn();
207 this->m_coinGrou[coin->m_x - 1][coin->m_y] = this-
>m_coinGrou[coin->m_x - 1][coin->m_y] == 0 ? 1 : 0;
208 }
209 //翻转上面
210 if(coin->m_y - 1 >= 0)
211 {
212 this->m_coinArray[coin->m_x][coin->m_y - 1]->coinTurn();
213 this->m_coinGrou[coin->m_x][coin->m_y - 1] = this-
>m_coinGrou[coin->m_x][coin->m_y - 1] == 0 ? 1 : 0;
214 }
215 //翻转下面
216 if(coin->m_y + 1 <= 3)
217 {
218 this->m_coinArray[coin->m_x][coin->m_y + 1]->coinTurn();
219 this->m_coinGrou[coin->m_x][coin->m_y + 1] = this-
>m_coinGrou[coin->m_x][coin->m_y + 1] == 0 ? 1 : 0;
220 }
221
222 //还有状元
223 for(int i = 0; i < 4; i++)
224 {
225 for(int j = 0; j < 4; j++)
226 {
227 this->m_coinArray[i][j]->m_judge = false;
228 }
229 }
230
231 //检查胜利
232 this->m_iswinP = true; //假设胜利
233 for(int i = 0; i < 4; i++)
234 {
235 for(int j = 0; j < 4; j++)
236 {
237 if(this->m_coinArray[i][j]->m_falg == false) //存
在一个银币 没有胜利
238 {
239 this->m_iswinP = false;
240 break;
241 }
242 }
243
244 if(this->m_iswinP == false)
245 break;
246 }
247
248 //让每个按钮下的都有胜利属性

```

```

249 if(this->m_iswinP == true)
250 {
251 palywin->play();
252
253 for(int i = 0; i < 4; i++)
254 {
255 for(int j = 0; j < 4; j++)
256 {
257 this->m_coinArray[i][j]->m_iswin = true;
258 }
259 }
260
261
262 //动画效果的移回胜利窗口
263 QPropertyAnimation *anim = new
QPropertyAnimation(winLabel, "geometry");
264 anim->setDuration(1000); //设置动画时间
265 anim->setStartValue(QRect(winLabel->x(), winLabel-
>y(), winLabel->width(), winLabel->height())); //设置开始位置
266 anim->setEndValue(QRect(winLabel->x(), winLabel->y() +
114, winLabel->width(), winLabel->height())); //设置结束位置
267 anim->setEasingCurve(QEasingCurve::OutBounce);
//设置缓和曲线
268
269 anim->start();
270
271
272 }
273 });
274 \
275 });
276 }
277 }
278
279 }
280
281
282 //重写绘图事件
283 void PalyScene::paintEvent(QPaintEvent *event)
284 {
285 QPixmap pix;
286 QPainter p(this);
287 pix.load(":/img/PlayLevelSceneBg.png"); //加载
288 p.drawPixmap(0,0,this->width(),this->height(),pix);
289
290 QImage img;
291 img.load(":/img/Title.png");
292 img = img.scaled(img.width() * 0.5, img.height() * 0.5);
293 p.drawImage(10,30,img);
294 }
295

```

## 15.5 金币类

```
1 //mycoin.h
2 #ifndef MYCOIN_H
3 #define MYCOIN_H
4
5 #include <QPushButton>
6 #include <QTimer>
7
8 class MyCoin : public QPushButton
9 {
10 Q_OBJECT
11 public:
12 //explicit MyCoin(QWidget *parent = nullptr);
13
14 //传入显示的图片
15 MyCoin(QString btnImg);
16
17 //改变标志函数
18 void coinTurn();
19
20 //重写 点击 事件 第一种消除用户快速点击时，动画没完成，出现贴图错误方法
21 void mousePressEvent(QMouseEvent *e);
22
23 int m_x; //记录行
24 int m_y; //记录列
25 bool m_falg; //记录正方
26
27 int m_min = 1; //图片的最小值
28 int m_max = 8; //图片的最大值
29 QTimer *m_t1; //定时器1 记录正翻反
30 QTimer *m_t2; //定时器2 记录反翻正
31
32 bool m_isAnimation = false; //执行动画的标志 默认没有 防止用户快速点击时，动画没完成，
 出现贴图错误
33 bool m_iswin = false; //胜利状态
34 bool m_judge = false; //记录胜利状态 防止有人点快出现bug
35 signals:
36 };
37
38 #endif // MYCOIN_H
39
40 //mycoin.cpp
41 #include "mycoin.h"
42 #include <QPixmap>
43 #include <QMessageBox>
44
45 MyCoin::MyCoin(QString btnImg)
46 {
47 QPixmap pix;
48 bool rel = pix.load(btnImg);
49 }
```

```

50 if(!rel)
51 {
52 QMessageBox::critical(this,"金币 错误提示框","图片加载失败");
53 return;
54 }
55
56 this->setFixedSize(pix.width(),pix.height());
57 this->setStyleSheet("QPushButton{border:0px;}");
58 this->setIconSize(QSize(pix.width(),pix.height()));
59 this->setIcon(QIcon(pix));
60
61 //初始化定时器
62 this->m_t1 = new QTimer(this);
63 this->m_t2 = new QTimer(this);
64
65 //监听定时器
66 connect(this->m_t1,&QTimer::timeout,[=]() {
67 QPixmap pix1;
68 pix1.load(QString(":/img/Coin000%1.png").arg(this->m_min++));
69 this->setFixedSize(pix1.width(),pix1.height());
70 this->setStyleSheet("QPushButton{border:0px;}");
71 this->setIconSize(QSize(pix1.width(),pix1.height()));
72 this->setIcon(QIcon(pix1));
73
74 this->m_t1->start(30);
75
76 //翻完
77 if(this->m_min > this->m_max)
78 {
79 this->m_min = 1;
80 this->m_t1->stop();
81 this->m_isAnimation = false;
82 }
83
84 });
85
86
87 connect(this->m_t2,&QTimer::timeout,[=]() {
88 QPixmap pix1;
89
90 pix1.load(QString(":/img/Coin000%1.png").arg(this->m_max--));
91 this->setFixedSize(pix1.width(),pix1.height());
92 this->setStyleSheet("QPushButton{border:0px;}");
93 this->setIconSize(QSize(pix1.width(),pix1.height()));
94 this->setIcon(QIcon(pix1));
95
96 this->m_t2->start(30);
97
98 if(this->m_min > this->m_max)
99 {
100 this->m_max = 8;
101 this->m_t2->stop();
102 this->m_isAnimation = false;

```

```

103 }
104
105 });
106 }
107
108 //改变标志函数
109 void MyCoin::coinTurn()
110 {
111
112 //正翻反
113 if(this->m_falg)
114 {
115 //启动定时器1
116 this->m_t1->start(30);
117 this->m_falg = !this->m_falg;
118 this->m_isAnimation = true;
119 }
120 else
121 {
122 //启动定时器2
123 this->m_t2->start(30);
124 this->m_falg = !this->m_falg;
125 this->m_isAnimation = true;
126 }
127 }
128
129 void MyCoin::mousePressEvent(QMouseEvent *e)
130 {
131 //如果进动画 或者 胜利 或者 改变周围方块时 将不能点击按钮
132 if(this->m_isAnimation || this->m_iswin || this->m_judge)
133 return;
134 else
135 QPushButton::mousePressEvent(e);
136 }
137
138
139

```

Fence 103

## 15.6 关卡数据类

```

1 //dataconfig,h
2 #ifndef DATACONFIG_H
3 #define DATACONFIG_H
4
5 #include <QObject>
6 #include <QMap>
7 #include <QVector>
8
9 class dataConfig : public QObject
10 {
11 Q_OBJECT

```

```

12 public:
13 explicit dataConfig(QObject *parent = nullptr);
14
15
16
17 QMap<int, QVector< QVector<int>>> m_data; //map中的int表示关卡号
18 QVector< QVector<int>>>表示金币的二维数组
19 signals:
20 };
21 #endif // DATACONFIG_H
22
23
24 //dataconfig.cpp
25 #include "dataconfig.h"
26
27
28 dataConfig::dataConfig(QObject *parent)
29 : QObject{parent}
30 {
31 //创建第一关的模板
32 //1为金币 0为银币
33 int array1[4][4] = {{1, 1, 1, 1},
34 {1, 1, 0, 1},
35 {1, 0, 0, 0},
36 {1, 1, 0, 1} } ;
37
38 QVector< QVector<int>> v;
39 for(int i = 0 ; i < 4;i++)
40 {
41 QVector<int>v1;
42 for(int j = 0 ; j < 4;j++)
43 {
44
45 v1.push_back(array1[i][j]);
46 }
47 v.push_back(v1);
48 }
49
50 m_data.insert(1,v);
51
52
53 int array2[4][4] = { {1, 0, 1, 1},
54 {0, 0, 1, 1},
55 {1, 1, 0, 0},
56 {1, 1, 0, 1} } ;
57
58 v.clear();
59 for(int i = 0 ; i < 4;i++)
60 {
61 QVector<int>v1;
62 for(int j = 0 ; j < 4;j++)
63 {

```

```

64 v1.push_back(array2[i][j]);
65 }
66 v.push_back(v1);
67 }
68
69 m_data.insert(2,v);
70
71
72
73 int array3[4][4] = { {0, 0, 0, 0},
74 {0, 1, 1, 0},
75 {0, 1, 1, 0},
76 {0, 0, 0, 0}} ;
77 v.clear();
78 for(int i = 0 ; i < 4;i++)
79 {
80 QVector<int>v1;
81 for(int j = 0 ; j < 4;j++)
82 {
83 v1.push_back(array3[i][j]);
84 }
85 v.push_back(v1);
86 }
87
88 m_data.insert(3,v);
89
90
91 int array4[4][4] = { {0, 1, 1, 1},
92 {1, 0, 0, 1},
93 {1, 0, 1, 1},
94 {1, 1, 1, 1}} ;
95 v.clear();
96 for(int i = 0 ; i < 4;i++)
97 {
98 QVector<int>v1;
99 for(int j = 0 ; j < 4;j++)
100 {
101 v1.push_back(array4[i][j]);
102 }
103 v.push_back(v1);
104 }
105
106 m_data.insert(4,v);
107
108
109 int array5[4][4] = { {1, 0, 0, 1},
110 {0, 0, 0, 0},
111 {0, 0, 0, 0},
112 {1, 0, 0, 1}} ;
113 v.clear();
114 for(int i = 0 ; i < 4;i++)
115 {
116 QVector<int>v1;

```

```
117 for(int j = 0 ; j < 4;j++)
118 {
119 v1.push_back(array5[i][j]);
120 }
121 v.push_back(v1);
122 }
123
124 m_data.insert(5,v);
125
126
127 int array6[4][4] = { {1, 0, 0, 1},
128 {0, 1, 1, 0},
129 {0, 1, 1, 0},
130 {1, 0, 0, 1}} ;
131
132 v.clear();
133 for(int i = 0 ; i < 4;i++)
134 {
135 QVector<int>v1;
136 for(int j = 0 ; j < 4;j++)
137 {
138 v1.push_back(array6[i][j]);
139 }
140 v.push_back(v1);
141 }
142
143 m_data.insert(6,v);
144
145
146 int array7[4][4] = { {0, 1, 1, 1},
147 {1, 0, 1, 1},
148 {1, 1, 0, 1},
149 {1, 1, 1, 0}} ;
150
151 v.clear();
152 for(int i = 0 ; i < 4;i++)
153 {
154 QVector<int>v1;
155 for(int j = 0 ; j < 4;j++)
156 {
157 v1.push_back(array7[i][j]);
158 }
159 v.push_back(v1);
160 }
161
162 m_data.insert(7,v);
163
164
165 int array8[4][4] = { {0, 1, 0, 1},
166 {1, 0, 0, 0},
167 {0, 0, 0, 1},
168 {1, 0, 1, 0}} ;
169
170 v.clear();
171 for(int i = 0 ; i < 4;i++)
172 {
173 QVector<int>v1;
```

```

170 for(int j = 0 ; j < 4;j++)
171 {
172 v1.push_back(array8[i][j]);
173 }
174 v.push_back(v1);
175 }
176
177 m_data.insert(8,v);
178
179 int array9[4][4] = { {1, 0, 1, 0},
180 {1, 0, 1, 0},
181 {0, 0, 1, 0},
182 {1, 0, 0, 1}} ;
183 v.clear();
184 for(int i = 0 ; i < 4;i++)
185 {
186 QVector<int>v1;
187 for(int j = 0 ; j < 4;j++)
188 {
189 v1.push_back(array9[i][j]);
190 }
191 v.push_back(v1);
192 }
193
194 m_data.insert(9,v);
195
196
197
198 int array10[4][4] = { {1, 0, 1, 1},
199 {1, 1, 0, 0},
200 {0, 0, 1, 1},
201 {1, 1, 0, 1}} ;
202 v.clear();
203 for(int i = 0 ; i < 4;i++)
204 {
205 QVector<int>v1;
206 for(int j = 0 ; j < 4;j++)
207 {
208 v1.push_back(array10[i][j]);
209 }
210 v.push_back(v1);
211 }
212
213 m_data.insert(10,v);
214
215
216 int array11[4][4] = { {0, 1, 1, 0},
217 {1, 0, 0, 1},
218 {1, 0, 0, 1},
219 {0, 1, 1, 0}} ;
220 v.clear();
221 for(int i = 0 ; i < 4;i++)
222 {

```

```

223 QVector<int>v1;
224 for(int j = 0 ; j < 4;j++)
225 {
226 v1.push_back(array11[i][j]);
227 }
228 v.push_back(v1);
229 }
230
231 m_data.insert(11,v);
232
233 int array12[4][4] = { {0, 1, 1, 0},
234 {0, 0, 0, 0},
235 {1, 1, 1, 1},
236 {0, 0, 0, 0}} ;
237 v.clear();
238 for(int i = 0 ; i < 4;i++)
239 {
240 QVector<int>v1;
241 for(int j = 0 ; j < 4;j++)
242 {
243 v1.push_back(array12[i][j]);
244 }
245 v.push_back(v1);
246 }
247
248 m_data.insert(12,v);
249
250
251 int array13[4][4] = { {0, 1, 1, 0},
252 {0, 0, 0, 0},
253 {0, 0, 0, 0},
254 {0, 1, 1, 0}} ;
255 v.clear();
256 for(int i = 0 ; i < 4;i++)
257 {
258 QVector<int>v1;
259 for(int j = 0 ; j < 4;j++)
260 {
261 v1.push_back(array13[i][j]);
262 }
263 v.push_back(v1);
264 }
265
266 m_data.insert(13,v);
267
268 int array14[4][4] = { {1, 0, 1, 1},
269 {0, 1, 0, 1},
270 {1, 0, 1, 0},
271 {1, 1, 0, 1}} ;
272 v.clear();
273 for(int i = 0 ; i < 4;i++)
274 {
275 QVector<int>v1;

```

```

276 for(int j = 0 ; j < 4;j++)
277 {
278 v1.push_back(array14[i][j]);
279 }
280 v.push_back(v1);
281 }
282
283 m_data.insert(14,v);
284
285
286 int array15[4][4] = { {0, 1, 0, 1},
287 {1, 0, 0, 0},
288 {1, 0, 0, 0},
289 {0, 1, 0, 1}} ;
290
291 v.clear();
292 for(int i = 0 ; i < 4;i++)
293 {
294 QVector<int>v1;
295 for(int j = 0 ; j < 4;j++)
296 {
297 v1.push_back(array15[i][j]);
298 }
299 v.push_back(v1);
300 }
301
302 m_data.insert(15,v);
303
304
305 int array16[4][4] = { {0, 1, 1, 0},
306 {1, 1, 1, 1},
307 {1, 1, 1, 1},
308 {0, 1, 1, 0}} ;
309
310 v.clear();
311 for(int i = 0 ; i < 4;i++)
312 {
313 QVector<int>v1;
314 for(int j = 0 ; j < 4;j++)
315 {
316 v1.push_back(array16[i][j]);
317 }
318 v.push_back(v1);
319 }
320
321 m_data.insert(16,v);
322
323
324 int array17[4][4] = { {0, 1, 1, 1},
325 {0, 1, 0, 0},
326 {0, 0, 1, 0},
327 {1, 1, 1, 0}} ;
328
329 v.clear();
330 for(int i = 0 ; i < 4;i++)
331 {
332 QVector<int>v1;

```

```
329 for(int j = 0 ; j < 4;j++)
330 {
331 v1.push_back(array17[i][j]);
332 }
333 v.push_back(v1);
334 }
335
336 m_data.insert(17,v);
337
338
339 int array18[4][4] = { {0, 0, 0, 1},
340 {0, 0, 1, 0},
341 {0, 1, 0, 0},
342 {1, 0, 0, 0}} ;
343
344 v.clear();
345 for(int i = 0 ; i < 4;i++)
346 {
347 QVector<int>v1;
348 for(int j = 0 ; j < 4;j++)
349 {
350 v1.push_back(array18[i][j]);
351 }
352 v.push_back(v1);
353 }
354
355 m_data.insert(18,v);
356
357 int array19[4][4] = { {0, 1, 0, 0},
358 {0, 1, 1, 0},
359 {0, 0, 1, 1},
360 {0, 0, 0, 0}} ;
361
362 v.clear();
363 for(int i = 0 ; i < 4;i++)
364 {
365 QVector<int>v1;
366 for(int j = 0 ; j < 4;j++)
367 {
368 v1.push_back(array19[i][j]);
369 }
370 v.push_back(v1);
371 }
372
373 m_data.insert(19,v);
374
375 int array20[4][4] = { {0, 0, 0, 0},
376 {0, 0, 0, 0},
377 {0, 0, 0, 0},
378 {0, 0, 0, 0}} ;
379
380 v.clear();
381 for(int i = 0 ; i < 4;i++)
382 {
383 QVector<int>v1;
384 for(int j = 0 ; j < 4;j++)
```

```

382 {
383 v1.push_back(array20[i][j]);
384 }
385 v.push_back(v1);
386 }
387
388 m_data.insert(20,v);
389
390
391 }
392

```

Fence 104

## 16. 打包

### 16.1 步骤1

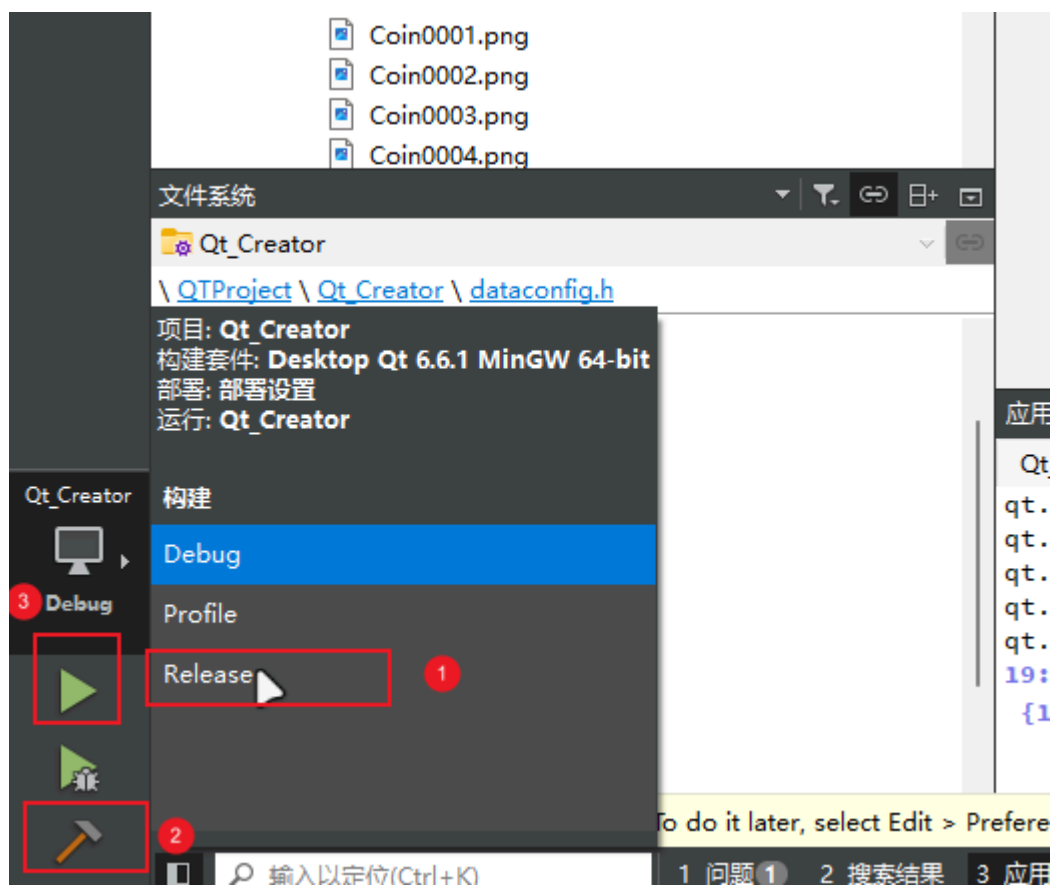


Figure 77

记得运行程序，检查是否正常。

16.2 步骤2(只做到这一步，需要使用者有对应的qt环境)

在项目的上一级目录，有Debug（调式）版本和Release（发布）版本，复制Release文件夹里面的.exe文件

|                                                           |                  |     |
|-----------------------------------------------------------|------------------|-----|
| build-Qt_Creator-Desktop_Qt_6_6_1_MinGW_64_bit-Debug      | 2025/3/5 13:54   | 文件夹 |
| build-Qt_Creator-Desktop_Qt_6_6_1_MinGW_64_bit-Release    | 2025/3/6 19:36   | 文件夹 |
| build-Signal_And_Slot-Desktop_Qt_6_6_1_MinGW_64_bit-Debug | 2023/12/17 20:43 | 文件夹 |
| build-student-Desktop_Qt_6_6_1_MinGW_64_bit-Debug         | 2023/12/31 21:21 | 文件夹 |
| Qt_Creator                                                | 2025/3/6 19:20   | 文件夹 |

Figure 78

|                                                                                              |                |      |          |
|----------------------------------------------------------------------------------------------|----------------|------|----------|
| 电脑 > 新加卷 (E:) > QTProject > build-Qt_Creator-Desktop_Qt_6_6_1_MinGW_64_bit-Release > release |                |      |          |
| <input type="checkbox"/> 名称                                                                  | 修改日期           | 类型   | 大小       |
| <input checked="" type="checkbox"/> Qt_Creator.exe                                           | 2025/3/6 19:36 | 应用程序 | 1,799 KB |

Figure 79

16.3 步骤3(这一步之后，就可以将这个文件夹压缩，发给使用者)

将复制的.exe文件,复制到新建的一个文件夹中

在该文件夹下，打开cmd 输入 windeployqt.exe 项目.exe 例如：windeployqt.exe Qt\_Creator.exe

确保在qt安装目录下能找到windeployqt.exe程序，E:\Qt\6.6.1\mingw\_64\bin\windeployqt.exe

|                                                    |                |      |          |
|----------------------------------------------------|----------------|------|----------|
| 电脑 > 新加卷 (E:) > QTProject > Qt_Creator小游戏打包        |                |      |          |
| <input type="checkbox"/> 名称                        | 修改日期           | 类型   | 大小       |
| <input checked="" type="checkbox"/> Qt_Creator.exe | 2025/3/6 19:36 | 应用程序 | 1,799 KB |

Figure 80

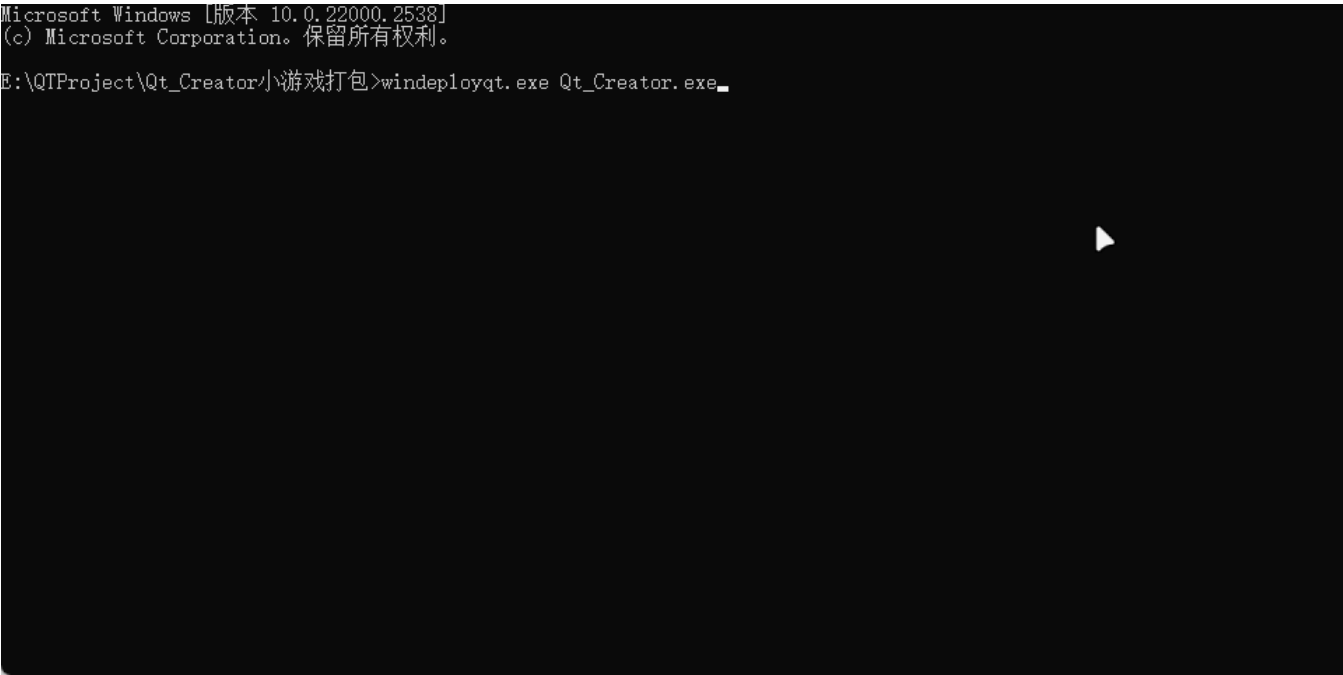


Figure 81

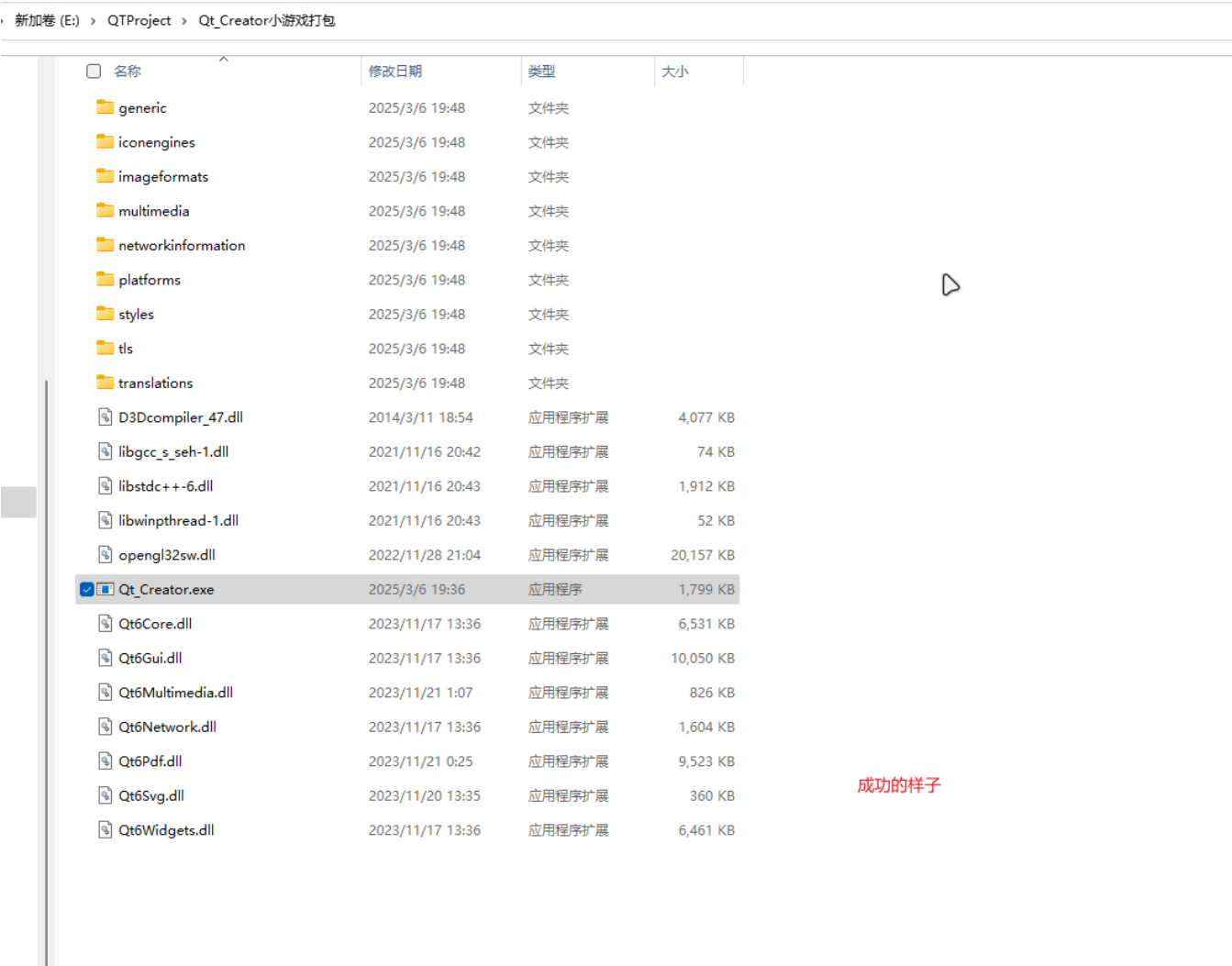


Figure 82

# 16.4 步骤4（可省略 使用第三方，形成安装程序 固定生成的是setup.exe的安装包）

- 1. 安装nsis与hm nis edit第三方打包软件
- 2. 启动hm nis edit

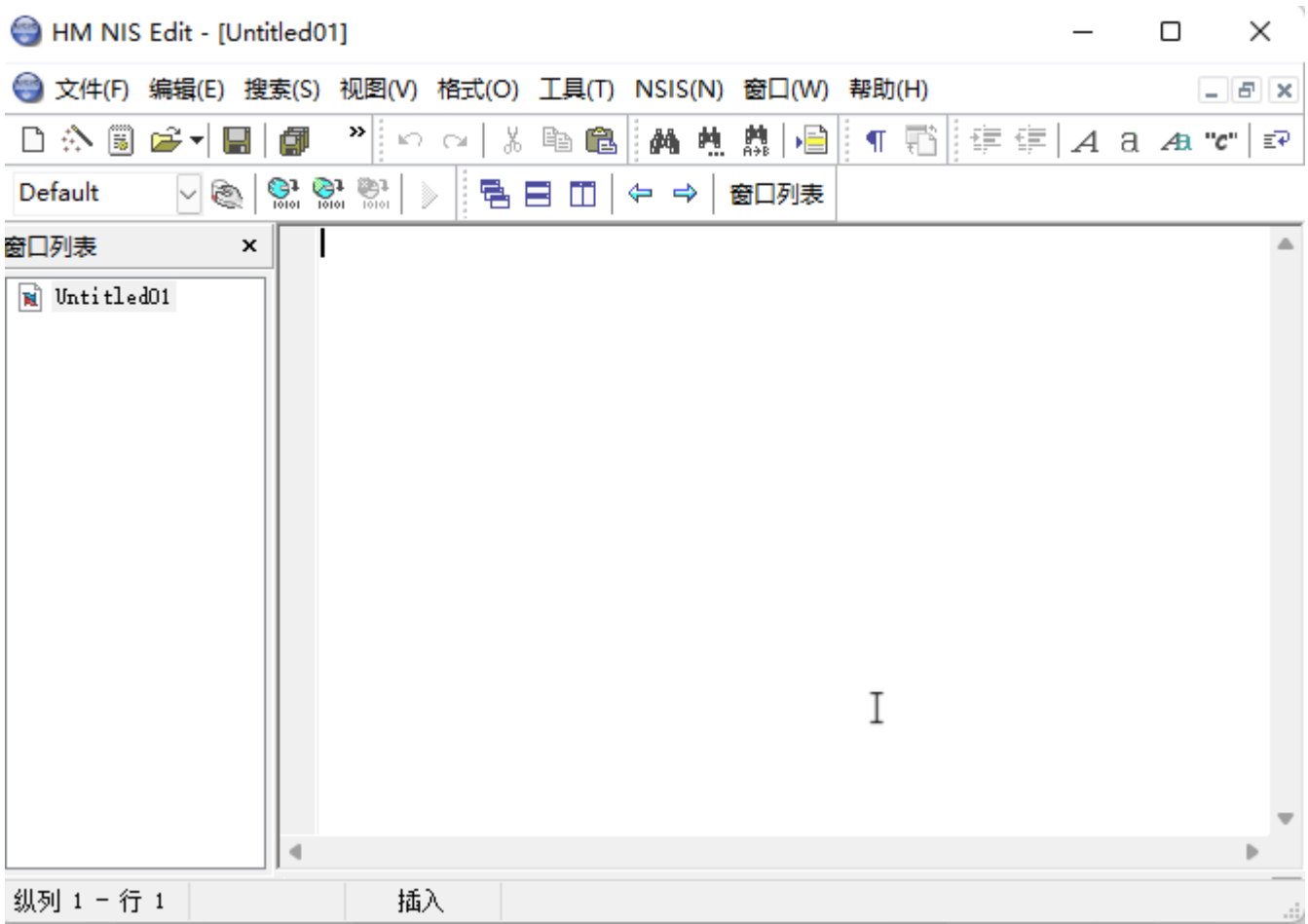


Figure 83

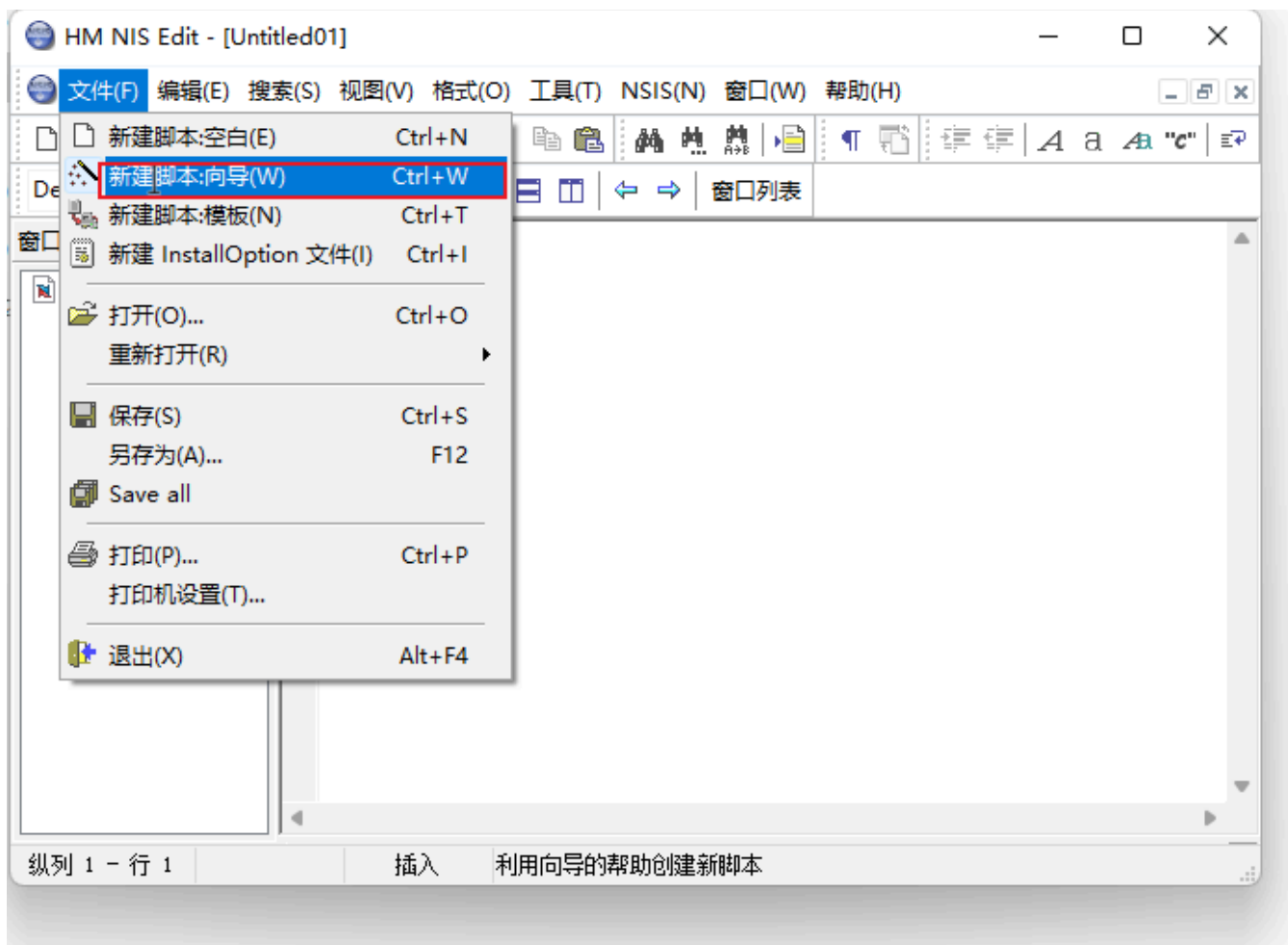


Figure 84

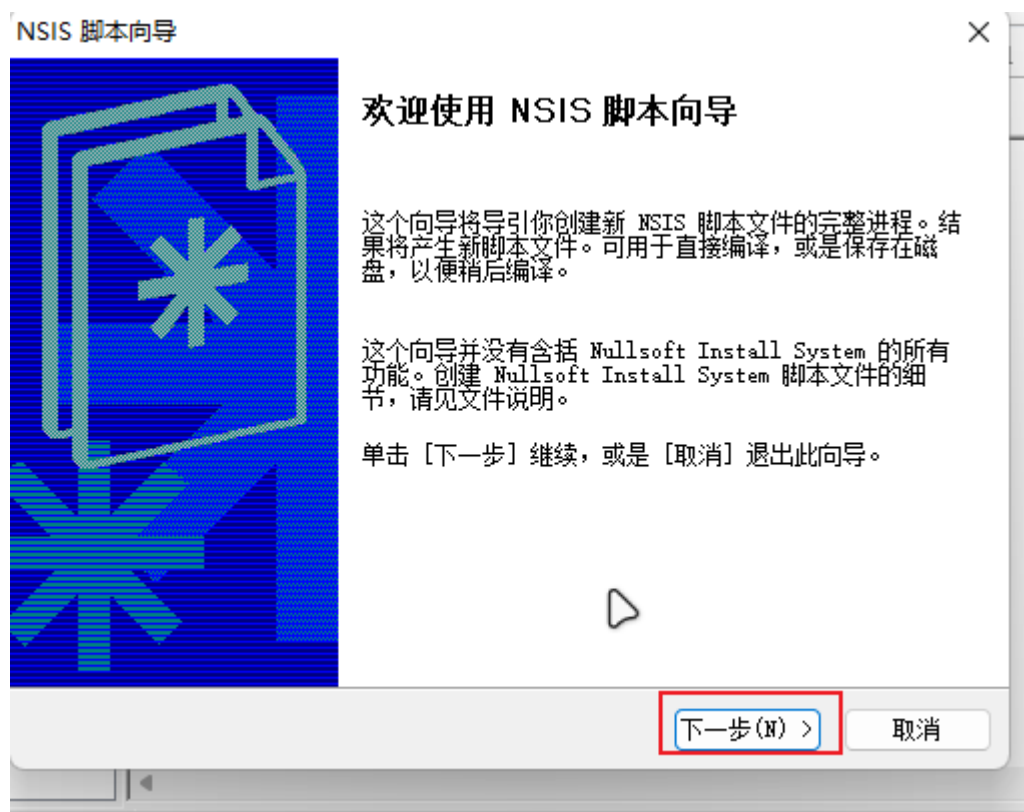


Figure 85

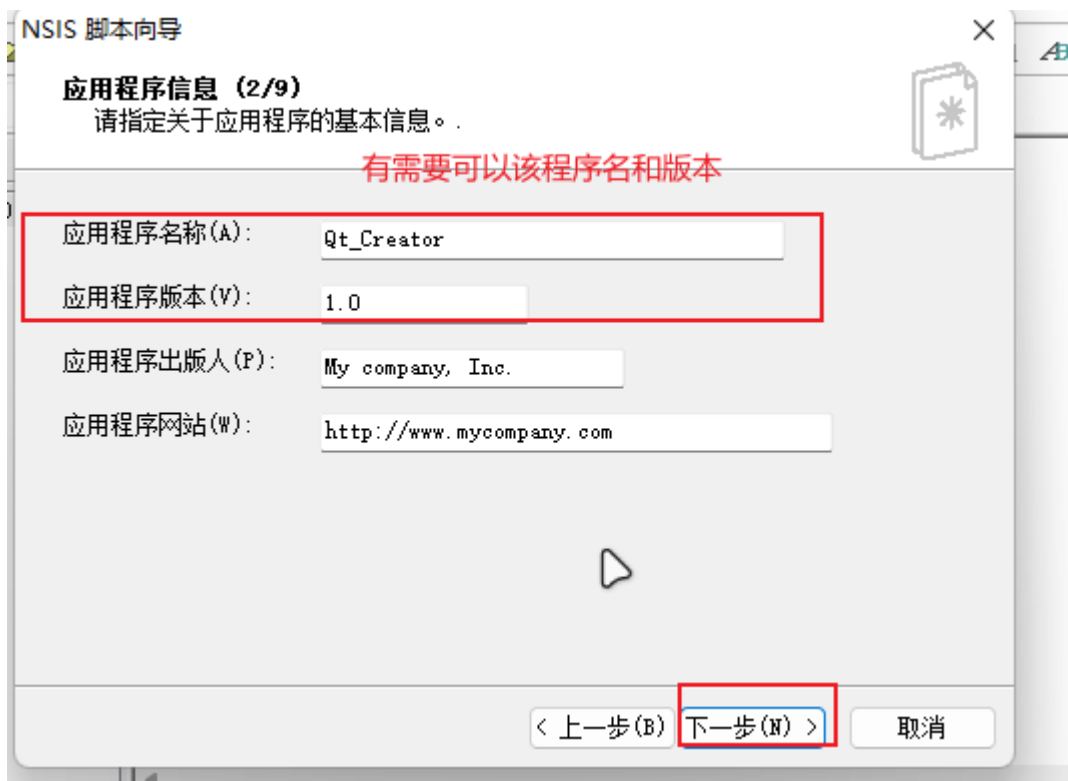


Figure 86

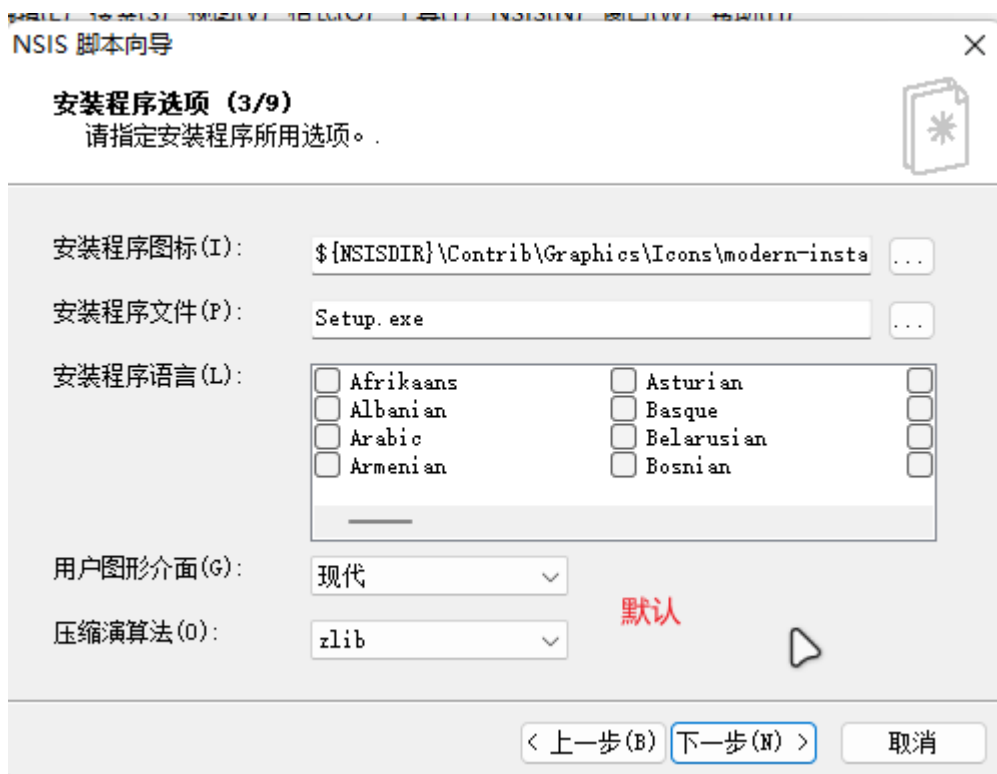


Figure 87

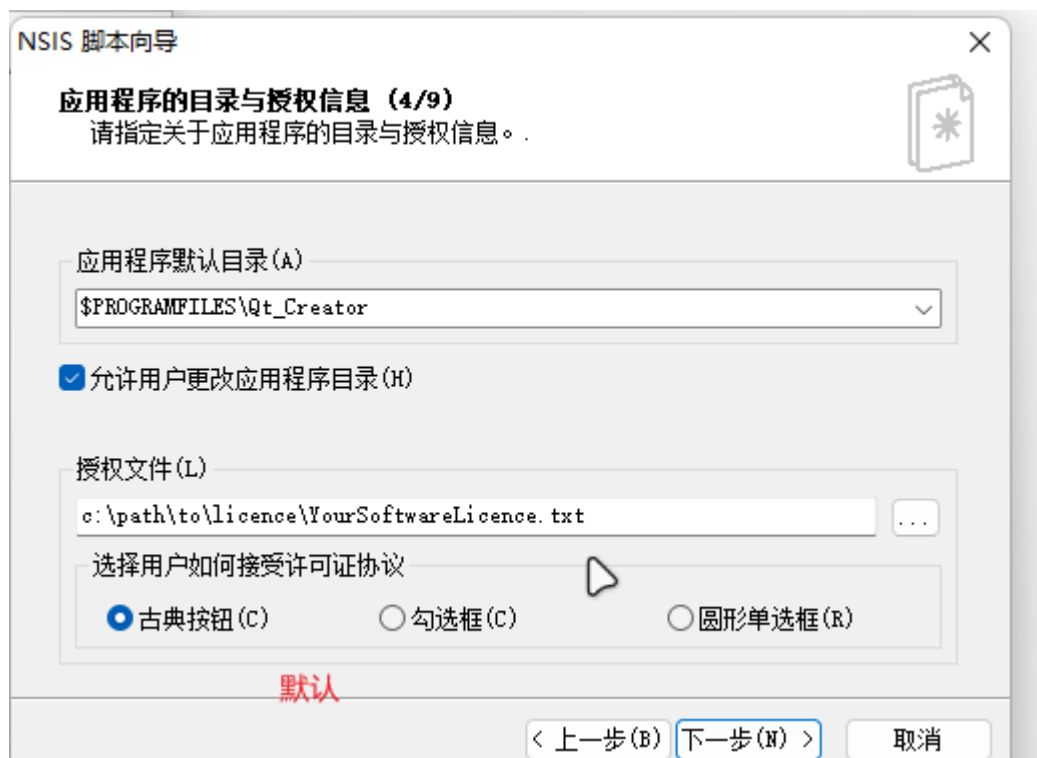


Figure 88

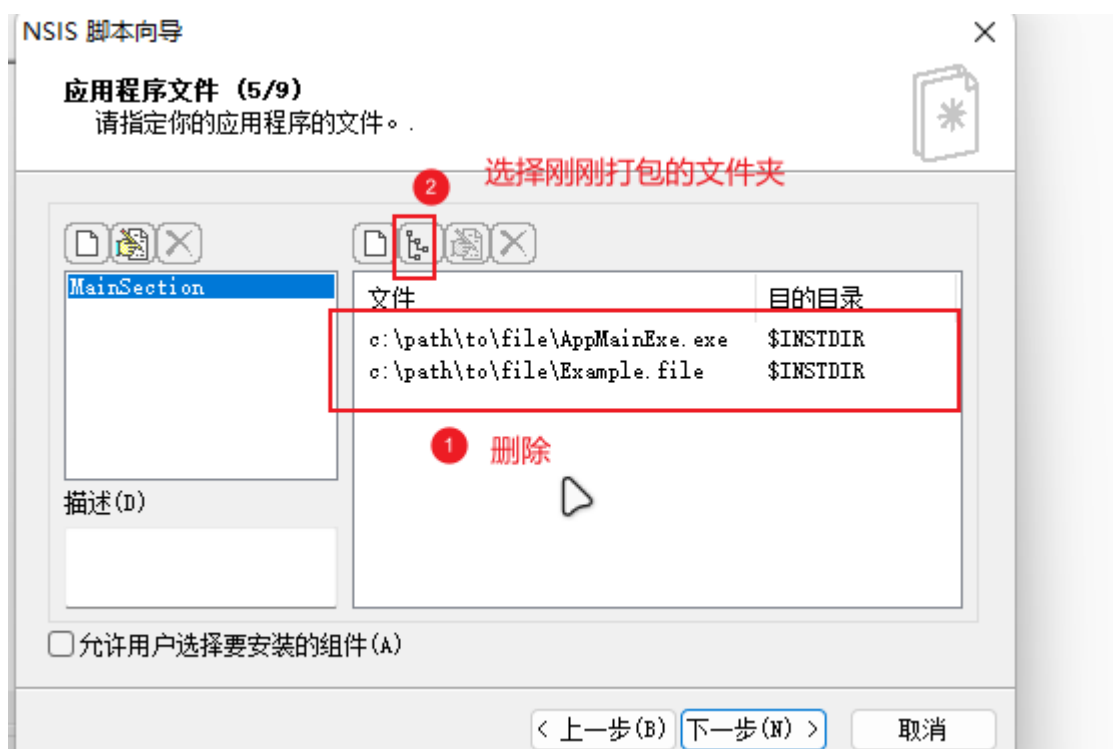


Figure 89

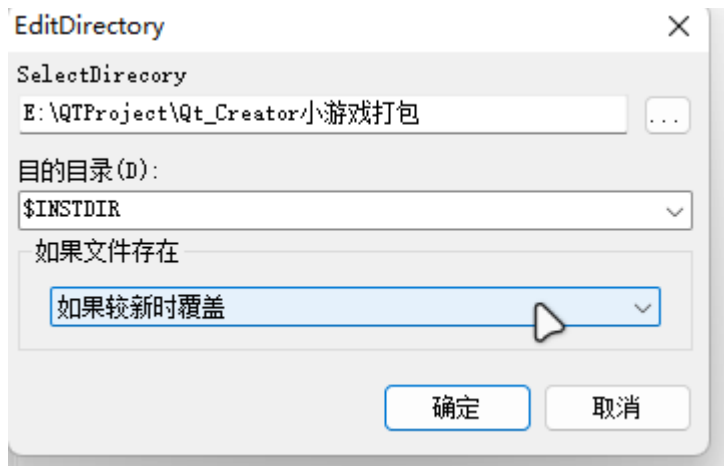


Figure 90



Figure 91



Figure 92

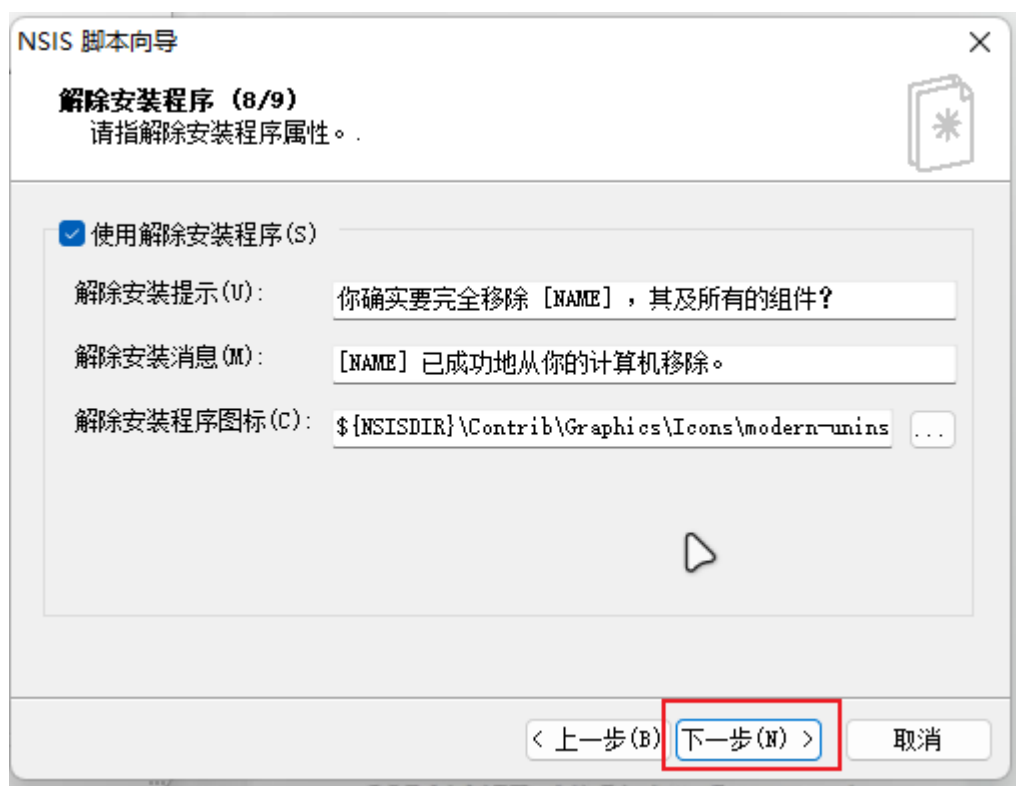


Figure 93



Figure 94

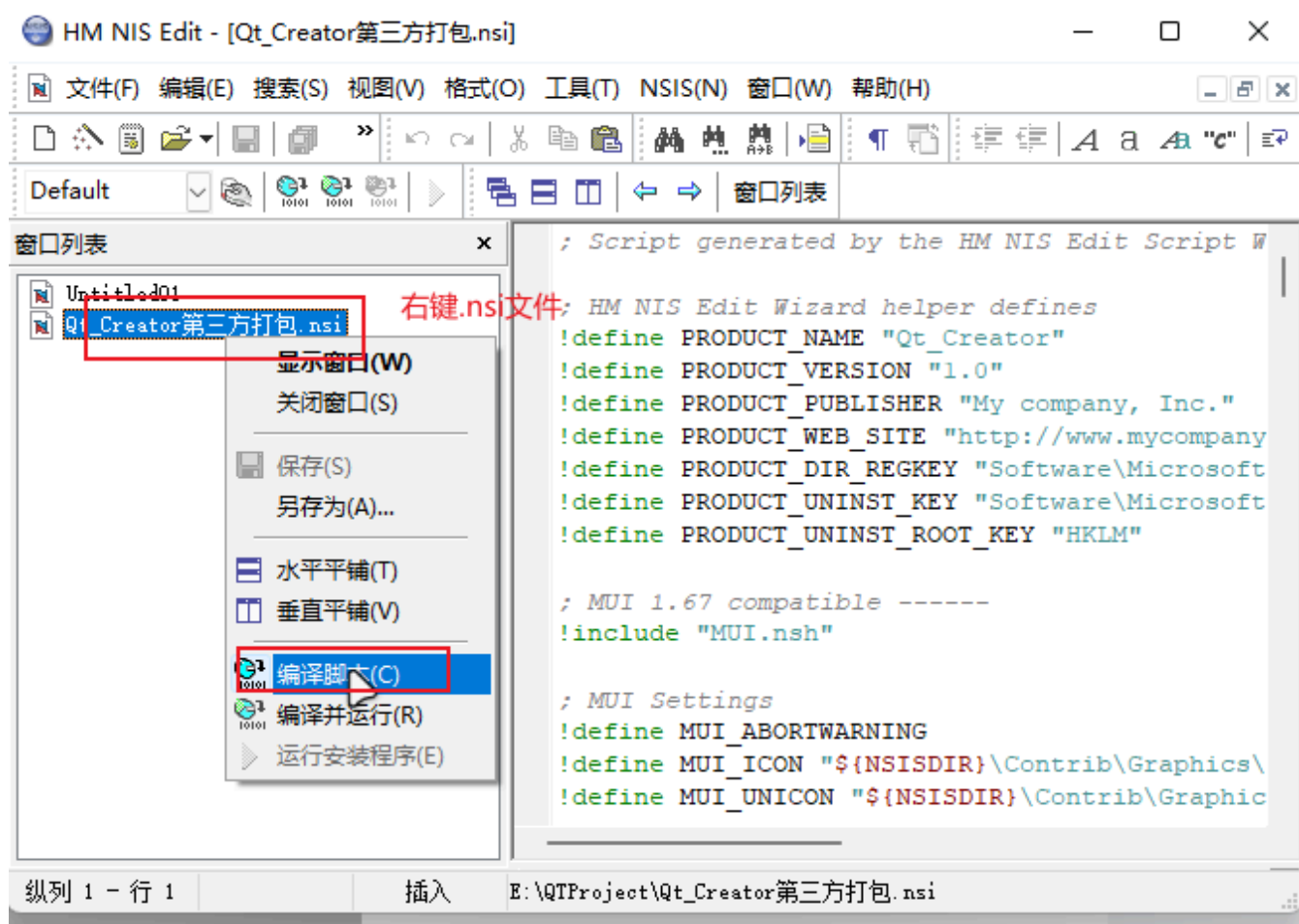


Figure 95

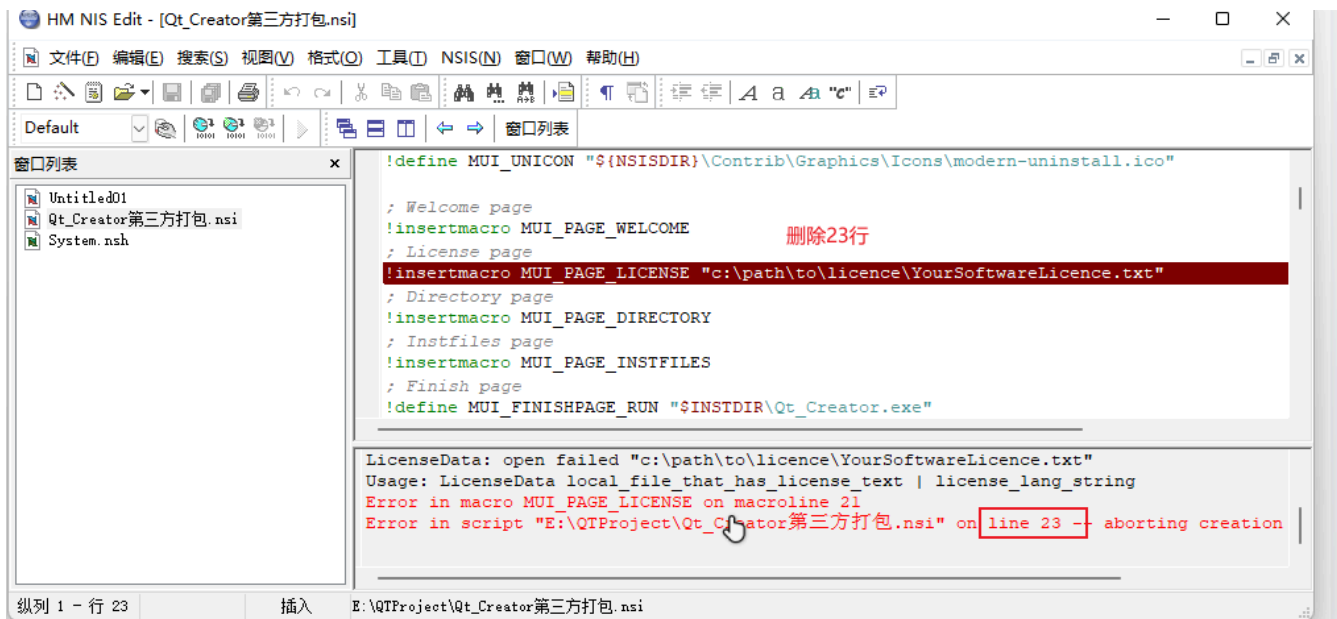


Figure 96

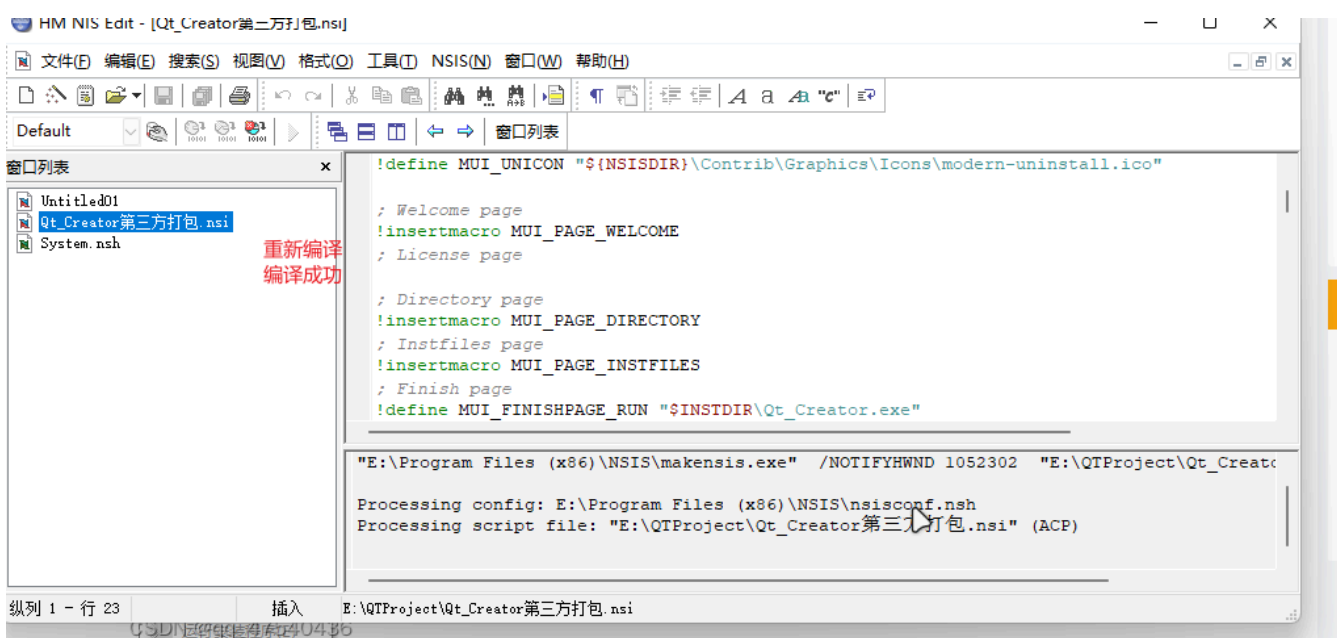


Figure 97

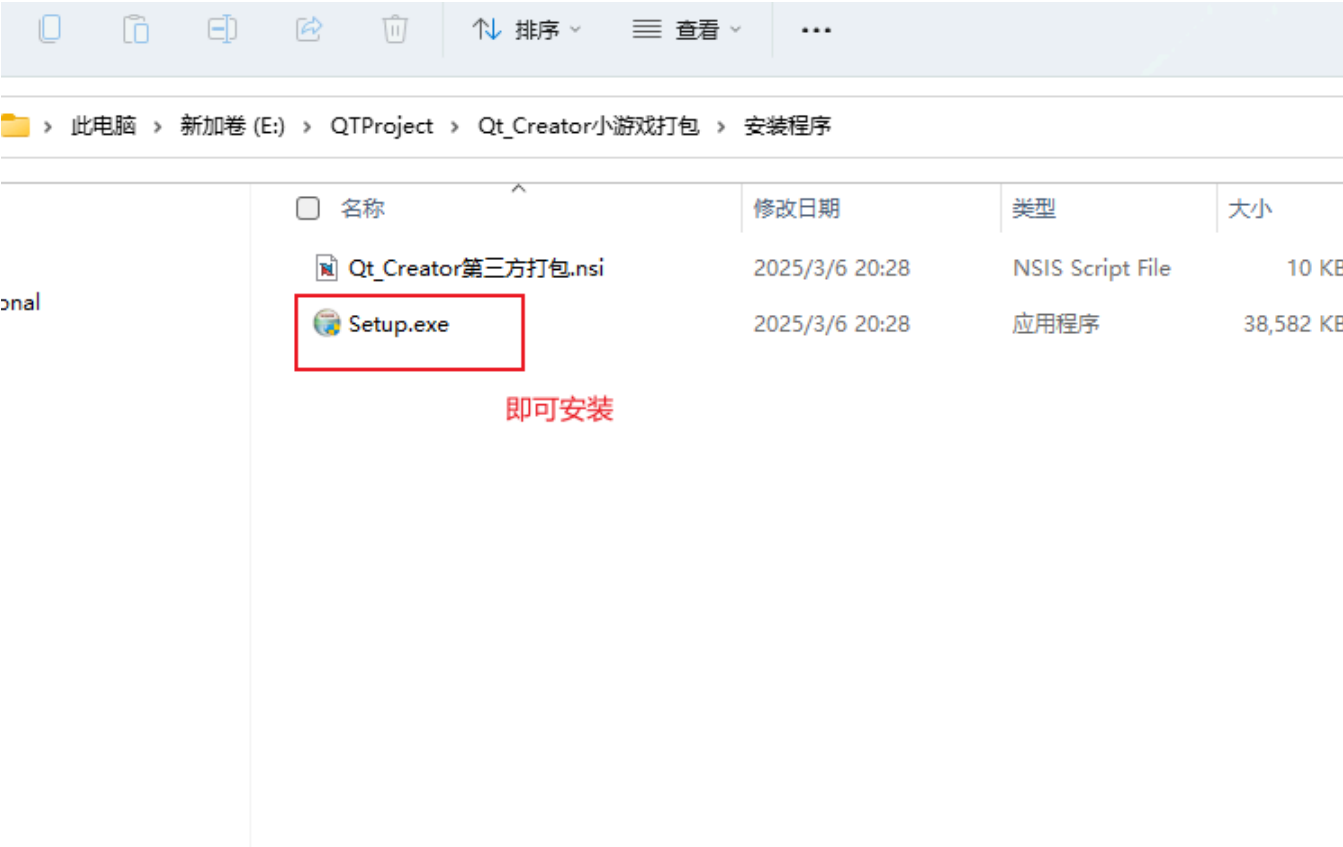


Figure 98